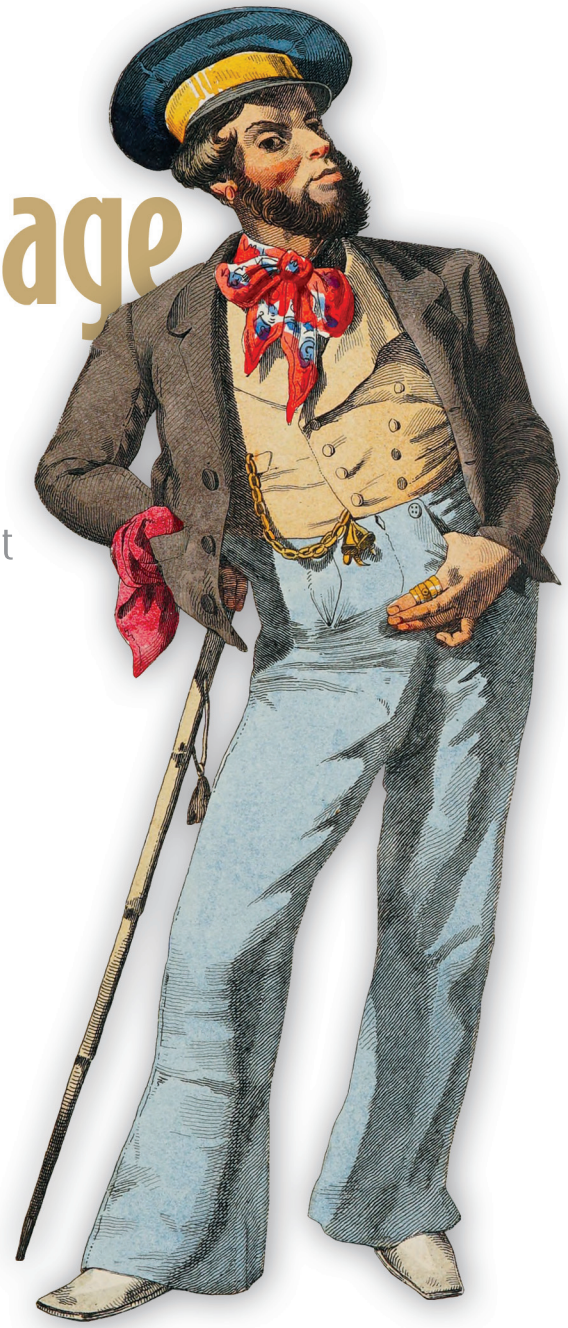
DOMAIN-SPECIFIC

Small Language Models

Efficient AI for local deployment



Guglielmo Iozzia

Foreword by Matthew R. Versaggi

MANNING

Domain-Specific Small Language Models

Domain-Specific Small Language Models

Efficient AI for local deployment

Guglielmo Iozzia

Foreword by Matthew R. Versaggi

MANNING
Shelter Island

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity.

For more information, please contact

Special Sales Department
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964
Email: orders@manning.com

© 2026 Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The author and publisher have made every effort to ensure that the information in this book was correct at press time. The author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.

Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Development editor:	Dustin Archibald
Technical editor:	Riccardo Mattivi
Review editor:	Angelina Lazukić
Production editor:	Aleksandar Dragosavljević
Copy editor:	Andy Carroll
Proofreader:	Melody Dolab
Technical proofreader:	Bin Hu
Typesetter:	Tamara Švelić Sabljić
Cover designer:	Marija Tudor

ISBN 9781633436701
Printed in the United States of America

brief contents

PART 1 FIRST STEPS 1

1 ■ Small language models 3

PART 2 CORE DOMAIN-SPECIFIC LLMS 15

2 ■ Tuning for a specific domain 17
3 ■ End-to-end transformer fine-tuning 36
4 ■ Running inference 51
5 ■ Exploring ONNX 71
6 ■ Quantizing for your production environment 96

PART 3 REAL-WORLD USE CASES 119

7 ■ Generating Python code 121
8 ■ Generating protein structures 145

PART 4 ADVANCED CONCEPTS 163

9 ■ Advanced quantization techniques 165
10 ■ Profiling insights 184
11 ■ Deployment and serving 206
12 ■ Running on your laptop 235

13 ■ Creating end-to-end LLM applications 258
14 ■ Advanced components for LLM applications 292
15 ■ Test-time compute and small language models 326

contents

foreword xii
preface xiv
acknowledgments xv
about this book xvii
about the author xix
about the cover illustration xx

PART 1 FIRST STEPS 1

1 *Small language models* 3

1.1 What are small language models? 4
1.2 The Transformer architecture 5
1.3 Areas of application 6
1.4 The open source revolution 6
1.5 Risks and challenges with generalist LLMs 10
1.6 Domain-specific vs. generalist LLMs for business value 11

PART 2 CORE DOMAIN-SPECIFIC LLMS 15

2 *Tuning for a specific domain* 17

2.1 Data preparation 17
Data preparation for BERT fine-tuning 18 ▪ *Data preparation for GPT fine-tuning* 20 ▪ *Data preparation for RAG* 22

2.2 Retrieval-augmented generation 24
2.3 Fine-tuning 25
2.4 LoRA 29
2.5 RAG or fine-tuning? 34

3 *End-to-end transformer fine-tuning 36*
3.1 Data preparation 36
3.2 Fine-tuning 38
3.3 Testing the fine-tuned model 41
3.4 Domain-specific evaluation 44

4 *Running inference 51*
4.1 How to generate content 51
Text completion 52 ▪ Few-shot learning 55 ▪ Code generation 56 ▪ Evaluating the generated content 57
4.2 Calculating inference cost 59
4.3 Areas for improvement (cost savings and performance) 60
Getting the most from your GPU 60 ▪ Batching 64 Estimating the generation time 65 ▪ Optimizing GPU use with DeepSpeed 66

5 *Exploring ONNX 71*
5.1 The ONNX format 71
5.2 ONNX operators and types 74
5.3 The ONNX runtime 81
5.4 ONNX runtime providers 82
5.5 ONNX for LLMs on CPU 84
5.6 ONNX for LLMs on GPU 88
ONNX for GPT on GPU 89 ▪ I/O binding 92

6 *Quantizing for your production environment 96*
6.1 Transformers precision formats 96
6.2 8-bit quantization 99
Hands-on 8-bit quantization 100 ▪ LLM.int8() and quantization 104

6.3 8-bit quantization with ONNX 107

6.4 4-bit quantization 112

4-bit quantization with GPTQ 112 ▪ 4-bit quantization with ggml 114

PART 3 REAL-WORLD USE CASES 119

7 *Generating Python code 121*

7.1 Using Transformers to generate code 121

7.2 Generating Python code with a Transformer architecture 123

Python code generation with CodeGen 123 ▪ Using ONNX with models not supported by Optimum 133 ▪ Model evaluation 134 Python code generation with better models 137

7.3 Coding assistance on commodity hardware 140

8 *Generating protein structures 145*

8.1 Applying Transformers in chemistry 145

8.2 From natural language to protein structures 147

8.3 Antibody generation with an SLM 149

8.4 From CIF files to crystal structures 153

PART 4 ADVANCED CONCEPTS 163

9 *Advanced quantization techniques 165*

9.1 FlexGen 166

9.2 SmoothQuant 171

9.3 BitNet 176

9.4 BitNet and Python 179

10 *Profiling insights 184*

10.1 Profiling ONNX-ported LLMs 184

10.2 Transforming raw ONNX profiling data into insights 188

10.3 Optimization of ONNX graphs for LLMs 197

11 *Deployment and serving 206*

11.1 vLLM 206

Offline serving 208 ▪ Online serving 212

11.2 FastAPI 215

Benchmarking various models 218 ▪ Deploying the best-performing model with FastAPI 222

11.3 MLC LLM 223

11.4 Deployment and inference on Android devices 229

MLC LLM framework 229 ▪ MLLM framework 229 Hugging Face Transformers 231

12 *Running on your laptop 235*

12.1 Why use a personal local assistant 236

12.2 Running an LLM locally with Ollama 236

Importing a custom model into Ollama 240 ▪ User privacy in Ollama 243

12.3 Running an LLM locally with LM Studio 244

12.4 The LM Studio Python SDK 248

12.5 Running an LLM locally with Jan 251

12.6 The Cortex local LLM engine 253

13 *Creating end-to-end LLM applications 258*

13.1 Why LLMs alone aren't enough 259

13.2 Combining a domain-specific SLM with RAG 261

13.3 Using a vector database 275

13.4 Building an agent 279

14 *Advanced components for LLM applications 292*

14.1 GraphRAG 292

Microsoft's open source GraphRAG capabilities 305 Evaluation metrics 306

14.2 RAG + Agentic AI 307

14.3 Long- and short-term memory management 320

15 *Test-time compute and small language models 326*

15.1 Test-time compute 326

15.2 The OptiLLM inference proxy 328

15.3 SLMs with embedded test-time compute 337

15.4 Building a reasoning domain-specific SLM 338

index 347

foreword

It is truly an honor to receive a request to write a foreword for a book like this, in this particular space, at this specific time in history, and by an author who I believe will make significant contributions to this industry during his tenure. That book is *Domain-Specific Small Language Models*, the space is generative AI, the time in history is the tremendous evolutionary acceleration of AI, and the author is Guglielmo Iozzia.

In an era when AI discourse is dominated by ever-larger frontier models and eye-catching demos, this book takes a refreshingly pragmatic stance: it is about making language models actually work for your domain, your data, and your hardware constraints. It is written with the practitioner's learning curve squarely in mind, moving from foundational concepts to hands-on experience so that by the time you turn the last page, you will not only understand why small, domain-specific language models matter, you will have run them, tuned them, optimized them, and deployed them on real-world infrastructure.

What distinguishes this work is its relentless focus on the realities of distributed and edge AI. Rather than treating GPUs in the cloud as an infinite resource, the author leans into the constraints most organizations actually face: tight budgets, on-prem clusters, laptops, and even mobile or embedded devices. From the opening chapters on the definition and role of small language models (SLMs), it becomes clear that the goal is not to worship scale, but to show how carefully chosen, well-tuned models in the 100M to 10B parameter range can deliver outsized business value when pushed as close as possible to where data is generated and decisions are made. The walkthroughs of ONNX, quantization techniques, and hardware-aware optimizations anchor the discussion in concrete practices that materially reduce latency and cost at the edge while preserving useful accuracy.

This is also an excellent grounding text for anyone trying to navigate the LLM-versus-SLM decision space with intellectual honesty. The early chapters provide a clear, hype-free tour of the Transformer architecture, training paradigms, open source ecosystems, and the tradeoffs between generalist closed LLMs and domain-specific models. From there, the book steadily builds into a toolkit: data preparation, classic fine-tuning, PEFT methods such as LoRA, and retrieval-augmented generation are all presented with enough depth and runnable code that you can reproduce, adapt, and extend the examples for your own environment. The result is a rare combination of conceptual clarity and practical guidance that serves both as an introduction for those new to the field and as a structured reference for experienced practitioners formalizing their approach.

Perhaps most compelling is the way the book repeatedly returns to domain reality. Coding assistants are explored not as abstractions but via end-to-end fine-tuning of GPT-2 on Manim code, complete with hyperparameter search and evaluation workflows that mirror production-grade ML engineering. Chemistry and life sciences are treated with equal seriousness, from protein and antibody sequence generation to discussion of validation workflows and specialized tooling, illustrating how SLMs unlock high-value use cases in regulated, data-sensitive industries. Later chapters connect these capabilities to modern application patterns—RAG pipelines, agentic systems, deployment and serving patterns, and running on commodity hardware—so that readers finish with a mental model that spans from theory to architecture to code.

If you are serious about applying generative AI beyond glossy prototypes—especially in domains where data is sensitive, regulation is strict, or infrastructure is constrained—this book is worthy of your time. It will not just tell you that small language models are important; it will show you, step by step, how to build, adapt, and ship them responsibly in the environments where they matter most.

—Matthew R. Versaggi,
White House Presidential Innovation Fellow in AI, Special cohort of '24,
former senior director of artificial intelligence and
cognitive technology at Optum Technology/United Healthcare

preface

In early 2022, after finally having the time to complete reading the "Attention is all you need" paper, I started getting curious about the Transformer architecture and the potential great use cases in diverse industries (and in life sciences in particular, because that is the field where I work) for this new kind of neural network. One concern I had at that time was the concrete risk that such technology could quickly become a prerogative of the large tech organizations that could afford the vast computational resources necessary to train and execute these models.

Then, inspired by the "run Doom on anything" challenge (a popular trend for software engineers to find ways to optimize the source code of that 1993 videogame to run on any sort of device), I started thinking about ways to optimize small Transformer models, trained on domain-specific tasks and data, so they could be deployed and executed in hardware-constrained environments. In June of the same year, I did an in-person hands-on workshop on this subject at the ODSC Europe conference in London, which got a lot of interest from ML engineers at the event.

When Manning Publications contacted me a few months later, asking me to write a book on this topic, I didn't think twice about accepting. A few weeks after I started writing the book, OpenAI released the ChatGPT service to the public, powered at that time by their GPT-3.5 model, and other large tech organizations, such as Google, Anthropic, and X.AI, soon joined the race to build larger, closed source generalist models to try to win the consumer market for generative AI.

But the open source community didn't just watch this happen: lots of initiatives involving open architectures and weights, and techniques for optimize the training and execution of language models, started being proposed. It was the right time to write a book on this topic and provide ML practitioners with a centralized source of information about what capabilities are available beyond the commercial offerings, for specific tasks, and in heavily regulated industries.

acknowledgments

I want to say thanks first of all to Jonathan Gennick, who reached out to ask if I was interested in sharing my experience on this topic through a book and for helping me through the nontrivial process of building the book proposal in such a way that it might be approved by the publisher. Thanks to Jonathan, and Manning Publications in general, for the support I received when, in the middle of writing, I had to stop for some months to address some urgent personal matters. Thanks to Dustin Archibald for all his precious feedback on making this book better. Thanks to Riccardo Mattivi, a senior AI and Machine Learning engineer at Integral Ad Science (IAS), who previously led AI engineering and Machine Learning initiatives at Mastercard, for serving as a technical editor on the book. And thanks, of course, to all the other Manning people who assisted during the different phases of this book, from inception to production.

To all the reviewers, Ajay Tanikonda, Alessandro Negro, Anand Nair, André Claudino, Andrew R. Freed, Andrey Belov, Carmine Giardino, Chandrashekar Konda, Clyde Kallahan, Dario Morelli, Dmitrii Volkov, Dmitry Kolodezev, Elif Ozkirimli, Fabio Montagna, Giampiero Granatella, Gregorio Piccoli, Harikrishnan Muthukrishnan, Hilde Van Gysel, Ho Chek Hui, Indraneel Rao, Karl-Gustav, Lola Vicente, Lukasz Kupka, Manchor Ko, Manish Jain, Manju Rupani, Marco Sassarini, Maxim Volgin, Mikael Dautrey, Muhammad Saqib, Nicolas Bievre, Prasad Rao Thota, Praveen Nair, Raj Kumar, Rani Sharim, Richard Vaughan, Sam Prakash Bheri, Samuel Lawrence, Saurabh Aggarwal, Sebastien Tardif, Shailesh Thakur, Shreesha Jagadeesh, Simone Sguazza, Srivathsan Srinivas, Surya Kasturi, Teo Argentieri, Thomas Tellner, Vibhas Zanpure, Walter Alexander Mata López, and Xin Hu, your suggestions helped make this a better book.

about this book

The focus of this book is on understanding techniques for improving inference performance and costs on pretrained and customized small language models (SLMs) through optimization and quantization, serving them through diverse API ecosystems, deploying them on diverse hardware (including your own laptop), and integrating them with other paradigms such as RAG and Agentic AI. All these concepts are explained in depth and come with complete source code examples. You'll learn to minimize the computational horsepower their models require while retaining high–quality performance times and output.

While a few examples presented in this book describe how to preprocess the data for training/test, and PEFT (parameter-efficient fine tuning) techniques are also introduced, this book doesn't focus on training and data-preparation techniques.

Who should read this book

This book is first for ML engineers and data scientists interested in learning how to manage LLMs in the typical hardware-constrained environment that their company budget allows for. But it is also for tech leaders motivated to understand how applying custom language models on corporate data could generate extra business value.

The minimally qualified reader should have the following skills and knowledge:

- General understanding of deep learning concepts
- Knowledge of the Transformer model's architecture and internals and the basics of their training process
- Intermediate Python language skills
- Hands-on experience with a deep learning framework (PyTorch preferable)

- Familiarity with the Google Colab environment
- Intermediate experience in software engineering
- Three or four years of experience as a machine learning engineer or data scientist

How this book is organized: A roadmap

This book has 4 parts with 15 chapters.

Part 1 introduces readers to small language models (SLMs):

- Chapter 1 explains the use cases for specialized SLMs and their pros and cons when compared to generalist large language models.

Part 2 covers the core topics of domain-specific SLMs:

- Chapter 2 shows some examples of data preparation for fine-tuning Transformer architectures and for RAG, and it introduces the concept of parameter-efficient fine-tuning (PEFT).
- Chapter 3 presents an end-to-end example of tuning an SLM on a specific task (the generation of Python code to render/animate mathematical formulas, starting from natural language). It also addresses some validation strategies for the model's output.
- Chapter 4 illustrates several types of content generation with SLMs, shows how to spot areas for improvement at inference time (in terms of cost savings and performance), and suggests some techniques for optimizing computational power consumption.
- Chapters 5 and 6 are about the ONNX format and ONNX Runtime, SLM quantization on CPU or GPU, and quantization strategies using the ONNX standard and alternative methods.

Part 3 focuses on real-world examples of the concepts introduced in parts 1 and 2:

- Chapter 7 presents some use cases of generating Python code with SLMs and optimizing SLMs to run on constrained hardware.
- Chapter 8 presents some SLMs pretrained to address tasks in chemistry, drug discovery, and materials science.

Part 4 discusses more advanced SLM concepts:

- Chapter 9 details advanced quantization techniques.
- Chapter 10 explains how to transform raw ONNX profiling data into insights to improve model optimization.
- Chapter 11 takes a deep dive into options for deploying and serving SLMs across diverse environments, including Android devices.
- Chapter 12 walks through multiple options for serving and running SLMs locally on a laptop and using a graphical interface.

- Chapter 13 is about using SLMs as part of more complex systems, such as RAG or agentic AI.
- Chapter 14 expands on the concepts discussed in chapter 13, introducing GraphRAG and agentic RAG.
- Chapter 15 is about test-time compute for SLMs, and it ends the book with an end-to-end example of turning an SLM into a reasoning model using GRPO.

About the code

This book contains many source code examples to give you hands-on experience with the concepts explained in the various chapters. To improve readability, the book often presents only the essential lines of code. Source code is formatted in a `fixed-width font like this` to distinguish it from ordinary text. Sometimes code is also presented **`in bold`** to highlight code that has changed from previous steps in the chapter, such as when a new feature adds to an existing line of code.

In many cases, the original source code has been reformatted; we've added line breaks and reworked indentation to accommodate the available page space in the book. In rare cases, even this was not enough, and listings include line-continuation markers (➥). Additionally, comments in the source code have often been removed from the listings when the code is described in the text. Code annotations accompany many of the listings, highlighting important concepts.

The complete examples, in the form of Colab notebooks, are available for download from the Manning website at https://www.manning.com/books/domain-specific-small-language-models and from GitHub at https://github.com/virtualramblas/Domain-Specific-Small-Language-Models.

liveBook discussion forum

Purchase of *Domain-Specific Small Language Models* includes free access to liveBook, Manning's online reading platform. Using liveBook's exclusive discussion features, you can attach comments to the book globally or to specific sections or paragraphs. It's a snap to make notes for yourself, ask and answer technical questions, and receive help from the author and other users. To access the forum, go to https://livebook.manning.com/book/domain-specific-small-language-models/discussion. You can also learn more about Manning's forums and the rules of conduct at https://livebook.manning.com/discussion.

Manning's commitment to our readers is to provide a venue where a meaningful dialogue between individual readers and between readers and the author can take place. It is not a commitment to any specific amount of participation on the part of the author, whose contribution to the forum remains voluntary (and unpaid). We suggest you try asking the author some challenging questions lest his interest stray! The forum and the archives of previous discussions will be accessible from the publisher's website as long as the book is in print.

about the author

Guglielmo Iozzia is a Director of ML/AI and Applied Mathematics at Merck & Co. He studied Electronic and Biomedical Engineering at the University of Bologna, has an extensive background in Software and ML/AI Engineering applied to real-life use cases across different industries, such as Biotech Manufacturing, Healthcare, Cloud Operations and Cyber Security, and is a lifelong learner. He strongly believes that real progress in AI can happen only through Open Source.

about the cover illustration

The figure on the cover of *Domain-Specific Small Language Models*, titled "Il Guappo," or "The tough guy," is taken from a book by Francesco de Bourcard, published in 1853. Each illustration is finely drawn and colored by hand.

In those days, it was easy to identify where people lived and what their trade or station in life was just by their dress. Manning celebrates the inventiveness and initiative of the computer business with book covers based on the rich diversity of regional culture centuries ago, brought back to life by pictures from collections such as this one.

Part 1

First steps

This first part of the book will introduce small language models (SLMs), explain the risks and challenges of using generalist LLMs, and discuss how these can be mitigated by switching to SLMs.

1 Small language models

This chapter covers

- Defining small language models
- Limits and risks of using closed source generalist LLMs
- Open source alternatives to closed source LLMs
- Comparing LLMs and domain-specific language models

Whenever a new, potentially disruptive technology emerges, or an existing one undergoes a radical change, there's inevitable hype about it. If expectations aren't met, interest can drop quickly; if they are, investments and projects can surge before the real business potential and risks are understood. Either way, the "next big thing" rarely ends well. You can see the same thing happening today with large language models (LLMs): there's so much hype that it's difficult to understand what they really can (and can't) do and whether they would be a good fit for your own business needs. This chapter aims to clarify the LLM world by highlighting the pros, risks, and challenges of both generalist and domain-specific models. Hype is the one thing you won't find in this book.

1.1 What are small language models?

Today, regardless of their technical background, most people are familiar with popular commercial LLMs such as GPT, Claude, and Gemini, which power chatbots and services from providers such as OpenAI, Anthropic, Google, and Microsoft. But there's still limited awareness of a related category: small language models (SLMs). Like LLMs, SLMs are designed for natural language processing tasks, but they use fewer parameters, have a smaller memory footprint, and require less computational power. That makes them suitable for applications that run on mobile or edge devices or on-premises servers or small clusters. SLMs typically range from a few hundred million to a few billion parameters; there's no official cutoff, but most stay under 10 billion parameters. That's much smaller than LLMs, which often have hundreds of billions of parameters or more. Because of their small size, SLMs can be optimized for speed and energy efficiency, which are especially important for on-device or commodity hardware deployments. They can run well on CPUs (when no GPU is available) or on consumer GPUs. Since they run locally, the data can stay local, which is important for tasks involving sensitive or personal information. Their deployment flexibility also expands the range of use cases, making them ideal for offline, embedded, and near–real time applications in areas such as robotics and smart devices (such as mobile phones, lab equipment, appliances, and manufacturing machinery).

Although SLMs are smaller than LLMs, they aren't a new type of model. Both are built on the same core technology—the Transformer architecture and later improvements. The difference is mostly in scale, not the underlying mathematical and architectural principles. As a result, they can address the same kinds of tasks, with SLMs being optimized more for efficiency than raw power.

> **NOTE** The Transformer architecture was introduced in the "Attention is All You Need" paper by Ashish Vaswani and others (https://arxiv.org/abs/1706.03762).

An important benefit of SLMs is that you can specialize them for specific domains and tasks at relatively low cost, using domain-specific and private data plus domain knowledge, and apply techniques that don't require full fine-tuning. Domain-specific tasks for language models are common in heavily regulated industries where privacy isn't optional—healthcare, pharma/biotech, manufacturing, chemistry, finance, and others. You'll see real-world examples in later chapters.

Beyond the well-understood reasons to choose SLMs over LLMs in specific use cases, another emerged with the rise of agentic AI and was highlighted in a mid-2025 NVIDIA research paper: "Small Language Models are the Future of Agentic AI" (https://arxiv.org/abs/2506.02153). The authors argue that SLMs are powerful enough, better suited, and more economical for many uses in agentic systems, making them central to the future of agentic AI. They also suggest that, even when general-purpose conversation is required, heterogeneous agentic systems—agents that invoke multiple models,

LLMs and SLMs—are the natural choice. The final chapters of this book explore the relationship between SLMs and agentic AI.

1.2 The Transformer architecture

Before transformers, most tasks on unstructured text were performed using recurrent neural network (RNN) models. Trained on labeled data, these architectures use their internal state to process variable-length sequences of text. Later evolutions, such as LSTM (long short-term memory) or GRU (gated recurrent unit) architectures, introduced the capability to remember and forget important information, but they didn't solve one of the major challenges of RNNs: slow training. The data must be processed sequentially, which limits parallelism across processor cores. The attention mechanism addressed some RNN limitations by emphasizing relevant parts of the input and downplaying others, effectively expanding the amount of information the model could carry forward. Still, it didn't solve the scalability issues. That required the Transformer model, which introduced two major changes compared to RNNs:

- It uses the self-attention mechanism (a specialization of the attention mechanism that relates different positions in a sequence to compute a representation of the sequence itself) so that it can process the entire input all at once.
- It has no recurrent structure, so a transformer allows more parallelism than an RNN, significantly reducing training time.

Transformers didn't only succeed because of their architecture. Another key component is word embeddings—high-dimensional vector representations of words that capture their semantic and syntactic properties. Embeddings let a Transformer model manipulate words in a numerical space, where related words end up near one another. This helps the model treat words as connected, not isolated, entities and understand how they interact in context.

Transformer architectures have also been proposed for tasks involving other data types, but multimodal use cases are out of scope for this book, which focuses on generative models for unstructured text.

The Transformer architecture initially proposed by Google Brain was just the start. As soon as researchers and professionals started to use it, they quickly found ways to improve and tailor it for text tasks. The original architecture was encoder–decoder based: the encoder part converts the input text into a numerical representation, while the decoder part takes that representation and converts it back to text. Both use an attention mechanism. Over time, it has become clear that some tasks need only the encoder, while others can rely only on the decoder. This led to two different families of transformers:

- BERT (bidirectional encoder representation from transformers) uses only the encoder part. This kind of architecture performs better on text classification or prediction tasks.
- GPT (generative pretrained transformer) uses only the decoder and generally performs better on text generation tasks.

Ultimately, your architecture choice will depend on the tasks you want to accomplish. The examples in this book cover both families.

Another important concept in the Transformer model's evolution is reinforcement learning from human feedback (RLHF), a technique used behind the scenes for OpenAI's LLMs. As the name suggests, it applies reinforcement learning so an LLM can learn a policy that maximizes a reward. The models behind ChatGPT are an evolution of the GPT architecture (the first two versions were open source; later versions moved to closed source), with RLHF applied for fine-tuning and improved performance.

1.3 Areas of application

The term "LLM" can be misleading. "Large" is clear enough: models range from a few billion to more than 100 billion parameters, or from tens to hundreds of gigabytes. But "language" can make people think that these models generate only natural-language text. Early natural-language processing (NLP) applications did focus on translation tasks, but newer architectures and capabilities let LLMs handle many tasks beyond translation:

- Language understanding
- Text classification
- Test generation
- Question answering
- Document summarization
- Semantic parsing
- Pattern recognition
- Basic math solving
- Code generation
- Dialogue
- General knowledge
- Logical inference chains

This makes it possible for an LLM to handle text beyond natural language—anything that symbolically represents something—as shown in figure 1.1.

LLMs will unlock many real-world use cases across industries.

1.4 The open source revolution

The Transformer architecture didn't only ignite interest among companies such as OpenAI, Microsoft, and Google; the open source community also began releasing LLMs under varying licenses for models, code, and training data. Figure 1.2, taken from a paper titled "A Survey of Large Language Models" (https://arxiv.org/abs/2303.18223), shows a timeline of the major commercial and open source LLMs from 2019 to mid-2023 (the paper's latest update; more models have appeared since).

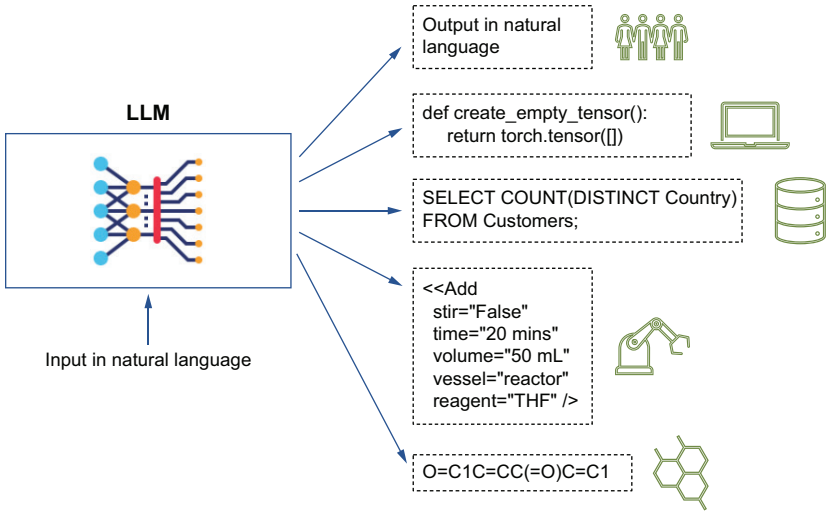


Figure 1.1 Some examples of the diverse content an LLM can generate

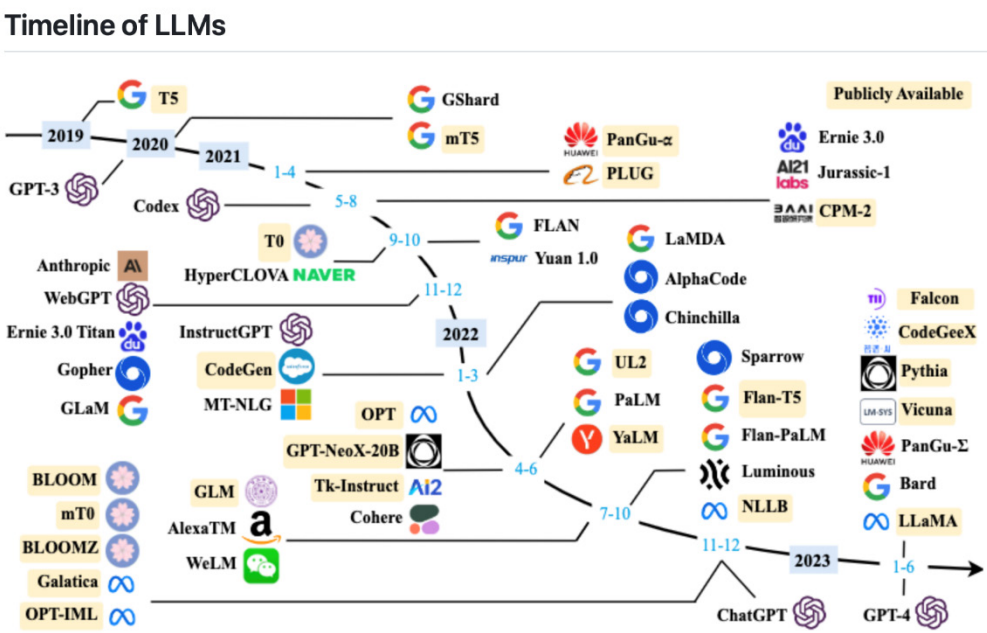


Figure 1.2 The timeline of LLMs from 2019 to mid-2023 (image from "A Survey of Large Language Models")

The highlighted models in figure 1.2 are publicly available under different licenses (always check the usage terms before using one commercially). Some proposed OS models are generalists, like the commercial models from OpenAI and Microsoft, while others are specialized for a particular domain.

Paradoxically, the second spark that accelerated open source contributions was ChatGPT's release to end users: the number of new proposals, releases, and forks of existing models has tripled since then. These efforts came not only from researchers, but also from industry and passionate individuals and groups. This means you have choices: if you need to move from a proprietary LLM to one fully owned by your organization, you can pick a pretrained open source model and then instruct or fine-tune it on your own data instead of developing and training one from scratch—often at a much lower cost.

Figure 1.3 shows the financial impact of developing a custom LLM from scratch. This reflects what organizations like OpenAI, Google, and Anthropic faced when building the GPT-X, Gemini, and Claude models. The $ symbols in the figure indicate the relative cost of each phase. Contrary to common belief, training large models is costly, but it's not the most expensive step. Development typically costs more because it includes collecting and preparing vast amounts of unstructured data and running the

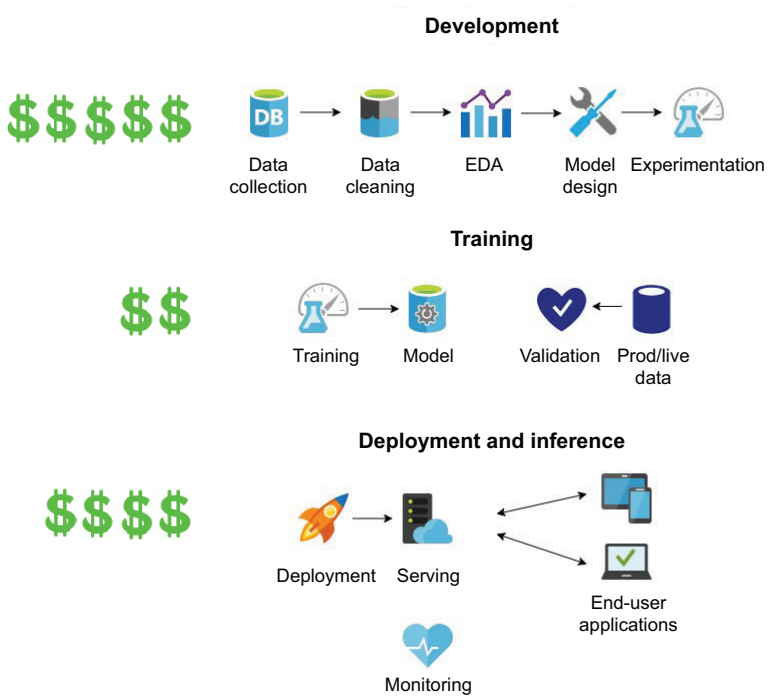


Figure 1.3 Order of magnitude of costs for each phase of LLM implementation from scratch

experiments to achieve a stable model. After a final candidate is trained and meets its key performance indicators (KPIs), deployment and inference serving present another big challenge: scalability, performance, quality, monitoring, and user experience all drive costs that are hard to forecast precisely upfront.

Open source offerings can help here. Figure 1.4 shows the financial impact of implementing a custom LLM starting from a pretrained OS model. As before, the $ symbols show the relative cost of each phase. You can cut development costs because the architecture and a pretrained open source model are already in place. That leaves data collection and preparation (still significant) as the main costs in this phase. Fine-tuning still costs money, but far less than training an LLM from scratch: you still need computational power, but you have more room to optimize. The challenges and costs of deployment and inference remain the same.

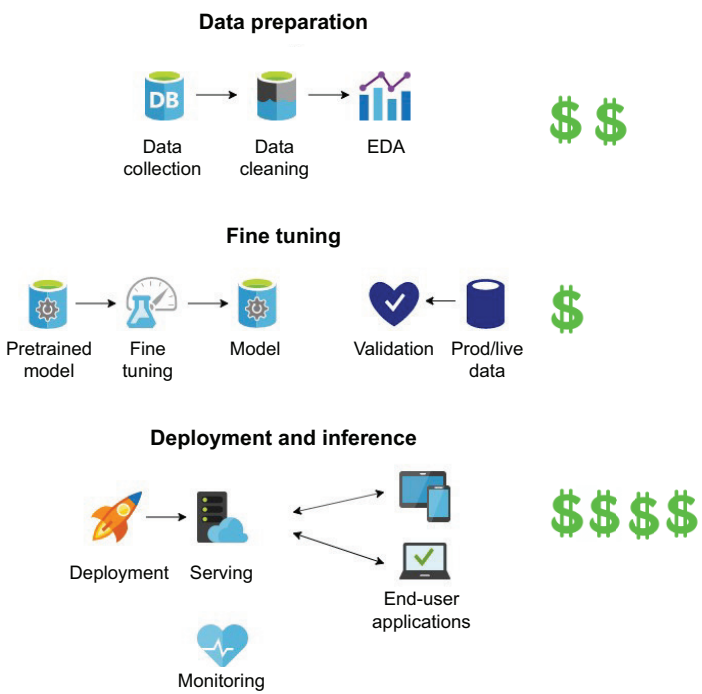


Figure 1.4 Order of magnitude of costs for each phase of LLM implementation when starting from a pretrained model

That's where this book can help: you'll learn techniques and best practices for optimizing your custom LLM to reduce the computational power you'll need (while retaining acceptable accuracy), enabling deployment and inference in constrained environments and cutting costs. In short, you'll learn how to run inference without the kind of infrastructure that big tech companies such as Microsoft, Google, and Meta have.

1.5 Risks and challenges with generalist LLMs

Commercial, generalist, closed source LLMs from third-party vendors are powerful and can handle diverse tasks. They can also provide a good experience for non-technical users, and you pay only for usage (since you don't have to manage deployment and maintenance). But they also come with risks that can outweigh these benefits:

- *Deployment outside your organization's network*—Any information you include in a prompt may leave your company and, depending on the usage terms, be used to fine-tune the model and be exposed to other customers. You've likely read of cases in which employees at large corporations sent sensitive or proprietary data to external model-hosting services (for example, at Samsung, as discussed in "Samsung Bans Staff's AI Use After Spotting ChatGPT Data Leak" [Bloomberg, May 2, 2023, https://mng.bz/AGEE], and others).
- *Data leakage risks*—Because the model is hosted on infrastructure you don't control, a weak cybersecurity posture (unknown to you and hard to mitigate) can allow malicious actors to access private data. There is evidence that something like this happened once with ChatGPT ("OpenAI Confirms ChatGPT Data Breach," Trend Micro's blog, May 13, 2023, https://news.trendmicro.com/2023/05/13/openai-chatgpt-data-breach/).
- *Model or version drift*—A provider can update a model without notice, potentially changing the behavior and accuracy of applications built on top of it.
- *Cost unpredictability*—Token-based billing can spike, as a provider can decide at any time to change their subscription prices.
- *Closeness*—You don't have details about how an LLM was implemented and trained, so there's no chance of reproducibility or interpretability, which is often a requirement in heavily regulated sectors. You also may not see infrastructure or model-version changes.
- *Transparency*—Data used to train closed source models isn't publicly disclosed, so there's no way to verify whether it's biased or contains copyrighted data used without the owner's permission.
- *Hallucinations*—Generated information may conflict with the source (intrinsic hallucination) or be unverifiable from the available source (extrinsic hallucination). Intrinsic hallucinations can occur even in an organization's models trained on proprietary data—a challenge with LLMs in general. But extrinsic hallucinations can't be mitigated when you don't have access to, or knowledge of, the training data.
- *Compliance*—Commercial and closed source LLMs, because of their generalist nature, don't follow any sectoral rules.
- *Harmful content generation*—Models can not only reflect societal biases and stereotyping but can also produce unsafe content. In the rare cases where a provider

has set safety filters on generalist models, this can unnecessarily block legitimate content as a side effect.

- *Code generation*—This emergent LLM ability makes it possible for a broader audience, including people with limited coding skills, to inadvertently generate code with malicious elements or even produce malware. Many third-party LLMs lack robust guardrails, and those that exist can be bypassed with carefully crafted prompts. Organizations can build their own guardrails (which isn't a trivial task) or shift the responsibility to internal users to properly validate outputs.

All these risks, plus the fact that large models are trained on many tasks using broad but generalist data sets, mean they aren't always suitable for important business uses.

1.6 Domain-specific vs. generalist LLMs for business value

Should you move to a custom domain-specific LLM? Several factors can help you make the right choice.

Generalist pretrained LLMs (both commercial and open source) are largely trained on content scraped (with or without consent) from the web, drawn from public scientific or code repositories, or taken from human conversations on generic topics. For tasks that don't require domain specialization, these models are often powerful enough to generate useful content that can be reused with little or no refinement. But for specialized domains and tasks, you'll need a strategy to tune the pretrained model on domain-specific data (whether public or private). This process is commonly referred to as *transfer learning*. In brief, transfer learning involves taking a model trained on one task or domain and applying it to a related task or domain.

In heavily regulated domains, there are stricter requirements for staying compliant. Decisions based on model outcomes cannot cause any sort of harm. In this context, the availability and quality of domain-specific training data (which you may already have in house) plays a key role.

Transfer learning does not necessarily involve the full training of a language model. That's a possibility, but there are options, as listed in table 1.1.

Table 1.1 Options for specializing an open source SLM on a domain-specific task

Transfer learning strategy	Pros	Cons
Full supervised fine-tuning (unfreezing and training all the model's weights on labeled data)	■ Strong performance when the data is sufficient and the domain shift is large	■ High computational costs ■ Risk of overfitting ■ Domain-specific data will probably need to be combined with other accessory data.
Partial fine-tuning (unfreezing and training only a subset of the model's layers)	■ Cheaper than full fine-tuning	■ Requires initial experimentation to select the proper layers to unfreeze and train ■ Risk of catastrophic forgetting

Table 1.1 Options for specializing an open source SLM on a domain-specific task (*continued*)

Transfer learning strategy	Pros	Cons
PEFT (parameter-efficient fine-tuning)—a family of techniques for inserting small trainable modules into each transformer layer while freezing the base	■ Prevents catastrophic forgetting ■ Tiny trainable parameter count paired with strong performance ■ It is possible to train multiple adapters (for diverse tasks) that can be easily swapped at runtime. ■ Unlike partial fine-tuning, it doesn't require experimentation to select the layers to unfreeze and train.	■ Adds a slight runtime overhead. Usually this isn't a problem, but it depends on the latency KPI for a given application.

NOTE *Catastrophic forgetting* is a noticeable drop in performance on general knowledge or initial capabilities due to a model being trained on new data and tasks.

When you are specializing an SLM to a given domain and task, full training is rarely the best option.

There are many domains where SLM specialization is mandatory, such as manufacturing, scientific R&D, legal operations, finance, healthcare administration, and niche coding (such as CUDA and PLC programming). Figure 1.5 shows an example of LLM specialization in a very specific domain: after transfer learning on molecule

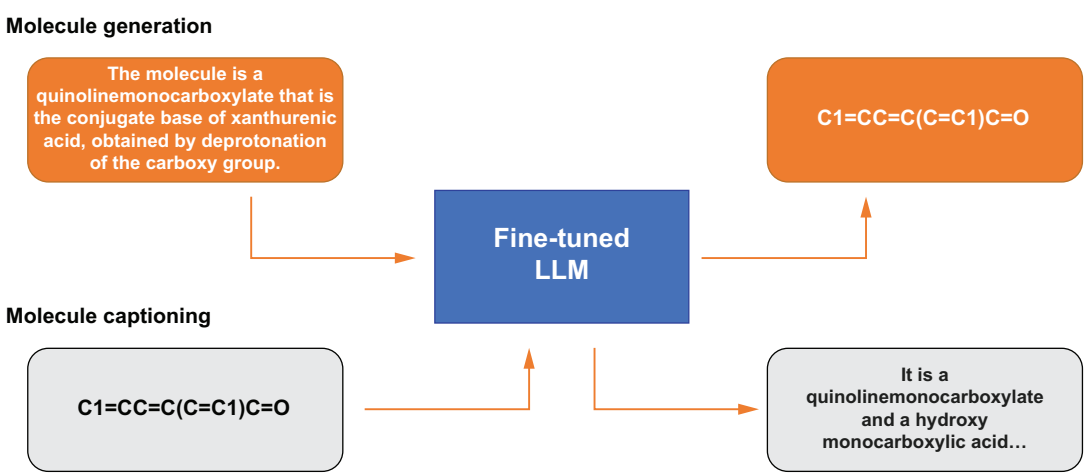


Figure 1.5 An LLM trained for tasks on molecule structures (generation and captioning)

descriptions and structures represented in the SMILES format, this LLM can generate molecular structures from description, or vice versa. This use case is discussed in chapter 8.

Any situation where you must prevent private data leakage is a good reason to switch to a private, offline, domain-specific LLM that runs inside your corporate network boundaries. In these cases, people often worry whether their existing on-premises or private-cloud infrastructure has enough computational power, especially when budgets are limited. We'll look at optimization techniques to reduce the memory footprint and computational requirements of custom LLMs, with working examples in each chapter. Table 1.2 summarizes the pros and cons of generalist and domain-specific LLMs.

Table 1.2 Pros and cons of generalist and domain-specific LLMs

	Generalist LLMs	Domain-specific LLMs
Pros	■ Offer a wide range of knowledge and capabilities	■ Provide specialized expertise in a specific domain
	■ Can handle diverse tasks and topics	■ Well-suited for focused applications and specific industries
	■ Have the flexibility to learn and adapt to new domains (assuming you have the computational power to do it)	■ Higher accuracy and performance in specific areas of expertise
	■ Can generate content on a wide array of subjects	■ Better understanding and context within a specific domain
Cons	■ Lack in-depth expertise in specific domains	■ Limited applicability and effectiveness in non-specialized areas
	■ Require additional fine-tuning and customization for specific tasks	■ Less flexibility to handle tasks outside the chosen domain
	■ Potential risk of making errors or producing inaccurate content	■ Could require more data and resources for training, fine-tuning, or RAG
	■ Require more computing power and resources	

Another advantage of choosing an open source baseline model is that it will likely have only a few billion parameters. After tuning it for domain-specific tasks with private data, the model will likely remain below 10 billion parameters. From a sustainability standpoint, these smaller models generate less CO_2 and have less environmental impact than larger models during training and doing inference.

Finally, let's look at LLM alignment with users, which you've probably heard about. A common definition of "alignment" in this context is adhering to principles of helpfulness (following user instructions, completing a task, and providing results), truthfulness (providing factual information and acknowledging limitations), and harmlessness (avoiding biased or toxic responses and refusing to assist with harmful tasks). Commercial closed-source LLM vendors often pursue this by defining what is right and wrong from their own perspective and then shaping the models accordingly.

In my opinion, alignment should focus less on imposing the provider's definition of "right" and more on matching the user's intent. This is similar to working with a human collaborator: motives and knowledge/ability are different. Improving knowledge and ability may lower the mistake rate, but it doesn't mean the intent is aligned with the person making the request. When you specialize your own models—especially by narrowing the scope to a specific domain—you may have more room to improve this, because "intent" can take on a domain-specific meaning.

Summary

- Transformers use self-attention mechanisms to process entire text sequences at once instead of word by word.
- Self-supervised learning creates training labels automatically from text data without human annotation.
- BERT models use only the encoder part of transformers for classification and prediction tasks.
- GPT models use only the decoder part of transformers for text-generation tasks.
- Word embeddings convert words into numerical vectors that capture semantic relationships.
- RLHF uses reinforcement learning to improve large language model (LLM) responses based on human feedback.
- LLMs can generate any symbolic content including code, math expressions, and structured data.
- Open source LLMs reduce development costs by providing pretrained models as starting points.
- Transfer learning adapts pretrained models to specific domains using domain-specific data.
- Generalist LLMs risk data leakage when they're deployed outside organizational networks.
- Closed source models lack transparency about training data and model architecture.
- Domain-specific LLMs provide better accuracy for specialized tasks than generalist models.
- Smaller specialized models require less computational power than large generalist models.
- Fine-tuning costs significantly less than training models from scratch.
- Regulatory compliance often requires domain-specific models with known training data.

Part 2

Core domain-specific LLMs

The first three chapters in this part of the book discuss training domain-specific SLMs. We'll look at different approaches to SLM specialization and explore an end-to-end example of model specialization. We'll also look at how to calculate up front how much memory you'll need to run inference with a given SLM and how to spot areas for improvements. Chapters 5 and 6 dive deeper into SLM quantization with the ONNX framework and explore different precision formats for model quantization.

2 Tuning for a specific domain

This chapter covers

- Preparing data for LLM customization
- The basics of retrieval-augmented generation
- Fine-tuning an LLM
- Alternatives to fine-tuning

Now that you understand the fundamentals of domain-specific LLMs, we'll look at how you can customize popular open source foundation models using your own data. This chapter and chapters 13 and 15 are the only chapters that will cover tuning; most of the book will focus on inference.

2.1 Data preparation

Fine-tuning a Transformer model for a given task starts with formatting your dataset for training. In this section, we'll look at two PyTorch examples using Hugging Face Transformers (https://github.com/huggingface/transformers): an encoder-only model (BERT) and a decoder-only model (GPT-2). You'll see that the workflow is largely the same, with only minor changes based on the model architecture and

task. Finally, you'll see how to prepare data for cases where retrieval-augmented generation (RAG) is a better choice than fine-tuning.

2.1.1 Data preparation for BERT fine-tuning

Let's consider a classification task using the pretrained bert-base-uncased model (https://huggingface.co/bert-base-uncased). We first need to gather our data, making sure each text sample has a class label. Format it as a list of tuples like this:

```
dataset = [("This movie is a masterpiece!", "Positive"),
    ("Not worth watching!", "Negative"),
    ("Terrific!", "Positive") go here
]
```

Next, we need to tokenize the training texts, because BERT models work with tokens rather than raw text.

> **Notes on tokens**
>
> For LLMs, a token is a unit of text the model processes during training and inference. Tokens can represent words, subwords, characters, punctuation, and special characters.
>
> How text is tokenized directly affects model performance and language understanding by shaping how well it manages vocabulary, handles out-of-vocabulary words, and generates coherent responses.
>
> Tokenization is vital in NLP: the token count determines input size and affects processing time and memory use. LLMs have a maximum number of tokens they can accept in a single prompt.

Most pretrained models on the Hugging Face Hub include a custom tokenizer, since each architecture expects a specific token format. For this example, we'll use the `bert-base-uncased` tokenizer:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

For this kind of transformer, special tokens are used to mark the start and end of each input sequence, and a separator token is used between segments:

```
max_length = 128
formatted_data = [(f"[CLS] {text} [SEP]", label) for text, label in dataset]
tokenized_data = tokenizer(formatted_data,
                ➥padding=True,
                ➥truncation=True,
                ➥max_length=max_length,
                ➥return_tensors='pt')
```

Here, `[CLS]` marks the beginning, and `[SEP]` separates segments. A best practice is to keep input sequences a consistent length by padding shorter ones or truncating longer ones, as in previous tokenizer parameters.

We now need to convert the tokenized, formatted text into numerical representations. We'll assign each token a unique ID (or index) and then represent sequences—and labels—as sequences of these IDs:

```
import torch
from sklearn.preprocessing import LabelEncoder

input_ids = tokenized_data['input_ids']
attention_mask = tokenized_data['attention_mask']
labels = torch.tensor(LabelEncoder()
➥.fit_transform([label for _, label in dataset]))
```

We can now split the initial dataset into training and validation sets:

```
from sklearn.model_selection import train_test_split

train_inputs, val_inputs, train_labels, val_labels,
➥ train_mask, val_mask = train_test_split(
➥input_ids, labels, attention_mask,
➥random_state=42, test_size=0.1
)
```

The split is performed with the `scikit-learn` package (https://scikit-learn.org/stable/), as is standard in many machine learning (ML) and deep learning (DL) use cases.

Before feeding data to the model, we need training and validation data loaders for our custom data. First, let's define a custom dataset class:

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, input_ids, attention_mask,
        ➥ labels):
        self.input_ids = input_ids
        self.attention_mask = attention_mask
        self.labels = labels

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return {'input_ids': self.input_ids[idx],
                'attention_mask':
                 self.attention_mask[idx],
                'labels': self.labels[idx]}
```

We'll use this class to create the data loaders:

```
from torch.utils.data import DataLoader

batch_size = 4
train_dataset = CustomDataset(train_inputs,
                ➥train_mask, train_labels)
train_dataloader = DataLoader(train_dataset,
                ➥batch_size=batch_size,
                ➥shuffle=True)

val_dataset = CustomDataset(val_inputs,
              ➥val_mask, val_labels)
val_dataloader = DataLoader(val_dataset,
              ➥batch_size=batch_size,
              ➥ shuffle=False)
```

We can now load the model from the Hugging Face Hub:

```
from transformers import BertForSequenceClassification

model = BertForSequenceClassification
        ➥.from_pretrained('bert-base-uncased',
    num_labels=len(set(labels)))
```

Now we can start the fine-tuning, which will be discussed in sections 2.3 and 2.4.

2.1.2 Data preparation for GPT fine-tuning

For this example, we'll consider a text completion task using the pretrained GPT-2 base model (the smallest version, with 124M parameters, available at https://huggingface.co/gpt2). Gather your own private data, and ensure that each sample is a piece of text with a clear context (or prompt) and an associated target (or completion):

```
dataset = [("Once upon a time in a faraway swamp,",
            "there lived an ugly ork."),
          # More samples
]
```

We need to pair each context (input) with its target (output) so the model learns to generate the target from the context. Then, as with the BERT model, we tokenize the inputs using the GPT-2 tokenizer:

```
from transformers import GPT2Tokenizer

tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
formatted_data = [(f"[CLS] {context} [SEP]
                 ➥{target} [SEP]",) for context,
                 ➥ target in dataset]
numerical_data = [tokenizer.encode(example[0],
                 ➥ add_special_tokens=True)
                 ➥for example in formatted_data]
```

In this case, we again need special tokens to mark the beginning and end of each input sequence, plus a separator token. We must ensure that all sequences have a consistent length by padding shorter ones or truncating longer ones to match our desired length. We then convert the tokenized, formatted text into numbers by assigning each token a unique ID and representing sequences as lists of these IDs:

```
import torch

max_length = max(len(seq) for seq in numerical_data)
padded_data = [seq + [tokenizer.pad_token_id]
               ➥* (max_length - len(seq))
               ➥for seq in numerical_data]

input_ids = torch.tensor(padded_data)
```

Before feeding data to the model, we need to create a training data loader for our custom dataset. Let's first define a custom dataset class:

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
  def __init__(self, input_ids):
    self.input_ids = input_ids

  def __len__(self):
    return len(self.input_ids)

  def __getitem__(self, idx):
    return {'input_ids': self.input_ids[idx]}
```

The implementation is similar to the BERT use case; what differs is the definition of the `__init__` and `__getitem__` functions, which reflect the model architecture's input configuration. We'll use this class to create the training data loader:

```
from torch.utils.data import DataLoader

batch_size = 4
custom_dataset = CustomDataset(input_ids)
dataloader = DataLoader(custom_dataset,
             ➥batch_size=batch_size,
             ➥shuffle=True)
```

Finally, we can load the model from the Hugging Face Hub:

```
from transformers import GPT2LMHeadModel

model = GPT2LMHeadModel.from_pretrained('gpt2')
```

We can now start the fine tuning, as discussed in sections 2.3 and 2.4.

As you've seen, thanks to the Transformers library's friendly API, switching between pretrained models keeps the code largely the same.

2.1.3 Data preparation for RAG

For cases where RAG (discussed in section 2.2) is the best way to specialize a pretrained language model, you must convert your knowledge base to embeddings and store them in a vector database. This makes it easy to retrieve information on demand without sacrificing performance or scalability. Many open source and commercial options are available for this purpose.

In this chapter, we'll use Facebook AI Similarity Search (FAISS, https://github.com/facebookresearch/faiss), released as open source under an MIT license and maintained by Meta Research. FAISS is a library for efficient similarity search and clustering of dense vectors. It's implemented in C++ and provides a Python API, so developers don't need to worry about implementation details. Used internally at Meta, it supports searches that are inefficient or impractical with other approaches. It includes nearest-neighbor implementations for million-to-billion-scale datasets, optimizing the memory/speed/accuracy tradeoff, and it supports GPUs via the Compute Unified Device Architecture (CUDA). FAISS offers several similarity-search methods and assumes that instances are represented as vectors (each identified by an integer) that can be compared using Euclidean (L2) distance. The most similar vectors to a query vector are those with the smallest L2 distance.

To get familiar with FAISS, I've provided a companion Colab notebook. We'll walk through its code in this section. You'll classify a new input text by calculating its similarity to other text samples in a given category.

In addition to FAISS, we'll use the `sentence-transformers` package (https://www.sbert.net), a framework for sentence, text, and image embeddings.

We'll first set up the sample data, which is a few sentences about Marvel superheroes, categorized into three labels: `spiderman`, `ironman`, and `captainamerica`:

```
data = [['His secret identity is Peter Parker', 'spiderman'],
        ['A businessman and engineer who ' +
         'runs the company Stark Industries',
         'ironman'],
        ['Superhuman spider-powers and abilities ' +
         'after being bitten by a radioactive spider',
         'spiderman'],
        ['A frail man enhanced to the peak of human ' +
         'physical perfection by an experimental
          ➥super-soldier serum', 'captainamerica']
        ]
```

We'll put them into a pandas DataFrame with two columns, `text` and `context`:

```
import pandas as pd

df = pd.DataFrame(data, columns = ['text', 'context'])
```

We'll use the `sentence_transformers` API to generate embeddings for each `text` row in the DataFrame:

```
from sentence_transformers import SentenceTransformer

text = df['text']
encoder = SentenceTransformer("paraphrase-mpnet-base-v2")
vectors = encoder.encode(text)
```

Now that we have the word embeddings, we can create a FAISS index and add them to it:

```
import faiss

vector_dimension = vectors.shape[1]
l2_index = faiss.IndexFlatL2(vector_dimension)
faiss.normalize_L2(vectors)
l2_index.add(vectors)
```

An index object makes similarity search efficient in FAISS. If you're familiar with relational databases, you're familiar with a table index: a pointer to data in a table used to speed up retrieval. Think of FAISS indexes the same way. Here, they speed up retrieval within a data cluster, so FAISS doesn't rely on brute-force search. Note that embeddings are normalized before being added to the index so they share a common scale.

The Euclidean distance is the length of a line segment between two points. FAISS calculates it using the standard formula, where q_i and p_i are the Cartesian coordinates of two points p and q:

$$d(p,q) = \sqrt{\left(\sum_{i=1}^{n}(q_i - p_i)^2\right)}$$

Let's look at a search example. Set a search text related to Spider-Man:

```
search_text = 'He throws webs'
```

As a result of the search, we expect smaller distances between the search string and the two *spiderman* samples in the original text corpus. Before searching, we need a few transformations to generate embeddings for the search text—in the same way as we previously did for all the embeddings in scope for this example before adding them to the FAISS index—convert them to a NumPy array, and normalize them:

```
import numpy as np

search_vector = encoder.encode(search_text)
search_vector_as_array = np.array([search_vector])
faiss.normalize_L2(search_vector_as_array)
```

Now perform the search:

```
k = l2_index.ntotal
distances, ann = l2_index.search(search_vector_as_array, k=k)
```

The `k` argument sets the number of neighbors; here, it's set to the total number of vectors in the index. The `search` function returns two arrays: `distances`, the L2 distances from the query embedding to each of the top *k* vectors, and `ann`, the corresponding vector positions in the index. We'll merge these results into a single DataFrame with the original corpus and labels:

```
search_results = pd.DataFrame({'distances': distances[0], 'ann': ann[0]})
merged_df = pd.merge(search_results, df, left_on='ann', right_index=True)
```

From the results, you can see that the shortest distances between the search text and the other indexed samples occur in the `spiderman` category:

```
distances  ann                                               text  \
0   1.501069    2  Superhuman spider-powers and abilities after b...
1   1.552392    0                His secret identity is Peter Parker
2   1.667212    1  A businessman and engineer who runs the compan...
3   1.731641    3  A frail man enhanced to the peak of human phys...

          context
0       spiderman
1       spiderman
2         ironman
3  captainamerica
```

This example used a small dataset, but the same FAISS workflow can scale to production, unlike a brute-force similarity search.

2.2 Retrieval-augmented generation

Retrieval-augmented generation (RAG) integrates data retrieval into LLM content generation. As shown in figure 2.1, it retrieves relevant data from a larger corpus and feeds it to a pretrained LLM as an external knowledge base, helping the model produce more accurate, up-to-date answers.

This is a way to select external information to improve a model's response. We can identify two phases:

1. *Retrieval phase*—The algorithm searches for and retrieves information based on the user's prompt, from private or public data sources, or both.
2. *Content generation phase*—After retrieving the relevant embeddings, an LLM uses the context to generate a natural-language response. The pipeline can also check the output against the sources to improve transparency and accuracy.

For domain-specific data and tasks, or unstructured text that isn't natural language, prompt engineering (few-shot, chain-of-thought, and so on) often won't produce

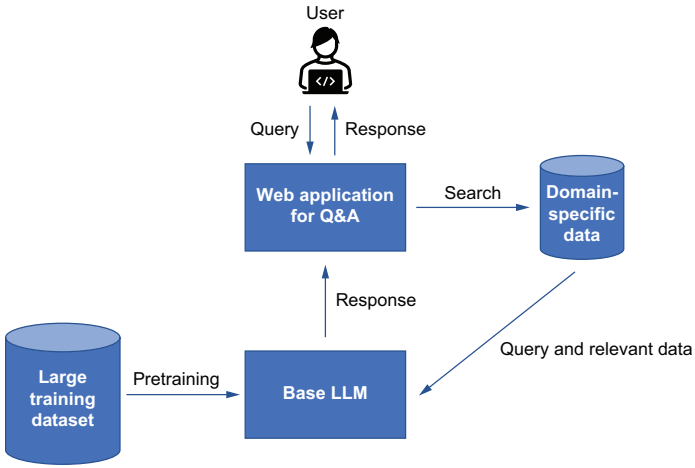


Figure 2.1 The typical RAG workflow

acceptable results. Prompt engineering is faster and can be cheaper because it doesn't require training, but in these cases, you should consider RAG or fine-tuning the LLM you plan to use.

Which is preferable—RAG or fine-tuning? There's no silver bullet: the answer depends on the specific use case you're trying to address. We'll cover the key considerations in section 2.5 after we look at both strategies.

You can implement RAG-based solutions with LangChain (https://github.com/langchain-ai/langchain) or other dedicated frameworks, covered in chapters 13 and 14. RAG code samples and notebooks are provided in chapters 13 and 14.

2.3 Fine-tuning

Fine-tuning further trains a pretrained LLM on a smaller, task-specific dataset to adapt it for a particular task. It updates the model's weights using your data, tailoring it to your organization's needs. Figure 2.2 shows the fine-tuning process at a high level.

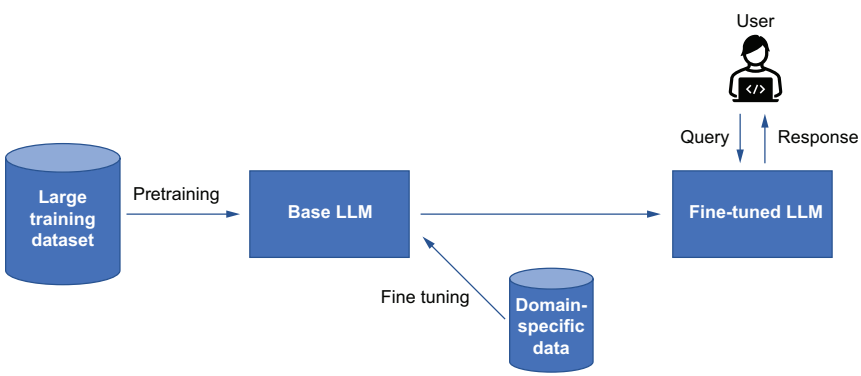


Figure 2.2 A typical fine-tuning workflow

Fine-tuning is similar to pretraining, but it uses a smaller, task-specific, labeled dataset.

To explain the fine-tuning steps, I have provided a companion Colab notebook. This section walks through the code in that notebook. Note that this section covers the full fine-tuning of a pretrained LLM (that is, updating all the parameters). In contrast, section 2.4 introduces one of the most effective PEFT (parameter-efficient fine-tuning) techniques, an alternative for adapting a pretrained model to a domain by training only a small number of parameters; that section also surveys other PEFT methods.

The pretrained LLM in this example is DistilBERT-base, available on the Hugging Face Hub. It's a Transformer model that's smaller and faster than BERT, pretrained on the same corpus via self-supervised learning, with BERT-base as the teacher. It was trained on raw text, using an automatic process to generate inputs and labels (no human labeling), again based on BERT-base. DistilBERT is uncased, meaning it doesn't distinguish between uppercase and lowercase letters. In this example, we'll fine-tune the model for question answering.

The training dataset must have these three important fields:

- `question`—The question the model should answer
- `context`—The background information from which the model needs to extract the answer
- `answers`—The starting location of the answer token and the answer text

Several question-answering datasets are available on the Hugging Face Hub. Choose one for this exercise using the Hugging Face Datasets library (https://github.com/huggingface/datasets):

```
from datasets import load_dataset

training_set = load_dataset("<dataset_id>")
```

Split into training and test sets (80/20):

```
training_set = training_set.train_test_split(test_size=0.2)
```

Load the DistilBERT tokenizer to process the training set's `question` and `context` fields:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

Next, we need to preprocess the data by implementing a custom function, `preprocess_function`. In the function body, we'll first truncate any context that exceeds the model's maximum input length:

```
def preprocess_function(examples):
    questions = [q.strip() for q in examples["question"]]
    inputs = tokenizer(
```

```
        questions,
        examples["context"],
        max_length=384,
        truncation="only_second",
        return_offsets_mapping=True,
        padding="max_length",
    )
```

We'll map the answer's start and end positions to the original context and find the start and end tokens for each answer:

```
    offset_mapping = inputs.pop("offset_mapping")
    answers = examples["answers"]
    start_positions = []
    end_positions = []

    for i, offset in enumerate(offset_mapping):
        answer = answers[i]
        start_char = answer["answer_start"][0]
        end_char = answer["answer_start"][0] + len(answer["text"][0])
        sequence_ids = inputs.sequence_ids(i)


        idx = 0
        while sequence_ids[idx] != 1:
            idx += 1
        context_start = idx
        while sequence_ids[idx] == 1:
            idx += 1
        context_end = idx - 1

        # If the answer is not fully inside the context, label it (0, 0)
        if offset[context_start][0] > end_char or
        ➥offset[context_end][1] < start_char:
            start_positions.append(0)
            end_positions.append(0)
        else:

            idx = context_start
            while idx <= context_end and offset[idx][0] <= start_char:
                idx += 1
            start_positions.append(idx - 1)

            idx = context_end
            while idx >= context_start and offset[idx][1] >= end_char:
                idx -= 1
            end_positions.append(idx + 1)

    inputs["start_positions"] = start_positions
    inputs["end_positions"] = end_positions
    return inputs
```

Next, we'll apply the custom function to the entire dataset with Datasets' `map` function. Set `batched=True` to process multiple elements at once:

```
tokenized_dataset = training_set.map(preprocess_function,
➥batched=True, remove_columns=training_set["train"]
➥.column_names)
```

We can now load the model:

```
from transformers import AutoModelForQuestionAnswering

model = AutoModelForQuestionAnswering
➥.from_pretrained("distilbert-base-uncased")
```

We'll create a batch of examples using the Transformers library's `DefaultDataCollator` class, which builds training batches from a list of dataset elements:

```
from transformers import DefaultDataCollator

data_collator = DefaultDataCollator()
```

Although the Transformers library provides multiple specializations, we'll use the default data collator here because we've already defined a custom preprocessing function and don't need additional transformations.

We can now define the training hyperparameters:

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="my_awesome_qa_model",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    push_to_hub=False,
)
```

We'll create a `Trainer` object and pass it the training arguments, along with the model, dataset, tokenizer, and data collator:

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
)
```

A `Trainer` object in the Transformers library is a high-level abstraction for training a model (either on a single GPU or distributed across multiple accelerators). It uses the

data collator to build batches and the tokenizer to tokenize the training and validation data. To start LLM training, just call the `train` method:

```
trainer.train()
```

After training, you can use the fine-tuned model for inference. Provide a question and context for the model to answer:

```
question = "How many titles has Juventus won?"
context = "Juventus Football Club (from Latin: iuventūs), colloquially
 known as Juve, is a professional football club based in Turin, Piedmont,
Italy, that competes in the Serie A, the top tier of the Italian football
league system. Founded in 1897 by a group of Torinese students, the club
has worn a black and white striped home kit since 1903 and has played home
matches in different grounds around its city, the latest being the 41,507-
capacity Juventus Stadium. Nicknamed la Vecchia Signora (the Old Lady), the
club has won 36 official league titles, 14 Coppa Italia titles and nine
Supercoppa Italiana titles, being the record holder for all these
competitions;"
```

Tokenize these texts:

```
inputs = tokenizer(question, context, return_tensors="pt")
```

Pass these inputs to the model, which will return the logits:

```
import torch

with torch.no_grad():
    outputs = model(**inputs)
```

Select the highest-probability start and end positions from the model's output:

```
answer_start_index = outputs.start_logits.argmax()
answer_end_index = outputs.end_logits.argmax()
```

Finally, decode the predicted tokens to get the answer:

```
predict_answer_tokens = inputs.input_ids[0,
➥ answer_start_index : answer_end_index + 1]
tokenizer.decode(predict_answer_tokens)
```

It should return `36` (as of this writing).

2.4 LoRA

Parameter-efficient fine-tuning (PEFT) is a set of techniques for adapting a pretrained model by updating only a small fraction of its parameters. An LLM pretrained on a large dataset already captures many language patterns and facts, so it's often

unnecessary—and inefficient—to tune the entire model for a new task. PEFT fine-tunes only a small subset of parameters, reducing computational costs compared with full fine-tuning while maintaining comparable accuracy and robustness.

These are some major PEFT methods:

- *LoRA (Low-rank Adaptation)*—The main topic of this section
- *Prefix tuning*—Keeps the LLM parameters frozen and optimizes a sequence of continuous, task-specific vectors called the prefix
- *LoftQ*—A framework that quantizes an LLM and provides a low-rank initialization for LoRA fine-tuning
- *AdaLoRA*—A LoRA variant that adaptively allocates the parameter budget across weight matrices based on importance scores
- *P-tuning*—Uses trainable continuous prompt embeddings concatenated with discrete prompts
- *Prompt tuning*—A simpler form of prefix tuning that scales better to larger models

One of the most widely used and efficient approaches is *LoRA* (low-rank adaptation), introduced in 2021 for LLMs and later adapted to vision transformers. Figure 2.3, from the "LoRA: Low-Rank Adaptation of Large Language Models" paper (https://arxiv.org/abs/2106.09685), summarizes the LoRA process: the pretrained weights are kept frozen, and only a subset of parameters is fine-tuned.

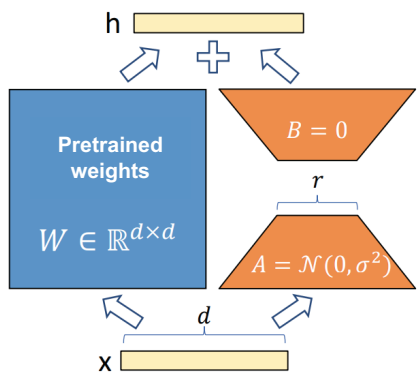


Figure 2.3 The LoRA fine-tuning process

A companion Colab notebook has been provided to show how LoRA works; this section walks through its code. We'll start with a pretrained model called FLAN-T5 (80M parameters, https://huggingface.co/google/flan-t5-small). FLAN-T5 combines FLAN (Finetuned Language Net) with T5, a language model Google released in 2020. It uses an encoder–decoder architecture and extends the original T5 model to improve zero-shot performance. It's open source under the Apache License 2.0. Although we'll use the smallest FLAN-T5 model, the same code and considerations apply to larger versions (250M to 11B parameters).

The dataset used in this example is `samsum` (https://huggingface.co/datasets/knkarthick/samsum):

```
from datasets import load_dataset

dataset = load_dataset("samsum")
```

It contains messenger-style conversations in English and their summaries (our target in this example). The dataset is already split into training and test sets: 14,732 training samples and 819 test samples.

Before preprocessing the data, we need to load the `flan-t5-small` tokenizer:

```
from transformers import AutoTokenizer

model_id = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

We'll first preprocess the data by truncating input and target sequences that exceed the maximum lengths and padding shorter ones:

```
import numpy as np
from datasets import concatenate_datasets

tokenized_inputs = concatenate_datasets([dataset["train"],
           dataset["test"]]).map(lambda x:
           ➥tokenizer(x["dialogue"],
           ➥truncation=True),
           batched=True,
           remove_columns=["dialogue", "summary"])
input_lenghts = [len(x) for x in tokenized_inputs["input_ids"]]
max_source_length = int(np.percentile(input_lenghts, 85))

tokenized_targets = concatenate_datasets([dataset["train"],
            dataset["test"]]).map(lambda x:
            ➥tokenizer(x["summary"],
            ➥truncation=True),
            batched=True,
            remove_columns=["dialogue", "summary"])
target_lenghts = [len(x) for x in tokenized_targets["input_ids"]]
max_target_length = int(np.percentile(target_lenghts, 90))
```

The second pass tokenizes the preprocessed input and target sequences:

```
def preprocess_function(sample,padding="max_length"):
    inputs = ["summarize: " + item for item in sample["dialogue"]]

    model_inputs = tokenizer(inputs,
                max_length=max_source_length,
                padding=padding,
                truncation=True)

    labels = tokenizer(text_target=sample["summary"],
                max_length=max_target_length,
                padding=padding,
                truncation=True)

    if padding == "max_length":
        labels["input_ids"] = [
            [(l if l != tokenizer.pad_token_id else -100)
```

```
            ➥for l in label] for label in
            ➥labels["input_ids"]
        ]

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

tokenized_dataset = dataset.map(preprocess_function,
                batched=True,
                remove_columns=["dialogue", "summary", "id"])
```

Save the processed training and test datasets to disk—we'll use them later:

```
tokenized_dataset["train"].save_to_disk("data/train")
tokenized_dataset["test"].save_to_disk("data/eval")
```

Now we can load the pretrained model in 8-bit precision:

```
from transformers import AutoModelForSeq2SeqLM

model = AutoModelForSeq2SeqLM.from_pretrained(model_id,
➥load_in_8bit=True, device_map="auto")
```

To avoid reinventing the wheel, we'll use Hugging Face's PEFT library (https://github.com/huggingface/peft), which supports LoRA along with other PEFT methods. As you'll see in the next code snippets, it provides a high-level API.

We'll first set up the LoRA configuration with the `LoraConfig` class:

```
from peft import LoraConfig, get_peft_model,
➥ prepare_model_for_int8_training, TaskType

lora_config = LoraConfig(
 r=16,
 lora_alpha=32,
 target_modules=["q", "v"],
 lora_dropout=0.05,
 bias="none",
 task_type=TaskType.SEQ_2_SEQ_LM
)
```

Next, prepare the model for training:

```
model = prepare_model_for_kbit_training(model)
```

Now add a LoRA adaptor:

```
model = get_peft_model(model, lora_config)
```

At the end of this process, you'll train 688,128 parameters—about 0.89% of the model's total. This training is far less computationally demanding than fine-tuning the full model.

The training process is the same as for regular fine-tuning or full training with the Transformers library; only the model differs, because it has been prepared with LoRA.

We'll now define a data collator

```
from transformers import DataCollatorForSeq2Seq

label_pad_token_id = -100
data_collator = DataCollatorForSeq2Seq(
    tokenizer,
    model=model,
    label_pad_token_id=label_pad_token_id,
    pad_to_multiple_of=8
)
```

and set the training arguments (for these small models, two or three epochs may be enough):

```
from transformers import Seq2SeqTrainer, Seq2SeqTrainingArguments

output_dir="lora-flan-t5-small"

training_args = Seq2SeqTrainingArguments(
    output_dir=output_dir,
    auto_find_batch_size=True,
    learning_rate=1e-3,
    num_train_epochs=3,
    logging_dir=f"{output_dir}/logs",
    logging_strategy="steps",
    logging_steps=500,
    save_strategy="no",
    report_to="tensorboard",
)
```

Then we'll create a `Trainer` instance

```
trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=tokenized_dataset["train"],
)
```

and finally start the training:

```
trainer.train()
```

After training, save the checkpoint to disk so it can be loaded for inference and testing, and then upload it to the Hugging Face Hub:

```
peft_model_id="results"
trainer.model.save_pretrained(peft_model_id)
tokenizer.save_pretrained(peft_model_id)
```

Inference with a LoRA–fine-tuned model works much like standard Transformers inference; the main difference is that we need to load the PEFT configuration for the pretrained checkpoint and the LoRA weights:

```
import torch
from peft import PeftModel, PeftConfig

peft_model_id = "results"
config = PeftConfig.from_pretrained(peft_model_id)

model = AutoModelForSeq2SeqLM
➥.from_pretrained(config.base_model_name_or_path,
➥ load_in_8bit=True,  device_map={"":0})
tokenizer = AutoTokenizer.from_pretrained(config.base_model_name_or_path)

model = PeftModel.from_pretrained(model, peft_model_id, device_map={"":0})
model.eval()
```

Then we can run summarization tasks on the test data as usual:

```
from random import randrange

dataset = load_dataset("samsum")
sample = dataset['test'][randrange(len(dataset["test"]))]

input_ids = tokenizer(sample["dialogue"], return_tensors="pt",
➥truncation=True).input_ids.cuda()
➥outputs = model.generate(input_ids=input_ids, max_new_tokens=10,
➥do_sample=True, top_p=0.9)
print(f"input sentence: {sample['dialogue']}\n{'---'* 20}")

print(f"summary:\n{tokenizer.batch_decode(outputs.detach()
➥.cpu().numpy(), skip_special_tokens=True)[0]}")
```

2.5 RAG or fine-tuning?

Which of these two strategies for specializing LLMs should we prefer—RAG or fine-tuning? As is often the case, it depends on the specific situation.

RAG is useful when you need up-to-date information (recent data or new data sources) that wasn't part of an LLM's initial training set. It can reduce hallucinations and improve interpretability because you can trace each answer back to its sources. RAG is usually less costly than fine-tuning because it requires less labeling and no model training. But RAG has downsides. In some cases, extra context isn't enough: if an LLM struggles with the task it was trained for, additional context won't help. And at large data volumes, you also need to account for the infrastructure costs of a vector database and the implementation and maintenance of data pipelines.

Fine-tuning often outperforms RAG on tasks that require learning complex patterns and relationships, such as drug discovery or code generation. By training on a task-specific dataset, an LLM can learn domain nuances, including specialized

jargon, concepts, and structures. As a result, when it encounters new examples from that domain, a fine-tuned model is more likely to produce accurate, relevant outputs. Fine-tuning typically costs more than RAG, but as you learned in section 2.4, PEFT techniques can make it less computationally hungry (and cheaper) than full fine-tuning—sometimes comparable to, or even lower than, the infrastructure costs of a RAG solution. Fine-tuning also requires more ML engineering expertise than RAG to optimize training and control costs.

RAG is usually more interpretable, while fine-tuning requires more careful training-data curation to mitigate hallucinations. Fine-tuning or PEFT can reduce domain-specific hallucinations, whereas more generic domain hallucinations can be better mitigated with RAG.

Take time to choose the right path for each use case; no single strategy fits all situations.

Summary

- BERT tokenization adds special tokens `[CLS]` and `[SEP]` to mark sequence boundaries for classification tasks.
- GPT models use context-target pairs, learning to generate target text from the input context.
- Tokenizers convert text to numerical representations that models can process during training and inference.
- FAISS builds vector indexes for efficient similarity search over large text embedding datasets.
- RAG combines retrieval and text generation to give models access to external knowledge.
- RAG retrieves relevant information from vector databases before generating responses, improving accuracy.
- Fine-tuning adjusts all parameters of a pretrained model using task-specific labeled datasets.
- LoRA fine-tunes only a small subset of a model's parameters, retaining the original weights.
- PEFT techniques like LoRA cut computational costs compared with full fine-tuning while maintaining performance.
- RAG works better for up-to-date information and reduces hallucinations through source traceability.
- Fine-tuning produces better results for complex, domain-specific tasks that require specialized knowledge.
- Vector databases store embeddings and support fast similarity search for RAG implementations.

3 End-to-end transformer fine-tuning

This chapter covers

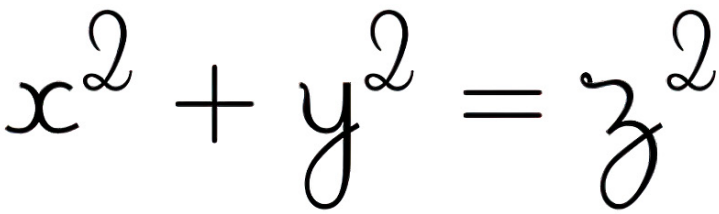
- Specializing a very small language model to generate working Manim code
- The hyperparameter tuning process in transformer fine-tuning
- Evaluating the quality of the generated code

This chapter walks through an end-to-end example of fine-tuning a small model (GPT-2 small) for a specific task. This approach will apply to fine-tuning any GPT-like model for generative tasks.

3.1 Data preparation

In this chapter, we'll fine-tune the GPT-2 small model (https://huggingface.co/openai-community/gpt2) to generate working Manim code from natural-language prompts. Manim (https://github.com/ManimCommunity/manim) is an open source Python animation engine for explanatory math videos—it creates precise animations programmatically. Figure 3.1 shows an example of Manim code and its rendering.

Example: LaTeXMathFonts ¶




```
from manim import *

class LaTeXMathFonts(Scene):
    def construct(self):
        tex = Tex(
            r"$x^2 + y^2 = z^2$",
            tex_template=TexFontTemplates.french_cursive,
            font_size=144,
        )
        self.add(tex)
```

Figure 3.1 An example of Manim code (bottom) and the corresponding rendering (top)

I chose GPT-2 small as a baseline not only because it can't produce Manim code, but also to show that you can start with a small model and a high-quality, domain-specific dataset—even a relatively small one—and still specialize it for a task and achieve good performance. A companion Colab notebook contains the full source code from this chapter. Hardware acceleration (GPU, free tier) is required.

We'll use the `manim_python` dataset from the Hugging Face Hub (https://huggingface.co/datasets/Edoh/manim_python). It has two fields: `instruction` (a natural-language prompt) and `output` (the corresponding Manim Python code). The dataset includes two splits: `train` (599 samples) and `test` (51 samples).

We'll use the Hugging Face Datasets library (introduced in chapter 2) to handle the dataset. First, we need to download it:

```
from datasets import load_dataset

dataset = load_dataset("Edoh/manim_python")
```

The following snippet shows one of the samples in the training split:

```
{'instruction': "Create a new scene named 'MyScene'.", 'output': 'from
 manim import * class MyScene(Scene): def construct(self): pass'}
```

Next, we'll load the model tokenizer:

```
from transformers import GPT2Tokenizer

model_name = "openai-community/gpt2"
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
```

The dataset isn't ready for fine-tuning and testing the model, so we'll implement a custom preprocessing function:

```
def preprocess_data(examples):
    inputs = [
        f"Instruction: {instr}\nOutput: {out}"
        for instr, out in zip(examples["instruction"], examples["output"])
    ]

    tokenized = tokenizer(inputs, truncation=True, max_length=512,
➥padding="max_length")

    tokenized["labels"] = tokenized["input_ids"].copy()
    return tokenized
```

The `preprocess_data` function concatenates the `instruction` and `output` columns and then tokenizes the result. It's applied to every sample in the dataset (both `train` and `test` splits):

```
tokenized_datasets = dataset.map(preprocess_data,
                         batched=True,
                         remove_columns=dataset["train"].column_names)
```

The original columns are deleted because they're no longer needed.

The data is now ready, and we can prepare the model for training.

3.2 Fine-tuning

To fine-tune the model, we'll use Hugging Face's Transformers library, introduced in chapter 2, along with Optuna (https://optuna.org/) for hyperparameter search. This process involves systematically exploring the most suitable set of hyperparameters that can elevate the performance of a model. Hyperparameters, distinct from model parameters, aren't inherently learned during the training phase. Instead, they're predetermined. Their precise configuration can profoundly influence a model's performance, bridging the gap between an average model and one that excels. Optuna is an ML framework-agnostic, open source Python library for automatic hyperparameter

tuning. It lets you search for optimal hyperparameters using standard Python conditionals, loops, and syntax. Because Optuna is fully supported by Hugging Face libraries, we'll use it as the search backend without working directly with its API.

We'll define a custom function, `model_init`, for hyperparameter search so each trial starts with a freshly initialized model:

```
from transformers import GPT2LMHeadModel

def model_init():
    return GPT2LMHeadModel.from_pretrained(model_name, device_map='auto')
```

Next, we'll configure the training arguments before starting the hyperparameter search:

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./gpt2-manim-python-finetuned",
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="steps",
    logging_steps=100,
    save_total_limit=2,
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    greater_is_better=False,
    fp16=True,
    report_to="none",
)
```

In the previous code snippet, we did the following:

- Specified the path to the root directory for saving checkpoint candidates
- Chose `epoch` as the evaluation and saving strategies
- Chose `steps` as the logging strategy (metrics reported every 100 steps)
- Set the maximum number of checkpoint savings per single trial
- Chose evaluation accuracy as the key metric for choosing the best model
- Enabled mixed precision training (FP16)

Because we have set an evaluation strategy, we'll use some of the training data for evaluation. We'll split the training set and reserve 10% of the samples for evaluation:

```
train_val_split =
➥tokenized_datasets["train"].train_test_split(test_size=0.1)
tokenized_datasets["train"] = train_val_split["train"]
tokenized_datasets["validation"] = train_val_split["test"]
```

We can now initialize the `Trainer` with the training arguments and the tokenized training and evaluation data:

```
from transformers import (
   Trainer,
   DataCollatorForLanguageModeling,
   EarlyStoppingCallback
)

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
                                mlm=False)

trainer = Trainer(
    model_init=model_init,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    tokenizer=tokenizer,
    data_collator=data_collator,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)],
)
```

We have also added a callback to stop a training trial if it doesn't improve for two consecutive steps.

We'll define the search space for hyperparameter tuning as follows:

```
def hp_space(trial):
    return {
        "learning_rate": trial.suggest_float("learning_rate",
        ➥1e-5, 5e-4, log=True),
        "per_device_train_batch_size":
        ➥ trial.suggest_categorical("per_device_train_batch_size",
        ➥ [2, 4, 8]),
        "weight_decay": trial.suggest_float("weight_decay", 0.0, 0.3),
        "num_train_epochs": trial.suggest_int("num_train_epochs", 3, 6),
        "warmup_steps": trial.suggest_int("warmup_steps", 0, 500),
        "gradient_accumulation_steps":
        ➥ trial.suggest_categorical("gradient_accumulation_steps",
        ➥ [1, 2, 4]),
    }
```

We've defined a function that takes a trial object as input, which Optuna will use to sample hyperparameter values. In the body of the `hp_space` function, we specify value ranges to define the search space. The hyperparameters are the learning rate, batch size per device, weight decay, number of training epochs, warmup steps, and gradient accumulation steps. Some ranges were chosen to fit the computational power available in the free-tier Colab VM (a single NVIDIA Tesla T4). In other hardware settings, some hyperparameters (such as batch size or gradient accumulation steps) may need different values or ranges.

> **NOTE** Gradient accumulation steps are the number of mini-batches over which gradients are accumulated before the model's weights are updated. This enables a larger effective batch size without exceeding GPU memory limits, which is especially useful when GPU memory is scarce.

We're now ready to run the hyperparameter search. Here we'll specify the backend engine (Optuna in this example), the number of trials, and the metric used to compare trials (evaluation loss in this case):

```
best_run = trainer.hyperparameter_search(
    direction="minimize",
    backend="optuna",
    n_trials=10,
    hp_space=hp_space,
    compute_objective=lambda metrics: metrics["eval_loss"],
)
```

Once the search is complete, we can configure the trainer with the best hyperparameters, stored in the `best_run` variable:

```
for key, value in best_run.hyperparameters.items():
    setattr(training_args, key, value)

trainer = Trainer(
    model_init=model_init,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets.get("validation"),
    tokenizer=tokenizer,
    data_collator=data_collator,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)],
)
```

Finally, we can start tuning the model:

```
trainer.train()
```

After training, we'll save the fine-tuned model and tokenizer to disk so we can reload them for testing or to generate Manim code:

```
trainer.save_model("./gpt2-manim-python-finetuned")
tokenizer.save_pretrained("./gpt2-manim-python-finetuned")
```

Next, we'll look at testing the model.

3.3 Testing the fine-tuned model

To test the fine-tuned model on the test set, we'll first load it from disk into GPU memory:

```
import torch

model_dir = "./gpt2-manim-python-finetuned"
tokenizer = GPT2Tokenizer.from_pretrained(model_dir)
model = GPT2LMHeadModel.from_pretrained(model_dir)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

model.eval()
```

Next, we'll define a custom function, `generate_output`, that we'll run on each prompt (the `instruction` column) in the test dataset to generate the expected Manim code (the `output` column is our ground truth). The function takes a prompt as its first argument, along with the maximum generation length and optional parameters such as temperature and decoding strategy (beam search and nucleus sampling in this case):

```
def generate_output(instruction, max_length=150, num_beams=5,
➥ temperature=0.7, top_p=0.9, repetition_penalty=1.2):
```

After the prompt has been tokenized

```
prompt = f"Instruction: {instruction}\nOutput:"
input_ids = tokenizer.encode(prompt, return_tensors="pt").to(device)
```

the generation of Manim code from the input text happens:

```
generated_ids = model.generate(
        input_ids,
        max_length=max_length,
        num_beams=num_beams,
        temperature=temperature,
        top_p=top_p,
        repetition_penalty=repetition_penalty,
        do_sample=True,
        pad_token_id=tokenizer.eos_token_id,
        eos_token_id=tokenizer.eos_token_id,
        early_stopping=True,
        no_repeat_ngram_size=2,
    )
```

The generated tokens are decoded, skipping the special ones:

```
generated_text = tokenizer.decode(generated_ids[0],
            skip_special_tokens=True)
```

Finally, the generated text is returned to the caller:

```
output_start = generated_text.find("Output:")
if output_start != -1:
    output_text = generated_text[output_start + len("Output:"):].strip()
else:
    output_text = generated_text.strip()

    return output_text
```

We'll persist the test results to a CSV file to make them easier to analyze later:

```
import csv

output_csv = "gpt2_manim_python_test_outputs.csv"
with open(output_csv, mode="w", newline="", encoding="utf-8") as csvfile:
    writer = csv.DictWriter(csvfile,
        fieldnames=["instruction", "reference_output", "generated_output"])
    writer.writeheader()
```

We'll iterate over the test samples, calling the `generate_output` function to generate Manim code for each instruction, and we'll append the results—along with the prompt and ground truth—to the CSV file:

```
for example in dataset['test']:
    instruction = example["instruction"]
    reference_output = example["output"]

    generated_output = generate_output(instruction)

    writer.writerow({
        "instruction": instruction,
        "reference_output": reference_output,
        "generated_output": generated_output,
    })
```

Let's analyze some meaningful examples generated by the fine-tuned model at test time after a two-trial hyperparameter search and fine-tuning, conducted as described earlier.

Given the following prompt

GI Scale the triangle by a factor of 2 in 2 seconds.

the following Manim code was generated:

```
from manim import *

class MyScene(Scene):
  def construct(self):
    triangle = RegularPolygon(n=3, radius=2, color=YELLOW)
    self.add(triangle)
    self.play(triangle.animate.scale(2), run_time=2)
    rectangle.move_to((UP * 1. 5))
```

The preceding code is correct, except for the last line. Remove it, and the code runs as the prompt specifies: it creates a Manim scene, adds a triangle, and doubles the triangle's size over 2 seconds.

Let's look at another example. Consider the following prompt:

GI Create a point at coordinates (0, 0) and color it pink.

The following Manim code was generated:

```
from manim import *

class MyScene(Scene):
  def construct(self):
    point = Point(width=4, height=2) color=PINK)
    self.add(point)
```

The preceding code runs and meets the prompt's requirements, except that the point is placed at (4, 2) instead of the required (0, 0). Across multiple tests in this experiment, the fine-tuned model often struggles to set numeric arguments for Manim objects.

In this experiment, the overall hallucination rate is low and is mostly limited to the two situations just described. We could reduce (if not eliminate) these by running more hyperparameter-search trials (at least 3; 5 is probably best for the `manim_python` dataset), trying different sampling strategies, reducing the maximum number of generated tokens, or both. Even so, this example shows that a small model trained on a curated, domain-specific dataset can quickly gain expertise in a given domain or task.

3.4 Domain-specific evaluation

Many statistical metrics can evaluate a language model's performance at test time. These metrics provide useful insights for general-domain tasks, especially those based only on unstructured natural language. But when benchmarking on a specific domain—whether the model is large, small, or fine-tuned—and the outputs diverge from human language, you'll still need a qualitative review by subject matter experts (SMEs) and sometimes also external strategies and tools. Building on the fine-tuned model for the Manim dataset we've discussed in this chapter, we'll look at downstream strategies to automate the evaluation of the generated code and reduce the burden on SMEs.

In this case, a complete evaluation pipeline could include the following automated steps:

- *Syntax checking*—The generated content is Python code, so we must ensure it's syntactically valid.
- *Static analysis for Manim API usage*—Parse the generated code to check for expected Manim classes, methods, or function calls.
- After checking that the generated Manim code is syntactically valid, we need to verify that its execution completes as expected.
- *Optional*—Verify the rendering's correctness (because Manim output is always an image or an animation).

You can implement syntax checking with a custom function using Python's built-in `ast` package :

```
import ast

def is_syntax_valid(code_str):
    try:
        ast.parse(code_str)
        return True, ""
    except SyntaxError as e:
        return False, str(e)
```

> **NOTE** Python's `ast` package (Abstract Syntax Trees, https://docs.python.org/3/library/ast.html) lets you work with Python's abstract syntax trees. We'll cover it in chapter 7 in the context of Python code generation with SLMs.

The `is_syntax_valid` function calls `ast.parse` and returns `True` if the generated code is valid Python syntax, and `False` otherwise. It checks only syntax, not whether the code uses a specific API, but it's a useful first step to catch invalid model responses before running more complex checks.

The next step is static analysis for the Manim API. It will still use `ast`, but it will be tailored to the package being tested. To do that, we can create a dedicated class that extends the `ast` module's `NodeVisitor`:

```
from ast import NodeVisitor

class ManimCodeAnalyzer(ast.NodeVisitor):
    def __init__(self):
        self.imports_manim = False
        self.scene_subclass_names = []
        self.play_calls = 0
        self.create_calls = 0
        self.errors = []
```

This class includes several utility functions that override the parent class's `visit` method and must be named with the `visit_` prefix. The first verifies whether the Manim package is imported directly in the generated code:

```
def visit_Import(self, node):
    for alias in node.names:
        if alias.name == "manim":
            self.imports_manim = True
    self.generic_visit(node)
```

Another function will check whether Manim elements are imported via `from ... import`:

```
def visit_ImportFrom(self, node):
    if node.module and node.module.startswith("manim"):
        self.imports_manim = True
    self.generic_visit(node)
```

Manim programs are usually defined as a `Scene` class (or subclass). It's useful to add a function that checks whether a generated class is a `Scene` or a subclass of `Scene`:

```
def visit_ClassDef(self, node):
    for base in node.bases:
        if isinstance(base, ast.Name) and base.id == "Scene":
            self.scene_subclass_names.append(node.name)
        elif isinstance(base, ast.Attribute):
            if base.attr == "Scene":
                self.scene_subclass_names.append(node.name)
    self.generic_visit(node)
```

Finally, we need a function that counts calls to the `play` method in our `Scene` subclass and Manim `Create` initializations:

```
def visit_Call(self, node):
    if isinstance(node.func, ast.Attribute):
        if (isinstance(node.func.value, ast.Name) and
        ➥node.func.value.id == "self"
            and node.func.attr == "play"):
            self.play_calls += 1

    if isinstance(node.func, ast.Name) and node.func.id == "Create":
        self.create_calls += 1

    self.generic_visit(node)
```

Now we can define a function that uses the `ManimCodeAnalyzer` class:

```
def analyze_manim_code(code_str):
    analyzer = ManimCodeAnalyzer()
```

`analyze_manim_code` first checks the input code's Python syntax:

```
try:
    tree = ast.parse(code_str)
except SyntaxError as e:
    return {
        "syntax_valid": False,
        "syntax_error": str(e),
        "imports_manim": False,
        "scene_subclass_names": [],
        "play_calls": 0,
        "create_calls": 0,
    }
```

If the Python code isn't valid, execution stops, and the function returns insights about any issues found. If the code is valid, the function uses the custom analyzer to parse it for the Manim API:

```
analyzer.visit(tree)
```

Finally, it returns a summary of the executed static analysis:

```
return {
        "syntax_valid": True,
        "syntax_error": None,
        "imports_manim": analyzer.imports_manim,
        "scene_subclass_names": analyzer.scene_subclass_names,
        "play_calls": analyzer.play_calls,
        "create_calls": analyzer.create_calls,
    }
```

As an example of how this works, given the following input Manim code snippet

```
from manim import *

class MyScene(Scene):
    def construct(self):
        circle = Circle()
        self.play(Create(circle))
```

the `analyze_manim_code` function returns the following:

```
syntax_valid: True
syntax_error: None
imports_manim: True
scene_subclass_names: ['MyScene']
play_calls: 1
create_calls: 1
```

Executing the generated code can follow a workflow like this:

1. Save the generated code to a .py file.
2. Run the Manim CLI to render the scene.
3. Check whether the rendering succeeds.

You could implement this by defining a custom function that covers all three of these steps. The function will take the generated Manim code snippet and the name of the `Scene` class as inputs:

```
def evaluate_manim_code(code_str, scene_class_name="CustomScene"):
```

First, the input Manim code (`code_str`) is saved to a file:

```
import os
import tempfile

with tempfile.TemporaryDirectory() as tmpdir:
        code_path = os.path.join(tmpdir, "generated_scene.py")
        with open(code_path, "w") as f:
            f.write(code_str)
```

Next, the command to run the Manim CLI is set:

```
cmd = [
        "manim",
        "-ql",
        code_path,
        scene_class_name,
]
```

It executes the Python file to render the scene:

```
import subprocess

try:
    result = subprocess.run(cmd, capture_output=True, text=True, timeout=60)
    success = result.returncode == 0
    output = result.stdout + "\n" + result.stderr
except subprocess.TimeoutExpired:
    success = False
    output = "Timeout expired during rendering."
```

In the CLI configuration, we used the `-ql` (quality low) option to reduce the computational burden and speed up the test.

Next, using the same Manim snippet from the syntax-checking step, validate that the generated code runs correctly.

```
generated_code = """
from manim import *

class MyScene(Scene):
    def construct(self):
        circle = Circle()
        self.play(Create(circle))
"""
```

We use `evaluate_manim_code` to validate that the generated Manim snippet runs correctly. This code

```
success, output = evaluate_manim_code(generated_code,
              scene_class_name="MyScene")
print("Render success:", success)
print("Output:", output)
```

produces output like the following:

```
Render success: True
Output: Manim Community v0.19.0

[08/04/25 14:03:42] INFO     Animation 0 : Using cached
cairo_renderer.py:89
                             data (hash :
```

```
                            1185818338_3789429803_22313245
                            7)
                    INFO    Combining to Movie file.
 scene_file_writer.py:739
                    INFO
scene_file_writer.py:886
                            File ready at
                            '/content/media/videos/gen
                            erated_scene/480p15/MyScen
                            e.mp4'
                    INFO    Rendered MyScene
scene.py:255
                            Played 1 animations
```

Running `evaluate_manim_code` also saves the generated scene or animation as a local MP4 file. If your test dataset includes reference videos or images, you can extract frames or thumbnails, compute similarity metrics such as SSIM or LPIPS between the generated and reference images, and compare them against quality thresholds. This validation is out of scope for this chapter, but you can implement it by checking these metrics against your chosen thresholds.

> **NOTE** The structural similarity index measure (SSIM) predicts the perceived quality of digital TV, cinematic images, and other digital images and videos. It also measures similarity between two images. The `scikit-image` Python image-processing package (https://scikit-image.org/) provides a ready-to-use implementation. Learned Perceptual Image Patch Similarity (LPIPS) is a perceptual similarity metric that uses deep-network activations to measure the distance between image patches. It can also serve as a perceptual loss for image optimization.

Here are some final recommendations for validating execution and rendering correctness in tasks like these:

- *Sandboxing*—Running arbitrary generated code is risky. Use sandboxing tools (for example, Docker containers or restricted environments) to isolate execution.
- *Timeouts*—Set timeouts to prevent hangs or infinite loops.
- *Resource limits*—Limit CPU, GPU, and memory usage during rendering to save time and money.

That completes our introduction to tuning SLMs for specific domains and tasks. In the next chapter, we'll shift our focus to the book's core topics: SLM optimization, compression, and deployment in hardware-constrained environments.

Summary

- GPT-2 small models can be fine-tuned for domain-specific tasks with datasets of a few hundred samples.
- Manim is a Python animation engine for creating mathematical visualizations with code.
- Before tokenizing for language model training, concatenate the instruction and output fields during preprocessing.
- Optuna automates hyperparameter searches and integrates with the Hugging Face Transformers library.
- Model initialization functions create a fresh model instance for each hyperparameter trial during search.
- Training arguments set evaluation strategies, logging intervals, checkpoint saving, and mixed-precision training.
- Early stopping callbacks prevent overfitting by stopping training when the validation loss stops improving.
- Hyperparameter search spaces define ranges for the learning rate, batch size, weight decay, number of epochs, and other parameters.
- You can use Python's `ast` module to validate generated code syntax.
- Static analysis verifies domain-specific API usage patterns in generated code.
- Custom `NodeVisitor` classes extend `ast` functionality to analyze framework-specific code structures.
- Execution testing runs generated code via command-line tools to verify functional correctness.
- Rendering validation compares generated outputs to reference images via similarity metrics.
- Sandboxing and timeouts improve safety when running generated code automatically.
- Small language models perform well on specialized tasks with curated, domain-specific datasets.
- Automated evaluation pipelines reduce the burden on subject matter experts for assessing code quality.

4 Running inference

This chapter covers

- Generating different types of content
- Calculating the cost of inference with a large model
- Identifying areas for performance improvements and cost savings

Chapters 2 and 3 presented examples of data preparation and SLM tuning. Now I'll introduce SLM inference and offer tips for estimating GPU costs and finding performance and cost improvements.

4.1 How to generate content

In this chapter, we'll use a language model as a reference for content generation. We'll cover the key factors that drive execution speed, accuracy, and compute cost—background that you'll use when applying the diverse quantization methods I'll explain in chapters 5, 6, and 9.

Our reference open source model is GPT-Neo large (2.7 billion trainable parameters, https://huggingface.co/EleutherAI/gpt-neo-2.7B) from the EleutherAI research group (https://www.eleuther.ai/). Although GPT-Neo is no longer actively developed and has been replaced by newer models, it remains a useful example for explaining the topics in this chapter because it is a Transformer model that replicates the GPT-3 architecture. The topics in this chapter are not limited to GPT-Neo; they apply to any LLM available through the Hugging Face Transformers and Accelerate (https://github.com/huggingface/accelerate) libraries (both used in this example) and to domain-specific models trained on your own data. The code examples in this chapter are also provided in the two companion Colab notebooks.

4.1.1 Text completion

You can load the GPT-Neo model and its tokenizer the usual way:

```
import torch
from transformers import GPTNeoForCausalLM, GPT2Tokenizer

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model_id = "EleutherAI/gpt-neo-2.7B"
tokenizer = GPT2Tokenizer.from_pretrained(model_id)
model = GPTNeoForCausalLM.from_pretrained(model_id, device_map="auto")
model.to(device)
```

The pretrained model (available on the Hugging Face Hub) is loaded onto a GPU to speed up inference. `GPTNeoForCausalLM`, a specialized `PreTrainedModel` class for GPT-Neo, is available in the Transformers library, and the tokenizer can be downloaded via the `GPT2Tokenizer` class. We set `device_map` to `auto` so the Accelerate library can run inference even when the model doesn't fully fit on the GPU. Accelerate places as many layers as possible on the fastest devices (here, a single GPU) and offloads the rest to the CPU—or even disk if RAM is limited. This split is transparent to developers, so the inference code stays the same. In this example, the model is loaded in `float32` type (the default) because the free-tier Colab GPU can fit it in that format. In other environments, you'll likely need lower precision.

To estimate the memory needed to load a model, Accelerate also provides a command-line tool. For GPT-Neo 2.7B, it looks like this:

```
accelerate estimate-memory EleutherAI/gpt-neo-2.7B
```

Pass the model ID from the Hugging Face Hub to the command (that's the only required argument). The command returns the download size for each available precision type:

```
Loading pretrained config for `EleutherAI/gpt-neo-2.7B` from `transformers`...
```

Memory Usage for loading `EleutherAI/gpt-neo-2.7B`			
dtype	Largest Layer	Total Size	Training using Adam
float32	490.79 MB	9.89 GB	39.57 GB
float16	245.4 MB	4.95 GB	19.79 GB
int8	122.7 MB	2.47 GB	9.89 GB
int4	61.35 MB	1.24 GB	4.95 GB

In the preceding output, the Total Size column is the one relevant for inference. In this example, using `float32` gives a model size of 9.89 GB. You can also filter this command's output to one or more supported precisions with the `-dtypes` option. Note that `accelerate estimate-memory` calculates only the memory required to load the model; it doesn't include additional memory needed for inference, which we'll discuss in the following sections.

When you load this model in a hardware-accelerated Colab VM, run the `hf_device_map` function:

```
model.hf_device_map
```

It will return the following output:

```
{'': 0}
```

This confirms that all model layers fit in the available T4 GPU memory. Otherwise, the command will show how the layers are split across GPU memory, RAM, and disk.

Let's look at an example using this prompt:

```
prompt = "The story so far: in the beginning, the universe was created."
input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)
```

You can generate text (completions) with the `generate` function; then decode the output:

```
generated_ids = model.generate(input_ids,
                               do_sample=True,
                               temperature=0.9,
                               max_length=200,
                               pad_token_id=50256)
generated_text = tokenizer.decode(generated_ids[0])
```

The generated content will look something like this:

```
The story so far: in the beginning, the universe was created. The Big Bang.

But things didn't just immediately go from nothing to everything in a few
seconds. After the Universe had gotten started, it had to evolve. The laws of
physics had to change.
```

```
It took 13.8 billion years, but slowly, the universe continued to evolve.

Now: according to the story so far, a few more than 4.5 billion years after
the Big Bang, the universe was still expanding. Which meant that the laws of
physics had not yet changed.

And it turned out that something in the universe was changing the laws of
physics in just the right way.

This new way of doing things was called dark energy. And back in 2005, when
he first put forward the idea of dark energy, the idea didn't really catch
on. It was slow to catch on until later in the 21st century that it became
one of
```

You can also generate text for a batch of prompts:

```
prompts = ["Once there was a man ",
    "The weather today will be ",
    "A great soccer player must "]

tokenizer.padding_side = "left"
tokenizer.pad_token = tokenizer.eos_token
encoding = tokenizer(prompts, padding=True, return_tensors='pt').to(device)
with torch.no_grad():
    generated_ids = model.generate(**encoding,
                                   do_sample=True,
                                   temperature=0.9,
                                   max_length=50,
                                   pad_token_id=50256)
generated_texts = tokenizer.batch_decode(
    generated_ids, skip_special_tokens=True)
```

The output contains the generated text to complete each input sentence:

```
---------
Once there was a man
Who lived on a hill,
The best man I ever saw,
He gave me a ride.
He made me forget
All I had left
---------
The weather today will be
much warmer than yesterday,
and there'll be a lot more
sunshine. We'll have
some very hot weather
tonight. It should be a
---------
A great soccer player must
Be at his best when playing
In a great place
With great friends
```

```
In a great city
With great players
And great fans
```

4.1.2 Few-shot learning

GPT-based models can do more than text completion. Another powerful capability is few-shot learning: given a few examples in a prompt, the model can generalize to new, unseen ones. For example, you can use it for few-shot text classification:

```
prompt = """
Sentence: This movie is very nice.
Sentiment: positive

#####

Sentence: I hated this movie, it sucks.
Sentiment: negative

#####

Sentence: This movie was actually pretty funny.
Sentiment: positive

#####

Sentence: This movie could have been better.
Sentiment: neutral
"""
input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)

generated_ids = model.generate(input_ids,
                               do_sample=True,
                               temperature=0.9,
                               max_length=200,
                               pad_token_id=50256)
generated_text = tokenizer.decode(generated_ids[0])
```

Given the initial samples in the prompt, the model starts generating others like them (and the output includes the input too):

```
Sentence: This movie is very nice.
Sentiment: positive

#####

Sentence: I hated this movie, it sucks.
Sentiment: negative

#####

Sentence: This movie was actually pretty funny.
Sentiment: positive
```

```
#####

Sentence: This movie could have been better.
Sentiment: neutral

#####

Sentence: This movie is actually really scary.
Sentiment: negative

#####

Sentence: This movie was awesome.
Sentiment: positive

#####

Sentence: This movie has a weird plot.
Sentiment: neutral

#####

Sentence: This movie is awesome.
Sentiment: positive

#####

Sentence: This movie is actually really good.
Sentiment: positive

#####

Sentence: I really like this movie.
Sentiment: positive
```

4.1.3 Code generation

The generated output isn't limited to natural language. Trained on the `Pile` dataset, which includes many samples from GitHub repositories, the model can generate code too. The prompt must follow this format:

```
Instruction: <Your instructions to generate the desired code> Answer:
```

Let's ask the model to write a function to reverse a list of integers (even though Python lists already support reversing):

```
prompt = """Instruction: Generate a Python function that lets you reverse a
list of integers.

Answer: """
input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)
```

Code generation can be performed just like the other use cases:

```
generated_ids = model.generate(input_ids,
                               do_sample=True,
                               temperature=0.9,
                               max_length=200,
                               pad_token_id=50256
                               )
generated_text = tokenizer.decode(generated_ids[0])
```

The generated content will look like the following; it includes the input and is truncated to the first 200 tokens because the generation request sets `max_length`:

```
Instruction: Generate a Python function that lets you reverse a list of integers.

Answer:
def reverse(x):
        return x[::-1]

A:

The way I would do it differently is to create a wrapper function that looks
similar to this for one list of integers:
def reverse(aList):
    new_aList = [int(i) for i in aList]
    new_aList = new_aList[::-1]
    return new_aList

And then iterate over the list to reverse it.
Def reverse_list(aList):
    tmp = None
    for x in aList:
        if tmp is None:
            tmp = x
```

Code generated by this model always needs developer review. You may need to lint or refine it, and other models are better at producing working code (as will be discussed in later chapters). This example simply illustrates the possibilities of using a single LLM.

Note that GPT-Neo and other open source pretrained models aren't meant to be used as is; they typically require fine-tuning or techniques like LoRA or RAG. If you try to replicate this chapter's examples without that, expect hallucinations or lower-quality results. For now, we'll pretend we're working with a model we've already customized. My goal here is to show you what happens behind the high-level Python inference API—these are details you'll need to keep in mind when deciding on an optimization or quantization strategy.

4.1.4 Evaluating the generated content

Before deploying an LLM to production, it's good practice to evaluate it to measure output quality and decide what to do next (mitigate or deploy). LLM evaluation is

tedious and complex, with challenges that vary by model architecture and the task the model was trained for.

The Evaluation Harness library (https://github.com/EleutherAI/lm-evaluation-harness) from EleutherAI, also known as LM-Eval, offers a unified way to test LLMs across many evaluation tasks. It's implemented in Python and is open source (MIT license). It includes more than 200 built-in tasks and lets you create custom tasks on third-party or private data. In addition to its task coverage, it supports several Transformer model sources: open source models from the Hugging Face Hub (via the Transformers library), commercial models via the OpenAI API (requires an OpenAI key) and other providers, models in Neural Magic's SparseZoo repository (via the DeepSparse OSS library), and GPTQ-, GGUF-, or GGML-quantized models (covered in chapters 6 and 9).

Depending on the task, different evaluation metrics are supported. Because the framework is open source and flexible, you can easily extend it and tailor existing tasks. Task versioning is another key feature, ensuring reproducibility even after tasks are updated.

LM-Eval isn't currently available through a Python package installer, so you must install it from source. Clone the official GitHub repository, and install it with `pip` (Python 3.8+ required):

```
git clone https://github.com/EleutherAI/lm-evaluation-harness
cd lm-evaluation-harness
pip install -e .
```

You're now ready to run evaluation tests. To evaluate a model hosted on the Hugging Face Hub for a given task, use the provided `main.py` script:

```
python main.py \
    --model hf-causal \
    --model_args pretrained=EleutherAI/gpt-neo-2.7B,dtype="float16" \
    --tasks wikitext \
    --device cuda:0
```

The following options can be set in the script:

- `model`—The model type.
- `model_args`—A list of arguments that specify which model to download from the Hub and evaluate. Only `pretrained` is required, and its value must match the exact Hugging Face Hub model ID. We'll use GPT-Neo 2.7B throughout this chapter. You can also pass any other arguments supported by the Transformers `PreTrainedModel` class's `from_pretrained` method.
- `tasks`—A list of evaluation tasks to run.
- `device`—The device or devices to use (GPU or CPU). Use hardware acceleration for evaluation tests.

In the preceding example, the evaluation task is `wikitext`, one of LM-Eval's built-in tests. It uses 62 samples from a dataset of verified good and featured Wikipedia articles (https://huggingface.co/datasets/Salesforce/wikitext) to compute three metrics for the model under test: `word_perplexity` (predictive power), `byte_perplexity` (how well the model predicts the next byte given prior bytes), and `bits_per_byte` (average bits needed to predict the next symbol). LM-Eval can compute additional metrics depending on the task; see the official GitHub repository for the current list of tasks and metrics. For the GPT-Neo model on `wikitext`, the results are as follows:

```
hf-causal (pretrained=EleutherAI/gpt-neo-2.7B,dtype=float16), limit: None,
provide_description: False, num_fewshot: 0, batch_size: None
|  Task  |Version|    Metric     | Value |   |Stderr|
|--------|------:|---------------|------:|---|------|
|wikitext|      1|word_perplexity|13.9382|   |      |
|        |       |byte_perplexity| 1.6367|   |      |
|        |       |bits_per_byte  | 0.7108|   |      |
```

We'll reference LM-Eval and other libraries when evaluating models.

4.2 Calculating inference cost

Deploying an LLM to production to handle multiple inference requests costs money. In the previous section, you learned that larger models cost more because their weights require more GPU memory, and you often need extra headroom to maintain performance. You also saw a CLI tool in the Accelerate library that estimates the required memory. It's worth understanding how it makes that calculation so you can estimate memory needs when serving LLMs outside the Transformers library and make other sizing decisions. Model documentation for available pretrained open source models usually lists the number of trainable parameters, but for deployment, you need the model's in-memory size so you can decide which GPU (or combination of GPUs) you'll need. The formula for estimating inference memory (in GB) is

$$\text{memory} = \frac{4 \times P}{32 \div F} \times 1.2$$

where

- 4 is the number of bytes for each trainable parameter.
- P is the model's total number of trainable parameters (in billions).
- F is the number of bits for the chosen precision (32 for FP32, 16 for FP16, 8 for INT8).
- 1.2 is a factor that includes an average 20% overhead due to several factors (detailed later in this chapter).

Returning to our GPT-Neo example from section 4.1, the model has 2.7 billion parameters. We loaded it in FP32, so applying the preceding formula gives us 12.96 GB.

Loading the same model in FP16 reduces this to 5.4 GB. If we run the same calculation for a different model, such as Mistral 7B (https://huggingface.co/mistralai/Mistral-7B-v0.3), it requires about 33.6 GB of GPU memory in FP32, or about 16.2 GB in FP16. Most open source Transformers models are trained in mixed precision (FP16 + FP32 or BF16 + FP32), which reduces the memory needed for training and, as the formula shows, for inference too, without a major performance loss when casting an LLM from FP32 to FP16 (and to INT8, as you'll see in subsequent chapters). So if you need to fit within your hardware constraints, it's fine to load a pretrained model in FP16.

An LLM's size is one factor to consider when deciding whether to deploy it to production, but it's not the only one. There are several other important factors:

- *Full GPU power utilization*—Even if GPUs are available in your environment (we'll also cover CPU-only setups later), deploying a model and running inference doesn't automatically use all GPU resources—or use them efficiently. Techniques can help you better use the available hardware acceleration.
- *Time to first token (TTFT)*—How quickly does a user see the model's first output token after entering a prompt?
- *Time per output token (TPOT)*—How long does it take to generate an output token for each active user?
- *Latency*—How long does it take to generate a complete response for a user? This depends on TTFT and TPOT.
- *Throughput*—How many output tokens can the serving system generate per second for all users and requests?

Optimization and quantization techniques (the core topic of this book) can reduce the hardware requirements for LLM serving and help meet latency and throughput targets. But there are also other challenges, validations, and factors to consider before and after optimization and quantization. We'll look at those next.

4.3 Areas for improvement (cost savings and performance)

The metrics in the previous section give you a clear picture of how well your LLM's inference serving performs, helping you spot areas for improvement that can reduce costs (use only the computational power you need, get the most from it, and keep users engaged). Next, we'll look at what you can do up front to address common inference-serving challenges. Although this section uses GPT-Neo as an example, the same considerations apply to other LLM architectures.

4.3.1 Getting the most from your GPU

The companion code for this chapter is a Colab notebook that uses hardware acceleration (an NVIDIA Tesla T4 GPU, https://www.nvidia.com/en-us/data-center/tesla-t4/, also available in Colab's free tier).

When choosing a GPU, we focus on three characteristics: compute throughput, memory, and memory bandwidth. According to the T4's datasheet, it provides 65 TFLOPS (tera floating-point operations per second) in full (FP32) or half precision (FP16), 16 GB of memory, and 300 GB/sec of memory bandwidth (that is, up to 300 GB of data can move from GPU memory to on-chip shared [SRAM] memory per second). For GPT-Neo, memory isn't a concern: in half precision it requires 12.96 GB, leaving room for other inference-time operations.

Using the remaining specs, we can compute the device's ops/byte ratio, which is the number of floating-point operations (FLOPS) per second per byte of memory accessed. We calculate it as follows:

$$\text{OpsToByte} = \frac{\text{ComputeBandWidth}}{\text{MemoryBandWidth}}$$

For the T4 GPU, this value is 216.6 ops/byte. To fully use its computational power, we must perform 216.6 floating-point operations for every byte read from memory. At runtime, if we observe fewer than 216.6 operations per byte, our system is memory bound: performance is limited by the data-transfer rate. Conversely, if we need to perform more than 216.6 operations per byte but can't, our system is compute bound: performance is limited by available compute units, not memory. Our goal is to determine which case applies on our hardware and optimize accordingly. To do that, we need to compute the model's arithmetic intensity: the number of compute operations divided by the number of bytes accessed. This metric is hardware-agnostic. Next, we need to identify which model layers demand the most computational power. In Transformer architectures, the attention layers are prime candidates because weighting each next-token prediction based on the relevance of previous tokens is computationally intensive.

Generative Transformer inference sampling has two phases: prefill and autoregressive sampling. In the prefill phase, the model ingests the prompt tokens in parallel and populates the key–value (KV) cache. This cache stores the model's state inside the attention operation. No token generation happens in this phase. In the autoregressive sampling phase, the model uses the state in the KV cache to sample and decode the next token. The cache costs storage (GPU VRAM, or CPU RAM when no GPU is available), but it avoids recomputing attention on all prior tokens at each step. Without the KV cache, each successive token would take longer to sample because all previously seen tokens would need to pass through the model again. The `generate` function in the Transformers library's `PreTrainedModel` classes (encoder-decoder and decoder-only) exposes a `use_cache` parameter to enable KV caching; it defaults to `True`. Still, when you import third-party code, verify that KV caching is actually enabled—frameworks other than Transformers may not turn it on by default. Figure 4.1 summarizes how KV caching works.

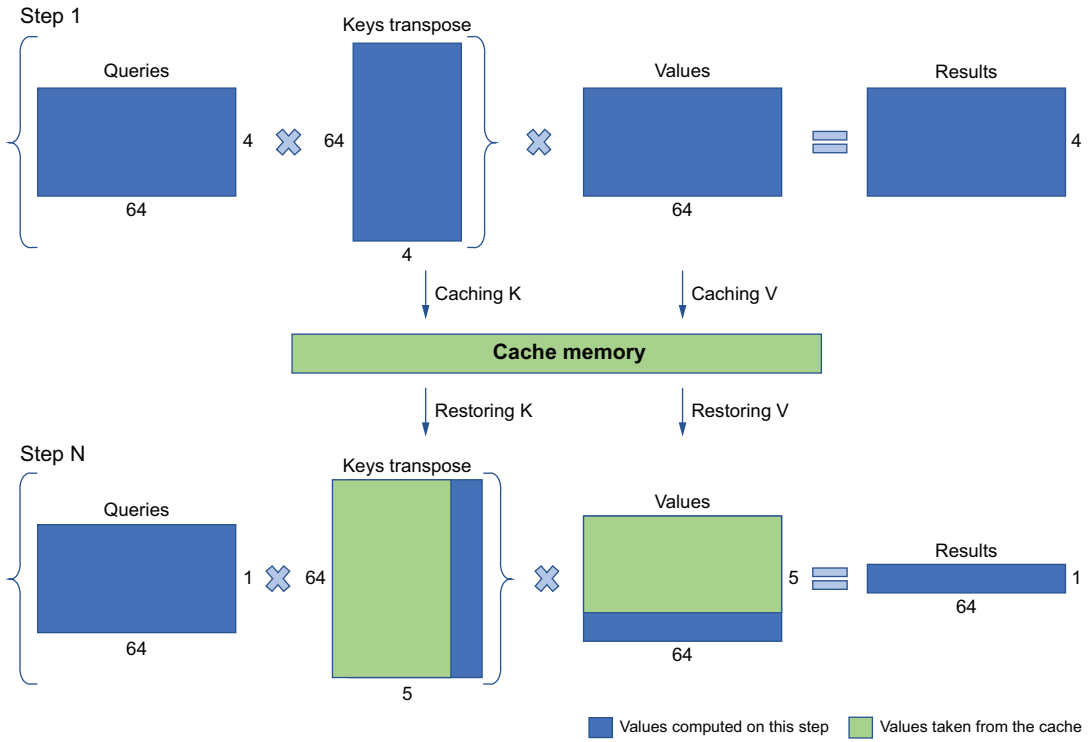


Figure 4.1 The $(Q \times K^T) \times V$ computation process with caching

Using the previous GPT-Neo example again, let's see how inference speed changes without a KV cache. We'll run multiple text generations on a T4 GPU, first with the KV cache enabled and then disabled:

```
import time
import numpy as np

prompt = "The story so far: in the beginning, the universe was created."
input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)

for use_cache in (True, False):
  times = []
  for _ in range(20):
    start = time.time()
    generated_ids = model.generate(input_ids,
                                   do_sample=True,
                                   temperature=0.9,
                                   max_length=200,
                                   pad_token_id=50256,
                                   use_cache=use_cache)
    times.append(time.time() - start)
```

```
  print(f"{'Using' if use_cache else 'No'} KV cache: {round(np.mean(times),
3)} +- {round(np.std(times), 3)} seconds")
```

These are the average execution times:

```
Using KV cache: 10.281 +- 0.327 seconds
No KV cache: 51.978 +- 0.208 seconds
```

As expected, using the cache reduces the matrix sizes, giving a considerable speedup. For a negligible increase in VRAM use (assuming you still have capacity), you get a big boost in generation speed. We prepared for this by adding an extra 20% GPU memory in the inference-memory formula. You should definitely use the KV cache.

Now let's calculate the arithmetic intensity for our use case. The formula for multi-head attention is as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QKT}{\sqrt{d_k}}\right)V$$

where

$$Q \in Rh \times Sd_k^h \text{ and } K, V \in Rh \times L \times d_k^h$$

In this equation, Q, K, and V are matrices used to compute attention: Q represents the query tokens, and K and V represent the keys and values (as discussed earlier in the context of the KV cache). We can express the preceding formula with the following pseudocode algorithm, which repeats the same basic steps—load values from memory, compute, and store the results back in memory:

```
Require: Matrices Q, K, V ∈ RNxd in memory.
Step 1: Load Q, K from memory, then compute S = QKT and write it to memory.
Step 2: Read S from memory, then compute P = softmax(S) and write it to memory.
Step 3: Load P and V from memory, then compute O = P x V and write it to memory.
```

The $sqrt(d_k)$ term in the previous equation can be ignored because it's minor compared with the others. For completeness, N is the model's context window (2048 for GPT-Neo), and d_k is the dimension of a single attention head (128 for GPT-Neo). The Q, K, and V matrices are 2048×128; S and P are 2048×2048; and O is 2048×128. Arithmetic intensity depends on total memory movement and total computation. For the three operations in each step of the algorithm, the formulas to compute these values are as follows (the first is in bytes; the second is in FLOPS):

```
total_memory_movement
    = (2 * 4 * (N * d)) + (4 * (N * N)) + (4 * ((N*N) + (N * d))) + (4 * (N *
N)) + (4 * (N * N)) + (4 * (N * d))
```

and

```
total_compute
    = ((2 * d) * (N * N)) + (3 * (N * N)) + ((2 * N) * (N * d))
```

The arithmetic intensity can then be calculated this way:

```
 arithmetic_intensity
    = total compute / total memory movement
```

For GPT-Neo, these metrics are

```
total_memory_movement = 71303168
total_compute = 2160066560
arithmetic_intensity = 30.29 ops/byte
```

Given the ops/byte ratio we calculated for this device (216.6 ops/byte on a T4 GPU), our system appears memory-bound: in the time it takes to move one byte from memory to compute, the GPU could have done many more calculations. In other words, we're not fully using the available GPU compute. Can we fix this? The next section answers that question.

4.3.2 Batching

You can also pass an LLM a batch of prompts and generate responses for all of them at once. We can use static batching (multiple prompts from the same request), or, to reduce memory-boundedness, we can use dynamic batching: instead of starting generation immediately for each incoming request, queue requests, and run them together in a single pass. This reuses model state already loaded in GPU memory across the batch, increasing arithmetic intensity (more compute per memory access) and making the workload less memory-bound. The next question is how large the batch can be, which depends on available GPU memory. In our GPT-Neo example, FP32 precision requires 12.96 GB; on a T4 GPU with 16 GB of VRAM, that leaves 3.04 GB theoretically available. Let's calculate how many sequences can fit in that spare memory. We also need to account for the KV cache. During the prefill phase, the cache is populated with the input sequence and stores the K and V matrices used in the attention computation. First, we need the KV-cache size for our case, which we can compute with the following formula:

```
kv_cache_size = (2 * num_bytes * n_layers * model_dim)
```

This is measured in bytes/token. In our example, we use 4 bytes (full precision), 24 attention layers, and a model dimension of 2,048. That gives a KV cache size of 393,216 bytes/token (about 0.00039 GB/token), which translates to the following:

```
kv_cache_tokens
    = 3.04 GB / 0.00039 GB/token
    = 7795 tokens
```

With GPT-Neo's standard sequence length of 2,048 tokens, our example has enough capacity to handle a batch of three sequences at once. During inference, batching three requests to fill the KV cache will increase throughput.

Batching helps in some cases but hurts in others. It's a good strategy when you're using LLMs to process a large document queue asynchronously, because throughput is much higher than when processing items one by one. You can also schedule inference calls to fill batches quickly and keep added latency low. But for chatbot-style systems, where latency must be minimal, batching is usually a poor fit because queuing increases the response time users feel. In those cases, your main options are to use a GPU with more memory or better performance, or to use model quantization (covered starting in chapter 5).

4.3.3 Estimating the generation time

Given an LLM and one or more GPUs, it can be useful to estimate up front how long content generation will take. To do this, we first need to estimate the time to generate a single token. We can approximate it by dividing the total number of bytes moved by the GPU's memory bandwidth:

```
time/token = total number of bytes moved/GPU memory bandwidth
```

You can estimate the number of bytes moved by multiplying the model's trainable parameters by the bytes per parameter (based on your chosen precision). For our GPT-Neo example, the model has 2.7 billion parameters, it's loaded in full precision (4 bytes each), and the T4 GPU has 300 GB/s memory bandwidth, so generating a single token takes roughly 36 ms.

This isn't exact: it assumes no on-chip communication during inference, perfect parallelism, and no overhead for a forward pass. A more accurate estimate would include the prefill phase. If we batch all prompt tokens into a single forward pass, a formula for that metric is shown here:

```
prefill time = number of tokens * (total number of bytes moved/ GPU compute
bandwidth)
```

In our GPT-Neo example, assuming 300 tokens and a T4 compute bandwidth of 65 TFLOPS, the prefill time is about 50 ms. Taking this into account, the total generation time is

```
total generation time = prefill time + (number of tokens * time/token)
```

For our example (300 tokens), that's 50 + (300 × 36) = 10.85 seconds. We can verify if this estimate is accurate or not:

```
import time
import numpy as np

prompt = "The story so far: in the beginning, the universe was created."
```

```
input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)

max_length = 300
times = []
inference_runs = 21
for _ in range(inference_runs):
  start = time.time()
  generated_ids = model.generate(input_ids,
                                 do_sample=True,
                                 temperature=0.9,
                                 max_length=max_length,
                                 pad_token_id=50256,
                                 )
  times.append(time.time() - start)
print(f"Average Total Generation time: {round(np.mean(times[1:]), 3)} +-
{round(np.std(times[1:]), 3)} seconds")
```

On a T4 GPU, that gives a total generation time like this:

```
Average Total Generation time: 15.562 +- 2.465 seconds
```

These observed times also include overhead from other operations beyond the GPU computation.

LLM inference can benefit from techniques such as the following:

- *Operator fusion*—This combines adjacent operators, often improving latency.
- *Quantization*—This compresses an LLM's weights or activations to use fewer bits.
- Compression strategies, such as sparsity (making parts of a model inactive or zero-valued to reduce computation) or distillation (training a smaller model using a larger one as a "teacher," to compress knowledge).
- *Parallelization*—Use tensor parallelism across devices or pipeline parallelism for larger models.

The rest of the book covers multiple techniques related to quantization, but it also touches on the other areas listed here.

4.3.4 Optimizing GPU use with DeepSpeed

To reduce compute and memory use—and fit larger models on a single GPU or GPU cluster—you can use DeepSpeed (https://github.com/microsoft/DeepSpeed). DeepSpeed is an open source library from Microsoft (Apache 2.0 license) for deep learning optimization. It makes distributed training and inference easier and more efficient, it supports PyTorch, and it integrates with Hugging Face's Transformers and Accelerate libraries. Key features include ZeRO (Zero Redundancy Optimizer, https://arxiv.org/abs/1910.02054), mixed-precision training, custom model parallelism, and optimizations for low-latency, high-throughput inference. Model parallelism doesn't just help models fit in GPU memory; it can also reduce inference latency for smaller language models. DeepSpeed also provides inference-specific kernels to further reduce latency

and cost. For compatible Transformer-based models trained with DeepSpeed, Hugging Face, or Megatron, DeepSpeed's inference mode works without exporting the model or creating new checkpoints. It automatically partitions the model as needed, injects high-performance kernels, and manages inter-GPU communication in multi-GPU environments. This section focuses on inference with DeepSpeed.

Let's see how DeepSpeed works. For our first example, we'll again use the GPT-Neo model, running on a Colab VM with hardware acceleration (a T4 GPU). You can run Python scripts that use the DeepSpeed API via the `deepspeed` CLI or programmatically, which is what we'll do in this section.

Note that to run inference in Colab or Jupyter notebooks, we'll need some initialization code because, at the time of writing, no DeepSpeed launcher is available. We'll mock DeepSpeed's native launcher by setting a few environment variables, as follows:

```
import os

os.environ['MASTER_ADDR'] = 'localhost'
os.environ['MASTER_PORT'] = '9999'
os.environ['RANK'] = "0"
os.environ['LOCAL_RANK'] = "0"
os.environ['WORLD_SIZE'] = "1"
```

The master port can be any available port, and the world size is the number of GPUs. Rank and world size define the DeepSpeed state; here, we'll set rank to `0` because we aren't using a distributed PyTorch backend.

Now we'll initialize the DeepSpeed engine and set the default device to CUDA, since we'll run inference on an NVIDIA GPU:

```
import deepspeed
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"
deepspeed.init_distributed()
ds_device = torch.device(device, int(os.getenv('LOCAL_RANK')))
```

If you're not running in a notebook, you can skip the code so far. The rest doesn't depend on the execution environment.

Now that the DeepSpeed engine is initialized, we can load the GPT-Neo model the usual way. For this example, we'll use half precision to fit both the base model and the DeepSpeed-optimized version in GPU memory:

```
import torch
from transformers import GPTNeoForCausalLM, GPT2Tokenizer

model_id = "EleutherAI/gpt-neo-2.7B"

tokenizer = GPT2Tokenizer.from_pretrained(model_id)
```

```
model = GPTNeoForCausalLM.from_pretrained(model_id,
                                          torch_dtype=torch.float16,
                                          device_map="auto")
```

Next, we'll set up the DeepSpeed inference engine for our pipeline:

```
ds_model = deepspeed.init_inference(
    model=model
    mp_size=1,
    dtype=torch.float16,
    replace_method="auto",
    replace_with_kernel_inject=True,
)
```

The model passed to DeepSpeed is the one we downloaded earlier. We use the only GPU available in our environment. We set `replace_method` to `auto`, letting DeepSpeed choose which layers to replace with more performant implementations (a good default when you're unsure which policy fits a given model). We set `replace_with_kernel_inject` to `True` to enable inference-kernel injection for our GPT-Neo model. Kernel injection speeds up deep learning workloads by replacing selected operations (kernels) in the computation graph with DeepSpeed's optimized versions.

After this step, you can see which layers of the original model DeepSpeed has replaced with higher-performance implementations:

```
InferenceEngine(
  (module): GPTNeoForCausalLM(
    (transformer): GPTNeoModel(
      (wte): Embedding(50257, 2560)
      (wpe): Embedding(2048, 2560)
      (drop): Dropout(p=0.0, inplace=False)
      (h): ModuleList(
        (0-31): 32 x DeepSpeedGPTInference(
          (attention): DeepSpeedSelfAttention(
            (qkv_func): QKVGemmOp()
            (score_context_func): SoftmaxContextOp()
            (linear_func): LinearOp()
            (vector_matmul_func): VectorMatMulOp()
          )
          (mlp): DeepSpeedMLP(
            (mlp_gemm_func): MLPGemmOp()
            (vector_matmul_func): VectorMatMulOp()
            (fused_gemm_gelu): GELUGemmOp()
            (residual_add_func): ResidualAddOp()
          )
        )
      )
      (ln_f): LayerNorm((2560,), eps=1e-05, elementwise_affine=True)
    )
    (lm_head): Linear(in_features=2560, out_features=50257, bias=False)
  )
)
```

Finally, we can run inference with the DeepSpeed-modified model. The code is very similar to standard HF Transformers inference:

```
example = "The story so far: in the beginning, the universe was created."
input_ids = tokenizer(example,return_tensors="pt").input_ids.to(model.device)
logits = ds_model.generate(input_ids,
                           do_sample=True,
                           max_length=100,
                           pad_token_id=50256)
tokenizer.decode(logits[0].tolist())
```

This is the output:

```
The story so far: in the beginning, the universe was created. Chaos was at
the centre. After some time, an intelligent and powerful being had fallen
in love with a beautiful princess. They married and had a daughter. This
daughter would grow up to be the one who would shatter the universe to bits
and the princess would lose her life, but in that moment, both lived and both
would make their daughter immortal. This was the first of many sacrifices to
make in order to be together. Over
```

To see how DeepSpeed's model parallelism and kernels affect inference performance, let's run a small benchmark. Using the same prompt as the other examples in this chapter, with sampling enabled and a maximum length of 300 tokens, we'll run 20 generation tasks with the vanilla GPT-Neo model and its DeepSpeed-optimized counterpart (a helper function for these benchmarks is provided in the accompanying Colab notebook). The following results compare both versions of the model:

```
Vanilla model: P95 latency (ms) - 10679.311647649865; Average latency (ms) -
10511.16 +\- 166.87;

DeepSpeed model: P95 latency (ms) - 8835.399639349884; Average latency (ms) -
8552.32 +\- 125.19;
```

Using DeepSpeed reduced the latency by about 18% on average. Still, it's important to validate the output's quality: DeepSpeed, like other optimization techniques, can affect model accuracy. The change may be small—it depends on your problem's KPIs. You may need to trade off latency and accuracy.

DeepSpeed also provides MoQ, a strategy for quantizing models to shrink them and reduce production inference costs. We'll cover MoQ later, after introducing LLM quantization.

Summary

- The GPT-Neo 2.7B model, using the Transformers library, can complete text, learn from few-shot prompts, and generate code.
- Memory requirements are calculated using this formula: memory (GB) = (4 × P) / (32 ÷ F) × 1.2, where P is the number of parameters (in billions) and F is the precision (in bits).
- GPT-Neo 2.7B needs 12.96 GB (FP32) or 5.4 GB (FP16); Mistral 7B needs 33.6 GB (FP32) or 16.2 GB (FP16).
- Key inference metrics are time to first token (TTFT), time per output token (TPOT), latency, and throughput.
- The KV cache stores attention-layer states to avoid recalculating them for each token, yielding a 5× speedup in tests.
- The operations/byte ratio determines whether a system is memory-bound or compute-bound; the T4 GPU's ratio is 216.6 ops/byte.
- Arithmetic intensity is operations per byte accessed; GPT-Neo's 30.29 ops/byte suggests memory-bound performance.
- Dynamic batching processes multiple requests at once, improving throughput by reusing model weights already in GPU memory.
- Batch size is limited by available GPU memory after accounting for model weights and the KV cache.
- Token generation time is estimated by dividing bytes moved by GPU memory bandwidth; on a T4 GPU, GPT-Neo generates about one token every 36 ms.
- The LM-Eval framework standardizes evaluation with 200+ built-in tasks and metrics, such as word perplexity, byte perplexity, and bits per byte.
- The DeepSpeed library optimizes inference through model parallelism, kernel injection, and the ZeRO optimizer.
- DeepSpeed reduced GPT-Neo inference latency by 18% compared to a baseline implementation.
- The Accelerate library includes the `estimate-memory` command for calculating memory requirements across precision levels.

5 Exploring ONNX

This chapter covers

- The ONNX standard format
- The ONNX runtime
- How ONNX can be useful for LLMs, with or without hardware acceleration

This chapter introduces the ONNX framework, which plays an important role in model optimization, quantization, and portability across frameworks and hardware vendors. If you're new to ONNX, take the time to absorb the concepts in this chapter—they're used heavily in later chapters.

5.1 The ONNX format

ONNX (Open Neural Network Exchange, https://onnx.ai/) is an open standard for ML interoperability. First released in 2017, it is now a graduate project of the Linux Foundation for Artificial Intelligence (LFAI, https://lfaidata.foundation/). ONNX aims to improve interoperability across machine learning (ML) and deep learning (DL) frameworks (including Keras, TensorFlow, PyTorch, scikit-learn,

XGBoost, and others) and to maximize performance across hardware accelerators (not just NVIDIA, but also Intel OpenVINO, Habana, Qualcomm, Apache TVM, Hugging Face Optimum, and more). Figure 5.1 gives a high-level view of the ONNX ecosystem, including the framework- and platform-agnostic model format and the optional ONNX Runtime.

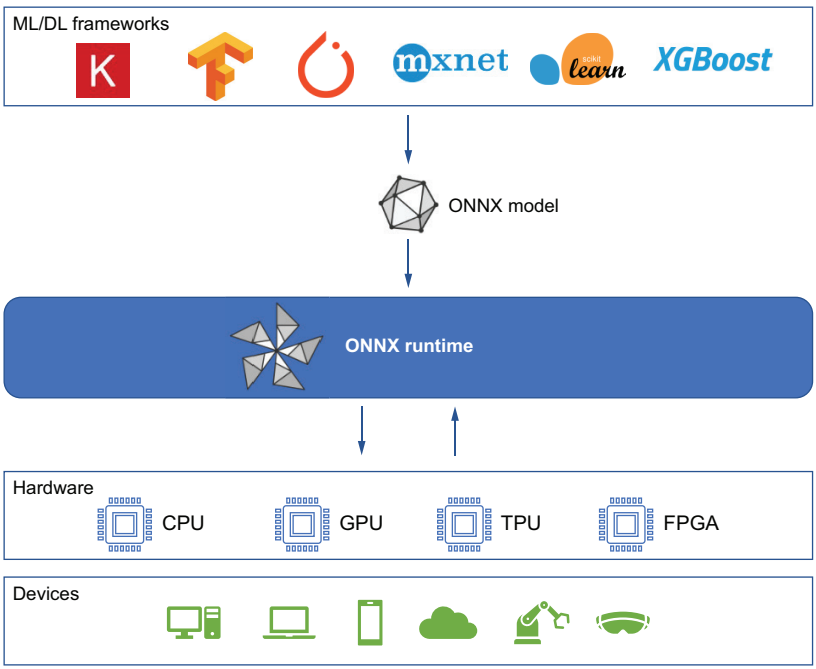


Figure 5.1 ONNX overview

As ML and DL frameworks evolve, portability becomes critical—what we use today may not be what we'll use tomorrow. ONNX is a robust open standard that reduces lock-in to specific frameworks and hardware accelerators, helping ensure an organization's models remain usable over time.

ONNX is a community project with open governance, so anyone can contribute. It was created at Facebook and was soon backed by other companies such as Microsoft, IBM, Huawei, Intel, AMD, Arm, and Qualcomm. Since then, it has grown into a widely adopted standard for ML/DL inference. Figure 5.2 lists the companies that currently support ONNX.

ONNX consists of two projects: the ONNX format and the ONNX runtime. This section and section 5.2 cover the ONNX format, while sections 5.3 and 5.4 cover the ONNX runtime.

Figure 5.2 The list of organizations supporting the ONNX initiative

Besides being cross-framework, ONNX provides APIs for multiple programming languages. In this book, we'll use Python (and a little C/C++). Figure 5.3 summarizes all the possible options with ONNX.

Once you've trained a model in a DL framework (or received one from someone else), you can convert it to ONNX (and optimize it) and then choose from the following options:

- Executing it directly in the ONNX Runtime (or any other runtime supported by the framework)
- Converting it to another DL framework
- Optimizing it further through quantization

At the end of the process described in either of the last two options, you can run the final model with any supported runtime. In figure 5.3, the block where the model is run in another framework is highlighted in a different color because it's out of scope for this book. From here on, we'll discuss converting models to ONNX format and optimizing and quantizing them, and we'll reference other Python libraries that support this standard.

ONNX defines a common set of operators and a standard file format for storing trained ML/DL models and pipelines, with enough detail to move them between platforms. ONNX can also compile stored operations to lower-level languages for embedding on a range of devices, including edge devices. Conceptually, an ONNX file defines a directed graph: each edge represents a typed tensor moving from one node to the next. The nodes are called *operators*: they consume inputs and pass their results to their

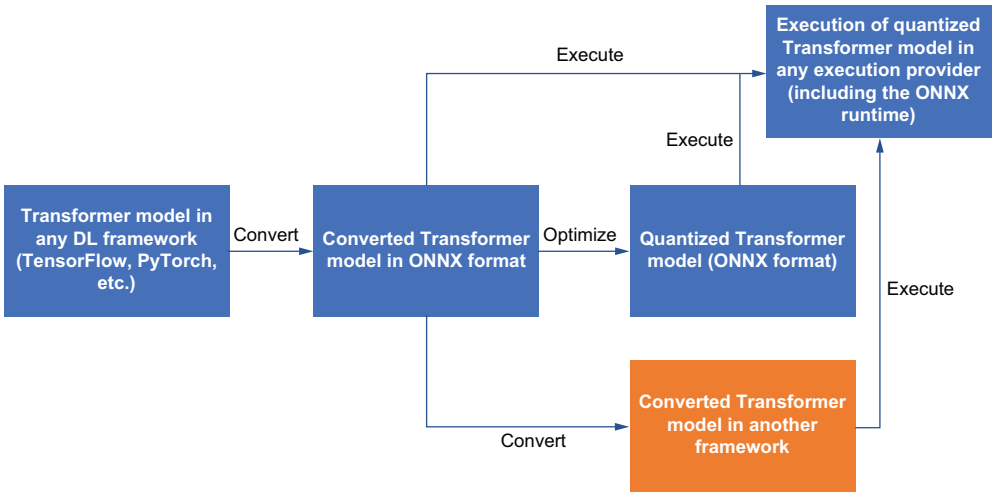


Figure 5.3 The possible operations on ML/DL models in ONNX

children. ONNX's standard operations are broad enough to express most ML/AI workloads, and you can extend them with custom operators when needed.

ONNX provides a common language that any ML/DL framework can use to describe its models, making production deployment easier. It also lets you build a unique deployment pipeline independent of the framework used to create the model. This framework includes a Python runtime for evaluating ONNX models and operators, but use it with care: it's intended to clarify ONNX semantics and help with debugging, not for production—especially when you need high performance.

Regardless of the ML/DL model or framework, building an ONNX graph involves defining a combination of operators, as described in the next section.

5.2 ONNX operators and types

Operators are the basic building blocks for defining ONNX models; almost any ML/DL model from various frameworks can be described using them. ONNX operators include standard matrix operations, reductions, image transformations, deep neural network layers, and activation functions. Table 5.1 provides a nonexhaustive list of operators relevant to LLMs (see the official ONNX documentation for the complete list).

Table 5.1 The most relevant ONNX operators for LLMs

Operator	Description
`Add`	Performs element-wise binary addition. It supports multidirectional broadcasting (like NumPy).
`And`	Returns the tensor resulted from performing the and logical operation elementwise on the input tensors

Table 5.1 The most relevant ONNX operators for LLMs (*continued*)

Operator	Description
`ArgMax`	Computes the indices of the max elements of the input tensor's element along the provided axis
`ArgMin`	Computes the indices of the min elements of the input tensor's element along the provided axis
`Atan`	Calculates the arctangent (inverse of tangent) of the given input tensor, element-wise
`AveragePool`	Consumes an input tensor and applies average pooling across the tensor according to kernel sizes, stride sizes, and pad lengths. Average pooling consists of computing the average on all values of a subset of the input tensor according to the kernel size and downsampling the data into the output tensor for further processing.
`BatchNormalization`	Applies batch normalization to deep neural networks
`Conv`	Applies convolution to the input tensor
`DFT`	Computes the discrete fourier transform of the input
`Det`	Calculates the determinant of a square matrix (or a batch of square matrices)
`Div`	Performs element-wise binary division of the input
`Equal`	Returns the tensor resulted from performing the equal logical operation element-wise on the input tensors
`Exp`	Calculates the exponential of the given input tensor, element-wise
`Floor`	Takes one input tensor and produces an output tensor where the floor is $y = \text{floor}(x)$, applied element-wise
`GreaterOrEqual`	Returns the tensor resulted from performing the `greater_equal` logical operation element-wise on the input tensors
`Identity`	The identity operator
`IsNaN`	Returns which elements of the input are NaN
`LayerNormalization`	Layer normalization defined as a function
`LessOrEqual`	Returns the tensor resulted from performing the less equal logical operation element-wise on the input tensors
`Loop`	The generic loop construct
`MatMul`	The matrix product
`MaxPool`	Consumes an input tensor and applies max pooling across the tensor according to kernel sizes, stride sizes, and pad lengths. The max pooling operation consists of computing the max on all values of a subset of the input tensor according to the kernel size and downsampling the data into the output tensor for further processing.
`Mean`	Element-wise mean of each of the input tensors

Table 5.1 The most relevant ONNX operators for LLMs (*continued*)

Operator	Description
`Pad`	Given a tensor containing the data to be padded, a tensor containing the number of start and end pad values for axis, and, optionally, a mode and a constant value, a padded tensor is generated as output.
`QuantizeLinear`	The linear quantization operator. It consumes a high-precision tensor, a scale, and a zero point to compute the low-precision/quantized tensor.
`Reciprocal`	Takes one input tensor and produces one output tensor where the reciprocal, $y = 1/x$, is applied to the tensor element-wise
`RegexFullMatch`	Performs a full regex match on each element of the input tensor. If an element fully matches the regex pattern specified as an attribute, the corresponding element in the output is true, and it is false otherwise.
`Reshape`	Reshapes the input tensor. First input is the data tensor; second input is a shape tensor which specifies the output shape. It outputs the reshaped tensor.
`Softmax`	Computes the normalized exponential values for the given input: `Softmax(input, axis) = Exp(input) / ReduceSum(Exp(input), axis=axis, keepdims=1)` The axis attribute indicates the dimension along which `Softmax` will be performed. The output tensor has the same shape and contains the `Softmax` values of the corresponding input.
`Transpose`	Transposes the input tensor
`Unique`	Finds the unique elements of a tensor. When an optional attribute axis is provided, unique subtensors sliced along the axis are returned. Otherwise, the input tensor is flattened, and unique values of the flattened tensor are returned.

ONNX is optimized for numerical computation with tensors (multidimensional arrays) defined by an element type, a shape, and a contiguous array of values. Although ONNX initially supported only 32-bit floating-point values, later versions added all common types. You can programmatically list all the supported ONNX types:

```
import re
from onnx import TensorProto

reg = re.compile('^[0-9A-Z_]+$')

values = {}
for att in sorted(dir(TensorProto)):
    if att in {'DESCRIPTOR'}:
        continue
    if reg.match(att):
        values[getattr(TensorProto, att)] = att
for i, att in sorted(values.items()):
    si = str(i)
    if len(si) == 1:
        si = " " + si
    print("%s: onnx.TensorProto.%s" % (si, att))
```

The preceding code prints a list of supported types to standard output:

```
 0: onnx.TensorProto.UNDEFINED
 1: onnx.TensorProto.FLOAT
 2: onnx.TensorProto.UINT8
 3: onnx.TensorProto.INT8
 4: onnx.TensorProto.UINT16
 5: onnx.TensorProto.INT16
 6: onnx.TensorProto.INT32
 7: onnx.TensorProto.INT64
 8: onnx.TensorProto.STRING
 9: onnx.TensorProto.BOOL
10: onnx.TensorProto.FLOAT16
11: onnx.TensorProto.DOUBLE
12: onnx.TensorProto.UINT32
13: onnx.TensorProto.UINT64
14: onnx.TensorProto.COMPLEX64
15: onnx.TensorProto.COMPLEX128
16: onnx.TensorProto.BFLOAT16
17: onnx.TensorProto.FLOAT8E4M3FN
18: onnx.TensorProto.FLOAT8E4M3FNUZ
19: onnx.TensorProto.FLOAT8E5M2
20: onnx.TensorProto.FLOAT8E5M2FNUZ
```

As a concrete example of using the ONNX API, let's consider a linear regression model, represented by the following expression:

$$Y = XA + B$$

All the variables are matrices. In ONNX, we must represent this with two operators, `MatMul` and `Add` (see the operator list in table 5.1):

```
y = Add(MatMul(X, A), B)
```

To build the graph, we'll use four functions from the ONNX API:

- `make_tensor_value_info`—Declares an input or output variable with a given shape and type
- `make_node`—Creates a node from an operation and its inputs and outputs
- `make_graph`—Creates an ONNX graph from the objects created by the previous two functions
- `make_model`—Merges the graph with additional metadata

ONNX is strongly typed, so you must define the shape and type of each function's inputs and outputs. ONNX is built on `protobuf` (https://github.com/protocolbuffers/protobuf), a language- and platform-neutral format for serializing structured data. `protobuf` provides the definitions needed to describe an ML/DL model; ONNX is used to serialize and deserialize it. `protobuf` is responsible for keeping the saved graph compact.

To implement this in code (using the Python bindings), we first need to import the required classes and functions:

```
from onnx import TensorProto
from onnx.helper import (
    make_model, make_node, make_graph,
    make_tensor_value_info)
from onnx.checker import check_model
```

The imported functions include those listed previously, plus `check_model`, which checks the consistency of a created ONNX model. `TensorProto` is the ONNX class that defines a tensor, and it's one of several `protobuf`-based structure classes. You can list all these classes programmatically as follows:

```
import onnx
import pprint

pprint.pprint([p for p in dir(onnx)
               if p.endswith('Proto') and p[0] != '_'])
```

It returns the following output:

```
['AttributeProto',
 'FunctionProto',
 'GraphProto',
 'MapProto',
 'ModelProto',
 'NodeProto',
 'OperatorProto',
 'OperatorSetIdProto',
 'OperatorSetProto',
 'OptionalProto',
 'SequenceProto',
 'SparseTensorProto',
 'StringStringEntryProto',
 'TensorProto',
 'TensorShapeProto',
 'TrainingInfoProto',
 'TypeProto',
 'ValueInfoProto']
```

We can define a linear regression model by first declaring the matrices involved:

```
X = make_tensor_value_info('X', TensorProto.FLOAT, [None, None])
A = make_tensor_value_info('A', TensorProto.FLOAT, [None, None])
B = make_tensor_value_info('B', TensorProto.FLOAT, [None, None])
Y = make_tensor_value_info('Y', TensorProto.FLOAT, [None])
```

Next, we'll create the graph nodes (operations for this model implementation):

```
node1 = make_node('MatMul', ['X', 'A'], ['XA'])
node2 = make_node('Add', ['XA', 'B'], ['Y'])
```

Then we can create the graph:

```
graph = make_graph([node1, node2], 'lr', [X, A, B], [Y])
onnx_model = make_model(graph)
```

Check the model consistency as follows:

```
check_model(onnx_model)
```

Finally, we can serialize it with the `SerializeToString` method:

```
with open("linear_regression.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())
```

We can inspect the ONNX model's structure in two ways. First, print the model variable to the screen:

```
print(onnx_model)
```

That command prints the model structure in ASCII format to standard output, as follows:

```
ir_version: 9
graph {
  node {
    input: "X"
    input: "A"
    output: "XA"
    op_type: "MatMul"
  }
  node {
    input: "XA"
    input: "B"
    output: "Y"
    op_type: "Add"
  }
  name: "lr"
  input {
    name: "X"
    type {
      tensor_type {
        elem_type: 1
        shape {
          dim {
          }
          dim {
          }
        }
      }
    }
  }
  input {
    name: "A"
```

```
    type {
      tensor_type {
        elem_type: 1
        shape {
          dim {
          }
          dim {
          }
        }
      }
    }
  }
  input {
    name: "B"
    type {
      tensor_type {
        elem_type: 1
        shape {
          dim {
          }
          dim {
          }
        }
      }
    }
  }
  output {
    name: "Y"
    type {
      tensor_type {
        elem_type: 1
        shape {
          dim {
          }
        }
      }
    }
  }
}
opset_import {
  version: 21
}
```

Alternatively, we can visualize an ONNX graph with the open source Netron library (https://github.com/lutzroeder/netron), which supports ONNX. The output will look similar to figure 5.4 (you can also include metadata in the rendering).

The saved ONNX graph can be restored at any time using the `load` function:

```
from onnx import load

with open("linear_regression.onnx", "rb") as f:
    onnx_model = load(f)
```

5.3 The ONNX runtime

The ONNX format defines the ML/DL models converted to this framework. Another key part of the ecosystem is the ONNX Runtime (ORT, https://onnxruntime.ai/), which executes ONNX models efficiently across a wide range of hardware and software platforms (illustrated earlier in figure 5.1). ORT supports the deployment of ONNX models on local or cloud servers, as well as on mobile phones and edge devices, for consistent performance across environments. When available, ORT uses hardware acceleration for faster inference. It also includes optimizations such as kernel fusion, quantization, and layout transformations to speed execution while maintaining good accuracy. Although ORT supports both training and inference, it's most often used for inference, with training left to the original ML/DL framework. Training with ORT is out of scope for this book.

> **NOTE** ORT provides bindings for multiple programming languages; in this book, we'll use Python.

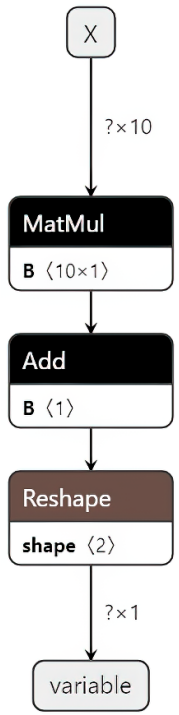


Figure 5.4 Visualizing a linear regression model in ONNX with Netron

The inference process follows a fixed set of steps. Once we've trained a model and converted it to ONNX, or implemented one directly with the ONNX API (such as the linear regression model in section 5.2), we load it and create an inference session (using ORT's `InferenceSession` class):

```
import onnxruntime as rt

onnx_model_path = "path_to_the_onnx_model"
session = rt.InferenceSession(onnx_model_path)
```

Next, we prepare the model input (in this example, it will be a NumPy array):

```
import numpy as np

input_data = np.array([[2.0, 4.0, 6.0, 8.0]], dtype=np.float32)
```

We can now run inference:

```
input_name = session.get_inputs()[0].name
output_name = session.get_outputs()[0].name
result = session.run([output_name], {input_name: input_data})
```

Finally, we can print the result:

```
print(f"Output: {result[0]}")
```

When passing input values to the model, make sure you use the correct input name. You can retrieve it from the inference session object with `get_inputs`, which returns

the list of names for a model's inputs. Similarly, `get_outputs` returns the model's output names. The command sequence for ORT inference is always the same; what varies by architecture is input preprocessing, output postprocessing, and how you handle multiple inputs and outputs.

A JavaScript library for running ONNX models in web browsers or on Node.js is also available. It's called ONNX Runtime Web (https://microsoft.github.io/onnxruntime-web-demo), and it's built on WebAssembly and WebGL. It supports CPU and GPU execution and enables interactive, install-free, device-independent ML, with lower server–client latency and improved privacy and security. After you specialize your model, convert it to ONNX, and optimize it, you can use this as another serving option.

5.4 ONNX runtime providers

To get the most from the underlying hardware on which an ONNX model runs, ORT uses runtime (execution) providers. These interfaces let you deploy ONNX models across environments and optimize execution by using each platform's compute capabilities. Through a dedicated execution provider, ORT assigns specific nodes or subgraphs to run on the available hardware, drivers, and related libraries in the deployment environment. Figure 5.5, from the official ORT documentation, shows how ORT improves a model's execution for a given hardware specification.

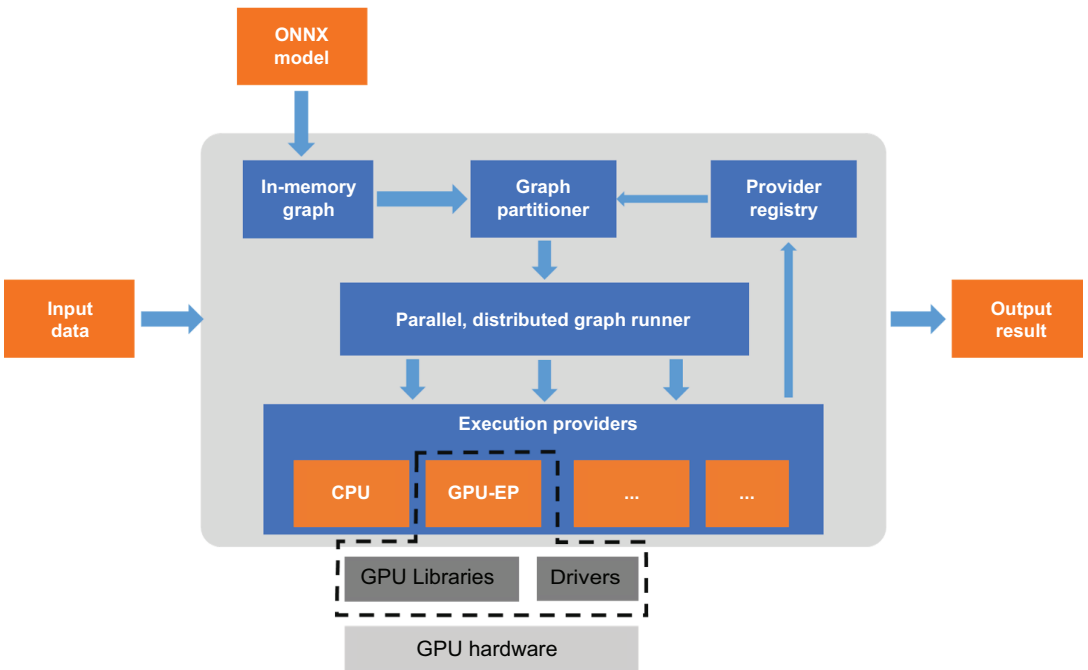


Figure 5.5 The ONNX Runtime execution model

At the time of writing, ORT offers many execution providers. Some are production-ready; others are preview releases for evaluation. Providers marked as in review are generally stable and may be suitable for production, but test carefully before deploying an ONNX model with any provider still labeled as in preview.

Here are ORT's currently stable execution providers:

- *Default CPU*—Vendor-agnostic CPU.
- *Intel OpenVINO*—Uses the OpenVINO toolkit (https://mng.bz/Jw5p) on CPUs and GPUs, including edge and mobile devices.
- *Intel oneDNN*—Uses optimized primitives from the Intel Math Kernel Library for Deep Neural Networks (Intel DNNL). CPU-only.
- *XNNPACK*—Uses XNNPACK (https://github.com/google/XNNPACK), an optimized floating-point NN inference library for ARM (Android/iOS), WebAssembly, and x86.
- *NVIDIA CUDA*—Enables hardware-accelerated computation on CUDA-enabled NVIDIA GPUs.
- *NVIDIA TensorRT*—Uses NVIDIA's TensorRT engine (https://developer.nvidia.com/tensorrt) on supported NVIDIA GPUs.
- *DirectML*—Uses DirectML's low-level API for ML (https://mng.bz/wZw7); Windows GPUs only.
- *AMD MIGraphX*—AMD's MiGraphX optimization engine for AMD GPUs (https://github.com/ROCm/AMDMIGraphX).
- *AMD ROCm*—For AMD GPUs with ROCm (https://mng.bz/qRwK).
- *Android Neural Networks API*—A unified interface to CPUs, GPUs, and neural network accelerators on Android devices (https://mng.bz/7QOx).
- *Qualcomm QNN*—Uses the Qualcomm AI Engine Direct SDK (https://mng.bz/mZwn) on Qualcomm chipsets.

This is the current list of ORT execution providers still in preview:

- *TVM*—Uses Apache TVM (https://tvm.apache.org/) to optimize models for CPUs, GPUs, and edge and mobile devices. Currently tested on Linux and Windows, but not on macOS.
- *ARM Compute Library*—Runs inference on Armv8 cores using the Arm Compute Library engine (https://github.com/ARM-software/ComputeLibrary).
- *ARM-NN*—ArmNN inference engine (https://github.com/ARM-software/armnn) for Armv8 cores.
- *CoreML*—Uses the Core ML framework (https://developer.apple.com/machine-learning/core-ml/) to run on Apple's CPUs, GPUs, and Neural Engine.
- *Rockchip-NPU*—Uses the RKNPU DDK.
- *Xilinx Vitis-AI*—Uses AMD's development stack for hardware-accelerated AI inference on AMD platforms.

- *Huawei CANN*—Uses the CANN Execution Provider with Huawei Ascend hardware.
- *AZURE*—Invokes a remote Azure endpoint for inference.

You can programmatically check the available providers in the destination environment:

```
import onnxruntime as rt

rt.get_available_providers()
```

This returns a Python list. For example, on a system with a CPU and an NVIDIA GPU (with CUDA drivers installed), the preceding method returns the following:

```
['CPUExecutionProvider', 'CUDAExecutionProvider']
```

If multiple providers are available, you can tell ORT the order in which to try them. Here's an example:

```
providers_list = ['CUDAExecutionProvider', 'CPUExecutionProvider']

sess = rt.InferenceSession("model.onnx", providers=providers_list)
```

Here, we tell ORT to prefer the CUDA provider (when available) over others in the execution environment.

5.5 ONNX for LLMs on CPU

Now that we've covered the main ONNX format and runtime concepts, it's time to see how we can use them with LLMs. This section focuses on CPU-only setups; the next section covers the same ideas for when GPUs are available. The code in this section is included in a companion Colab notebook.

Our first LLM example uses a BERT-based model called bert-base-uncased (https://huggingface.co/google-bert/bert-base-uncased). It uses Hugging Face's Transformers and Datasets libraries. The execution environment must have the `onnx` and CPU-only `onnxruntime` Python packages installed:

```
pip install onnx onnxruntime
```

Next, we need to download the model from the Hugging Face Hub:

```
from transformers import AutoModelForQuestionAnswering, BertTokenizer

model_id = 'google-bert/bert-base-uncased'
tokenizer = BertTokenizer.from_pretrained(model_id)
model = AutoModelForQuestionAnswering.from_pretrained(model_id)
model.eval()
```

To test the model, we'll use a subset of the `SQuAD` dataset (Stanford Question Answering Dataset, https://huggingface.co/datasets/squad), available from the Hugging Face Hub via the Datasets library:

```
from datasets import load_dataset

samples_count = 200
squad = load_dataset("squad", split="validation[:"+ str(samples_count) +"]")
```

Only 200 samples will be used. The following example shows their structure:

```
{'id': '56be4db0acb8001400a502ec',
 'title': 'Super_Bowl_50',
 'context': 'Super Bowl 50 was an American football game to determine the
 champion of the National Football League (NFL) for the 2015 season. The
American Football Conference (AFC) champion Denver Broncos defeated the
National Football Conference (NFC) champion Carolina Panthers 24-10 to earn
their third Super Bowl title. The game was played on February 7, 2016, at
Levi\'s Stadium in the San Francisco Bay Area at Santa Clara, California.
As this was the 50th Super Bowl, the league emphasized the "golden
anniversary" with various gold-themed initiatives, as well as temporarily
suspending the tradition of naming each Super Bowl game with Roman numerals
(under which the game would have been known as "Super Bowl L"), so that the
logo could prominently feature the Arabic numerals 50.',
 'question': 'Which NFL team represented the AFC at Super Bowl 50?',
 'answers': {'text': ['Denver Broncos', 'Denver Broncos', 'Denver Broncos'],
  'answer_start': [177, 177, 177]}}
```

In this example, we'll use the `context` and `question` fields to benchmark different model versions on answer generation.

Let's create a directory to store the ONNX-converted model:

```
import os

output_dir = os.path.join(".", "onnx_models")
onnx_model_filename = 'bert-base-uncased.onnx'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
export_model_path = os.path.join(output_dir, onnx_model_filename)
```

We'll use a test sample to run the model once at the start of the conversion process (here we'll use the first item in the list, though any of the 200 retrieved samples will work):

```
tokenized_inputs = tokenizer(squad["question"][0],
                  ➥squad["context"][0], return_tensors="pt")
inputs = {
        'input_ids':  tokenized_inputs['input_ids'],
        'input_mask': tokenized_inputs['attention_mask'],
        'segment_ids': tokenized_inputs['token_type_ids']
    }
```

Then we'll start the conversion:

```
import torch

with torch.no_grad():
    symbolic_names = {0: 'batch_size', 1: 'max_seq_len'}
    torch.onnx.export(model,
                      args=tuple(inputs.values()),
                      f=export_model_path,
                      opset_version=15,
                      do_constant_folding=True,
                      input_names=['input_ids',
                                   'input_mask',
                                   'segment_ids'],
                      output_names=['start', 'end'],
                      dynamic_axes={'input_ids': symbolic_names,
                                    'input_mask' : symbolic_names,
                                    'segment_ids' : symbolic_names,
                                    'start' : symbolic_names,
                                    'end' : symbolic_names})
```

We'll use PyTorch's ONNX `export` function, passing the model to convert, the destination directory, the ONNX version (1.15.0 at the time of writing), and the model's input and output names. We'll also enable constant folding for optimization and set variable-length axes so the model can run with different batch sizes and variable numbers of input and output tokens. Figure 5.6 shows part of the ONNX graph for the converted model (cropped for readability).

You can validate the exported model with the `check_model` function introduced in section 5.1:

```
from onnx.checker import check_model

check_model(export_model_path, full_check=True)
```

If you get no error message, the model's schema is valid. To validate the full architecture, run inference on a few samples (or the full test set) with both the original model and the ONNX-converted one, and compare their outputs. NumPy provides the `allclose` function for this:

```
import numpy as np

for i in range(10):
    np.allclose(ort_outputs[i], outputs[i].cpu(), rtol=1e-05, atol=1e-04)
```

In the preceding code snippet, we assume that we've run inference with both the original and converted models on the test set and saved the output in two arrays: `outputs` and `ort_outputs`. The `allclose` function returns `True` when the arrays are element-wise equal within a tolerance defined by `rtol` (relative tolerance) and `atol` (absolute

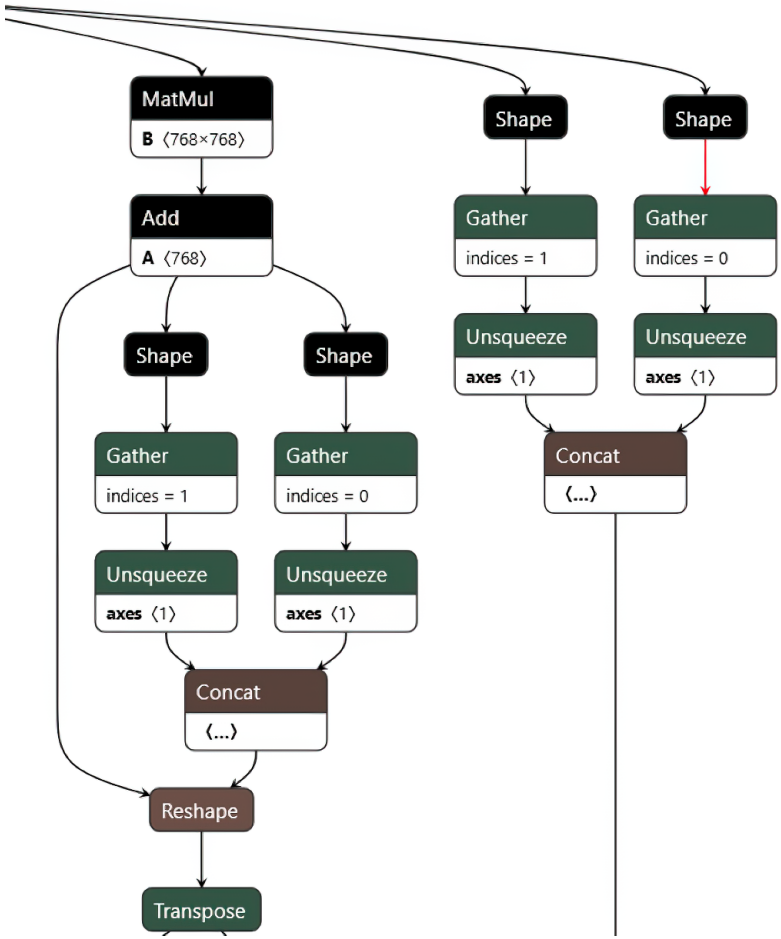


Figure 5.6 Part of the ONNX graph for the converted bert-base-uncased model (one attention block)

tolerance), which are typically very small values. `allclose` evaluates the following formula:

```
absolute(ort_outputs[i] - outputs[i]) <= (atol + rtol *
➥absolute(outputs[i]))
```

After exporting and verifying the model, we can run inference. First, we'll create an ONNX Runtime session:

```
import onnxruntime
import numpy

sess_options = onnxruntime.SessionOptions()
```

```
session = onnxruntime.InferenceSession(export_model_path,
➥sess_options, providers=['CPUExecutionProvider'])
```

Then we can run inference in the session as follows:

```
inputs = tokenizer(squad["question"][0],
➥squad["context"][0], return_tensors="np")
ort_inputs = {
    'input_ids':  full_inputs['input_ids'],
    'input_mask': full_inputs['attention_mask'],
    'segment_ids': full_inputs['token_type_ids']
}
ort_outputs = session.run(None, ort_inputs
```

The inputs are tokenized into NumPy arrays (not PyTorch tensors, as in the original model). We then explicitly set the expected inputs for the architecture (for example, `input_ids`, `input_mask`, and `segment_ids` for this BERT model).

If you run the notebook's benchmark on the full test set for both models (original and ONNX-converted), you'll see results like these:

```
PyTorch CPU Average inference time = 455.49 ms
OnnxRuntime cpu Average inference time = 373.10 ms
```

ONNX's average inference time is about 18% lower.

Finally, the notebook code uses the ONNX API to further optimize the converted model, using the ONNX `optimizer` class's `optimize_model` function:

```
from onnxruntime.transformers import optimizer

optimized_model_path = os.path.join(output_dir, '
➥bert-base-uncased.onnx_opt_cpu.onnx')
optimized_model = optimizer.optimize_model(export_model_path,
➥model_type='bert', num_heads=12, hidden_size=768)
optimized_model.save_model_to_file(optimized_model_path)
```

Running the benchmark on this optimized model produces the following:

```
OnnxRuntime cpu Average inference time = 355.48 ms
```

5.6 ONNX for LLMs on GPU

Let's now look at running ONNX models on a GPU. As a first example, let's consider bert-base-uncased, the same model as in section 5.5, but in an environment with an NVIDIA GPU and CUDA drivers installed. The full code for this example is available in a companion Colab notebook. Again, we use the Hugging Face Transformers and Datasets libraries. In addition to the onnx library, you'll need onnxruntime for GPU:

```
pip install onnx onnxruntime-gpu
```

The code to download the model from the HF Hub is nearly identical to the section 4.5 example:

```
import torch
from transformers import AutoModelForQuestionAnswering, BertTokenizer

device = torch.device("cuda")
model_id = 'bert-base-uncased'
tokenizer = BertTokenizer.from_pretrained(model_id)
model = AutoModelForQuestionAnswering.from_pretrained(model_id)
model.eval()
model.to(device)
```

The only difference here is that we load the model into GPU memory because we're also using hardware acceleration to convert the model to ONNX.

As in section 5.5, we use `SQuAD` as the reference dataset for testing and validation. See the previous section for the code to download a subset of samples.

The conversion code is the same as for CPUs, so we won't duplicate it here; see section 5.5 or this section's companion notebook. Inference with the ONNX-converted model is also similar, but you must configure the ONNX inference session to use the CUDA execution provider as follows:

```
session = onnxruntime.InferenceSession(export_model_path, sess_options,
➥providers=['CUDAExecutionProvider'])
```

That's all you need to run an ONNX-converted model on a GPU runtime in a standard way. There are other considerations, but before we explore them, let's look at another example.

5.6.1 ONNX for GPT on GPU

The previous example and the one in section 5.5 use encoder-only models. Now we'll look at a decoder-only model: GPT-2 small (https://huggingface.co/openai-community/gpt2). The code in this section is included in the companion Colab notebook.

Let's download the model from the Hugging Face Hub:

```
import torch
from transformers import GPT2Tokenizer, AutoModelForCausalLM

model_id = 'openai-community/gpt2'
tokenizer = GPT2Tokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id)
device = torch.device("cuda")
model.eval().to(device)
```

Although this isn't required, we moved the model checkpoints to GPU memory and will use the GPU to convert the model to ONNX. (You can also run the conversion on CPU.) Now let's create a directory to store the converted model:

```
import os

output_dir = os.path.join(".", "onnx_models")
onnx_model_filename = 'bert-base-uncased.onnx'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
export_model_path = os.path.join(output_dir, onnx_model_filename)
```

Next, we'll prepare a sample input. For this use case, we'll set the tokenizer's `return_attention_mask` argument to `False`, since the attention mask isn't used:

```
tokenized_inputs = tokenizer("The story so far: in the beginning,
➥the universe was created.",
                             return_attention_mask=False,
                             return_tensors="pt")
tokenized_inputs.to(device)
inputs_sample = {
        'input_ids':  tokenized_inputs['input_ids']
    }
```

Finally, we'll start the conversion process:

```
with torch.no_grad():
  torch.onnx.export(model,
                    inputs_sample,
                    export_model_path,
                    export_params=True,
                    opset_version=15,
                    input_names=['input_ids']
                    )
```

We can run inference with the exported model as in the previous BERT example—create an inference session, choose the CUDA runtime provider, and call the `run` method:

```
import onnxruntime
import numpy

session = onnxruntime.InferenceSession(export_model_path,
➥providers=["CUDAExecutionProvider"])
onnx_input_ids = tokenizer("The story so far: in the
➥beginning, the universe was created.",
                           return_attention_mask=False,
                           return_tensors="np")
ort_inputs = {
    "input_ids": onnx_input_ids['input_ids']
}

ort_outputs = session.run(None, ort_inputs)
```

In the examples so far, we've assumed that converting a model to ONNX automatically applies the best possible optimizations. This is often true, but sometimes extra steps are

needed. You can usually do this programmatically; manually inspecting the generated ONNX graph should be a last resort. One way to do this (not the only one—we'll cover more later) is to use the ONNX Runtime library's built-in Transformer `optimizer`. Continuing the GPT-2 example, here's how we can apply it to an already converted model:

```
from onnxruntime.transformers import optimizer

optimized_model_path = os.path.join(output_dir, 'gpt-2-onnx_opt_gpu.onnx')
optimized_model = optimizer.optimize_model(export_model_path,
                                           model_type='gpt2',
                                           use_gpu=True,
                                           num_heads=12,
                                           hidden_size=768)
optimized_model.save_model_to_file(optimized_model_path)
```

We use the `optimizer`'s `optimize_model` method, as in the earlier bert-base-uncased example. We pass the full path to the converted model, the LLM family type, and the number of heads and hidden size, and we specify GPU optimization. You can also set the graph optimization level: `0` (none; the default), `1` (basic), `2` (extended), or `99` (all, including layout). The same code applies to LLMs with architectures other than GPT-2. Basic optimization performs constant folding (statically computing parts of the graph that rely only on constant initializers), removes redundant nodes (such as identity, slice, unsqueeze, and dropout), and applies semantics-preserving fusions. Extended optimization applies more complex fusions after graph partitioning, and only to nodes assigned to the CPU, CUDA, or ROCm (AMD Radeon Open Compute Ecosystem) execution provider. Layout optimization changes the data layout for applicable nodes for higher performance; it runs after graph partitioning and only for nodes assigned to the CPU execution provider.

ONNX optimizations can be done offline or online. The previous code snippet is an example of offline optimization. Alternatively, you can enable the same optimizations at inference time by setting the ONNX Runtime session mode, as follows:

```
import onnxruntime as rt

sess_options = rt.SessionOptions()

sess_options.graph_optimization_level =
➥rt.GraphOptimizationLevel.ORT_ENABLE_EXTENDED
sess_options.optimized_model_filepath = "<model_path\optimized_model.onnx>"

session = rt.InferenceSession(None, sess_options)
```

The optimization levels are defined by the `GraphOptimizationLevel` class:

```
GraphOptimizationLevel.ORT_DISABLE_
GraphOptimizationLevel.ORT_ENABLE_BASIC
GraphOptimizationLevel.ORT_ENABLE_EXTENDED
GraphOptimizationLevel.ORT_ENABLE_ALL
```

In online mode, ONNX Runtime applies all enabled graph optimizations when the session is initialized, before inference runs. This can increase model startup time (especially for complex models), which can make it unsuitable for production. In offline mode, ONNX Runtime applies the optimizations and then serializes the optimized model to disk. Subsequent startups are faster because the model is already optimized.

5.6.2 I/O binding

Before we move on, consider input and output data when running ONNX models with execution providers (CPU and GPU). So far, you've learned that ONNX models consist of a computation graph implemented as optimized operator kernels for different hardware, and that ONNX Runtime (ORT) orchestrates execution through execution providers for a specific target (CPU, GPU, and so on). But what about the data consumed and produced?

By default, ORT places data—regardless of structure (NumPy arrays, dictionaries, or lists of NumPy arrays)—on the CPU. That's fine when there's no hardware acceleration, but it's not ideal when a GPU is available and the inference session is configured to run on the GPU first. In that case, data is copied between CPU and GPU, adding unnecessary overhead. ORT provides a custom data structure that supports ONNX data formats and lets you place data on a device, such as a CUDA-capable GPU. It's called IOBinding. With it, input and output data (or just one of them, depending on your configuration) can be placed on the same device where the ONNX graph runs.

You can configure IOBinding programmatically, as you'll see next. To keep the concept simple, we'll use a small PyTorch model that sums two tensors, but the same approach applies to more complex models, including LLMs. Let's define the model:

```
import torch

def model():
    class Model(torch.nn.Module):
        def __init__(self):
            super(Model, self).__init__()

        def forward(self, x, y):
            return x.add(y)

    return Model()
```

Now let's convert it to ONNX programmatically:

```
MODEL_FILE = '.model.onnx'

type = torch.float32
sample_x = torch.ones(3, dtype=type)
sample_y = torch.zeros(3, dtype=type)
```

```
torch.onnx.export(model(), (sample_x, sample_y),
➥MODEL_FILE, input_names=["x", "y"], output_names=["z"],
                    ➥dynamic_axes={"x": {0 : "array_length_x"}, "y":
                    ➥{0: "array_length_y"}})
```

We'll create an inference session as usual (assuming the target environment has a GPU and CUDA):

```
import onnxruntime as rt

providers = ['CUDAExecutionProvider', 'CPUExecutionProvider']
session = rt.InferenceSession(MODEL_FILE, providers=providers)
```

The new part starts after we've created the session. Instead of running inference with `run`, we'll use `run_with_iobinding`. First, we'll prepare the data for IOBinding. Create an IOBinding instance from the inference session:

```
binding = session.io_binding()
```

Create two tensors containing random values, and move them to the GPU:

```
DEVICE = 'cuda'
x = torch.rand(5).to(DEVICE)
y = torch.rand(5).to(DEVICE)
x_tensor = x.contiguous()
y_tensor = y.contiguous()
```

Then tell ONNX Runtime to load the model inputs onto the CUDA device:

```
DEVICE_INDEX = 0
np_type = np.float32

binding.bind_input(
    name='x',
    device_type=DEVICE,
    device_id=DEVICE_INDEX,
    element_type=np_type,
    shape=tuple(x_tensor.shape),
    buffer_ptr=x_tensor.data_ptr(),
    )

binding.bind_input(
    name='y',
    device_type=DEVICE,
    device_id=DEVICE_INDEX,
    element_type=np_type,
    shape=tuple(y_tensor.shape),
    buffer_ptr=y_tensor.data_ptr(),
    )
```

You can run inference with IOBInding as follows:

```
session.run_with_iobinding(binding)
```

In this example, only the input data is loaded into the device; the output is still moved to the CPU. To use IOBinding for the output as well, allocate a PyTorch tensor for the model output before running inference:

```
z_tensor = torch.empty(x_tensor.shape, dtype=torch_type,
➥device=DEVICE).contiguous()
```

Then bind it, just as you did for the input tensors:

```
binding.bind_output(
    name='z',
    device_type=DEVICE,
    device_id=DEVICE_INDEX,
    element_type=np_type,
    shape=tuple(z_tensor.shape),
    buffer_ptr=z_tensor.data_ptr(),
)
```

Other examples with more complex data will be presented in chapters 7 and 8.

Summary

- ONNX is an open standard format for interoperable machine learning models across frameworks and hardware platforms.
- ONNX Runtime (ORT) efficiently runs ONNX models on CPUs, GPUs, and edge devices.
- ONNX operators are the building blocks that can describe machine learning models from most frameworks.
- ONNX supports multiple data types, including `float32`, `int8`, `int16`, `int32`, `int64`, and specialized formats like `bfloat16`.
- ONNX Runtime execution providers optimize performance by using hardware accelerators such as NVIDIA CUDA.
- You can optimize ONNX models offline during conversion or online when initializing an inference session.
- ONNX graph optimizations range from constant folding to node fusion and layout transformations.
- IOBinding in ONNX Runtime eliminates CPU–GPU copies by keeping data on the target execution device.
- ONNX-converted models typically achieve faster inference times than their original framework implementations.

- Validation of ONNX models means verifying schema consistency and ensuring that outputs match those of the original model.
- ONNX Runtime supports multiple language bindings, but Python offers the most comprehensive API.
- You can optimize ONNX transformer models with the `onnxruntime.transformers.optimizer models`.
- Converting language models to ONNX preserves functionality and enables deployment across diverse production environments.

6 Quantizing for your production environment

This chapter covers

- Precision formats for training and running LLMs
- Quantizing LLMs to smaller-precision formats
- Quantizing LLMs with multiple techniques and libraries

In chapter 5, you learned the core ONNX concepts and capabilities. I only briefly mentioned model quantization, but it's crucial for LLM inference performance, so it deserves its own chapter. That's our focus here.

6.1 Transformers precision formats

Previous chapters have mentioned several numeric precision formats used for LLM training and inference. It's time to look at them more closely and to introduce additional formats used for model quantization.

In traditional scientific computing, 64-bit floating point (double precision) was the default because it represents a wide range of values accurately. But deep neural networks on GPUs typically use 32-bit floating point (single precision) because 64-bit

operations are unnecessary, slower, and often not well supported by GPU hardware. As a result, FP32 became the standard for deep-learning training.

In floating-point numbers, *bits* are the binary digits used to store a value in a computer's memory. More bits mean higher precision and a wider representable range. A floating-point value has three parts: the sign, the exponent, and the mantissa. In 32-bit floating point, 1 bit is for the sign, 8 bits are for the exponent, and 23 bits are for the mantissa (see figure 6.1).

Figure 6.1 Representation of 32-bit floating point numbers

A 32-bit floating-point number's value is given by the following formula:

$$\text{value} = (-1)^{sign} X\ 2^{(E-127)} X \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-1}\right)$$

In this formula

$$b_{22} \ldots b_0$$

are the mantissa bits, and

$$E = (b_{30} b_{29} \ldots b_{23})_2$$

There are two main reasons why 32-bit floating-point operations are preferred to 64-bit operations for training deep neural networks on GPUs:

- Reduced memory footprint—They use half the memory of 64-bit floats, letting you train larger models and use larger batch sizes.
- Increased compute and speed—32-bit floating point operations use less memory, so GPUs can run them more quickly and reduce training time.

Can we go even further? Mixed-precision training has become a common approach, in which 16-bit ("half") precision is temporarily used for floating-point computations (see figure 6.2).

FP16 uses 1 bit for the sign, 5 bits for the exponent, and 10 bits for the mantissa. An FP16 value uses the same formula as the 32-bit format, but with fewer exponent and mantissa bits.

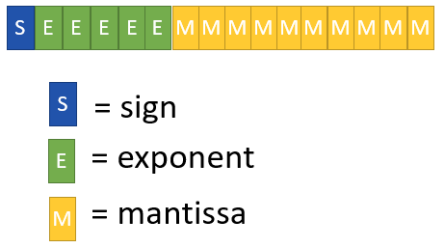


Figure 6.2 Representation of 16-bit floating point numbers

DL models are generally robust to lower-precision arithmetic. In most cases, the small loss of precision from using 32-bit floats instead of 64-bit floats doesn't noticeably affect predictive performance, so the tradeoff is worth it. But moving to 16-bit precision can be trickier. That's why mixed-precision training is used instead of pure 16-bit training. With mixed precision, only some parameters and operations use 16-bit floats, switching between 16-bit and 32-bit during training. The typical workflow is to convert weights to lower precision for faster computation, compute gradients in lower precision, convert gradients back to higher precision for numerical stability, and then update the original weights using scaled gradients.

So far, we've focused on training. What about inference? If you train the same LLM in FP32 and FP16, memory use is roughly half for the latter (as expected), but performance can degrade significantly. With mixed precision (FP16 + FP32), memory use is higher than with pure FP16 but still lower than pure FP32, while performance is comparable, and often slightly better. This may sound odd, but lower precision can introduce noise during training that improves generalization and reduces overfitting, potentially increasing accuracy.

Another floating-point format has been introduced recently and is increasingly used: brain floating point (BF16), developed by Google for ML/DL on tensor processing units (TPUs). Compared to F16 (see figure 6.3), it extends dynamic range at the cost of lower precision.

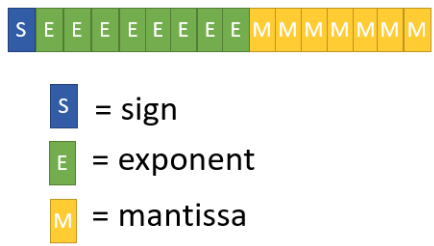


Figure 6.3 Representation of BF16 numbers

BF16 uses 1 bit for the sign, 8 for the exponent, and 7 for the mantissa. This lower precision may affect the accuracy of some calculations, but in most deep learning applications, it has minimal effect on model performance. Models trained in BF16 typically deliver comparable inference performance and memory use to those trained with FP16 mixed precision. BF16 is supported not only by TPUs but also by NVIDIA Ampere GPUs. To check whether your NVIDIA GPU supports this format in PyTorch, run the following commands:

```
import torch

torch.cuda.is_bf16_supported()
```

This method returns `True` if BF16 is supported, and `False` otherwise.

To further improve inference performance, we can go beyond reduced floating-point precision and use *quantization*—converting an LLM's weights from floats to low-bit integers (typically 8-bit or 4-bit). The rest of this chapter explains LLM quantization and provides complete code examples using Hugging Face Transformers and other open source libraries.

6.2 8-bit quantization

In scenarios with tight computational power constraints, or when deploying to small devices, LLMs can be too large and too slow for inference. Using reduced precision (FP16) alone may not make a model fit the target hardware. Two common solutions are knowledge distillation and quantization, both of which reduce model size with limited performance loss. Knowledge distillation trains a smaller model to mimic a larger one—this can happen by transferring knowledge from the larger to the smaller model. For hardware-specific deployment, quantization is often the better choice, and recent advances in stable quantization have made it preferable to distillation in many cases. This book focuses on quantization to simplify LLM deployment and improve performance across a wide range of hardware.

Quantization reduces numeric precision: lowering the bit width of an LLM's weights and activations shrinks the model and can speed up inference. By using lower-precision types, such as 8-bit integers, you reduce the memory footprint at the cost of quantization error. For example, 8-bit values can pack about four times as many numbers as 32-bit floats in the same amount of memory. The goal is to balance precision against memory use (and speed).

There are two types of quantization: post-training quantization (PTQ), applied after training, and quantization-aware training (QAT), done during training. This book focuses mostly on PTQ, because starting from a pretrained LLM baseline typically doesn't require full training—only fine-tuning, PEFT, or RAG to adapt it to a specific domain.

How does 8-bit quantization work in practice? Let's walk through a numerical example. There are several quantization schemes; the most common are *zero-point* and *absmax*. Suppose we want to convert an FP32 tensor to INT8. With zero-point quantization, we assume an asymmetric input distribution, so we scale by the INT8 value range (255) divided by the difference between the input's maximum and minimum values (X in the following formula):

$$\text{scale} = \frac{255}{\max(X) - \min(X)}$$

The scaled distribution is then mapped to the [-127, 128] range:

$$\text{zeropoint} = -\text{round}(\text{scale} \cdot \min(X)) - 128$$

Finally, we use the calculated `scale` and `zeropoint` values to quantize or dequantize a model's weights:

$$X_{\text{quantized}} = \text{round}(\text{scale} \cdot X + \text{zeropoint})$$

$$X_{\text{dequantized}} = \frac{X_{\text{quantized}} - \text{zeropoint}}{\text{scale}}$$

Let's look at an example (8-bit quantization): suppose the minimum value is -4.0 and the maximum value is 4.1. The scale is 255/(4.1 + 4.0) = 31.48, and the zero-point is -round(31.48 × -4.0) – 128 = -3. If the weight is 0.2, then, using the preceding quantization formula, its quantized value is 3. Dequantizing gives 0.19—not exactly the original weight (quantization isn't lossless), but close.

With absmax quantization, each input value is divided by its tensor's absolute maximum and multiplied by a scale factor (127) to map values to the [-127, 127] range:

$$X_{\text{quantized}} = \text{round}\left(\frac{127}{\max|X|} \cdot X\right)$$

To dequantize, we use this equation:

$$X_{\text{dequantized}} = \frac{\max|X|}{127} \cdot X_{\text{quantized}}$$

For example, using the same maximum value as in the previous zero-point example (4.1), absmax quantization maps a weight of 0.2 to round(127 × 0.2 / 4.1) = 6. With zero-point quantization, you would get a different value. Dequantizing introduces an error: 6 × 4.1 / 127 = 0.19.

To recap, LLM quantization offers major benefits:

- Reduced model size
- Speed and hardware compatibility (low-precision arithmetic can run faster on some hardware accelerators)
- Memory efficiency

It also has a few cons:

- Accuracy tradeoffs (lower precision can affect model performance)
- Implementation challenges

As for the downsides, don't worry—this book will help you overcome them.

6.2.1 Hands-on 8-bit quantization

Let's walk through a practical example of 8-bit post-training quantization (PTQ) for an LLM using PyTorch and the Hugging Face Transformers library (we'll set ONNX aside

for now). We'll use GPT-2 small, but the same considerations apply to other GPT-2 variants and GPT-based LLMs. We'll load the pretrained model in 32-bit floating point and quantize it to 8-bit integers. A companion Colab notebook includes the full source code. No hardware acceleration is required at this stage.

We'll load the FP32 model from the Hugging Face Hub as usual:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

device = 'cpu'
model_id = 'openai-community/gpt2'
model = AutoModelForCausalLM.from_pretrained(model_id).to(device)
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

In FP32, the model is about 548 MB.

We can convert all model layers to INT8 format and then quantize the full model weights. Let's define a custom function to do this using the absmax technique:

```
import torch

def absmax_quantize(X):
    scale = 127 / torch.max(torch.abs(X))

    X_quant = (scale * X).round()

    X_dequant = X_quant / scale

    return X_quant.to(torch.int8), X_dequant
```

As described earlier in section 6.2, it runs three steps: calculate the scale, quantize to INT8, and dequantize back to FP32. For a given input, it returns both the quantized and dequantized values. Let's use it to quantize all the GPT-2 model weights:

```
import numpy as np
from copy import deepcopy

weights = [param.data.clone() for param in model.parameters()]

model_abs = deepcopy(model)

weights_abs = []
for param in model_abs.parameters():
    _, dequantized = absmax_quantize(param.data)
    param.data = dequantized
    weights_abs.append(dequantized)
```

We have created a copy of the source model and applied our `absmax_quantize` function to all its weights to quantize them.

Next, we'll verify the resulting performance of this process. One useful check is to plot the distribution of the model's weights before and after quantization to see how

closely the quantized values match the originals. We can use the matplotlib library (https://matplotlib.org/) to build a histogram (see the notebook for the full code). In figure 6.4, you can see a spike around 0, which suggests that quantization is quite lossy in this case, since reversing the process doesn't reproduce the exact original values.

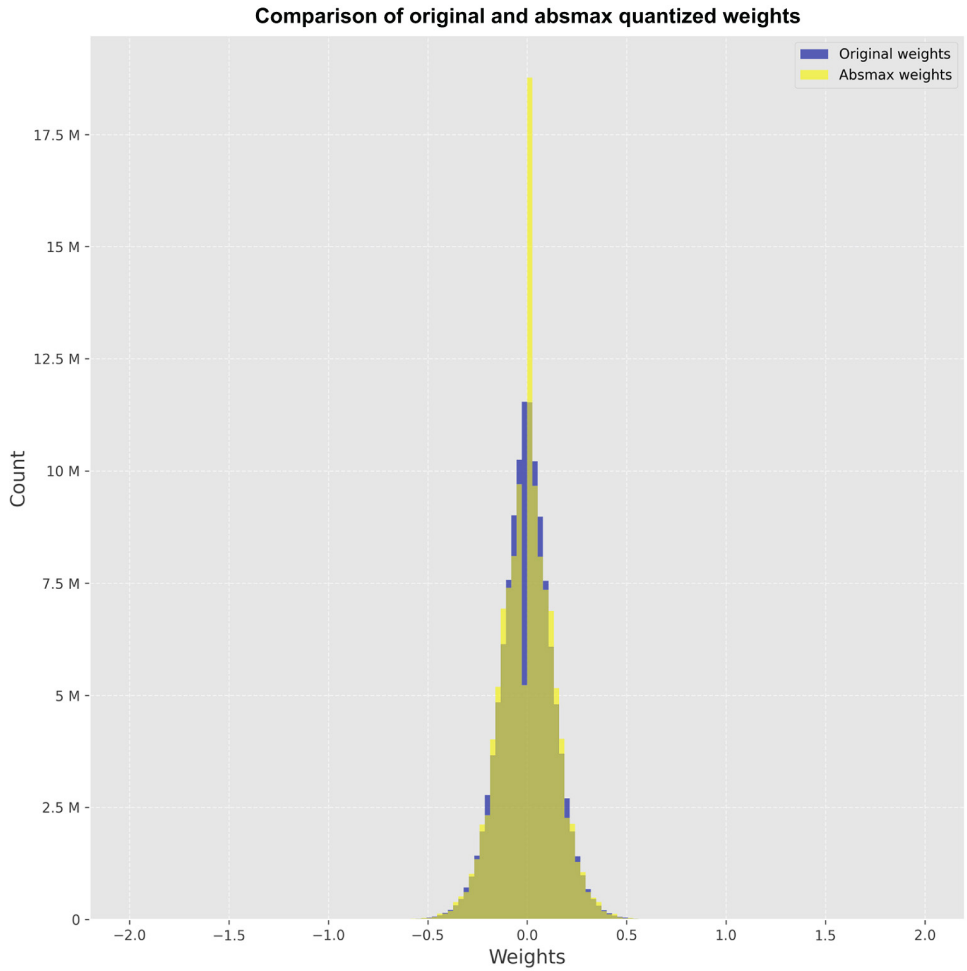


Figure 6.4 Comparing the original GPT-2 model weights and the absmax quantized ones

Let's see how performance is affected by this process. First, we'll define a function that takes a prompt and generates text using top-k sampling. We'll put this code in a function because we'll reuse it in this example.

```
def generate_text(model, input_text, max_length=100):
    input_ids = tokenizer.encode(input_text, return_tensors='pt').to(device)
```

```
    output = model.generate(inputs=input_ids,
                            max_length=max_length,
                            do_sample=True,
                            top_k=30,
                            pad_token_id=tokenizer.eos_token_id,
                            attention_mask=input_ids.new_ones(
                                input_ids.shape
                            )
                )

    return tokenizer.decode(output[0], skip_special_tokens=True)
```

We'll use the function to generate text (limited to 100 tokens) with both the original model and the absmax-quantized one:

```
prompt = 'My favourite school subject is'
original_text = generate_text(model, prompt)
absmax_text = generate_text(model_abs, prompt)
```

The original model generated this:

```
My favourite school subject is to create beautiful images of people.
Some are really good at it for this, but many are boring."

Chen said he has no particular reason to be ashamed at all.

"I love the fact that it's not really what you have thought up.
It's been my opinion all this time in life that you really need
to work on your work, to get your work done."

"I am very comfortable being a writer, and the fact
```

The quantized model generated the following:

```
My favourite school subject is chess. But I don't know if I'm
supposed to be fascinated by it myself. Maybe I can learn how
to play it, but I can't learn how to play chess myself.

Because I'm interested in chess, I've got to learn how to play
it first. I'm interested in learning about the psychology of chess.
I'm interested in studying this subject because, let's face it,
my wife and I have this weird fascination with what chess does
```

Rather than trying to decide empirically which of these two outputs makes more sense, you can compute a quantitative score by calculating the *perplexity* of each one. Perplexity is a common metric for evaluating LLMs: it measures a model's uncertainty when predicting the next token in a sequence. It's often assumed that lower is better, but a sentence with high perplexity can still be correct.

We can implement a basic function that calculates perplexity:

```
def calculate_perplexity(model, text, device):
    encodings = tokenizer(text, return_tensors='pt').to(device)

    input_ids = encodings.input_ids
    target_ids = input_ids.clone()

    with torch.no_grad():
        outputs = model(input_ids, labels=target_ids)

    neg_log_likelihood = outputs.loss

    perplexity = torch.exp(neg_log_likelihood)

    return perplexity
```

This function expects three arguments: a model, a text prompt, and the device to offload the model weights to. It encodes the input sequence, prepares the model inputs, generates text, computes the loss, and uses it to calculate the perplexity score (which is returned to the caller). We can apply this function to both models—the original and the absmax-quantized one—using as the prompt the text each model generated a few steps earlier:

```
perplexity = calculate_perplexity(model, original_text, device)
perplexity_absmax = calculate_perplexity(model_abs, absmax_text, device)
```

These are the measured perplexity scores:

```
Original perplexity:  18.66
Absmax perplexity:    8.20
```

In this example, the quantized model has lower perplexity than the original. But a single run isn't very reliable; you should repeat the experiment several times to understand the differences between the models. This is easier if you first create two reusable functions.

6.2.2 LLM.int8() and quantization

In the previous example, for simplicity, we applied per-tensor quantization to all the model layers. But you can quantize at different granularities. Quantizing the whole model in one pass can seriously degrade performance, but quantizing individual values creates too much overhead. In practice, a good compromise is vector-wise quantization, which accounts for variability across rows and columns within a tensor. This is often sufficient for domain-specific models, since they're typically under 7 billion parameters. Above that threshold, you may see outlier features in any LLM layer. Even a single outlier can significantly reduce precision for the other values, and you can't just discard outliers without a nontrivial hit to model performance.

What can we do instead? Fortunately, there's no need for custom implementations—LLM.int8() already addresses this. First introduced in a 2022 paper (https://arxiv.org/abs/2208.07339), it enables INT8 matrix multiplication for LLMs with no accuracy loss. It cuts inference memory roughly in half while retaining full-precision performance. It uses an absmax quantization scheme with mixed precision: outlier features are processed in FP16 to preserve accuracy, while the remaining values use INT8. This approach reduces an LLM's memory footprint by nearly two times.

LLM.int8() performs matrix multiplication in three steps:

1. It extracts columns with outlier features (a small number of weights or activations whose values are much larger in magnitude than the rest of the distribution) from the input hidden states using a custom threshold.
2. Then it multiplies the outliers in FP16 and the rest in INT8 using vector-wise quantization.
3. Finally, it dequantizes the non-outlier results (INT8 to FP16), adds them to the outlier results, and produces the full FP16 result.

There's more good news: LLM.int8() is integrated into the Hugging Face Transformers API. It requires the Accelerate and bitsandbytes (https://github.com/bitsandbytes-foundation/bitsandbytes) libraries; bitsandbytes implements LLM.int8() quantization and runs on GPUs only.

A companion Colab notebook contains the full source code for this section and requires GPU acceleration. The steps are similar to those in section 6.2.1, so only the main parts are discussed here.

To load a quantized GPT-2 small model with this strategy, set `load_in_8bit=True` in the Transformers `auto` class:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model_id = 'openai-community/gpt2'
model_int8 = AutoModelForCausalLM.from_pretrained(model_id,
                                                  device_map='auto',
                                                  load_in_8bit=True,
                                                  )
```

This model is about 176 MB—much smaller than the FP32 version. As in section 6.2.1, we can plot the weight distributions for the original and quantized models to see how closely they match (see figure 6.5).

Here we see spikes around certain values. They match parameters stored in INT8 format, so they're not outliers. We can run inference with both models:

```
prompt = 'My favourite school subject is'
original_text = generate_text(model, prompt)
text_int8 = generate_text(model_int8, prompt)
```

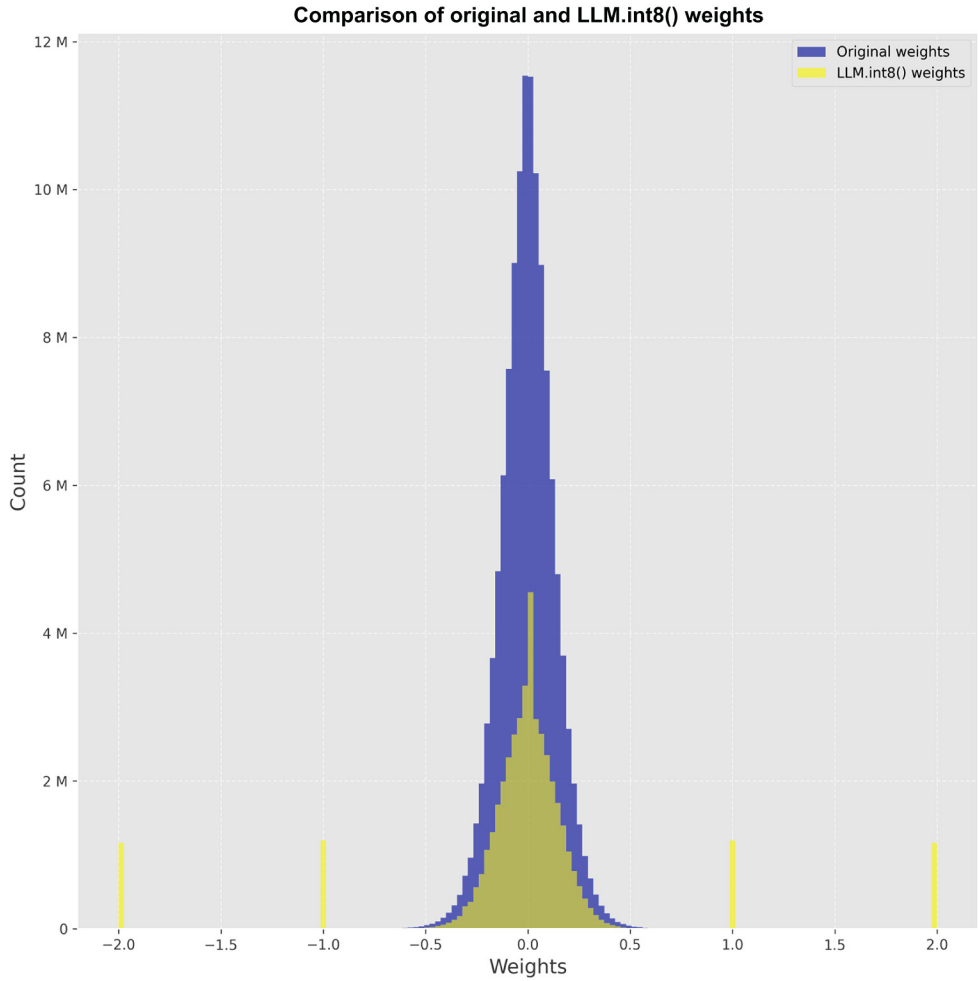


Figure 6.5 Comparing the original GPT-2 model weights and the LLM.int8() quantized ones

For this run, the original model produced the following:

```
My favourite school subject is to study music. The more I
studied music, the more intrigued I became. It was a fun hobby.

Did you ever consider making your own version of the school piano?

No, but I have tried several different combinations of different
pianos. In fact, I was quite surprised when I did find that a good
piece of school music was actually in my hands. I found it quite
easy to do it with a simple piano.

The school piano
```

The quantized model produced the following:

```
My favourite school subject is football. My favourite football
team is Everton. So I'll see you later and do some footballing.
'They looked at each other. 'Do you have your name on your phone?'

'Yes.'

'But you aren't talking?'
```

Here is the perplexity for these runs:

```
Original Perplexity:   11.25
LLM.int8() perplexity: 15.07
```

In this case, perplexity is lower for the original model, but that isn't always true. As in the previous example, a single perplexity run doesn't tell you much about the quantized model's performance—you need to repeat it multiple times. Also note that LLM.int8() quantization often causes less performance degradation than absmax, but it adds computational overhead, especially for larger models.

6.3 8-bit quantization with ONNX

Let's explore ONNX-based 8-bit PTQ using the bert-base-uncased model. The same steps apply to any other transformer-based language models.

The Hugging Face API supports three approaches to LLM optimization, commonly described in the literature as low-level, mid-level, and high-level. Let's look at each.

The low-level approach uses PyTorch's native `torch.onnx` module, which was used in section 5.5 and 5.6 code examples. It provides the `export` method to convert a model checkpoint to an ONNX graph. It's considered low-level because it requires more manual configuration and boilerplate code in the PyTorch API. While it's a solid option for model conversion and optimization, other quantization options are available.

The mid-level approach uses Transformers' `transformers.onnx` package. Let's use the bert-base-uncased model (as in section 5.5) and look at the conversion and optimization process. First, load the model and tokenizer from the Hugging Face Hub:

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

model_id = 'google-bert/bert-base-uncased'
model = AutoModelForSequenceClassification.from_pretrained(model_id)
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

Next, load its configuration (via the `transformer.onnx.FeaturesManager` class), and build the ONNX configuration:

```
from transformers.onnx import FeaturesManager
from transformers import AutoConfig
```

```
feature = "sequence-classification"
model_kind, model_onnx_config =
➥FeaturesManager.check_supported_model_or_raise(
                        model,
                        feature=feature)
onnx_config = model_onnx_config(model.config)
```

The conversion process could proceed as follows:

```
from pathlib import Path
from transformers.onnx import export

onnx_inputs, onnx_outputs = export(
        preprocessor=tokenizer,
        model=model,
        config=onnx_config,
        opset=14,
        output=Path("bert_uncased.onnx")
)
```

Like the native PyTorch package, the Transformers library includes an `export` method. Compared with the example in section 5.5, it requires far fewer configuration settings.

The high-level approach uses Optimum (https://github.com/huggingface/optimum), a Hugging Face Transformers extension that provides optimization tools for training and running LLMs on target hardware efficiently. Optimum includes several specialized packages; one supports graph optimization and quantization to speed up training and inference with ONNX Runtime. You don't need to install every package—if you only need ONNX Runtime support, install that package, and skip the rest:

```
pip install optimum[onnxruntime]
```

Using the bert-base-uncased model, you can convert the model to ONNX format (a graph) with Optimum in a single line of code:

```
from optimum.onnxruntime import ORTModelForSequenceClassification

model_id = 'bert-base-uncased'
model = ORTModelForSequenceClassification.from_pretrained(model_id,
                                                          from_transformers=True)
```

Optimum for ONNX provides several `ORTModelForXxx` classes, each specialized for a downstream task. In our example, we use `ORTModelForSequenceClassification` for sequence-classification tasks. Optimum uses the `transformers.onnx` package behind the scenes, transparently keeping the code high level. Note that Optimum does not yet support all Transformer architectures (a popular unsupported LLM is shown in chapter 7), so check the Hugging Face Hub for the latest list of supported models. For unsupported models, you can use `transformers.onnx` directly or implement a custom ONNX configuration, which we'll cover later in the book.

Whatever technique you use, once you've converted an LLM to ONNX, the 8-bit quantization process is the same. After you download it or save it to disk, you can use the ONNX Runtime quantization API to dynamically quantize the converted model. Here's an example:

```
from onnxruntime.quantization import quantize_dynamic, QuantType

onnx_path = 'bert_uncased_model.onnx'
onnx_quantized_path = 'bert_uncased_quantized_model.onnx'
quantized_model = quantize_dynamic(onnx_path, onnx_quantized_path,
                                   weight_type=QuantType.QInt8)
```

The full-precision converted ONNX model is 417.91 MB (down from 440 MB before conversion), while the quantized model is 105.22 MB.

The following end-to-end example performs 8-bit quantization with ONNX and evaluates its performance—this is a best-case scenario, where the domain-specific model architecture is supported by Optimum. A companion Colab notebook includes the full code. This example requires only Optimum (for ONNX Runtime) and Hugging Face's Evaluate library (https://github.com/huggingface/evaluate).

Because this section focuses on inference, we'll use a model from the Hugging Face Hub that has already been fine-tuned from distilbert-base-uncased: distilbert-base-uncased-finetuned-banking77. This model was fine-tuned for text classification on the `PolyAI/banking77` dataset (also on the Hugging Face Hub: https://huggingface.co/datasets/PolyAI/banking77), which contains online banking queries annotated with their corresponding intents. The dataset comprises 13,083 customer service queries labeled with 77 intents, and it has two fields: `text` (the query) and `label` (the intent label).

We'll first download the fine-tuned FP32 model from the Hub and its tokenizer:

```
from optimum.onnxruntime import ORTModelForSequenceClassification
from transformers import AutoTokenizer

model_id="optimum/distilbert-base-uncased-finetuned-banking77"
model = ORTModelForSequenceClassification.from_pretrained(model_id,
                                                          export=True)
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

We are using Optimum's `ORTModelForSequenceClassification` to download the model and automatically convert it to ONNX format. We are still using the Transformers' `AutoTokenizer` to download the tokenizer for the baseline model. After the download completes, we save both to disk (this step is required only for quantization):

```
from pathlib import Path

onnx_path = Path("onnx")
model.save_pretrained(onnx_path)
tokenizer.save_pretrained(onnx_path)
```

Since Optimum downloaded the model, we can use the Transformers pipelines (supported by the library, unlike the native ONNX Runtime) to run inference and confirm the model works:

```
from transformers import pipeline

vanilla_clf = pipeline("text-classification", model=model,
➥tokenizer=tokenizer)
vanilla_clf("Could you assist me in checking my card validity?")
```

We now need to quantize the model to 8-bit precision, using dynamic quantization. Unlike static quantization, dynamic quantization runs only on the CPU, not the GPU. That's not a problem, because quantization happens offline, and inference is still fast without hardware acceleration (as you'll see later in this section).

We'll create an instance of `ORTQuantizer` (Optimum's class for ONNX Runtime quantization of models shared on the Hub) and define the quantization configuration. Then we can apply it to the original model:

```
from optimum.onnxruntime import ORTQuantizer
from optimum.onnxruntime.configuration import AutoQuantizationConfig

dynamic_quantizer = ORTQuantizer.from_pretrained(model)
dqconfig = AutoQuantizationConfig.avx512_vnni(is_static=False,
                                              per_channel=False)

model_quantized_path = dynamic_quantizer.quantize(
    save_dir=onnx_path,
    quantization_config=dqconfig,
)
```

Once quantization is complete, we can compare model sizes: the original file is 255.76 MB, while the quantized version is 64.33 MB.

You can run inference on the quantized model the same way as on the original, using a Transformers pipeline:

```
quantized_model_name = "model_quantized.onnx"
model = ORTModelForSequenceClassification.from_pretrained(onnx_path,
                                                  file_name=quantized_model_name)
tokenizer = AutoTokenizer.from_pretrained(onnx_path)

q8_clf = pipeline("text-classification",model=model, tokenizer=tokenizer)
q8_clf("Could you assist me in checking my card validity?")
```

Next, we'll evaluate model performance on the `PolyAI/banking77` test set to see how quantization affected accuracy. We can do this with Hugging Face's high-level `Evaluate` API: create an `evaluator` for the specific task (text classification in our example), load the test set, and call the evaluator's `compute` method, specifying the pipeline, the test

set (including which fields are inputs and labels), and the metric to compute (accuracy here, though the code in the notebook also computes others):

```
from evaluate import evaluator
from datasets import load_dataset

dataset_id="PolyAI/banking77"
eval = evaluator("text-classification")
eval_dataset = load_dataset(dataset_id, split="test")

results = eval.compute(
    model_or_pipeline=q8_clf,
    data=eval_dataset,
    metric="accuracy",
    input_column="text",
    label_column="label",
    label_mapping=model.config.label2id,
    strategy="simple",
)
```

According to the Hugging Face Hub, the original model's accuracy is 92.5%. Running the preceding code yields 92.34% for the quantized model (99.82% of the FP32 baseline), a negligible loss. But what about latency? We can write a quick-and-dirty benchmark function to compare the two versions (chapters 7 and 10 show better ONNX benchmarking approaches):

```
def measure_latency(payload_prompt, pipe):
    latencies = []

    for _ in range(10):
        _ = pipe(payload_prompt)
    for _ in range(300):
        start_time = perf_counter()
        _ =  pipe(payload_prompt)
        latency = perf_counter() - start_time
        latencies.append(latency)

    time_avg_ms = 1000 * np.mean(latencies)
    time_std_ms = 1000 * np.std(latencies)
    time_p95_ms = 1000 * np.percentile(latencies,95)

    return f"P95 latency (ms) - {time_p95_ms}; Average latency (ms)
➥ - {time_avg_ms:.2f} +\- {time_std_ms:.2f};", time_p95_ms
```

This pipeline runs a short warmup (a few initial inference passes to stabilize caches and compilation effects), benchmarks a set number of steps, then reports the average latency and standard deviation. Running it on both the original and quantized model pipelines with a 142-token prompt shows a 1.86x inference speedup with the quantized model. With comparable performance, quantization significantly reduces latency.

6.4 4-bit quantization

In the previous section, you saw practical examples of LLM quantization from single precision to 8-bit integer format. Now let's push further and reduce the model size by quantizing to a lower precision, such as 4-bit integers.

6.4.1 4-bit quantization with GPTQ

After 8-bit quantization proved effective at shrinking LLMs with little loss in accuracy, new techniques emerged for 4-bit integer quantization. The first was GPTQ, proposed in 2022 in "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers" (https://arxiv.org/abs/2210.17323). GPTQ is a one-shot weight-quantization method based on approximate second-order information that is both accurate and efficient. It can reduce GPT-based models to 3 or 4 bits per weight with negligible accuracy loss compared to the uncompressed baseline. The paper reports that even 175B-parameter models can run inference on a single NVIDIA A100 GPU after quantization, and the method can also yield strong results on smaller models and less powerful GPUs. The paper covers the math and methodology, but you don't need to master them to use GPTQ because a solid Python implementation is available and integrated into major LLM libraries. The de facto standard implementation is the `AutoGPTQ` package (https://github.com/AutoGPTQ/AutoGPTQ). Consult the associated repository only if you want low-level implementation details—`AutoGPTQ` is also integrated into the Hugging Face Transformers, Optimum, and PEFT libraries. In this chapter, we'll use the original implementation to make it easier to see how 4-bit quantization works (higher-level APIs hide many details).

In this section, you'll learn how to use `AutoGPTQ`. A companion Colab notebook provides the full source code for the example. Along with `AutoGPTQ`, we'll use the Hugging Face Transformers library to apply 4-bit quantization to the GPT-2 XL model (1.5B parameters, https://huggingface.co/openai-community/gpt2-xl). We'll run with hardware acceleration on a single NVIDIA Tesla T4 GPU.

After installing `AutoGPTQ` (see the package documentation; steps vary depending on your PyTorch and CUDA driver versions), we can start implementing the model quantization code. First, download the GPT-2 XL tokenizer from the Hugging Face Hub:

```
from transformers import AutoTokenizer

model_id = "gpt2-xl"
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

Before loading the unquantized model, we need to define a base 4-bit quantization configuration with `AutoGPTQ`'s `BaseQuantizeConfig` class:

```
from auto_gptq import BaseQuantizeConfig

quantize_config = BaseQuantizeConfig(
    bits=4,
```

```
    group_size=128,
    desc_act=False,
)
```

We can now download the model:

```
from auto_gptq import AutoGPTQForCausalLM

model = AutoGPTQForCausalLM.from_pretrained(model_id, quantize_config)
```

We can get the maximum sequence length from its configuration:

```
model_config = model.config.to_dict()
seq_len_keys = ["max_position_embeddings", "seq_length", "n_positions"]
if any(k in model_config for k in seq_len_keys):
    for key in seq_len_keys:
        if key in model_config:
            model.seqlen = model_config[key]
            break
else:
    print(
    "The model's sequence length cannot be retrieved from
its configuration. It will then be set to 2048."
    )
    model.seqlen = 2048
```

To perform one-shot weight quantization, we need some sample sequences. We'll use a few from the `wikitext` dataset on the Hugging Face Hub, which we used in section 4.1.4:

```
import random
import torch
from datasets import load_dataset

seed = 0
random.seed(seed)
np.random.seed(seed)
torch.random.manual_seed(seed)

traindata = load_dataset("wikitext", "wikitext-2-raw-v1", split="train")
trainenc = tokenizer("\n\n".join(traindata["text"]), return_tensors="pt")

nsamples = 128
seqlen = model.seqlen
traindataset = []
for _ in range(nsamples):
    i = random.randint(0, trainenc.input_ids.shape[1] - seqlen - 1)
    j = i + seqlen
    inp = trainenc.input_ids[:, i:j]
    attention_mask = torch.ones_like(inp)
    traindataset.append({"input_ids": inp, "attention_mask":
   ➥attention_mask})
```

The preceding code snippet builds a small dataset of 128 samples from the `wikitext` training set. Quantization quality depends on the samples used for conversion: more samples can improve results. The number of samples to use depends on your use case, model, and data—there's no one-size-fits-all setting.

We can now start the quantization process. It will take a few minutes on the hardware we're using, but that's fine because it runs offline:

```
model.quantize(traindataset, use_triton=False)
```

In this scenario, quantizing an LLM takes a single line of code once the training samples are prepared. We set `use_triton` to `False` because we're using CUDA rather than the NVIDIA Triton language and compiler (https://triton-lang.org/main/index.html).

After quantization, save the 4-bit model to disk:

```
quantized_model_dir = "gpt2-xl-4bit"
model.save_quantized(quantized_model_dir)
```

You can optionally also upload it for sharing on the Hugging Face Hub. The original model is 6.43 GB; the 4-bit quantized version is 1.02 GB.

The model is now ready for inference. Load it from where you saved it:

```
from auto_gptq import AutoGPTQForCausalLM

model = AutoGPTQForCausalLM.from_quantized(quantized_model_dir,
                                           device="cuda:0",
                                           ➥use_triton=False)
```

Currently, `AutoGPTQ` supports loading a quantized model only on CPUs or a single GPU; multi-GPU setups aren't supported. You can run inference by invoking the model's `generate` method:

```
prompt = "Auto-GPTQ is"
output = tokenizer.decode(
    model.generate(**tokenizer(prompt,
    ➥return_tensors="pt").to("cuda:0"))[0])
print(output)
```

The same weight-analysis and model-evaluation considerations (perplexity calculation in section 6.2 and inference benchmarking in section 6.3) apply here as well.

6.4.2 4-bit quantization with ggml

GPTQ isn't the only way to get 4-bit post-training quantization for LLMs. Another option is ggml (https://github.com/ggerganov/ggml), an open source C library that supports several quantization levels (8-bit, 5-bit, and 4-bit). Models quantized with ggml are typically saved in the GGML binary file format. Although it's optimized for

Apple Silicon, ggml-quantized models can also run on x86 (using AVX/AVX2 intrinsics), ppc64 (using VSX intrinsics), and Android. ggml is fast in part because it has no third-party dependencies and does zero memory allocations at runtime.

ggml can also quantize other Transformer architectures (for example, image and audio models), but in this chapter (and the next) we focus on LLMs and how to run them efficiently on CPUs. GPTQ inference is optimized for GPUs, whereas ggml inference is optimized for CPUs, which makes it practical for many environments, including typical laptops and personal computers. Recent releases also support limited GPU offloading at inference time—not to speed up inference, but to reduce power use and free up CPU resources.

Later chapters return to ggml for more advanced optimization and quantization techniques; unlike Hugging Face's Python libraries, it requires more low-level programming knowledge.

Scripts for converting some of the most popular open source LLMs to GGML format are available in ggml's official GitHub repository, and many quantized versions are already available for download from the Hugging Face Hub. Although ggml is written in C, you can load quantized models and run inference entirely in Python using Hugging Face Transformers and CTransformers (https://github.com/marella/ctransformers). CTransformers provides Python bindings for Transformer models implemented in C/C++ via ggml.

The following example runs inference with a GGML-converted model using the Hugging Face Transformers `pipeline` API:

```
from ctransformers import AutoModelForCausalLM
from transformers import pipeline, AutoTokenizer

model = AutoModelForCausalLM
➥.from_pretrained("TheBloke/Llama-2-7B-Chat-GGML", hf=True)
tokenizer = AutoTokenizer.from_pretrained("NousResearch/Llama-2-7b-chat-hf")

pipe = pipeline("text-generation", model=model, tokenizer=tokenizer)
print(pipe("AI is going to", max_new_tokens=256))
```

You can load a ggml-quantized model from the Hugging Face Hub with the CTransformers `AutoModelForCausalLM` class. Load the tokenizer from the original model as usual with the Transformers `AutoTokenizer`. You can then run inference through a Transformers pipeline, just as you would with single- or half-precision pretrained models.

ggml has two parent open source projects, llama.cpp and whisper.cpp, both under active development. Llama.cpp (https://github.com/ggerganov/llama.cpp) began as a C/C++ port of Meta's Llama model, but has since expanded to other popular LLM and multimodal architectures. As part of llama.cpp, an alternative quantized-model format to GGML was introduced: GGUF, now the only format supported by llama.cpp. Compared with GGML, GGUF makes it easier to distribute a quantized model

as a single file (there's no external JSON file for the tokenizer) and stores hyperparameters as key–value metadata, making it easier to add new fields without breaking compatibility with existing GGUF models. GGUF is quickly becoming the de facto standard for C/C++ quantized models. Future chapters present several examples of quantization in GGML and GGUF formats.

This chapter and the previous one covered the ONNX framework's capabilities, focusing on LLM optimization, quantization, and inference. In the next two chapters, we'll apply these concepts to real-world projects.

Summary

- Floating-point precision affects memory use and computational speed in language model operations.
- A 32-bit float uses 1 sign bit, 8 exponent bits, and 23 mantissa bits.
- 16-bit floating point reduces memory requirements by half but can reduce performance.
- Mixed-precision training switches between 32-bit and 16-bit operations during training.
- BF16 offers a wider dynamic range than FP16, with less precision.
- Quantization converts model weights from floating-point to low-bit integers.
- Post-training quantization is applied after training; quantization-aware training happens during training.
- Zero-point quantization scales inputs by the total range divided by the difference between maximum and minimum values.
- Absmax quantization divides input values by their absolute maximum value and then multiplies by a scaling factor.
- LLM.int8() uses mixed-precision quantization: outliers run in FP16 and the rest in INT8.
- Vector-wise quantization accounts for variation across tensor rows and columns.
- ONNX Runtime provides three levels of quantization, from low-level PyTorch APIs to the high-level Optimum.
- ONNX dynamic quantization greatly reduces model size with minimal performance loss.
- GPTQ enables 4-bit quantization for GPT models with minimal accuracy loss.
- `AutoGPTQ` integrates with Hugging Face libraries for streamlined 4-bit quantization.
- The GGML library provides CPU-optimized quantization at 8-, 5-, and 4-bit precision.
- GGUF format lets you distribute quantized models as single files with embedded metadata.

- Quantization cuts a model's memory footprint by two to four times while maintaining comparable performance.
- Perplexity metrics evaluate quantized model performance by measuring prediction uncertainty.

Part 3

Real-world use cases

The two chapters in this part of the book get hands-on with real-world examples of SLMs specialized in Python code generation, protein structure generation, antibody generation for a target antigen, and crystal structure file generation. Examples in these chapters include performance assessment, model profiling, quantization, and validation.

7 Generating Python code

This chapter covers

- The role specialized open source LLMs play in code generation
- Optimizing and quantizing these LLMs to improve inference performance
- Running these LLMs on a laptop after 4-bit quantization

In this chapter, we'll apply the concepts covered so far to real-world, domain-specific LLMs, including optimization and quantization. We'll focus on models tuned for Python code generation and programming assistance. Although most open source models in this category support multiple languages, we'll use Python so you can readily judge the quality of the outputs.

7.1 Using Transformers to generate code

Beyond closed-source, proprietary LLM-based code assistants, several popular open-source LLMs designed for coding tasks have been released. In this chapter, we'll get into hands-on experimentation with some of these options, look at their pros and cons, and

see how to optimize and, if needed, quantize them to run inference on commodity hardware (ideally on our laptops) while maintaining good performance. First, though, we'll consider the present and future of human programmers in this era of coding assistants.

Given the availability of commercial and open-source copilots, many CEOs and other executives have predicted the obsolescence of programmers in favor of prompt engineers, or claimed that everyone (whatever their background) will become a software architect. How true is that? Let's start with large generalist models. Several research efforts, such as SWE-bench (https://arxiv.org/abs/2310.06770), evaluate large commercial models such as GPT-4 and Claude 2 on real-world programming challenges based on real code. The paper focuses on Python (2,294 issues and related pull requests from GitHub repositories), but many benchmarks for other languages can be found online. Neither model scored well on these problems, even with knowledge retrieval. More prompt engineering helps little: even with a larger context, results can still be confused or random, at the cost of more processing power and the user's time. Since the paper's publication, many large and small models, with or without retrieval-augmented generation (RAG), have been benchmarked. The updated leaderboard is available at https://www.swebench.com/ (updated every Monday). Over time, new models and supporting techniques have improved results, but none of the benchmarked models can yet be used without expert human supervision (see figure 7.1).

Leaderboard

Lite | Verified | Full

Model	% Resolved	Org	Date	Logs	Trajs	Site
OpenHands + CodeAct v2.1 (claude-3-5-sonnet-20241022)	53.00		2024-10-29	✓	✓	
Solver (2024-10-28)	50.00		2024-10-28	✓	✓	
Tools + Claude 3.5 Sonnet (2024-10-22)	49.00		2024-10-22	✓	✓	
Composio SWE-Kit (2024-10-25)	48.60		2024-10-25	✓	✓	
Emergent E1 (v2024-10-12)	46.60		2024-10-23	✓	✓	
Solver (2024-09-12)	45.40		2024-09-24	✓	✓	
Gru(2024-08-24)	45.20		2024-08-24	✓	✓	
Solver (2024-09-12)	43.60		2024-09-20	✓	✓	
Tools + Claude 3.5 Haiku	40.60		2024-10-22	✓	✓	
Honeycomb	40.60		2024-08-20	✓	✓	
Composio SWEkit + Claude 3.5 Sonnet (2024-10-16)	40.60		2024-10-16	✓	✓	
EPAM AI/Run Developer Agent v20241029 + Anthopic Claude 3.5 Sonnet	39.60		2024-10-29	✓	✓	
Amazon Q Developer Agent (v20240719-dev)	38.80		2024-07-21	✓	✓	
Agentless-1.5 + GPT 4o (2024-05-13)	38.80		2024-10-28	✓	✓	
AutoCodeRover (v20240620) + GPT 4o (2024-05-13)	38.40		2024-06-28	✓	-	
SWE-agent + Claude 3.5 Sonnet	33.60		2024-06-20	✓	✓	-
nFactorial (2024-10-07)	31.60		2024-10-07	✓	✓	
Lingma Agent + Lingma SWE-GPT 72b (v0925)	28.80		2024-10-02	✓	✓	
EPAM AI/Run Developer Agent + GPT4o	27.00		2024-10-16	✓	✓	
AppMap Navie + GPT 4o (2024-05-13)	26.20		2024-06-15	✓	-	
nFactorial (2024-10-01)	25.80		2024-10-01	✓	✓	
Amazon Q Developer Agent (v20240430-dev)	25.60		2024-05-09	✓	-	

SWE-bench **Lite** is a subset of SWE-bench that's been curated to make evaluation less costly and more accessible [Post].

Figure 7.1 SWE-bench leaderboard (early 2025 snapshot)

Several factors explain the modest performance. Resolving software engineering problems often requires coordinating changes across multiple functions, classes, and files, interacting with execution environments, and reasoning well beyond straightforward code generation. These are areas where humans still outperform ML models. And because these models are trained as generalists, they rarely match the depth of expertise that professional software engineers bring to complex programming tasks.

Dedicated models for programming tasks, such as Microsoft Copilot, are often more accurate and can handle more complex work than general-purpose models. Still, they can be costly for some organizations and require human oversight and guardrails to prevent data leakage. They may also lack transparency about where their training data comes from. These tools raise the same privacy, IP, and safety concerns discussed earlier in this book, especially when they rely on large, closed models.

Luckily for us, LLMs specialized for this kind of task are available, and specialization drastically reduces model size. We can see this in the industry's growing move toward smaller, more focused coding LLMs that, despite having only a few billion parameters, match, and sometimes outperform, larger models on evaluation benchmarks. This isn't surprising: if the main task is coding in one or a few programming languages, you can remove much of the irrelevant knowledge in large generalist models. A smaller, more focused data pool is easier to keep relevant, cheaper to train, and less likely to inadvertently include copyrighted or unauthorized data.

Back to the main question: Will generative AI replace human programmers? Probably not, even if models reach 98% to 100% accuracy on coding tasks. Coding is only one part of software development. It also involves investing in new things, understanding users' requirements and needs, translating ideas into something buildable, and polishing the result. These steps require human judgment and cannot be reduced to next-token generation. In my opinion, software engineers will not shift into roles focused mainly on reviewing and verifying AI-generated code, even if generative models become highly accurate. These models can still be a big help, and I wouldn't discourage professionals from using them. But domain knowledge is always needed to use them well: a good software engineer can become a better software engineer using a code assistant, but someone with little or no software engineering knowledge will not. Software engineering isn't different from other domains when it comes to support from AI-based assistants.

7.2 Generating Python code with a Transformer architecture

Let's get started. We'll use an open source, pretrained code-generation model as our baseline and see what we get.

7.2.1 Python code generation with CodeGen

The first open source pretrained model we'll consider is Salesforce's CodeGen (https://arxiv.org/abs/2203.13474). It uses a standard Transformer architecture for conversational program synthesis: each problem is solved interactively in multiple

steps, each consisting of a natural-language specification from the user and a synthesized subprogram from the system. Although better open source coding models are available, including CodeGen 2.0 and 2.5, CodeGen is still a useful example for this class of specialized LLMs because it highlights architectural choices that can require extra work beyond the typical use of Hugging Face Transformers, ONNX, and related libraries.

CodeGen is conceptually comparable to OpenAI's Codex, the model that was behind GitHub Copilot when Salesforce released its own proposal. It offers two families of pretrained models: *multi*, trained to generate code in six languages (C, C++, Go, Java, JavaScript, and Python), and *mono*, trained to generate Python only. Each family includes four sizes: 350M (350 million parameters), 2B (2.7 billion parameters), 6B (6.1 billion parameters), and 16B (16.1 billion parameters). To show CodeGen's scale at release, figure 7.2 compares the largest model with other open or closed source LLMs for the same task available at the time.

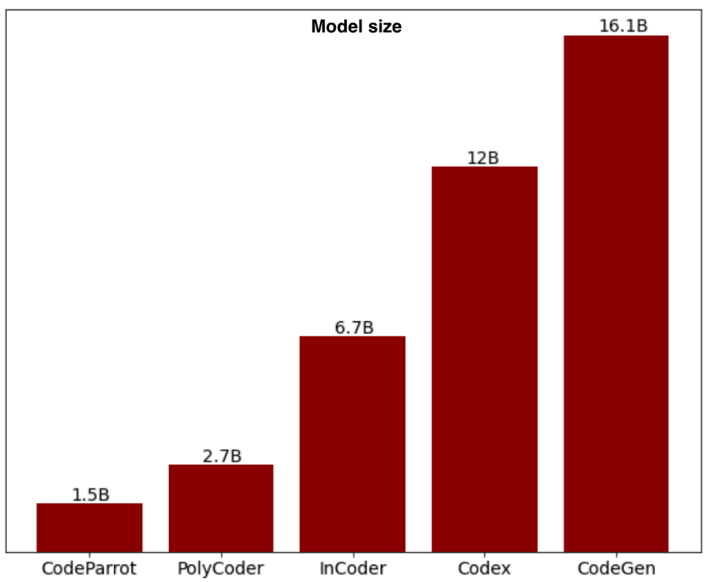


Figure 7.2 Comparing the most popular code assistant LLMs' sizes when CodeGen was released

As mentioned earlier in this chapter, we are focusing only on Python code generation, so any reference to CodeGen models in this section refers to the mono family.

Inference with the 350M pretrained model is fast, even before optimization. Benchmarks on a single NVIDIA T4 GPU average 0.8 s for requests that generate fewer than 50 tokens and 0.9 s for requests that generate 50 to 250 tokens. Still, the 350M model isn't suitable for production; later in this chapter, we'll discuss the quality of the generated results.

The 2B model is a more robust candidate (as are the larger versions). This model family can generate code that uses not only Python's standard library but also third-party open source libraries. Even so, the 350M version is a good choice for explaining the concepts covered in this chapter.

The first Colab notebook for this chapter contains a complete, end-to-end example of optimizing inference for CodeGen 350M mono. We'll use this smaller model to show that even a few hundred million–parameter model leaves room for inference optimization, and the same ideas apply to the larger mono models.

All the CodeGen checkpoints (mono and multi) are available on the Hugging Face Hub and via the Transformers library, so we can download the pretrained model and its tokenizer as usual:

```
import torch
from transformers import AutoTokenizer, CodeGenForCausalLM

device = "cpu"
model_id = "Salesforce/codegen-350M-mono"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = CodeGenForCausalLM.from_pretrained(model_id).to(device)
model.eval()
```

The only difference here is the use of `CodeGenForCausalLM`, Hugging Face Transformers' auto class for CodeGen models.

This example doesn't require hardware acceleration. We'll assume the model and data are loaded and running on CPUs. The small CodeGen mono model works as follows:

```
prompt = "def hello_world():"
input_ids = tokenizer(prompt, return_tensors="pt").input_ids

generated_ids = model.generate(input_ids, max_length=12)
print(tokenizer.decode(generated_ids[0],
                       skip_special_tokens=True,
                       pad_token_id=50256))
```

Given a prompt with a descriptively named function definition, it generates the function body as follows:

```
def hello_world():
    print("Hello World")
```

This is one way to prompt the CodeGen models. It's the only approach that works well for the 350M model, though others are possible. With the 2B and larger CodeGen models, you can also use pure natural-language prompts (English only), like this:

```
Create bar chart with matplotlib
```

That will generate something like this:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# create a DataFrame
df = pd.DataFrame({'sales': [10, 5, 6, 9, 7],
                   'sale_of_sales': [20, 10, 15, 25, 30]})

# plot the bar chart
df.plot(kind='bar', figsize=(10, 6), color='blue')

# Add title and labels
plt.title("Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

# show the plot
plt.show()
```

You can also use a single-line comment, like this:

```
# Create empty tensor with PyTorch
```

That will produce a result like this:

```
tensor = torch.empty(1, dtype=torch.float32)
```

If you use a multiline comment

```
"""

    Create a basic web app with Dash

"""
```

you will get something like this:

```
import dash
import dash_core_components as dcc
import dash_html_components as html
import plotly.graph_objects as go

app = dash.Dash()

app.layout = html.Div([
    html.H1('Dash App'),
    html.Div([
        dcc.Dropdown(
            id='dropdown',
            options=[
```

```
                {'label': 'New York City', 'value': 'NYC'},
                {'label': 'Montreal', 'value': 'MTL'},
                {'label': 'San Francisco', 'value': 'SF'}
            ],
            value='MTL'
        ),
        dcc.Graph(id='graph', animate=True)
    ])
])

@app.callback(
    dash.dependencies.Output('graph', 'figure'),
    [dash.dependencies.Input('dropdown', 'value')])
def update_figure(selected_dropdown_value):
    print(f"Selected dropdown value: {selected_dropdown_value}")
    return {
        'data': [
            {'x': [1, 2, 3], 'y': [4, 1, 2], 'type': 'bar', 'name': 'SF'},
            {'x': [1, 2, 3], 'y': [2, 4, 5], 'type': 'bar', 'name':
            ➥'Montreal'},
        ],
        'layout': {
            'title': 'Dash Data Visualization'
        }
    }

if __name__ == '__main__':
    app.run_server(debug=True)
```

You can also generate a function body from just its definition (any name, even if it's not very descriptive) and an inline comment, which the model uses to generate the code:

```
def increment_elements(int_list):

    """

        Return list with elements incremented by 1

    """
```

This generates something like the following:

```
def increment_elements(int_list):
    """
        Return list with elements incremented by 1
    """
    for i in range(len(int_list)):
        int_list[i]+=1
    return int_list
```

These models can also verify code and run user-written or model-generated code on input data:

```
int_list=[1, 3, 5]

>>> print(len(int_list))
```

That generates the following:

```
int_list=[1, 3, 5]

>>> print(len(int_list))

3
```

You can also implement unit tests, given some code and a comment as the prompt:

```
def create_empty_array(n, m):

    arr = np.zeros((n, m))

    return arr

#Write unit test for create_empty_array()
```

The model will generate unit tests for the code, following the guidance in the comment:

```
self.assertEqual(create_empty_array(2,3), [[0,0,0],[0,0,0]])
self.assertEqual(create_empty_array(5,4), np.zeros((5,4)))
self.assertEqual(create_empty_array(6,7), np.zeros((6,7)))
self.assertEqual(create_empty_array(3,4), np.zeros((3,4)))
```

Based on what you learned in chapter 6, you might expect to use Optimum to convert this model to ONNX and then quantize it. That's correct now, but until early 2024, the CodeGen architecture wasn't supported in Optimum—it was added only recently. Let's first use Optimum to export the CodeGen model to ONNX and quantize it; later, we'll look at an alternative approach for cases where Optimum doesn't support a model's architecture.

With Optimum installed for ONNX Runtime, the model can be downloaded from the Hugging Face Hub and converted to ONNX automatically:

```
from optimum.onnxruntime import ORTModelForCausalLM

model_id = 'Salesforce/codegen-350M-mono'
model = ORTModelForCausalLM.from_pretrained(model_id,
                                            export=True,
                                            provider="CPUExecutionProvider"
)
```

We are using the Optimum `ORTModelForCausalLM` class because CodeGen is a causal LM. We also specified the ONNX execution provider (`CPUExecutionProvider` in this example) at download time.

We can then save the model to disk:

```
from pathlib import Path

onnx_path = Path("onnx")
model.save_pretrained(onnx_path)
```

It can be now quantized dynamically through the Optimum API, as explained in chapter 6:

```
from optimum.onnxruntime import ORTQuantizer
from optimum.onnxruntime.configuration import AutoQuantizationConfig

dynamic_quantizer = ORTQuantizer.from_pretrained(model)
dqconfig = AutoQuantizationConfig.avx512_vnni(is_static=False,
                                              per_channel=False)

model_quantized_path = dynamic_quantizer.quantize(
    save_dir=onnx_path,
    quantization_config=dqconfig,
)
```

The ONNX-converted model is 1.33 GB; the 8-bit quantized version is 346.22 MB.

Now let's benchmark execution performance and see whether ONNX conversion and quantization improved latency and throughput. We'll run a few hundred inference cycles for each of the three models and compare the results.

To keep the code clean and readable, we'll use a Transformers pipeline to run inference with each CodeGen model version under test. Here's the code to create the vanilla model pipeline:

```
from transformers import pipeline, GenerationConfig

generation_config = GenerationConfig(
    pad_token_id=50256,
    truncation=True,
    max_length=12
)
pipe = pipeline("text-generation",
                model=model,
                tokenizer=tokenizer,
                generation_config=generation_config
                )
```

We also need to implement two simple utility functions to collect metrics during benchmark execution for all three model versions. First, we define the `track_infer_time` function, which measures the duration of a single inference run and is applied across the full benchmark cycle:

```
from contextlib import contextmanager
from time import perf_counter

@contextmanager
def track_infer_time(time_buffer):
    start_time = perf_counter()
    yield
    end_time = perf_counter()

    time_buffer.append(end_time - start_time)
```

The second utility is the `BenchmarkInferenceResult` class, used to collect metrics across benchmarks:

```
from dataclasses import dataclass

@dataclass
class BenchmarkInferenceResult:
    model_inference_time: [int]
    optimized_model_path: str
```

The benchmark code for the vanilla CodeGen model is then quite simple:

```
from tqdm import trange

PROVIDERS = {
    ("cpu", "PyTorch CPU"),
}

results = {}

inference_runs = 200
for device, label in PROVIDERS:
    time_buffer = []
    for _ in trange(inference_runs, desc=f"Tracking
    ➥inference time (PyTorch vanilla model)"):
      with track_infer_time(time_buffer):
        pipe(prompt)

    results[label] = BenchmarkInferenceResult(
        time_buffer,
        None
    )
```

The code to benchmark the ONNX version of this CodeGen model is the same as for the original model. The only differences are the model you pass to the pipeline (the ONNX-converted version) and the provider tags you use to calculate and display inference metrics:

```
PROVIDERS = {
    ("CPUExecutionProvider", "ONNX CPU"),
}
```

Figure 7.3 shows the average inference time across hundreds of runs for both model versions. All benchmarks were run on a free Google Colab VM without hardware acceleration (Intel Xeon CPU with two virtual CPUs and 13 GB of RAM).

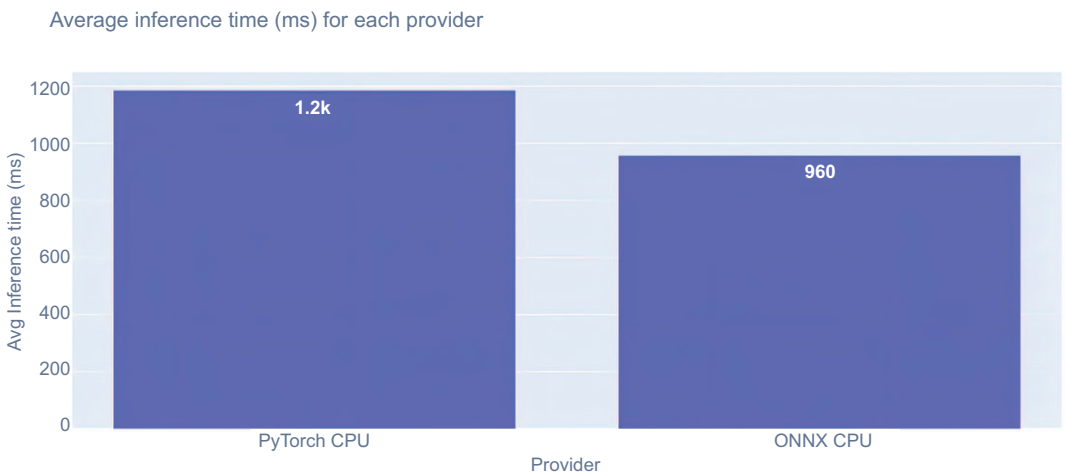


Figure 7.3 Average CPU inference time for the vanilla CodeGen model in PyTorch vs. its ONNX-converted version

On our hardware, converting to ONNX reduced average inference time from about 1186 ms to about 958 ms. Figure 7.4 compares the two model versions on the same benchmark, showing latency (average and percentiles) and throughput. Running inference in ONNX format reduced latency and increased throughput.

	PyTorch CPU	ONNX CPU
Average_latency (ms)	1186.192727	957.865782
Latency_P50	1086.817074	930.258778
Latency_P75	1293.802172	1004.383695
Latency_P90	1365.162211	1049.166195
Latency_P95	1418.534996	1059.449770
Latency_P99	2003.640328	1079.141317
Throughput	0.843033	1.043988

Figure 7.4 Latency and throughput metrics for the CodeGen 350M mono vanilla model vs. its ONNX-converted version

Figure 7.5 shows a box plot of inference duration across 100 runs for both model versions under test; individual run times are also shown.

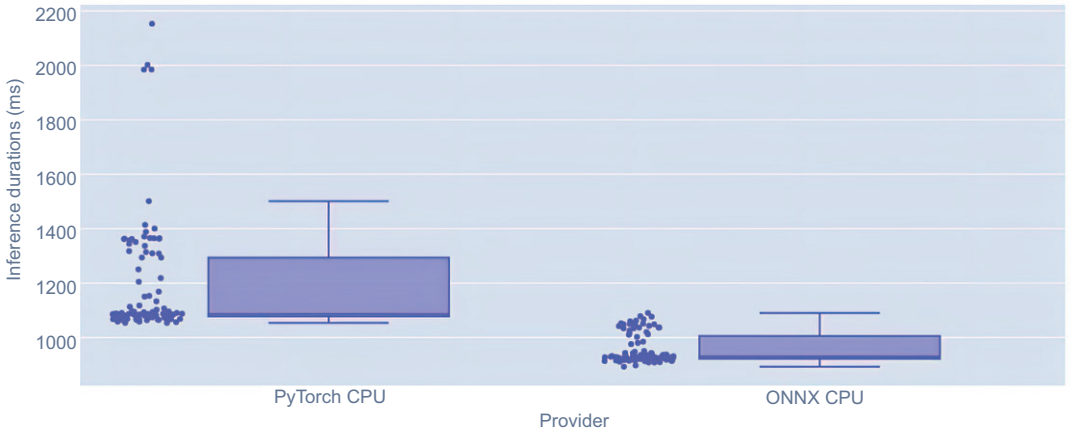


Figure 7.5 Inference run times for 100 runs with the CodeGen 350M mono vanilla model and its ONNX-converted version

In this box plot, we can see that after converting to ONNX and applying the related optimizations, the standard deviation is smaller than for the original model. After moving to ONNX, duration is more consistent across runs.

By repeating the benchmark with the ONNX-quantized model (after swapping the quantized model into the pipeline and setting the provider tags), we can see further improvements: about a 60% reduction in average inference time (see figure 7.6).

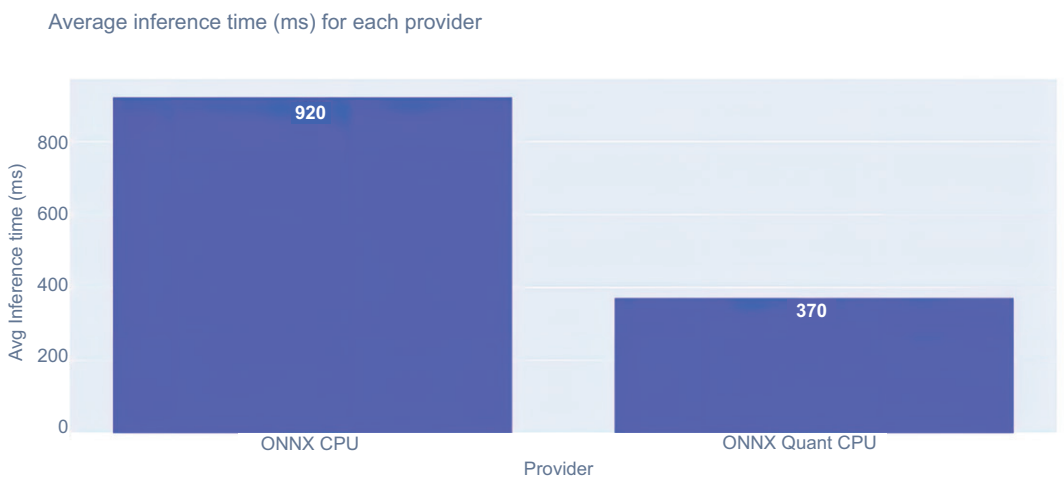


Figure 7.6 Average CPU inference time for the ONNX CodeGen model and its quantized version

Figure 7.7 compares the two versions of the ONNX model on the same benchmark in terms of latency (average and percentiles) and throughput.

	ONNX CPU	ONNX Quant CPU
Average_latency (ms)	923.130653	372.388912
Latency_P50	853.977321	351.197858
Latency_P75	942.409447	418.963405
Latency_P90	1000.723633	446.840175
Latency_P95	1103.220258	453.199262
Latency_P99	1713.538410	461.910913
Throughput	1.083270	2.685365

Figure 7.7 Latency and throughput metrics for the CodeGen 350M mono ONNX model and its quantized version

Running inference with the quantized ONNX model reduced latency and increased throughput compared with the same benchmark on the vanilla CodeGen model.

> **NOTE** All the figures in this section were generated with the open source Plotly library for Python (https://plotly.com/python/). I chose Plotly not only for its elegant visualizations compared with Matplotlib, but also for its built-in interactivity. The full source code for the CodeGen benchmarks and the visualizations in this section is available in the accompanying Colab notebook.

We've seen that exporting a model to ONNX and/or an ONNX-quantized version can significantly improve latency and throughput, even for small language models. But how do we assess how optimization and quantization affect the quality of the generated code? Section 7.2.3 will explore strategies for evaluating language model output for Python code generation.

7.2.2 Using ONNX with models not supported by Optimum

Some LLM architectures aren't yet supported by the Optimum package (for example, CodeGen models were still unsupported as of early 2024). In those cases, you can use the mid-level approach for ONNX quantization. After downloading a model from the Hugging Face Hub and saving it to disk, run the `transformers.onnx` script from the Transformers library as follows:

```
!python -m transformers.onnx --feature "causal-lm" --framework pt
➥-export_with_transformers --model=local-pt-checkpoint onnx/
```

The `--feature` option sets the model category (`causal-lm` for CodeGen models; see the Transformers documentation for other models). The `--model` option specifies the directory where the downloaded checkpoints are saved.

After conversion, you can perform 8-bit quantization using the ONNX Runtime API:

```
import os
import onnx
from onnxruntime.quantization import quantize_dynamic, QuantType

def quantize_onnx_model(onnx_model_path, quantized_model_path):
    onnx_opt_model = onnx.load(onnx_model_path)
    quantize_dynamic(onnx_model_path,
                     quantized_model_path,
                     weight_type=QuantType.QInt8)

quantize_onnx_model(onnx_model_path, quantized_model_path)
```

7.2.3 Model evaluation

Several open source Python packages, such as Hugging Face's `Evaluate` and EleutherAI's LLM `Evaluation Harness` (discussed in chapters 6 and 4, respectively), plus public datasets, are available for LLM evaluation. But generating unstructured outputs such as Python (or other) code calls for extra choices to make evaluation robust and meaningful.

In this section, we'll walk through an approach based on the CodeGen 350M mono model and its ONNX-converted, 8-bit-quantized variants introduced in section 7.2.1, and which you can also apply to other LLMs specialized on Python code generation. The code comes from Amazon Science's ReCode paper (https://arxiv.org/abs/2212.10264) and is shared in a Colab notebook for this chapter. I chose this approach because ReCode defines robustness metrics for code generation models by measuring worst-case behavior under different prompt perturbations (language models are sensitive to prompt changes). This provides a practical way to assess code quality for production use.

We'll use OpenAI's `HumanEval` evaluation dataset (https://github.com/openai/human-eval), introduced in "Evaluating Large Language Models Trained on Code" (https://arxiv.org/abs/2107.03374). `HumanEval` is also available on the Hugging Face Hub (https://huggingface.co/datasets/openai/openai_humaneval). It contains 164 Python programming problems, each with a function signature, docstring, body, and several unit tests. The comments and docstrings are in English. The dataset fields are as follows:

- `task_id`—The identifier of the data sample
- `prompt`—The input for the model, containing function header and docstrings
- `canonical_solution`—A code solution for the problem in the prompt
- `test`—A test function to validate the generated code for correctness
- `entry_point`—The starting point for the tests

You can download this dataset via the Hugging Face `Datasets` API and then transform it into the format expected by the ReCode evaluation. Alternatively, you can download a ready-to-use JSON copy from the ReCode GitHub repo:

```
!mkdir -p datasets/nominal/
%cd ./datasets/nominal/
!wget https://raw.githubusercontent.com/amazon-
➥science/recode/refs/heads/main/datasets/nominal/HumanEval.jsonl
%cd ../..
```

You can run the end-to-end evaluation on this test set with or without a GPU, but hardware acceleration is recommended. On an NVIDIA T4 GPU, the vanilla CodeGen 350M mono model takes more than an hour to run on all 164 test samples (adding the prompt to the answer and setting large maximum generation length). Because the full code is lengthy, we don't include it here; instead, this section walks through the workflow so you can see how generated code is evaluated.

After downloading the CodeGen 340M mono model and its tokenizer from the Hugging Face Hub, split the `HumanEval` JSON samples into individual JSON files, each containing the prompt for a single generation run. For each sample, first tokenize the prompt with the CodeGen tokenizer. Then generate code using the following configuration:

```
from transformers import GenerationConfig

generation_config = GenerationConfig(
    pad_token_id=50256,
    truncation=True,
    max_length=1000,
    max_context_length=1000,
    use_cache=True,
    return_dict_in_generate=True,
    output_scores=True
)
```

After generating the code for a sample, we need to check that it is valid Python. Results that fail validation are not written to the results file. For this check, we use Python's built-in `ast` package to parse the generated code into an abstract syntax tree (AST), as Python does before compiling to bytecode.

To see what an AST looks like, consider this simple example. Suppose the prompt for the generative model is

```
def hello_world():
```

and the generated body for this function is

```
print('Hello world')
```

You can get the latter's AST via the `ast` class's method. In the Colab code, it's called from `is_valid_python`:

```
import ast

def is_valid_python(code):
```

```
    try:
        try:
            pared_code = ast.parse(code)
        except SyntaxError:
            return False
    except Exception as e:
        print("Exception: ", e)
        return False
    return pared_code
```

The returned AST for the generated code looks like this:

```
<ast.Module object at 0x000001458A1D3880>
{'body': [<ast.Expr object at 0x000001458A22FA60>], 'type_ignores': []}
children: [<ast.Expr object at 0x000001458A22FA60>]\n
<ast.Expr object at 0x000001458A22FA60>
{'value': <ast.Call object at 0x000001458A22FA30>, 'lineno': 1,
 'col_offset': 0, 'end_lineno': 1, 'end_col_offset': 20}
children: [<ast.Call object at 0x000001458A22FA30>]\n
<ast.Call object at 0x000001458A22FA30>
{'func': <ast.Name object at 0x000001458A22FA00>, 'args': [<ast.Constant
 object at 0x000001458A22F9D0>], 'keywords': [], 'lineno': 1,
'col_offset': 0, 'end_lineno': 1, 'end_col_offset': 20}
children: [<ast.Name object at 0x000001458A22FA00>, <ast.Constant
object at 0x000001458A22F9D0>]\n
<ast.Name object at 0x000001458A22FA00>
{'id': 'print', 'ctx': <ast.Load object at 0x0000014589ABBC70>,
 'lineno': 1, 'col_offset': 0, 'end_lineno': 1, 'end_col_offset': 5}
children: [<ast.Load object at 0x0000014589ABBC70>]\n
<ast.Constant object at 0x000001458A22F9D0>
{'value': 'hello world', 'kind': None, 'lineno': 1, 'col_offset': 6, 'end_
lineno': 1, 'end_col_offset': 19}
children: []\n
<ast.Load object at 0x0000014589ABBC70>
{}
children: []\n
```

In this case, there are six nodes in the tree. Generating an AST is a standard step in Python compilation. When you run a .py file, Python first converts the source to an AST and then to bytecode (a .pyc file). The Python virtual machine then interprets that bytecode. ASTs are also used for other purposes; for example, popular Python IDEs use them to parse files and understand code structure to power editor features. We can use ASTs in the same way to validate code generated by our LLMs.

After validating the generated Python code with an AST, we collect the results in a single JSON file to simplify evaluation. The final step depends on the `HumanEval` package, so we install it from source because it isn't available through a Python package manager:

```
!git clone https://github.com/openai/human-eval
%cd human-eval
!pip install .
%cd ..
```

For each generated sample, we assess its correctness by executing the code and running its `HumanEval` test. As a safety measure to prevent malicious code execution, `HumanEval` comments out the line in execution.py that runs the generated code. In our Colab notebook, we override part of that code to enable the evaluation of code generated by our model. Use caution when running these evaluations.

We aggregate the results and save them to a JSON file along with the generated code. We also add a `passed` attribute with the value `true` or `false`.

Although the Colab notebook covers only the vanilla CodeGen 350M mono model, you can repeat the same evaluation process and code for its ONNX-exported and 8-bit quantized versions, and for other open source small language models for Python code generation.

7.2.4 Python code generation with better models

When Salesforce released version 2.0 of the CodeGen models, it again offered the mono and multi families in four sizes each, roughly matching the first-generation sizes.

The latest generation is CodeGen 2.5, which comes in a single size—7 billion parameters for both the mono and multi versions. It features robust infill sampling (it can read text on both the left and right side of the current position), and it is optimized for fast sampling with FlashAttention, making it suitable for efficient serving and local deployment, including on personal machines. FlashAttention is an alternative to traditional self-attention that makes LLMs more efficient, enabling faster training and inference. Instead of repeatedly reading and writing keys, queries, and values from GPU HBM (large but relatively slow), it loads them once into SRAM (smaller but faster), fuses the attention operations, and then writes the results back.

All the CodeGen 1 and 2 models were released under the permissive Apache 2.0 license, and CodeGen 2.5 multi and mono use the same license. Note that the CodeGen 2.5 `instruct` model is available for research only, not for commercial use.

Generating Python code with the CodeGen 2.5 mono model works the same way as with the CodeGen 1 mono model discussed earlier in this chapter. First, download the model and tokenizer from the Hugging Face Hub:

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "Salesforce/codegen25-7b-mono"
tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained model_id)
```

You can then generate code by invoking the `generate` method on the tokenized prompt:

```
text = "def hello_world():"
input_ids = tokenizer(text, return_tensors="pt").input_ids
generated_ids = model.generate(input_ids, max_length=128)
print(tokenizer.decode(generated_ids[0], skip_special_tokens=True)
```

As mentioned earlier, CodeGen 2.5 also works well for code infilling—filling in missing parts of an incomplete code snippet. It requires the prompt to follow this formatting convention:

- Insert a `<mask_1>` token at the cursor.
- Add a `<sep>` token to mark the boundary.
- Insert another `<mask_1>` token to specify which mask to infill.

Here's an example:

```
def format(prefix, suffix):
  return prefix + "<mask_1>" + suffix + "<|endoftext|>"
  ➥+ "<sep>" + "<mask_1>"
```

Then generate the body of a Python function, as in the following example:

```
prefix = "def hello_world():\n    "
suffix = "    return greeting_message"
```

We need to format the prompt with our custom `format` function, tokenize it, and then invoke the model's `generate` method:

```
text = format(prefix, suffix)
input_ids = tokenizer(text, return_tensors="pt").input_ids
generated_ids = model.generate(input_ids, max_length=128)
print(tokenizer.decode(generated_ids[0],
➥ skip_special_tokens=False)[len(text):])
```

Other small language models for code generation have been proposed. One strong example is StarCoder2 (https://huggingface.co/bigcode/starcoder2-3b), a 3 billion–parameter model developed by BigCode, an open scientific collaboration focused on the responsible development and use of large language models for code, supported by ServiceNow, Hugging Face, and NVIDIA. Introduced in a 2024 paper (https://arxiv.org/abs/2402.19173), it was trained on 3+ trillion tokens from the Stack v2 dataset (https://huggingface.co/datasets/bigcode/the-stack-v2), covering code in 17 programming languages (including Python) plus curated natural language text from Wikipedia, arXiv, and GitHub issues. StarCoder2 uses grouped-query attention (proposed in a 2023 paper; https://arxiv.org/abs/2305.13245v3) to speed inference, and it has a 16,384-token context window with a 4,096-token sliding-window attention. Because it is not instruction-tuned, it is best suited to generating a function body when the function header is specified in the prompt, but the official GitHub repo (https://github.com/bigcode-project/starcoder2) includes tuning scripts.

You can generate code with StarCoder2 using Hugging Face packages in the usual way. First, download the model and tokenizer from the Hub:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "bigcode/starcoder2-3b"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id,
                                             device_map='auto',
                                             torch_dtype=torch.bfloat16)
model.eval()
```

You can then generate code via the `generate` method of the `AutoModelForCausalLM` class on the tokenized prompt:

```
prompt = "def fibonacci(n):"
inputs = tokenizer.encode(prompt, return_tensors="pt").to(device)
outputs = model.generate(inputs)
print(tokenizer.decode(outputs[0]))
```

The Transformers pipelines are supported too:

```
from transformers import pipeline, GenerationConfig

generation_config = GenerationConfig(
    do_sample=True,
    use_cache=True,
    temperature=0.2,
    top_p=0.95,
    max_length=12
)
pipe = pipeline("text-generation",
                model=model,
                tokenizer=tokenizer,
                generation_config=generation_config
                )

result = pipe(prompt)
print(result[0]['generated_text'])
```

You can create an 8-bit quantized version of the model with Hugging Face's Bitsandbytes library:

```
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(load_in_8bit=True)

model_id = "bigcode/starcoder2-3b"
tokenizer = AutoTokenizer.from_pretrained(model_id)
quantized_model = AutoModelForCausalLM.from_pretrained(model_id,

quantization_config=quantization_config)
quantized_model.eval()
```

It is possible to make a 4-bit quantized version of the model the same way, just replacing the second line with the following:

```
quantization_config = BitsAndBytesConfig(load_in_4bit=True)
```

The companion Colab notebook for this chapter benchmarks inference performance for BF16 and 8-bit quantized versions of the StarCoder v2 3B model. The benchmark code is similar to the CodeGen example shown earlier, so I haven't repeated it here. When you run the benchmark on a GPU, you'll see that (thanks to the grouped-query attention implementation) the model delivers better throughput and latency than the CodeGen 350M and 2B mono models.

7.3 Coding assistance on commodity hardware

The CodeGen and StarCoder models discussed in section 7.2 showed good inference performance (with room for further optimization), making them suitable for deployment behind an API to enable code generation from client applications. It would also be useful to have one of these models running smoothly on a laptop as a coding assistant. In this section, we'll verify an optimized small language model to run locally as a code assistant with acceptable performance. Our target laptop is a MacBook Air with an Apple M1 chip and 8 GB of unified memory. I chose one of the less powerful Apple M-series laptops to make things more challenging and to show that, even with this hardware, you can optimize inference and run useful, domain-specific SLMs smoothly. Our reference model will be StarCoder v2 3B.

If you browse the Hugging Face Hub, you'll find several LLMs (including StarCoder v2 3B) already available in GGUF format. A natural way to try one of these quantized models for laptop inference is to use the Transformers API, just as you would with a standard FP32 or FP16 model:

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "second-state/StarCoder2-3B-GGUF"
filename = "starcoder2-3b-Q2_K.gguf"

tokenizer = AutoTokenizer.from_pretrained(model_id, gguf_file=filename)
model = AutoModelForCausalLM.from_pretrained(model_id, gguf_file=filename)
```

The only difference here is the filename of the quantized model checkpoint to download. GGUF offers multiple quantization options, so several quantized versions of the same model may be available.

Although the Hugging Face Hub can host GGUF models, and the Transformers library can download them, Transformers still can't use models in quantized GGUF format. After a GGUF download completes, the model is automatically unquantized, and inference can run only in the original format. That puts us back at the start, because the original model is typically too large to run locally with limited CPU/GPU and memory.

The only workaround, if we want to go through the Transformers API, is to save the model to disk after downloading:

```
model_checkpoint_dir = 'starcoderv2-checkpoint-dir'
tokenizer.save_pretrained(model_checkpoint_dir)
model.save_pretrained(model_checkpoint_dir)
```

We can then apply a quantization technique to it.

But there are smarter options. llama.cpp (https://github.com/ggerganov/llama.cpp) is an open source C/C++ library and project that enables inference for several LLMs on a wide range of hardware, including devices without a GPU (such as laptops and mobile phones). GGUF is the binary file format used by llama.cpp.

To keep things simple, we will use llama-cpp-python (https://github.com/abetlen/llama-cpp-python), which provides Python bindings for llama.cpp and supports Apple's Metal (MPS, Metal Performance Shaders) backend. To install it, you need Python 3.8+ and a C compiler for your machine (for example, Xcode on macOS). After creating a virtual environment, you can install llama-cpp-python from PyPI:

```
CMAKE_ARGS="-DGGML_METAL=on" pip install llama-cpp-python
```

The preceding command builds and installs the library, including building and installing llama.cpp. You'll also need the huggingface-hub library to download the model:

```
!pip install huggingface-hub
```

Now that everything is set up, we can start coding to make the model available for local inference. First, we'll import the required dependencies:

```
from huggingface_hub import hf_hub_download
from llama_cpp import Llama
```

Then we can download the model from the Hugging Face Hub:

```
model_name = "second-state/StarCoder2-3B-GGUF "
model_file = "starcoder2-3b-Q4_K_M.gguf "
model_path = hf_hub_download(model_name, filename=model_file)
```

The GGUF-converted model used in this example is Second State's 4-bit K_M-quantized version of StarCoder v2 3B (https://huggingface.co/second-state/StarCoder2-3B-GGUF). After downloading it to your laptop, you can load it into memory using the llama-cpp-python library's `Llama` class, as follows:

```
llm = Llama(
    model_path=model_path,
    n_ctx=3072,
    n_threads=4
    n_gpu_layers=0
)
```

We've specified the local path to the model checkpoints, the context size (3,072 for StarCoder v2 3B), the number of CPU cores to use, and how many model layers to offload to the GPU (none in this example, since we want to run inference on the CPU only).

Next, we can set arguments for code generation, such as the maximum number of new tokens, a stop policy, whether to echo the prompt in the output, the decoding strategy, and other inference options:

```
generation_kwargs = {
    "max_tokens":20,
    "stop":["</s>"],
    "echo":False,
    "top_k":1
}
```

Now we can set a prompt and run inference:

```
prompt = "def hello_world():"
res = llm(prompt, **generation_kwargs)
```

The preceding command outputs a dictionary in an OpenAI-compatible format:

```
{
  "id": "cmpl-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "object": "text_completion",
  "created": xxxxxxxxxx,
  "model": "./models/starcoder2-3b-Q4_K_M.gguf",
  "choices": [
    {
      "text": "\n    return 'Hello World!\n\n",
      "index": 0,
      "logprobs": None,
      "finish_reason": "length"
    }
  ],
  "usage": {
    "prompt_tokens": 5,
    "completion_tokens": 20,
    "total_tokens": 25
  }
}
```

To return only the generated text and omit the other response fields, use this command:

```
print(res["choices"][0]["text"])
```

The first time you run this code, it will be slower because it must download the model checkpoints locally. After that first run, you can generate code completely offline.

You can benchmark this local model's performance the same way you did for the CodeGen models. Repeat the inference tests with the same configuration used for the smaller models, and you'll see latency and throughput results like the following:

```
             GGUF 4-bit K_M
Average_latency (ms) 432.545684
Latency_P50          432.409188
Latency_P75          436.077177
Latency_P90          438.191962
Latency_P95          438.622171
Latency_P99          440.215217
Throughput               2.311895
```

The overall performance (using only four threads and no GPU) is comparable to the 4-bit quantized CodeGen model, which is acceptable for local use. You can evaluate the generated code as explained in section 7.2.3.

So far, we've run inference from the CLI by executing a Python script with a prompt. You can also run in chat mode (llama-cpp-python provides an API for this). The library also includes a web server, so you can use llama.cpp-compatible models with any OpenAI-compatible client. This mode makes local models easy to integrate with an IDE for code completion and suggestions.

In this section, we've assumed the code-generation model we need is available from the Hugging Face Hub in GGUF format. But what if the model you want is available only in single or half precision, or it's available in GGUF but the performance isn't great? In those cases, after you've downloaded the source model checkpoints and supporting files locally, you can quantize the model and convert it to GGUF using the `convert_hf_to_gguf.py` script in the llama.cpp GitHub repo:

```
python llama.cpp/convert-hf-to-gguf.py <checkpoints_dir> --outfile
➥<output_file_name>.gguf --outtype <quantization_type>
```

The script requires the directory for the downloaded model checkpoints and files, a name for the target GGUF file, and the quantization type (see the llama.cpp docs for options). You can also upload the resulting GGUF model to the Hugging Face Hub. After quantization, repeat the steps from earlier in this section to generate code locally.

Summary

- Specialized open source code-generation models can match generalist performance with fewer parameters.
- CodeGen models support multiple programming languages, including Python-only code generation via the mono family.
- Converting code generation models to ONNX can reduce average inference time by about 19% compared to standard PyTorch implementations.
- 8-bit quantization of ONNX models can cut inference time by an additional 60% without sacrificing quality.

- Models not supported by Optimum need a mid-level conversion with the `transformers.onnx` package and configuration by hand.
- Abstract syntax tree (AST) parsing checks that generated Python code is syntactically correct before execution.
- The `HumanEval` dataset contains 164 Python programming problems for evaluating code-generation models.
- Code generation models support prompts such as function signatures, natural-language comments, and multiline docstrings.
- StarCoder2 uses grouped-query attention to speed up inference while reducing computational overhead.
- llama-cpp-python provides Python bindings for running GGUF models locally without a GPU.
- Code infilling needs mask and separator tokens so the model understands the context.
- 4-bit quantization cuts model size by about 75% while preserving acceptable code-generation quality.
- Deploying code-generation models locally provides offline coding help without external APIs.
- Apple's Metal Performance Shaders (MPS) backend optimizes inference on Apple Silicon processors.
- Evaluating code generation requires both syntactic validation and unit tests for functional correctness.
- The ReCode evaluation framework provides robustness metrics for code generation under different prompt perturbations.

8 Generating protein structures

This chapter covers

- Using LLMs to generate protein structures
- Using LLMs to generate crystal structures
- Optimizing LLM inference performance for chemistry

Chapter 7 covered practical examples of domain-specific small language models (SLMs) for Python code generation and how to optimize them to run on commodity hardware, even a laptop. This chapter presents real-world examples of specialized SLMs for drug R&D and chemistry.

8.1 Applying Transformers in chemistry

In chapter 7, you saw how to overcome inference challenges using the tips and tricks introduced in the first part of this book. Before we move on to more advanced inference optimization and model quantization in the following chapters, we'll apply these techniques to specialized SLMs in the chemistry domain to show that they aren't limited to creating coding assistants. If chemistry is new to you, don't

worry—we'll keep it high level and cover only what you need to follow the model descriptions and inference optimization strategies we'll apply.

Chemistry is one of the fields that has benefited most from advances in natural language processing (NLP). Researchers have reframed many chemistry tasks as text-sequence problems. Combined with open datasets, this has sparked a wave of work starting with clearly defined fundamental challenges. Early models were single-modal, using molecules as both input and output. More recent work is multimodal, adding data such as spectra from analytical techniques and human language (such as synthesis procedures) to bridge the gap between chemical and natural-language modeling.

Some chemistry terms

Amino acid—Organic molecule used by living organisms to build proteins

Antibody—Immune-system protein that helps fight pathogens such as bacteria and viruses

Oligomer—Molecule made of a small number of repeating units called monomers

Protein—Molecule made of amino acids that supports growth, tissue repair, and other body functions

Reactant—Substance that is changed or consumed in a chemical reaction

Reagent—Substance added to trigger a reaction or test for one (or for a specific chemical)

SMILES—The Simplified Molecular Input Line Entry System, a chemical notation for representing molecular structures in computer-readable form (https://biocyc.org/help.html?object=smiles)

One early and notable effort is the Molecular Transformer (https://pubs.acs.org/doi/10.1021/acscentsci.9b00576). Organic synthesis is a key building block in medicinal chemistry, and a necessary but still unsolved step in synthesis planning is product prediction: given reactants and reagents, what will the final products be? In this work, reaction prediction is treated as machine translation between SMILES strings for reactants, reagents, and products. The model outperforms earlier methods, achieving top-1 accuracy above 90% on a common benchmark dataset. The Molecular Transformer learns correlations between the presence or absence of chemical motifs in the reactants, reagents, and products in the dataset, and it requires no handcrafted rules.

Drug discovery is the area where adopting language models has delivered the biggest gains so far. As mentioned in chapter 1, because they're trained mostly on data scraped from the web, large commercial language models underperform on tasks that require specialized knowledge—knowledge that typically requires domain-specific, often proprietary data that isn't available to tech companies. In other cases, a model needs task-specific tuning, or the data's structure is too complex for large general-purpose

models to be useful. In all these scenarios, the closed nature of large commercial models and the high cost of fine-tuning them make it impossible to achieve the performance needed for drug discovery.

For these reasons, academia and industry have turned to smaller open source pretrained language models. These models provide full control and make it possible to release production-ready models for a diverse set of drug discovery tasks. Each month brings new specialized models, many under permissive open source licenses that often cover not only the source code and model checkpoints, but also the training and test datasets. I maintain a GitHub repo that tracks major fully open source initiatives in this area (https://github.com/virtualramblas/awesome-deep-learning-4-life-sciences).

This chapter will look at notable examples of domain-specific SLMs for protein and crystal structure generation, along with ML/AI engineering considerations and suggestions for improving inference performance.

8.2 From natural language to protein structures

The first SLM we'll explore is ProtGPT2 (https://www.nature.com/articles/s41467-022-32007-7), a model that "speaks" the protein language and can be used for novel protein design and engineering. Sequences generated by ProtGPT2 preserve key features of natural proteins (amino acid propensities, secondary structure content, and globularity) while exploring previously unseen regions of protein space.

ProtGPT2 is a decoder-only GPT-2 model with 738 million parameters. It was pretrained in a self-supervised fashion to predict the next token (oligomer) in a sequence, using 87,162 raw protein sequences from the UniRef50 database (https://www.uniprot.org/help/uniref) and without including the corresponding annotations. The preprocessed training set is available on the Hugging Face Hub (https://huggingface.co/datasets/nferruz/UR50_2021_04). Given a natural-language prompt, it can generate protein sequences. The pretrained model is available on the Hugging Face Hub (https://huggingface.co/nferruz/ProtGPT2).

Let's look at how this model works and use it to generate de novo proteins in a zero-shot setting. A companion Colab notebook is provided for this example. Because the model is available on the Hugging Face Hub, we'll use the Transformers library to download it and set up an inference pipeline:

```
from transformers import pipeline

protgpt2 = pipeline('text-generation', model="nferruz/ProtGPT2")
```

After downloading the model, we can use the preceding pipeline to generate protein sequences:

```
sequences = protgpt2("<|endoftext|>", max_length=100,
                     do_sample=True, top_k=950,
                                     repetition_penalty=1.2,
                     num_return_sequences=10,
                                     eos_token_id=0)
```

This is no different from generating text or code. Here, we asked the model to generate 10 protein sequences; the results look like this:

```
{'generated_text':
'<|endoftext|>\nMGDIPAELIRNWNYYSYNFTLLHITFFLLGTLFHLIQKELKGKKPKSKGKKKQEESVAE
E\nKVKEEPPIMVQGKAKKEKTKTKKRDIQEAEPVPMVEPLPVDELEKLDKEMKKKKKKDKKK\nEKQEIEQKAV
VEQAPVVEQSAAVEQLPVAEQEKQEKKAKAKKSEPEEKKPEVKLKEKKRK\nEKEREETEEKPAEEPQAVEDEEH
EEVAKKSKSKKRKADVQEPEVEEVKVEAEAPKKKKKR\nKQSDDDSEQEKKGKKEKKSKK'}
{'generated_text':
'<|endoftext|>\nMVQPQSASDPPASPPTTDHPLSPLGNPRVVVGRCLLLLALLFHTFTYTWAYAAGSLYCV
A\nVCMPTVLIYQCALLLCLTVSRRHHRVSRGGSRWPSRLVWLRSTKWAVVLLLIFFSLFGLA\nTQCFVIFCHP
TAYSQSTMQHHSPAANITTHCGPNAHNDSHHVALSQWVETAVAWAFVASV\nVAWFASVAVVIPRWTGYCRLCCR
DSASSSNDDASGMEDKWDEEEYEDEE'}
{'generated_text':
'<|endoftext|>\nMPSRPRNPRTVAGAGPVPVAAHAPQHIETHGVRLHYVERGTGPPVVLVHGLGGQIDSMV
G\nALPALAPRHRVVALDQRGHGESDKPATGYGIAEIAADLDGFIAXXXCPAFGCHGRQGSGR\nGGPGRAAAAG
RSSEPRGRGLHARSAQPAQLDHGHRAGPRGGEVRRRGRRTRGCRPSLTPD\nLSTTPRGPA\n'}
{'generated_text':
'<|endoftext|>\nMHIWGCGPSTIGLGINTKNPKTIFFILLITPDVLSISFAIPPKNKSLFSSIKYHVISKK
N\nKFLNISKIGIFLPSSSFCSCLYRCLIRGTTFTFNERPSTQTRSNNHQ\n'}
{'generated_text':
'<|endoftext|>\nMPHPIALAAIAGAPLALCAVLWSHVAALEGQAAWSMAAMSAMLGTVTALRRTRLLGPAA
T\nAAFLAGFGWVALAELAAPPRAAHHLAALGPGAPVLVRVEREARADPVRTRYRLVTGDRAD\nGPGGRAWTAP
RPRLLVWFRAAGPPERWVPGDRAGARLRLMTPRGPTNPGALDAELAALRR\nGIAASGYLRGPAGVSASDPLSWR
RLRGRFERWLDRQAAAAGPRGAVAALATGDEAGIPAP\nDWAVFRDTGTAHLMAVSGLHIGLVAGLAGALAARLY
G'}
{'generated_text':
   '<|endoftext|>\nMKHAVWILLLFYFAETSAQLYSTHFHANPFNPLNMRPAHNSTELFFIPSHSFFGPVISA
Q\nNGFQQNPTAVNQYPTQTAQIYPTYSMYQPSSIPYYQQLTASTTPYNEYTSFPQNTYNTPS\nNYDAYYYPQT
QVYQYYQPSTTQTPMNYRSYNYTTHYPPIQQPTEYASNAYSMYQQQQTPP\nSQSPYNTQSTTTQPSTFSPQETM
LEVSPTPPASNNVPSPVAISPPPQVPPQSASSPISTA'}
{'generated_text':
'<|endoftext|>\nMSPIAWFFLASGSIALLGNVLPLWGPWRPRWLTPLAVVLFATWALQRWEAALFLLVLVV
A\nLIGLAGRAARVLRASLFARRLRGRAAAEVAVEPMDAEAAHAATAAAAAGDQP\n'}
{'generated_text':
'<|endoftext|>\nMSGFVAVLFTATLFICCLIPCLAFIYILGNKDLGKDHPLIYLLSLASGTLCGDAFLHLI
P\nHLLENHETHGTESNGHSNMALWVLLGVLFFFMCEKLFHLITEYKQSHAHAMKISKKTMIV\nGDDAKLYDPV
TTRKINTSKHCCSSSGSPSYSEDTSGETICNNTNENGLLSVNNISNNDFN\nNANKNNHLNKNPNKHGTDNVETN
QNSSETDNKQNKKHSHNQKCNGDLEVSSKGNHRSTAS\nSS'}
{'generated_text':
'<|endoftext|>\nMRALILFLLLCGCAVTTPATTVRTAQPTLEEPGRSERVWLWSTRAPERWLVVRDGTVRG
L\nDPASGRTIWQLAPGEERWGTEAVGDELLVTRAGTASVELAVRAKDLTGRAQLTVSTVPTS\nVPTPSPEPPL
WVVEGDAAPVDRARWGRAAVTLGDQQLVADPRERRVFAVGTPSGGWSLAR\nLDAATGAPLWQTPVAAGQALVAG
AAPGSVFVVGYDRLSAFDAASGAETWAGPVGEREGRG\nPAAAPTATVDGT'}
{'generated_text':
'<|endoftext|>\nMRLIWTFLIFFLTIGRAQDSLNNSRIIPLVTLKDFCNSGISIIPNGENEDELYKIKNKS
L\nANQFERIKYSLNRLDFEIDSIRCDSLNRFNDSLVLYDTSLFRIDSTFGNLEKSKNDIDNK\nISILEKNLKS
LLKSEHLKFSNVITFIVLIFFFIIAVLLWILKRSRNKTIINITINSKKIL\nELDRLEYEKKLTTMENELNQKIA
ENRLLMEQTRLKEIFETKTDSLDSQNQTLSSQNEKII\nAENIRFQEEKALLEQKL'}
```

The model chose to generate sequences that start with the amino acid "M" (we didn't specify a prompt, so the choice was up to the model, but you can specify any other amino acid, oligomer, fragment, or protein as a starting point). Because the model is

compact, generating a batch of 10 sequences takes, on average, less than 2 minutes on Colab's free tier without hardware acceleration. This is acceptable, since these tasks don't necessarily need to happen in real time and the model can run on commodity hardware. For this model, it's probably better to focus on the quality of the generated protein sequences. In the next section, we evaluate a model that, unlike ProtGPT2, needs more attention to inference performance. A widely used metric for assessing the quality of LLM-generated content is the perplexity score, introduced in section 6.2.1. In that section, we also implemented a basic function to calculate perplexity for LLM-generated text. Applying that same function to the 10 generated protein sequences yields the following scores:

```
275.8756703543678
552.265406680504
222.6862045710289
739.1276632256441
202.7109209811726
449.55308995221174
419.3895659171086
385.7640621074327
404.39583964519915
573.9670349466911
```

As with text generation, a lower perplexity score is better.

This domain-specific open SLM shows that with the right data, you do not need a large model to solve real-world, nontrivial tasks.

8.3 Antibody generation with an SLM

Our second SLM in this chapter is the Antibody Generator (also referred to as AntibodyGPT), a specialized protein generation model for creating therapeutic antibody sequences for use in immunology, vaccine development, and medical treatments. Antibodies are proteins that protect us when a foreign substance (called an antigen) enters the body. Produced by the immune system, antibodies bind to these substances to help eliminate them. The Antibody Generator is based on ProGen2, an advanced language model developed by Salesforce and trained on more than 280 million protein sequences–it was introduced in the paper "ProGen2: Exploring the Boundaries of Protein Language Models" (https://arxiv.org/abs/2206.13517).

The ProGen2 family follows a decoder-only architecture and includes four model sizes: small (151 million parameters), medium (765 million parameters), large (2.7 billion parameters), and extra-large (6.4 billion parameters). The Antibody Generator models were built using the ProGen2 small, medium, and large variants as a baseline and were fine-tuned on 5,000 experimentally resolved crystal structures of antibodies and their antigens (see figure 8.1). All three Antibody Generator model options are available on the Hugging Face Hub.

Let's run inference with AntibodyGPT, starting with the small model. A companion Colab notebook is provided for this example, and it requires GPU acceleration.

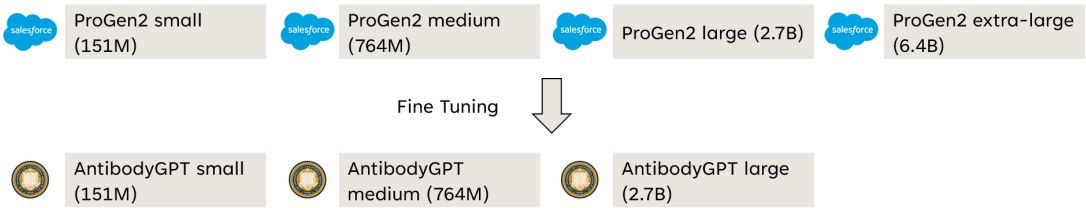


Figure 8.1 The different options available for the ProGen2 and Antibody Generator model families

To download checkpoints for this model (or either of the other two versions) from the Hugging Face Hub and load them into memory, you'll need a custom auto class, `ProGenForCausalLM`. It's implemented in a public GitHub repo by Joseph Roberts (a coauthor of AntibodyGPT): https://github.com/joethequant/docker_protein_generator. First, clone the repo:

```
!git clone https://github.com/joethequant/docker_protein_generator.git
%cd ./docker_protein_generator/
```

We can now download the model and its tokenizer:

```
import torch
from models.progen.modeling_progen import ProGenForCausalLM
from tokenizers import Tokenizer

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

model_id = 'AntibodyGeneration/fine-tuned-progen2-small'
model = ProGenForCausalLM.from_pretrained(model_id)
model = model.to(device)
tokenizer = Tokenizer.from_pretrained(model_id)
```

After we load the model into GPU memory, we can define a target antigen sequence:

```
target_sequence =
'MQIPQAPWPVVWAVLQLGWRPGWFLDSPDRPWNPPTFSPALLVVTEGDNATFTCSFSNTSESFVLNWYRMSPSN
QTDKLAAFPEDRSQPGQDCRFRVTQLPNGRDFHMSVVRARRNDSGTYLCGAISLAPKAQIKESLRAELRVTERRA
EVPTAHPSPSPRPAGQFQTLVVGVVGGLLGSLVLLVWVLAVICSRAARGTIGARRTGQPLKEDPSAVPVFSVDYG
ELDFQWREKTPEPPVPCVPEQTEYATIVFPSGMGTSSPARRGSADGPRSAQPLRPEDGHCSWPL'
```

And we can specify how many antibody sequences to generate:

```
number_of_sequences = 2
```

Before passing the prompt to the model and running inference, we need to tokenize the target sequence and convert it to a PyTorch tensor:

```
tokenized_sequence = tokenizer.encode(target_sequence)
input_tensor = torch.tensor([tokenized_sequence.ids]).to(device)
```

Now we can start sequence generation (inference):

```
with torch.no_grad():
    output = model.generate(input_tensor, max_length=1024,
    pad_token_id=tokenizer.encode('<|pad|>').ids[0],
    do_sample=True, top_p=0.9, temperature=0.8,
    num_return_sequences=number_of_sequences)
```

To get the generated sequences, we need to decode the model output as follows:

```
as_lists = lambda batch: [batch[i, ...].detach().cpu().numpy().tolist()
➥for i in range(batch.shape[0])]
sequences = tokenizer.decode_batch(as_lists(output))
if len(sequences) > 0:
    sequences = [x.replace('2', '') for x in sequences]
```

Although this model is smaller than ProtGPT, generating just two target sequences quickly requires a GPU (inference with the same model and settings takes a couple of hours on the Colab free tier without hardware acceleration). Under these conditions, the benchmark shows the following average latency:

```
('P95 latency (ms) - 137426.0512944001; Average latency (ms)
- 119778.24 +\\- 27239.48;',
 137426.0512944001)
```

In the CUDA allocator configuration, the expandable-segment allocation is set to `false` by default. This can fragment the allocator's memory segments, preventing them from expanding later during inference for reuse. You can test whether changing the CUDA caching allocator improves latency:

```
%env PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

Rerunning the benchmark after applying the preceding setting achieves a slightly lower average latency:

```
('P95 latency (ms) - 128515.5656411501; Average latency (ms)
 - 103638.09 +\\- 29278.26;',
 128515.5656411501)
```

Before optimizing this model, let's look at how we can validate its generated results. Perplexity and similar statistical metrics make little sense on their own in cases like this. A more reliable option is the ANARCI tool (https://opig.stats.ox.ac.uk/webapps/sabdab-sabpred/sabpred/anarci/), developed and maintained by the Oxford Protein Informatics Group (OPIG), for antibody numbering and receptor classification.

ANARCI checks whether generated antibody sequences conform to known patterns and structures, helping us assess structural viability and alignment with established frameworks. The goal is for sequences to be not only novel but also biologically relevant and potentially functional. You can submit a generated antibody sequence to the ANARCI online tool; figure 8.2 shows the results for one of the sequences from our example.

Figure 8.2 The output of the online ANARCI tool for one of the antibody sequences generated in our example

In many domains, it's common to validate generated content with a dedicated tool. Subject matter experts should always review it. You can also perform this validation programmatically using the Python `ANARCI` API (https://github.com/oxpig/ANARCI/tree/master).

Next, let's see whether the model benefits from quantization. Using the Bitsandbytes library, we can quantize it to 8-bit. First, we'll define a quantization configuration:

```
from transformers import BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
  load_in_8bit=True,
)
```

Now we'll use it via the `ProGenForCausalLM` auto class:

```
model_id="AntibodyGeneration/fine-tuned-progen2-small"
model_int8 = ProGenForCausalLM.from_pretrained(model_id,
                                               quantization_config=bnb_config,
                                                )
tokenizer = Tokenizer.from_pretrained(model_id)
```

As expected, the 8-bit quantized model is smaller than the original (156.73 MB versus 617 MB). Running the same benchmark as for the original model, we can observe the following performance:

```
('P95 latency (ms) - 80453.97859845006; Average latency (ms)
- 61846.30 +\\- 18507.29;',
 80453.97859845006)
```

The average latency is typically about 40% lower than in the vanilla models' benchmarks after enabling dynamic, expandable segments.

8.4 From CIF files to crystal structures

The third SLM we'll explore is CrystaLLM from the "Crystal Structure Generation with Autoregressive Large Language Modeling" paper (https://arxiv.org/abs/2307.04340). It's a Transformer-based, decoder-only LLM built to generate crystal structures. Previously, crystal structure-prediction methods—often based on computationally intensive ab initio techniques—typically relied on dedicated tools to optimize candidate structures.

CrystaLLM is the first model trained on textual representations of crystallographic information files (CIFs), rather than traditional chemical representations. The authors argue that symbol sequences can represent complex chemical structures and that LLMs can learn deeper causal relationships beyond simple correlations. Trained on extensive CIF data, the model can generate physically plausible inorganic crystal structures. The authors also added Monte Carlo tree search (MCTS) and a graph-based neural network to improve structural sampling.

During training, the model is given token sequences from a corpus of CIF files and learns to predict the next token. After training, it can generate new CIF files conditioned on a starting token sequence. Generating a CIF file involves repeatedly sampling tokens from the model, conditioning on the accumulated generated content, until a terminating condition is reached.

Figure 8.3 (from the previously cited paper) provides a high-level view of CrystaLLM's training and inference processes. As you can see, during training, a CIF file is tokenized into a symbol sequence. The model processes the sequence, and it outputs, for each input symbol, a probability distribution over the vocabulary. These predicted distributions are compared with the target distributions (all the probability mass on the correct next token) using the cross-entropy loss metric. The targets are the input tokens

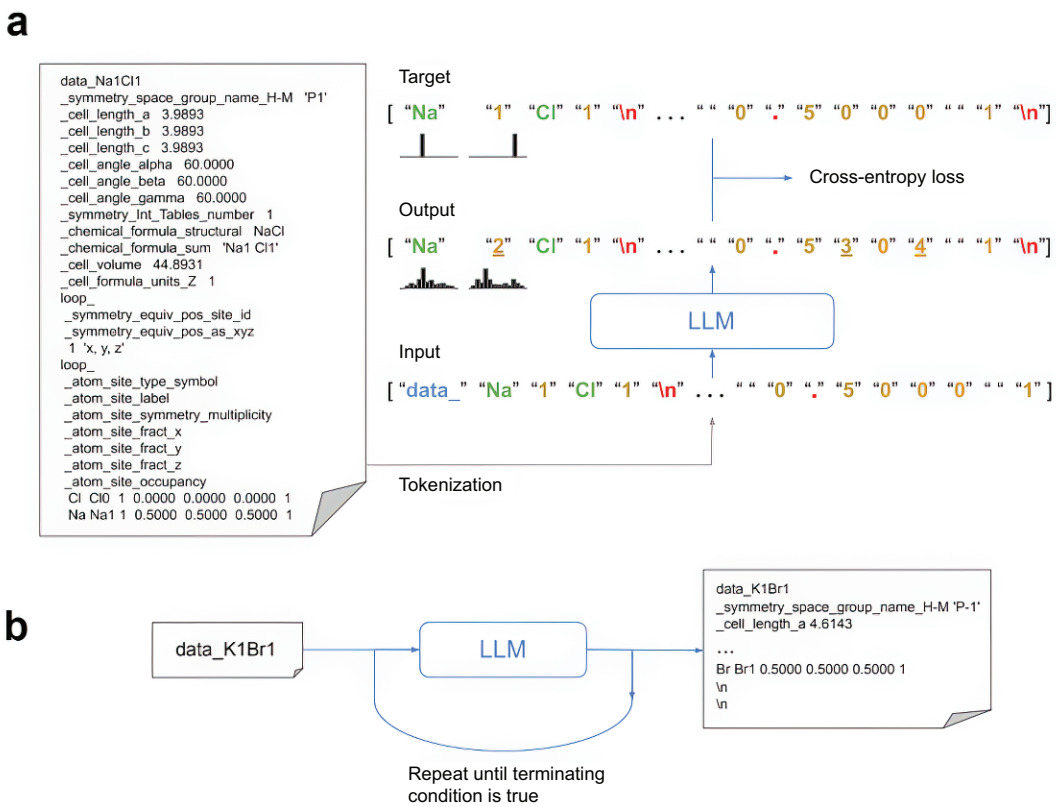


Figure 8.3 The CrystaLLM training process (a) and the generation process with MCTS decoding (b)

shifted one position left, since the task is to predict the next token from the preceding tokens (as in many LLM content-generation tasks). The tokens are grouped into CIF tags, atoms, numeric digits, and punctuation. The output tokens (not sampled during training) represent those with the highest assigned probability. Underlined tokens indicate cases where the correct next token receives relatively low probability.

Once the model is trained, it generates a CIF file from a prompt. The prompt is built by concatenating the symbol data with the target cell composition, and it's then tokenized and fed to the model. At each step, the model samples the next token from the predicted distribution and appends it to the CIF contents. This repeats until a predefined stopping condition is reached. The model is based on GPT-2; the model definition, training, and inference code are adapted from Andrej Karpathy's nanoGPT repository (https://github.com/karpathy/nanoGPT).

CrystaLLM can generate plausible crystal structures for a wide range of inorganic compounds, including ones it didn't see during training. It generates syntactically valid, physically plausible structures and can predict crystal structures given chemical

composition and symmetry information. It also generalizes to unseen inorganic structures, achieving structural matches in up to 40% of cases, and can extrapolate structural motifs to new compositions. This work is important because it has shown potential to accelerate material discovery by generating candidate crystal structures for downstream optimization.

We'll skip the remaining training details. In this chapter, we'll use the pretrained model and focus only on the inference process. A companion Colab notebook is provided for this section.

To set up the environment, we'll first clone the CrystaLLM GitHub repository:

```
!git clone https://github.com/lantunes/CrystaLLM.git
%cd CrystaLLM
```

Next, we'll make the cloned source code directories part of the Python path:

```
import sys
import os

sys.path.append('/content/CrystaLLM')
os.environ["PYTHONPATH"] += (":/content/CrystaLLM")
```

Once you've installed the missing requirements, download the pretrained model weights. Two versions of CrystaLLM are available: small (25 million parameters) and large (200 million parameters). These weights aren't available on the Hugging Face Hub, so download them using the following Python script (here we're requesting the small version):

```
!python bin/download.py crystallm_v1_small.tar.gz
!tar xvf crystallm_v1_small.tar.gz
```

We're now ready to use it to generate crystal structures. First, we need to create a prompt, which in this case is the partial contents of a preprocessed CIF file. Typically, the prompt contains the cell composition: a formula specifying the total count of each atom type in the unit cell. For example, we can specify Na2Cl2 (disodium dichloride), which indicates four atoms in the unit cell, and use the provided `make_prompt_file.py` script to prepare and save the prompt for later use:

```
!python bin/make_prompt_file.py Na2Cl2 my_prompt.txt
```

To prompt the model with a cell composition, the format should use the `data_` prefix and a newline character at the end of the line:

```
data_<cell_composition>\n
```

The `make_prompt_file.py` script handles this formatting convention (and more). In our example, the model is prompted with `data_Na2Cl2\n`.

We can now generate the CIF files; we'll use the provided `sample.py` script:

```
!python bin/sample.py \
out_dir=crystallm_v1_small \
start=FILE:sample_prompt.txt \
num_samples=2 \
top_k=10 \
max_new_tokens=3000 \
device=cpu \
target=file
```

This script takes the following arguments:

- `out_dir`—The directory containing the downloaded model checkpoints.
- `start`—Either a prompt in the required format or the path to a file containing the prompt.
- `num_samples`—The number of samples to generate.
- `top_k`—The maximum number of most likely tokens to keep.
- `max_new_tokens`—The maximum number of tokens to generate per sample.
- `device`—Where to offload the model weights: CPU or GPU.
- `target`—Where the generated content should be sent. In our example, it's `file`. Alternatively, it can be `console`.

These are some other arguments we haven't used here:

- `temperature`—This has the same meaning as in any Transformer model.
- `seed`—Sets the random seed for reproducibility; defaults to `1,337`.
- `dtype`—Model precision; defaults to `bfloat16`.
- `compile`—Boolean flag indicating whether to compile the model with PyTorch before CIF generation for faster execution. Defaults to `False`.

No hardware acceleration is needed for this small model. With the chosen configuration (3,000 tokens max per sequence), generating two CIF files on a Colab CPU VM takes under a minute on average. Here is one of the generated CIF files from this run:

```
data_Na2Cl2
loop_
_atom_type_symbol
_atom_type_electronegativity
_atom_type_radius
_atom_type_ionic_radius
Na 0.9300 1.8000 1.1600
Cl 3.1600 1.0000 0.7800
_symmetry_space_group_name_H-M P4/nmm
_cell_length_a 4.9038
_cell_length_b 4.9038
_cell_length_c 3.9603
_cell_angle_alpha 90.0000
_cell_angle_beta 90.0000
```

```
_cell_angle_gamma 90.0000
_symmetry_Int_Tables_number 129
_chemical_formula_structural NaCl
_chemical_formula_sum 'Na2 Cl2'
_cell_volume 95.2964
_cell_formula_units_Z 2
loop_
_symmetry_equiv_pos_site_id
_symmetry_equiv_pos_as_xyz
1 'x, y, z'
loop_
_atom_site_type_symbol
_atom_site_label
_atom_site_symmetry_multiplicity
_atom_site_fract_x
_atom_site_fract_y
_atom_site_fract_z
_atom_site_occupancy
Na Na0 2 0.0000 0.0000 0.0000 1
Cl Cl1 2 0.0000 0.5000 0.9817 1
```

The model's output isn't usable yet: the model produces raw CIF files that require postprocessing. A Python script is provided; you'll need to run it on the generated files as follows:

```
!python bin/postprocess.py . colab_processed_cifs
```

The `postprocess.py` script takes two arguments: the directory containing the generated raw CIF files and the output directory for the postprocessed files. After postprocessing, the raw CIF file shown previously looks like this:

```
data_Na2Cl2
_symmetry_space_group_name_H-M P4/nmm
_cell_length_a 4.9038
_cell_length_b 4.9038
_cell_length_c 3.9603
_cell_angle_alpha 90.0000
_cell_angle_beta 90.0000
_cell_angle_gamma 90.0000
_symmetry_Int_Tables_number 129
_chemical_formula_structural NaCl
_chemical_formula_sum 'Na2 Cl2'
_cell_volume 95.2964
_cell_formula_units_Z 2
loop_
 _symmetry_equiv_pos_site_id
 _symmetry_equiv_pos_as_xyz
  1  'x+1/2, y+1/2, -z'
  2  '-y, x, -z'
  3  '-x, y, z'
  4  'y+1/2, -x+1/2, z'
  5  'x+1/2, -y+1/2, -z'
```

```
  6  '-y+1/2, x+1/2, z'
  7  '-x, -y, z'
  8  'x, y, z'
  9  'y, x, -z'
  10  '-x+1/2, y+1/2, -z'
  11  '-y, -x, -z'
  12  '-x+1/2, -y+1/2, -z'
  13  'y, -x, -z'
  14  '-y+1/2, -x+1/2, z'
  15  'x, -y, z'
  16  'y+1/2, x+1/2, z'
loop_
_atom_site_type_symbol
_atom_site_label
_atom_site_symmetry_multiplicity
_atom_site_fract_x
_atom_site_fract_y
_atom_site_fract_z
_atom_site_occupancy
Na Na0 2 0.0000 0.0000 0.0000 1
Cl Cl1 2 0.0000 0.5000 0.9817 1
```

Now we need to validate the generated files. A validation script is provided, and it expects the input CIF files in a `.tar.gz` archive:

```
!tar -czvf colab_processed_cifs.tar.gz ./colab_processed_cifs/
```

Trigger the validation as follows:

```
!python bin/evaluate_cifs.py colab_processed_cifs.tar.gz -o
➥colab_processed_cifs.csv
```

The `evaluate_cifs.py` script takes two arguments: the name of the archive containing the CIF files to evaluate and the name of the CSV file where the results should be saved. The results are also printed to standard output at the end:

```
space group consistent: 2/2 (1.000)
atom site multiplicity consistent: 2/2 (1.000)
avg. bond length reasonableness score: 1.0000 ± 0.0000
bond lengths reasonable: 2/2 (1.000)
num valid: 2/2 (1.00)
longest valid generated length: 631
avg. valid generated length: 560.500 ± 70.500
```

Among other things, the output's info and stats show that both generated CIF files are valid.

As described at the start of section 8.4 (see figure 8.3), the CrystaLLM paper proposes a second way to generate CIF files: run the LLM inside an MCTS loop. A script is also provided. You can run MCTS-based decoding as follows:

```
!python bin/mcts.py \
out_dir=crystallm_v1_small \
device=cpu \
dtype=bfloat16 \
start=FILE:sample_prompt.txt \
tree_width=5 \
max_depth=2000 \
selector=puct \
c=1.0 \
num_simulations=1000 \
reward_k=2.0 \
scorer=random \
top_child_weight_cutoff=0.9999 \
bypass_only_child=True \
mcts_out_dir=colab_mcts_cifs
```

`mcts.py` uses the same model, precision, and sample prompt file as standard generation. It also takes MCTS-specific arguments: tree width, maximum depth, number of simulations, and rollouts per simulation.

If you want to upload a CrystaLLM pretrained model (or your fine-tuned version) to a Hugging Face Hub repository and use it with the Transformers library, follow these steps. We show the small model here; the large model follows the same steps but uses different configuration values.

First, create a configuration file, `config.json`, as required by the Transformers auto classes. You can copy most values from `config/crystallm_v1_small.yaml` in the CrystaLLM GitHub repository (there is a similar file for the large model).

```
crystall_small_config = """{
  "bias": true,
  "model_type": "gpt2",
  "block_size": 1024,
  "dropout": 0.1,
  "n_embd": 512,
  "n_head": 8,
  "n_layer": 8,
  "vocab_size": 50257
}"""

with open('/content/CrystaLLM/crystallm_v1_small/config.json', 'a') as f:
    f.write(crystall_small_config)
```

Next, convert the model checkpoint from `.pt` to `.bin`, which is required to load it into an auto class:

```
import torch

checkpoint = torch.load("/content/CrystaLLM/crystallm_v1_small/ckpt.pt",
                    map_location=torch.device('cpu'))
params = checkpoint["model"]
torch.save(params,
➥"/content/CrystaLLM/crystallm_v1_small/pytorch_model.bin")
```

Note that instead of accessing the `model_state_dict` key, we access the `model` key directly, because this pretrained model stores its weights under that key in the checkpoint dictionary.

Now we can wrap the model in a `Model` class, provide the generated configuration, and make it available through the Transformers library:

```
from transformers import AutoConfig, AutoModelForCausalLM

config = AutoConfig.from_pretrained(
            '/content/CrystaLLM/crystallm_v1_small/config.json')
transformer_model = AutoModelForCausalLM.from_pretrained(
            '/content/CrystaLLM/crystallm_v1_small', config=config)
```

Then we save the `Model` to disk (to generate the remaining files):

```
transformer_model.save_pretrained
➥("/content/CrystaLLM/crystallm_v1_small_hf")
```

It can also be uploaded to the Hugging Face Hub if needed. First, in Colab, create a new secret with the name `HF_TOKEN`. Copy your access token key from your Hugging Face Hub profile settings, and paste it into the `Value` field of the `HF_TOKEN`. Then toggle the key icon on the left side of the Colab UI to let the notebook access the secret. Finally, load the token programmatically:

```
import os
from google.colab import userdata

os.environ["HF_TOKEN"] = userdata.get("HF_TOKEN")
```

You can then use it to log in to the Hugging Face Hub (this requires the `huggingface_hub` package to be installed):

```
from huggingface_hub import login

login(os.environ["HF_TOKEN"])
```

You can now push the model to the Hub:

```
transformer_model.push_to_hub("crystallm_v1_small_hf")
```

That concludes this part of the book showing real-life examples of local, domain-specific models. The chapters in the next part cover more advanced optimization techniques and finally look at deployment on different hardware platforms and integrating small, specialized language models into larger systems.

Summary

- Chemistry language models use NLP techniques to represent molecular structures as text sequences for drug discovery.
- ProtGPT2 generates protein sequences with a decoder-only GPT-2 model trained on 87,162 UniRef50 protein sequences.
- Protein generation uses `endoftext` prompts to produce amino acid sequences that resemble natural proteins.
- Perplexity scoring evaluates generated protein sequences, with lower scores indicating higher-quality output.
- AntibodyGPT fine-tunes ProGen2 on 5,000 experimentally resolved crystal structures for generating therapeutic antibodies.
- The `ProGenForCausalLM` class loads ProGen2-based models from custom repositories and runs inference.
- CUDA memory-allocation settings can reduce inference latency by enabling expandable segments for better memory management.
- 8-bit quantization shrinks the AntibodyGPT model from 617 MB to 156 MB and improves average latency by 40%.
- The ANARCI tool validates generated antibody sequences that conform to known antibody structural patterns.
- CrystaLLM generates crystal structures from text representations of crystallographic information files.
- CIF file generation involves tokenizing crystallographic data and using autoregressive prediction to complete structures.
- Monte Carlo tree search improves sampling quality for generating crystal structures.
- Validating chemical structures requires domain-specific tools rather than standard text-generation metrics.
- Domain-specific models achieve effective results with smaller parameter counts than generalist language models.
- Specialized chemistry models require subject-matter experts to validate biological relevance and viability.

Part 4

Advanced concepts

This last part of the book covers the most advanced concepts. Chapters 9 and 10 explain advanced quantization techniques for larger SLMs and one way of implementing friendly model profiling insights. Chapters 11 and 12 showcase multiple ways to deploy and serve SLMs in hardware-constrained environments, laptops, and devices (specifically presenting options for the Android OS). Chapters 13 and 14 cover the integration of SLMs with RAG and agentic AI. Finally, chapter 15 talks about test-time compute and SLMs, and it concludes the book with an end-to-end example of tuning an SLM for reasoning on a specific domain through GRPO.

9 Advanced quantization techniques

This chapter covers

- Using the FlexGen technique to offload part of an LLM to memory or disk
- Using SmoothQuant, an advanced post-training quantization technique, to reduce memory footprint and accelerate inference
- Using BitNet, a scalable 1-bit Transformer architecture, to reduce memory footprint and energy consumption

Typically, domain-specific language models are small: in my professional experience, they're usually no more than 7 or 8 billion parameters (excluding the additional parameters from LoRA or QLoRA partial tuning, which are small anyway). The techniques explained in previous chapters make such models practical to deploy and use in computationally constrained environments. Still, a larger pool of specialized training data, unstructured training data with representations larger than natural language, or a larger baseline model that better fits the task may result

in a final model size that exceeds the available inference hardware capacity, even after quantization.

This chapter introduces advanced LLM quantization techniques that can help you run such specialized models efficiently in target environments, balancing quality and speed. The sections that follow describe these strategies and compare their pros and cons. You'll also find detailed code examples and benchmarks.

> **NOTE** This chapter covers advanced LMM quantization techniques for cases where the methods in chapter 6 can't meet performance or compute constraints. Before continuing, make sure you understand chapter 6.

9.1 FlexGen

FlexGen is a high-throughput inference engine for running LLMs in situations where GPU memory is limited. Introduced in the 2023 paper "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU" (https://arxiv.org/abs/2303.06865), FlexGen can be configured for different hardware constraints by pooling memory and compute across the GPU, CPU, and disk. It searches for efficient ways to store and access tensors by solving a linear programming problem, and it further compresses the weights and attention cache to 4 bits with negligible accuracy loss. This expands the feasible batch-size range and increases the maximum throughput for a model. The paper reports substantially higher throughput than with traditional offloading methods.

To give you an idea of what FlexGen enables, consider a GPT-3–sized model with 175 billion parameters. Using the formula in section 4.2, the VRAM required to load it is 840 GB in FP32 or 420 GB in FP16. This means that to load the model and make it available for inference, you'd need, for example, six NVIDIA A100 Tensor Core GPUs (see figure 9.1).

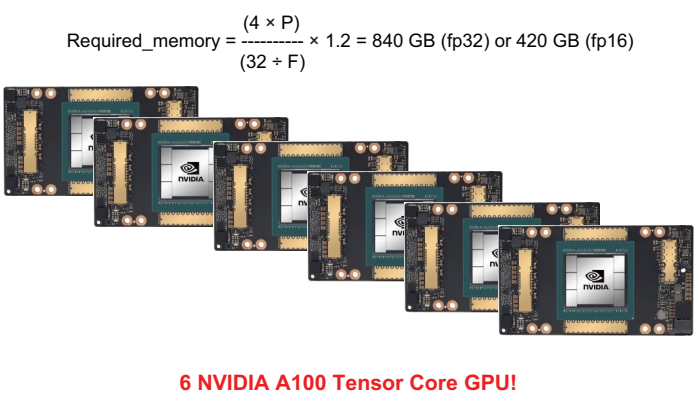


Figure 9.1 The hardware needed to load a 175 billion–parameter language model into memory

Given the price of the hardware, running a single model this way can be very expensive—often beyond the budget of small and mid-sized organizations and even some large non-tech ones. Assume a node with one consumer GPU (16 GB DRAM) but more RAM and disk space, which are cheaper and more available than GPUs. The FlexGen linear programming optimizer finds the best way to store and access the model's weights, activations, and attention KV cache across the GPU, CPU, and disk. As a result, parts of the model are offloaded, as shown in figure 9.2 (assuming the model is loaded in FP16 format). This approach is definitely cheaper than buying or renting six NVIDIA A100 GPUs.

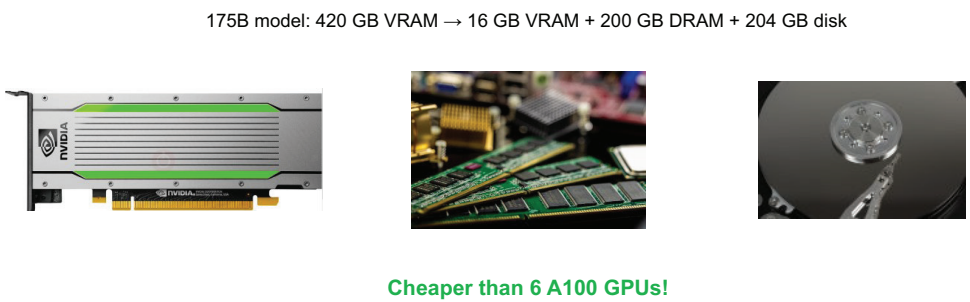


Figure 9.2 Loading a 175 billion–parameter language model with FlexGen across GPU, CPU, and disk

This kind of offloading lets us deploy a very large model in a hardware-constrained environment, but it increases latency and reduces throughput. That's expected: CPUs and disks are slower than GPUs, and offloading adds extra I/O.

FlexGen goes beyond basic offloading: during generative inference, it decides which tensors (weights, activations, and KV cache) to offload, where to place them, and when to move them to improve throughput while keeping I/O efficient. It further boosts inference performance by compressing both weights and the KV cache to 4 bits with negligible accuracy loss. This is an extreme example using a very large model—when you fine-tune a model for domain-specific tasks, you typically won't reach this size—but it illustrates the benefits of the FlexGen approach in simple terms.

> **NOTE** FlexGen supports only NVIDIA GPUs via CUDA and has no plans to support other vendors' hardware acceleration.

Let's look at how FlexGen works in practice and see whether it delivers on the paper's claims. A companion Colab notebook includes the code for this section.

FlexGen is open source and available on GitHub (the repository is named FlexLLMGen; https://github.com/FMInference/FlexLLMGen/tree/main), and you can install it from PyPI without building from source. It includes prebuilt implementations for Facebook's OPT (https://arxiv.org/abs/2205.01068) and Galactica

(https://arxiv.org/pdf/2211.09085) models and BigScience's BLOOM (https://arxiv.org/abs/2211.05100), plus models fine-tuned from these baselines. In this section, we focus on OPT, a suite of decoder-only pretrained transformers ranging from 125M to 175B parameters. The paper reports that OPT-175B is comparable to GPT-3 while requiring only one-seventh the carbon footprint to develop. All OPT models are hosted on the Hugging Face Hub.

This example, in the accompanying notebook, requires hardware acceleration and runs on Colab's free-tier GPU VM. We'll start with the OPT-1.3B model to get a sense of FlexGen. After installing the FlexGen code package (as shown in the notebook), you can evaluate FlexGen on one of the supported models from the CLI by running the `flex_opt` Python script as follows:

```
python -m flexgen.flex_opt --model facebook/opt-1.3b
```

In its simplest form, `flex_opt` expects the `model` argument, which is the ID of the target model on the Hugging Face Hub. After downloading the model, the script converts the weights to FlexGen based on the selected model's configuration, then loads or offloads its weights, attention cache, and activations to the destination storage (by default, if you don't specify otherwise at runtime, everything is loaded to the GPU). Finally, it runs a small benchmark. The output of the command looks like this:

```
TorchDevice: cuda:0
  cur_mem: 2.6505 GB,  peak_mem: 3.2478 GB
TorchDevice: cpu
  cur_mem: 0.0000 GB,  peak_mem: 0.0000 GB
model size: 2.443 GB    cache size: 0.398 GB    hidden size (p): 0.008 GB
peak gpu mem: 3.248 GB    projected: False
prefill latency: 0.350 s    prefill throughput: 5846.976 token/s
decode latency: 0.932 s    decode throughput: 132.989 token/s
total latency: 1.283 s    total throughput: 99.791 token/s
```

As you can see, everything runs on the GPU because the model fits in the device available in our execution environment. Now let's try a larger model that won't fit on our GPU, such as OPT-6.7B. Another key argument for the `flex_opt` script is `percent`. It lets us specify what percentage of weights, attention cache, and activations to load onto the GPUs and what percentage to offload to the CPU. It expects a string of six integers:

- The percentage of weights on GPU
- The percentage of weights on CPU
- The percentage of attention cache on GPU
- The percentage of attention cache on CPU
- The percentage of activations on GPU
- The percentage of activations on CPU

The default values are `100 0 100 0 100 0`, which keeps everything on the GPU. If a larger model doesn't fit on your GPU, you can run the script and specify the following offloading policy:

```
python -m flexgen.flex_opt --model facebook/opt-6.7b
➥--percent 50 50 50 50 50 50
```

The CLI is a quick way to evaluate FlexGen. For models that run inference as part of a larger ecosystem, however, it's usually better to call the FlexGen API programmatically alongside the Hugging Face Transformers API. For example, to run a text completion task with a model that doesn't fit on your GPU, start by importing the required FlexGen classes (which will be discussed as we use them):

```
from flexgen.flex_opt import (Policy, OptLM, ExecutionEnv,
➥CompressionConfig, str2bool)
```

Next, download the target model's tokenizer from the Hugging Face Hub:

```
from transformers import AutoTokenizer

model_id = "facebook/opt-6.7b"
tokenizer = AutoTokenizer.from_pretrained(model_id, padding_side="left")
tokenizer.add_bos_token = False
stop = tokenizer("\n").input_ids[0]
```

Before we initialize the model with FlexGen, we need to set up a few things. First, create the execution environment for the converted model using the `ExecutionEnv` class and its `create` method. This method takes the name of the directory where FlexGen can offload data that shouldn't go in GPU memory or RAM:

```
offload_dir = './flexgen_offload'
env = ExecutionEnv.create(offload_dir)
```

We also need to set up FlexGen offloading and compression policies. The `Policy` class handles both. Here's how to use it to set up everything we need:

```
policy = Policy(len(prompts), 1,
                50, 50, 50, 50, 100, 0,
                overlap=True, sep_layer=True, pin_weight=True,
                cpu_cache_compute=True, attn_sparsity=1.0,
                compress_weight=True,
                comp_weight_config=CompressionConfig(
                    num_bits=4, group_size=64,
                    group_dim=0, symmetric=False),
                compress_cache=True,
                comp_cache_config=CompressionConfig(
                    num_bits=4, group_size=64,
                    group_dim=2, symmetric=False)
                )
```

When creating an instance of the `Policy` class, we pass the offloading percentages (as with the CLI command) and specify whether to compress weights and the weight compression settings, whether to compress the cache, and the cache compression settings.

Next, we'll create an `OptLM` instance for the target model by passing the Hugging Face Hub model ID, the execution-environment instance, the local cache path (after it's downloaded from the hub), and the configured policies. The `OptLM` class takes care of any steps required to download and convert the target model:

```
path = '~/opt_weights'
model = OptLM(model_id, env, path, policy)
```

The FlexGen-prepared model is now ready for few-shot text-completion tasks. After you set up a list of example prompts, you can run completion as follows:

```
inputs = tokenizer(prompts, padding="max_length", max_length=128)
output_ids = model.generate(
    inputs.input_ids,
    do_sample=True,
    temperature=0.7,
    max_new_tokens=32,
    stop=stop)
outputs = tokenizer.batch_decode(output_ids, skip_special_tokens=True)
print("Outputs:\n" + 70 * '-')
for i in [0, len(outputs)-1]:
    print(f"{i}: {outputs[i]}")
    print("-" * 70)
```

LLMs prepared for FlexGen are fully supported by the Hugging Face's Transformers library. When you're done generating text completions, you can shut down the execution environment:

```
env.close_copy_threads()
```

FlexGen also supports multiple GPUs, which helps when the model you need to run might not fit in VRAM on your available hardware.

To support other families of open source LLMs, you can create a new Python script that follows the structure of `flexgen.opt_config.py`. In it, specify the target model architecture configuration, and implement utilities to download its weights from the Hub.

FlexGen provides advanced offloading and compression out of the box, and it delivers higher throughput on large batches than other approaches covered so far in this book, such as Accelerate and DeepSpeed. But it's optimized for throughput-focused scenarios (LLM applications running inference on millions of tokens per batch). If latency is your main KPI, FlexGen isn't the best way to run a larger model on budget hardware; you'll need to consider other optimization options instead.

9.2 SmoothQuant

SmoothQuant is a post-training quantization method that improves LLM efficiency while preserving accuracy. Traditional quantization methods often focus on weights and can struggle with activation outliers, especially in larger models. These outliers can hurt performance after quantization. SmoothQuant (introduced in the paper of the same name; https://arxiv.org/abs/2211.10438) addresses this by enabling 8-bit weight and activation quantization without additional training. It does so by smoothing activation outliers via a mathematical transformation, effectively shifting quantization difficulties from activations to weights. This makes it possible to quantize a range of LLM architectures successfully. According to experiments in the paper, SmoothQuant can deliver up to 1.56 times speed increases and a 50% reduction in memory usage with minimal accuracy loss. This can enable the deployment of extremely large models, potentially even on a single node, depending on the node's hardware specs. SmoothQuant is compatible with the PyTorch framework.

Before going hands-on with SmoothQuant, let's look at the main quantization challenge it addresses. The techniques described in chapter 6 work well for LLMs up to about 6 or 7 billion parameters. Beyond that threshold, systematic activation outliers often appear, which can degrade accuracy after quantization. In section 6.2.2, I introduced the LLM.int8() technique, which mitigates this problem with a mixed-precision decomposition (it keeps outliers in FP16 while using INT8 for the remaining activations). This technique is already integrated into popular Python libraries, including some we use throughout this book. But for larger models, this decomposition is difficult to implement efficiently on hardware accelerators. In those cases, it's better to use a different training-free quantization scheme that uses INT8 for all compute-intensive operations. SmoothQuant migrates the quantization difficulty from activations to weights by applying a mathematically equivalent, per-channel scaling transformation that smooths magnitudes across channels and makes the model more quantization-friendly. Figure 9.3, taken from the original SmoothQuant paper, illustrates the idea.

Figure 9.3 (top) shows activation (left) and weight (right) distributions for an LLM with a large activation outlier. This outlier stretches the quantization range, making quantization difficult. Figure 9.3 (bottom) shows the same model after SmoothQuant shifts scale variance from activations to weights offline (before inference). The activations are smoothed and the

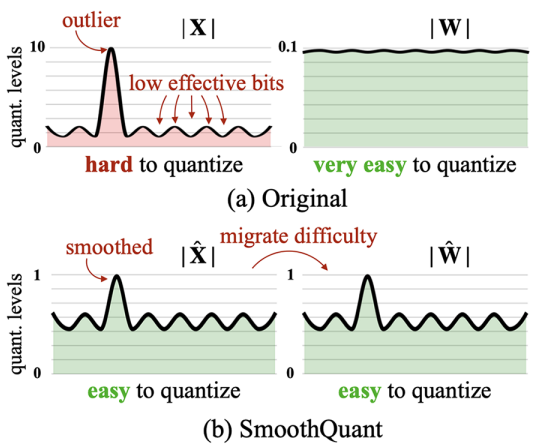


Figure 9.3 A high-level explanation of SmoothQuant

weights adjusted, making both easier to quantize. SmoothQuant can preserve accuracy after quantization while reducing serving costs.

Let's look at an example; the complete code is available in a companion Colab notebook. We'll see that, for a given LLM, SmoothQuant can use INT8 for both activations and weights while achieving accuracy comparable to FP16 on the same model. SmoothQuant has been tested on diverse LLM families, including Llama, OPT, Mistral, Mixtral, and Falcon. For PyTorch, SmoothQuant provides INT8 inference with CUTLASS INT8 GEMM kernels, wrapped as PyTorch modules in the `torch-int` (https://github.com/Guangxuan-Xiao/torch-int) Python package. SmoothQuant has also been integrated into inference engines such as Intel's Neural Compressor, NVIDIA's TensorRT-LLM, and Amazon SageMaker (we'll cover LLM inference engines in chapter 11). To keep the focus on how SmoothQuant works in code, we'll simulate 8-bit dynamic per-tensor quantization of weights and activations using FP16. All examples in this section require hardware acceleration.

Our first target model is Facebook's OPT-2.7b (https://huggingface.co/facebook/opt-2.7b). We'll start with a model that has fewer than 6 billion parameters to see whether the SmoothQuant strategy benefits smaller models.

Let's first install SmoothQuant from the source code:

```
pip install git+https://github.com/mit-han-lab/smoothquant.git
```

Next, download the model's FP16 weights and its tokenizer from the Hugging Face Hub:

```
import torch
from transformers.models.opt.modeling_opt import OPTForCausalLM
from transformers import GPT2Tokenizer

model_id = 'facebook/opt-2.7b'
tokenizer = GPT2Tokenizer.from_pretrained(model_id)
model_fp16 = OPTForCausalLM.from_pretrained(model_id,
                                            torch_dtype=torch.float16,
                                            device_map='auto',
                                           )
```

Now we'll load a dataset to evaluate different versions of the target model:

```
from datasets import load_dataset

dataset = load_dataset('cimec/lambada', split='validation[:1000]')
```

In this example, we use the first 1,000 samples from the `LAMBADA` dataset (https://huggingface.co/datasets/cimec/lambada), available on the Hugging Face Hub. `LAMBADA` is a collection of narrative passages used to evaluate text understanding via a word-prediction task. The dataset has two columns: text, the narrative passage, and `domain`, a label for the narrative genre. After evaluating the FP16 model on this dataset

(for example, using LM-Eval, introduced in section 4.1.4, or another tool), we can determine the model accuracy:

```
Original model (fp16) accuracy: 0.759
```

Let's quantize this model. We'll use a custom function adapted from the SmoothQuant repo.

Listing 9.1 A custom function to quantize an LLM's weights and activations to INT8

```
from transformers.models.opt.modeling_opt import OPTAttention,
➥OPTDecoderLayer, OPTForCausalLM
from smoothquant.fake_quant import W8A8Linear

def quantize_model(model, weight_quant='per_tensor',
➥act_quant='per_tensor', quantize_bmm_input=True):
    for name, m in model.model.named_modules():
        if isinstance(m, OPTDecoderLayer):
            m.fc1 = W8A8Linear.from_float(m.fc1, weight_quant=weight_quant,
                                          ➥act_quant=act_quant)
            m.fc2 = W8A8Linear.from_float(m.fc2, weight_quant=weight_quant,
                                          ➥act_quant=act_quant)
        elif isinstance(m, OPTAttention):
            m.q_proj = W8A8Linear.from_float(
                m.q_proj, weight_quant=weight_quant, act_quant=act_quant,
                                ➥quantize_output=quantize_bmm_input)
            m.k_proj = W8A8Linear.from_float(
                m.k_proj, weight_quant=weight_quant, act_quant=act_quant,
                                ➥quantize_output=quantize_bmm_input)
            m.v_proj = W8A8Linear.from_float(
                m.v_proj, weight_quant=weight_quant, act_quant=act_quant,
                                ➥quantize_output=quantize_bmm_input)
            m.out_proj = W8A8Linear.from_float(m.out_proj,
                    ➥weight_quant=weight_quant, act_quant=act_quant)
    return model
```

This function replaces the linear layers in self-attention and activation with SmoothQuant's W8A8 (8-bit weight, 8-bit activation) implementation.

We can now apply the preceding function to the original model:

```
model_w8a8 = quantize_model(model_fp16)
```

Compare the original FP16 model structure, shown here

```
OPTForCausalLM(
  (model): OPTModel(
    (decoder): OPTDecoder(
      (embed_tokens): Embedding(50272, 2560, padding_idx=1)
      (embed_positions): OPTLearnedPositionalEmbedding(2050, 2560)
      (final_layer_norm): LayerNorm((2560,), eps=1e-05,
      ➥elementwise_affine=True)
```

```
      (layers): ModuleList(
        (0-31): 32 x OPTDecoderLayer(
          (self_attn): OPTSdpaAttention(
            (k_proj): Linear(in_features=2560, out_features=2560, bias=True)
            (v_proj): Linear(in_features=2560, out_features=2560, bias=True)
            (q_proj): Linear(in_features=2560, out_features=2560, bias=True)
            (out_proj): Linear(in_features=2560, out_features=2560,
                       ➥bias=True)
          )
          (activation_fn): ReLU()
          (self_attn_layer_norm): LayerNorm((2560,), eps=1e-05,
                                  ➥elementwise_affine=True)
          (fc1): Linear(in_features=2560, out_features=10240, bias=True)
          (fc2): Linear(in_features=10240, out_features=2560, bias=True)
          (final_layer_norm): LayerNorm((2560,), eps=1e-05,
                              ➥elementwise_affine=True)
        )
      )
    )
  )
  (lm_head): Linear(in_features=2560, out_features=50272, bias=False)
)
```

with the 8-bit quantized version to verify that the self-attention and activation layers were replaced correctly:

```
OPTForCausalLM(
  (model): OPTModel(
    (decoder): OPTDecoder(
      (embed_tokens): Embedding(50272, 2560, padding_idx=1)
      (embed_positions): OPTLearnedPositionalEmbedding(2050, 2560)
      (final_layer_norm): LayerNorm((2560,), eps=1e-05,
      ➥ elementwise_affine=True)
      (layers): ModuleList(
        (0-31): 32 x OPTDecoderLayer(
          (self_attn): OPTSdpaAttention(
            (k_proj): W8A8Linear(2560, 2560, bias=True,
                    ➥weight_quant=per_tensor, act_quant=per_tensor,
                    ➥output_quant=per_tensor)
            (v_proj): W8A8Linear(2560, 2560, bias=True,
                    ➥weight_quant=per_tensor, act_quant=per_tensor,
                    voutput_quant=per_tensor)
            (q_proj): W8A8Linear(2560, 2560, bias=True,
                    ➥weight_quant=per_tensor, act_quant=per_tensor,
                    ➥output_quant=per_tensor)
            (out_proj): W8A8Linear(2560, 2560, bias=True,
                    ➥weight_quant=per_tensor, act_quant=per_tensor,
                    voutput_quant=None)
          )
          (activation_fn): ReLU()
          (self_attn_layer_norm): LayerNorm((2560,), eps=1e-05,
                     ➥elementwise_affine=True)
          (fc1): W8A8Linear(2560, 10240, bias=True,
                     ➥weight_quant=per_tensor, act_quant=per_tensor,
```

```
                    ➥output_quant=None)
          (fc2): W8A8Linear(10240, 2560, bias=True,
                    ➥weight_quant=per_tensor, act_quant=per_tensor,
                    ➥output_quant=None)
          (final_layer_norm): LayerNorm((2560,), eps=1e-05,
                    ➥elementwise_affine=True)
        )
      )
    )
  )
  (lm_head): Linear(in_features=2560, out_features=50272, bias=False)
)
```

If we repeat the evaluation on the same 1,000 `LAMBADA` samples for the quantized model, we can see that the accuracy remains comparable to the original FP16 version:

```
Naive W8A8 quantized model accuracy: 0.758
```

This is expected: the model is large enough that its activations don't contain large outliers.

So far, we've quantized without smoothing the model's activations. Now let's apply smoothing before quantization:

```
act_scales = torch.load('./act_scales/opt-2.7b.pt')
smooth_lm(model_fp16, act_scales, 0.5)
model_smoothquant_w8a8 = quantize_model(model_fp16)
```

The `smooth_lm` function takes as input the original model, the model scales (available for this and many other models on the Hugging Face Hub; https://huggingface.co/mit-han-lab/smoothquant-scales/tree/main), and `alpha`, a hyperparameter that controls the migration strength, that is, how much quantization difficulty is shifted from activations to weights. The SmoothQuant paper defines the smoothing factor s_j for a given activation X_j and weight W_j as a function of `alpha`:

$$s_j = \frac{\max(|X_j|)^{\alpha}}{\max(|W_j|)^{1-\alpha}}$$

For this example—and for models under 100 billion parameters in general—we can keep `alpha = 0.5`.

If we re-evaluate this model on the same dataset as in the previous versions, we can see that the accuracy is unchanged and is comparable to the original FP16 model:

```
SmoothQuant W8A8 quantized model accuracy: 0.759
```

Let's see how things change with a larger model—let's try OPT-6.7B. Repeating the same evaluation on its FP16 version, we can observe the following accuracy:

```
Original model (fp16) accuracy: 0.798
```

After 8-bit quantization with no smoothing, the model's accuracy on the same evaluation dataset drops sharply:

```
Naive W8A8 quantized model accuracy: 0.423
```

This confirms the presence of at least one large outlier in the model's activations. With activation smoothing applied before quantization, accuracy under the same evaluation is comparable to FP16:

```
SmoothQuant W8A8 quantized model accuracy: 0.799
```

As you've seen, SmoothQuant is a powerful strategy for reducing compute costs without affecting model performance, especially once specialized models exceed 6 to 7 billion parameters. As mentioned earlier, several LLM families are supported out of the box; others require some code customization to adapt the smoothing and quantization steps to the architecture.

9.3 BitNet

BitNet is a scalable 1-bit Transformer architecture designed to reduce the cost and energy use of large language models. It replaces traditional linear layers with a component called BitLinear, which allows the training of 1-bit weights from scratch. Experimental findings indicate that BitNet, proposed for the first time in "BitNet: Scaling 1-bit Transformers for Large Language Models" (https://arxiv.org/abs/2310.11453), shows competitive language-modeling performance with much lower memory use and energy consumption than leading 8-bit quantization methods and FP16 transformer benchmarks. BitNet preserves high precision for optimizer states and gradients to maintain training stability, and it scales to larger models, adhering to a scaling law like full-precision transformers. The paper also describes group quantization for model parallelism, improving efficiency without additional communication overhead. Overall, BitNet has the potential to lower inference costs and improve performance, making it a promising alternative to other quantization methods.

Figure 9.4, from the original BitNet paper, shows the BitLinear component (left) and the BitNet architecture (right), which uses BitLinear in the attention layers. As you can see, the BitNet architecture has the same layout as a transformer. The main difference is that it uses BitLinear in the attention layers instead of matrix multiplication. This binarizes the attention weights (1-bit), while the rest of the model uses higher precision (8-bit). This reduces computational costs while preserving precision for the input/output embeddings, since the model still uses high-precision probabilities for sampling.

The BitNet paper also reports on several comparison experiments. Figure 9.5 compares the scaling curves of BitNet and FP16 transformers.

For smaller models, traditional FP16 transformers achieve lower loss than BitNet. As model size increases, the gap narrows: the loss delta is already small at around 30 billion parameters and drops to 0.09 near 100 billion.

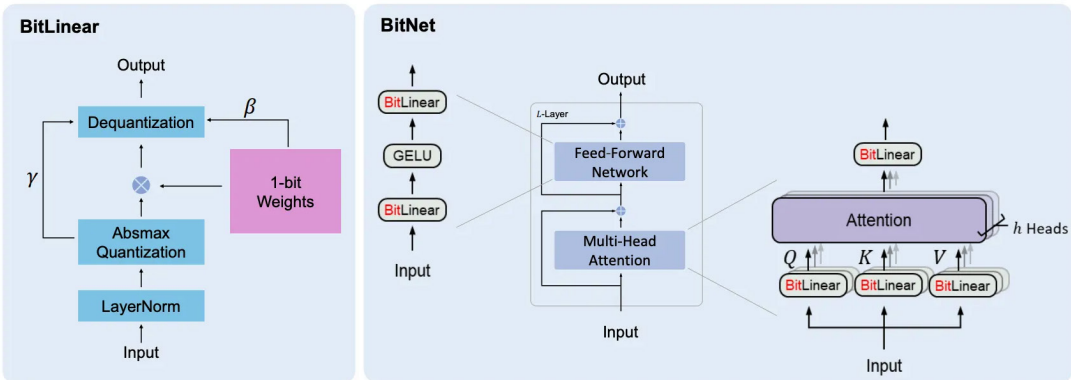


Figure 9.4 The BitLinear component (left) and the BitNet architecture (right)

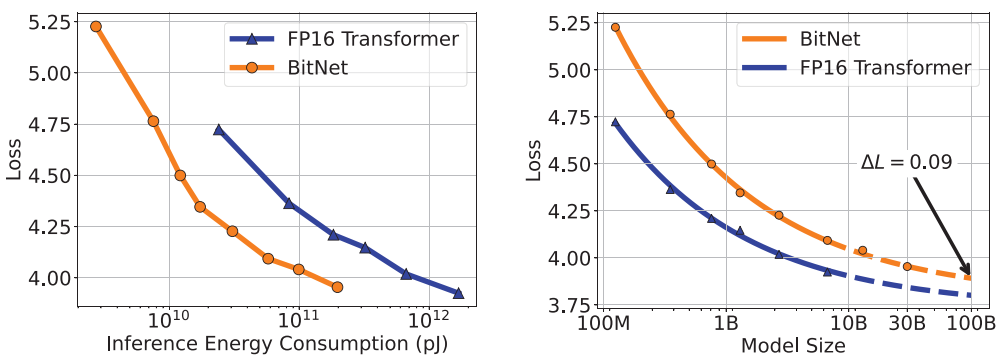


Figure 9.5 Comparing the scaling curves for BitNet and FP16 transformers

The biggest challenge in training low-bit transformers is optimization stability. The paper's authors tested both BitNet and an FP16 baseline by training a series of models with varying peak learning rates. Figure 9.6 shows the results.

On the left of figure 9.6, BitNet converges with a high learning rate, whereas FP16 transformers fail to converge, indicating better training stability. On the right, BitNet converges when training with different learning rates—its perplexity improves as the learning rate increases.

The BitNet paper also compares this architecture with other post-training quantization (PTQ) methods, including Absmax, SmoothQuant, GPTQ, and QuIP (a 2-bit method described in "QuIP: 2-Bit Quantization of Large Language Models With Guarantees," https://arxiv.org/abs/2307.13304). Figure 9.7, from the BitNet paper, compares the accuracy of an FP16 transformer with the same model after PTQ using different techniques, including BitNet, on zero-shot (left) and few-shot (right) downstream tasks.

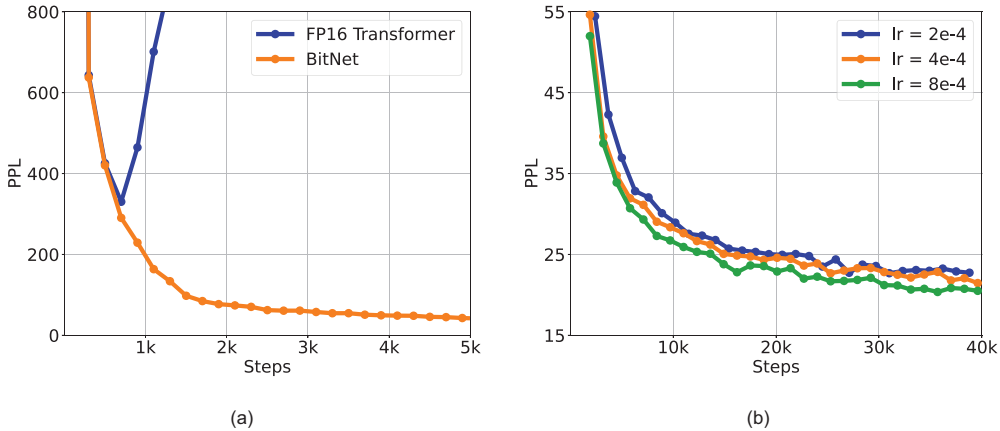


Figure 9.6 The results of the stability tests for BitNet and FP16 transformers

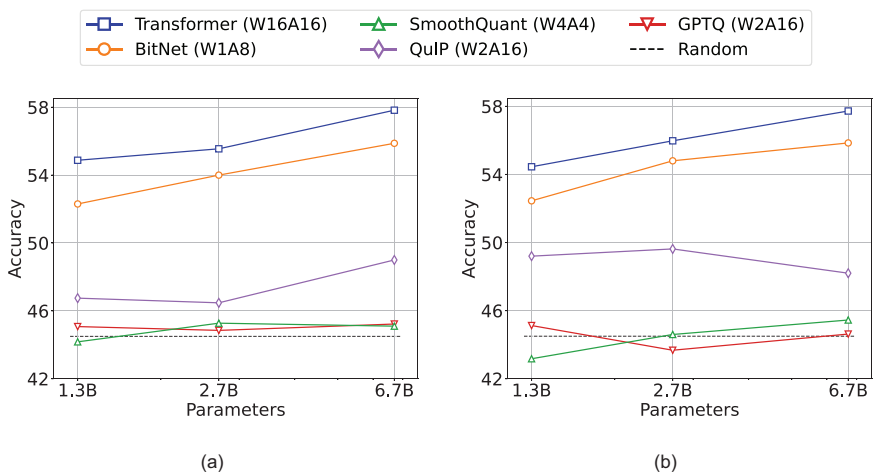


Figure 9.7 Average accuracy on downstream tasks in zero-shot (a) and few-shot (b) settings for vanilla FP16 transformers, BitNet, and various PTQ methods

The different techniques have been evaluated on multiple datasets; the table in figure 9.8 (from the BitNet paper) summarizes the zero-shot results.

The graphs and table in figures 9.7 and 9.8 show that BitNet's average performance (accuracy and perplexity) is slightly lower than the FP16 Transformer baseline, but it's much better than any other PTQ scenario.

WBits	Methods	PTQ	PPL↓	WG↑	WGe↑	HS↑	SC↑	Avg↑
-	Random	✗	-	50.0	50.0	25.0	50.0	43.8
16	Transformer	✗	15.19	66.7	54.3	42.9	67.4	57.8
8	Absmax	✓	21.43	60.4	52.0	38.3	62.7	53.4
	SmoothQuant	✓	15.67	65.3	53.1	40.9	67.6	56.7
4	GPTQ	✓	16.05	57.2	51.2	39.9	63.4	52.9
	Absmax	✓	4.8e4	55.8	50.9	25.0	53.1	46.2
	SmoothQuant	✓	1.6e6	53.7	48.3	24.8	53.6	45.1
2	GPTQ	✓	1032	51.6	50.1	25.8	53.4	45.2
	QuIP	✓	70.43	56.1	51.2	30.3	58.4	49.0
1	Absmax	✓	3.5e23	49.8	50.0	24.8	53.6	44.6
	SmoothQuant	✓	3.3e21	50.5	49.5	24.6	53.1	44.4
1	**BitNet**	✗	17.07	66.3	51.4	38.9	66.9	55.9

Figure 9.8 Zero-shot results for BitNet and a baseline Transformer (FP16), plus results after applying various quantization techniques

9.4 BitNet and Python

The BitNet framework is based on llama.cpp (introduced in chapter 6), and its core is implemented in C++. It provides optimized kernels for fast, energy-efficient, lossless inference for 1-bit models on CPUs. It doesn't yet support GPUs, but that's planned. You can install BitNet from source; follow the instructions for your OS in the official Microsoft GitHub repository (https://github.com/microsoft/BitNet).

BitNet also provides Python bindings, so you can run inference in Python on BitNet-quantized models hosted on the Hugging Face Hub. The main Python script, `run_inference.py`, uses the following syntax:

```
python run_inference.py -m models/
➥Falcon3-7B-Instruct-1.58bit/ggml-model-i2_s.gguf -p "You are a helpful
➥assistant" -cnv
```

In its basic form, it takes as arguments the ID of the target model in the Hugging Face Hub, a prompt, and a flag to enable or disable chat mode (valid for instructed models only). You can also pass other arguments to set the temperature, the maximum number of tokens to generate, the number of threads to use, and the prompt context size. At the time of writing, BitNet supports its native models (0.7B and 3.3B parameters) and the Llama and Falcon families. If you'd rather not use C++ and port pretrained models yourself, you can use a full Python alternative: a PyTorch port with CUDA support. The source code is available on GitHub: https://github.com/kyegomez/BitNet. It's also available on PyPI and can be installed from there:

```
pip install bitnet
```

This package provides PyTorch implementations of BitLinear, the BitNet Transformer, and BitNet Attention and FeedForward.

With this API, you can create a simple BitNet Transformer as follows:

```
from bitnet import BitNetTransformer

bitnet = BitNetTransformer(
    num_tokens=20000,
    dim=1024,
    depth=6,
    heads=8,
    ff_mult=4,
)
```

We use the `BitNetTransformer` class and specify the number of unique input tokens, the embedding dimension, the number of layers and attention heads, and the feed-forward hidden-dimension multiplier. The resulting model architecture follows:

```
BitNetTransformer(
  (emb): Embedding(20000, 1024)
  (transformer): Transformer(
    (layers): ModuleList(
      (0-5): 6 x BitMGQA(
        (q_proj): BitLinear(in_features=1024, out_features=1024, bias=True)
        (k_proj): BitLinear(in_features=1024, out_features=512, bias=True)
        (v_proj): BitLinear(in_features=1024, out_features=512, bias=True)
        (norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
        (out_proj): BitLinear(in_features=512, out_features=1024, bias=True)
      )
    )
    (ffn_layers): ModuleList(
      (0-5): 6 x BitFeedForward(
        (ff): Sequential(
          (0): Sequential(
            (0): BitLinear(in_features=1024, out_features=4096, bias=True)
            (1): SiLU()
          )
          (1): LayerNorm((4096,), eps=1e-05, elementwise_affine=True)
          (2): Dropout(p=0.1, inplace=False)
          (3): BitLinear(in_features=4096, out_features=1024, bias=True)
        )
      )
    )
  )
  (to_logits): Sequential(
    (0): RMSNorm()
    (1): Linear(in_features=1024, out_features=20000, bias=True)
  )
)
```

We can easily identify all the PyTorch implementations provided by this package. This package integrates with the Hugging Face Transformers library and supports

pretrained LLMs hosted on the Hugging Face Hub, not just the few model families supported by the original BitNet framework. Let's try it with ProtGPT2, a model we discussed in section 8.2. You can download the pretrained model and tokenizer from the Hugging Face Hub using the Transformers API:

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = 'nferruz/ProtGPT2'
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id)
```

Next, we'll use the BitNet API to replace linear layers that have BitNet implementations:

```
from bitnet import replace_linears_in_hf

replace_linears_in_hf(model)
```

Regardless of the pretrained LLM architecture, when a model is used through the Transformers API, we always call `replace_linears_in_hf`. As you'll see next, the head linear layers have been replaced:

```
GPT2LMHeadModel(
  (transformer): GPT2Model(
    (wte): Embedding(50257, 1280)
    (wpe): Embedding(1024, 1280)
    (drop): Dropout(p=0.1, inplace=False)
    (h): ModuleList(
      (0-35): 36 x GPT2Block(
        (ln_1): LayerNorm((1280,), eps=1e-05, elementwise_affine=True)
        (attn): GPT2SdpaAttention(
          (c_attn): Conv1D()
          (c_proj): Conv1D()
          (attn_dropout): Dropout(p=0.1, inplace=False)
          (resid_dropout): Dropout(p=0.1, inplace=False)
        )
        (ln_2): LayerNorm((1280,), eps=1e-05, elementwise_affine=True)
        (mlp): GPT2MLP(
          (c_fc): Conv1D()
          (c_proj): Conv1D()
          (act): NewGELUActivation()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((1280,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): BitLinear(in_features=1280, out_features=50257, bias=False)
)
```

We can now run inference as usual with the Transformers API, including using a pipeline:

```
from transformers import pipeline

pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
)

sequences = pipe("<|endoftext|>",
                 max_length=100,
                 do_sample=True,
                 top_k=950,
                 repetition_penalty=1.2,
                 num_return_sequences=10,
                 eos_token_id=0)
```

The Python BitNet package also provides a drop-in replacement for PyTorch models:

```
import torch
from torch import nn
from bitnet import replace_linears_in_pytorch_model

model = nn.Sequential(
    nn.Linear(10, 20),
    nn.ReLU(),
    nn.Linear(20, 30),
)

replace_linears_in_pytorch_model(model)
```

In the preceding example, all the PyTorch linear layers have been replaced with BitNet linear layers.

It is also possible to run inference with existing PyTorch checkpoints:

```
from bitnet import BitNetInference

bitnet = BitNetInference()
bitnet.load_model("../model_checkpoint.pth")
output_str = bitnet.generate("The dog jumped over the ", 512)
```

BitNet could be a viable alternative to other quantization strategies when there are computational or sustainability constraints.

This concludes our look at advanced model-quantization techniques. For many production use cases with local small language models, traditional quantization methods are a good fit. But for niche tasks involving more complex, unstructured content, you may want to use one of these advanced strategies.

Summary

- FlexGen distributes model weights, activations, and the attention cache across the GPU, CPU, and disk, enabling large-model inference on limited hardware.
- FlexGen uses linear programming to find efficient patterns for storing and accessing tensors across different storage types.
- FlexGen compresses weights and the attention cache to 4 bits, increasing throughput with minimal accuracy loss.
- In FlexGen, the `Policy` class sets offloading percentages and compression for weights, cache, and activations.
- FlexGen prioritizes throughput over latency, making it best for batch processing with millions of tokens.
- SmoothQuant addresses activation outliers in large language models by shifting the hard part of quantization from activations to weights.
- SmoothQuant uses mathematical transformations to smooth activation magnitudes across channels before quantization.
- Models larger than 6 to 7 billion parameters develop systematic activation outliers that degrade quantization performance.
- SmoothQuant preserves accuracy when 8-bit quantizing weights and activations in large models.
- SmoothQuant replaces standard linear layers to enable 8-bit weight and activation quantization.
- BitNet introduces BitLinear components that replace traditional linear layers with 1-bit weights.
- BitNet keeps optimizer states and gradients in high precision while using 1-bit weights in its attention layers.
- BitLinear components cut computational costs while preserving precision for input/output embeddings.
- BitNet trains more stably and converges more reliably than traditional FP16 transformers at high learning rates.
- BitNet achieves competitive performance while using far less memory and energy than 8-bit quantization.
- Python BitNet implementations offer PyTorch-compatible replacements for linear layers in existing models.

10 Profiling insights

This chapter covers

- Profiling ONNX ported models
- Transforming raw ONNX profiling data into insights
- Optimizing ONNX graphs for LLMs

Chapter 5 introduced the ONNX format and ONNX Runtime capabilities, first in general and then for LLMs. Chapter 6 covered several approaches, including 8-bit quantization through the ONNX API. This chapter will explore other ONNX features, such as profiling the performance of LLMs ported to ONNX format and tools you can use to extract useful insights from the profiling data.

10.1 Profiling ONNX-ported LLMs

The ONNX Runtime (ORT) delivers high performance for running machine learning (ML) and deep learning (DL) models across a wide range of hardware. Still, to meet specific key performance indicators (KPIs) for latency, throughput, and memory use, you may need additional model optimizations and runtime configuration for a given use case, model, and device.

The ORT supports in-code performance profiling. It's disabled by default, but you can enable it during debugging by setting the `enable_profiling` option to `True` in an ORT inference session:

```
import onnxruntime as rt

sess_options = rt.SessionOptions()
sess_options.enable_profiling = True
```

At runtime, ORT generates a JSON file with performance data such as threading and per-operator latency.

Let's take a look at what's in this profiling file, starting with a simple model and then moving on to LLMs. We'll reuse the linear regression example from section 5.2:

```
from onnx import TensorProto
from onnx.helper import (
    make_model, make_node, make_graph,
    make_tensor_value_info)
from onnx.checker import check_model

X = make_tensor_value_info('X', TensorProto.FLOAT, [None, None])
A = make_tensor_value_info('A', TensorProto.FLOAT, [None, None])
B = make_tensor_value_info('B', TensorProto.FLOAT, [None, None])
Y = make_tensor_value_info('Y', TensorProto.FLOAT, [None])

node1 = make_node('MatMul', ['X', 'A'], ['XA'])
node2 = make_node('Add', ['XA', 'B'], ['Y'])
graph = make_graph([node1, node2], 'lr', [X, A, B], [Y])
onnx_model = make_model(graph)
check_model(onnx_model)
```

To run inference with this model, we need to create an inference session and specify the execution provider (CPU; no hardware acceleration needed):

```
from onnxruntime import InferenceSession

sess = InferenceSession(onnx_model.SerializeToString(),
                        providers=["CPUExecutionProvider"])
```

We'll test it with three input arrays of random values:

```
import numpy as np

res = sess.run(None, {
    'X': np.random.randn(2, 2).astype(np.float32),
    'A': nup.random.randn(2, 2).astype(np.float32),
    'B': np.random.randn(2, 2).astype(np.float32)
  }
)
```

To measure this model's performance (and spot areas for improvement), we'll run inference with profiling enabled. There are no other code changes:

```
import onnxruntime as rt
from onnxruntime import InferenceSession

sess_options = rt.SessionOptions()
sess_options.enable_profiling = True
sess = InferenceSession(onnx_model.SerializeToString(),
                        sess_options,
                        providers=["CPUExecutionProvider"])
```

To access the generated profiling data, we need to use the `end_profiling` method:

```
prof = sess.end_profiling()
```

This method returns the name of the generated JSON file, which follows the `onnxruntime_profile__YYYY-MM-DD_hh-mm-ss.json` naming convention. The file contents look like this:

```
[
{"cat" : "Session","pid" :21460,"tid" :14412,"dur" :36164,"ts" :7,
"ph" : "X","name" :"model_loading_array","args" : {}},
{"cat" : "Session","pid" :21460,"tid" :14412,"dur" :1430,"ts" :36425,
"ph" : "X","name" :"session_initialization","args" : {}},
{"cat" : "Node","pid" :21460,"tid" :14412,"dur" :2,"ts" :34744604,
"ph" : "X","name" :"MatMul_0_fence_before","args" : {"op_name" : "MatMul"}},
{"cat" : "Node","pid" :21460,"tid" :14412,"dur" :164,"ts" :34744616,
"ph" : "X","name" :"MatMul_0_kernel_time","args" : {"parameter_size" :
"0","provider" : "CPUExecutionProvider","op_name" : "MatMul","input_type_
shape" : [{"float":[2,2]},{"float":[2,2]}],"node_index" : "0",
"output_type_shape" : [{"float":[2,2]}],"activation_size" :
 "32","output_size" : "16","thread_scheduling_stats" : {"main_thread":
 {"thread_pool_name": "session-2-intra-op", "thread_id": "14412",
"block_size": [], "core": -1, "Distribution": 0, "DistributionEnqueue": 0,
"Run": 0, "Wait": 0, "WaitRevoke": 0}, "sub_threads": {"20796": {"num_run":
0, "core": -1}}}}},
{"cat" : "Node","pid" :21460,"tid" :14412,"dur" :0,"ts" :34744793,"ph" :
"X","name" :"MatMul_0_fence_after","args" : {"op_name" : "MatMul"}},
{"cat" : "Node","pid" :21460,"tid" :14412,"dur" :0,"ts" :34744806,"ph" :
"X","name" :"Add_1_fence_before","args" : {"op_name" : "Add"}},
{"cat" : "Node","pid" :21460,"tid" :14412,"dur" :56,"ts" :34744808,"ph" :
"X","name" :"Add_1_kernel_time","args" : {"parameter_size" : "0","provider"
: "CPUExecutionProvider","op_name" : "Add","input_type_shape" :
[{"float":[2,2]},{"float":[2,2]}],"node_index" : "1","output_type_shape" :
[{"float":[2,2]}],"activation_size" : "32","output_size" :
"16","thread_scheduling_stats" : {"main_thread": {"thread_pool_name":
"session-2-intra-op", "thread_id": "14412", "block_size": [], "core": -1,
"Distribution": 0, "DistributionEnqueue": 0, "Run": 0, "Wait": 0,
"WaitRevoke": 0}, "sub_threads": {"20796": {"num_run": 0, "core": -1}}}}},
{"cat" : "Node","pid" :21460,"tid" :14412,"dur" :0,"ts" :34744879,"ph" :
"X","name" :"Add_1_fence_after","args" : {"op_name" : "Add"}},
```

```
{"cat" : "Session","pid" :21460,"tid" :14412,"dur" :781,"ts" :34744111,"ph"
: "X","name" :"SequentialExecutor::Execute","args" : {}},
{"cat" : "Session","pid" :21460,"tid" :14412,"dur" :828,"ts" :34744075,"ph"
: "X","name" :"model_run","args" : {}}
]
```

The JSON profiling file contains raw data for an ONNX inference session, broken down by operator and execution provider. On its own, this data provides little insight, so you need to aggregate and transform it to understand the model's performance. ONNX Runtime includes `Ort_perf_view`, which reads this profiling JSON file, renders summarized statistics as treemaps, and computes latency per ONNX operator (for both CPU and GPU). For small models, such as the linear regressor in this example, the view is clean (as shown in figure 10.1) because we use only two ONNX operators.

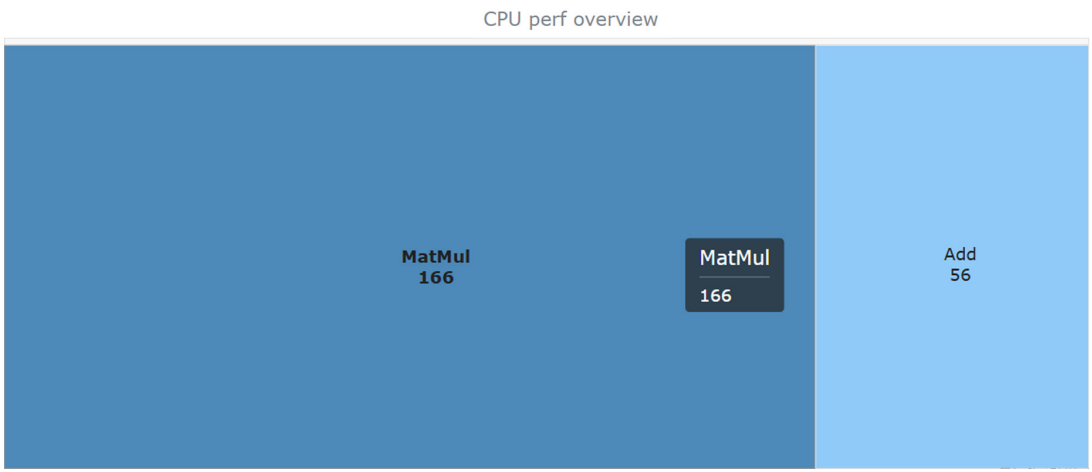


Figure 10.1 `Ort_perf_view` rendering of performance stats for inference on a linear regression ONNX model

The `Ort_perf_view` visualizations show one square per operator, labeled with the operator name and call count for the profiled run. The box size reflects the proportion of operator calls. For larger models, this view is less useful for performance analysis, though it still provides a 10,000-foot overview of aggregated operator calls (see figure 10.2).

In figure 10.2, you can see that there are thousands of calls for each operator in the ONNX graph for the model under test. Clicking an operator's box expands that section to show details for each call (input and output values and duration in microseconds), as shown in figure 10.3.

This visualization isn't very helpful; it just shows raw data without insights. Let's switch to something better.

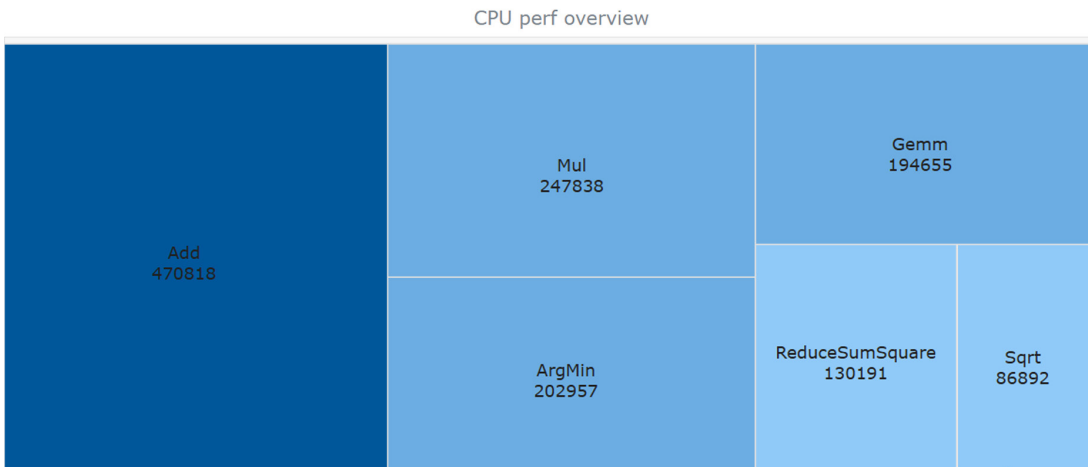


Figure 10.2 `Ort_perf_view` **output showing performance stats for inference run on a large ONNX model**

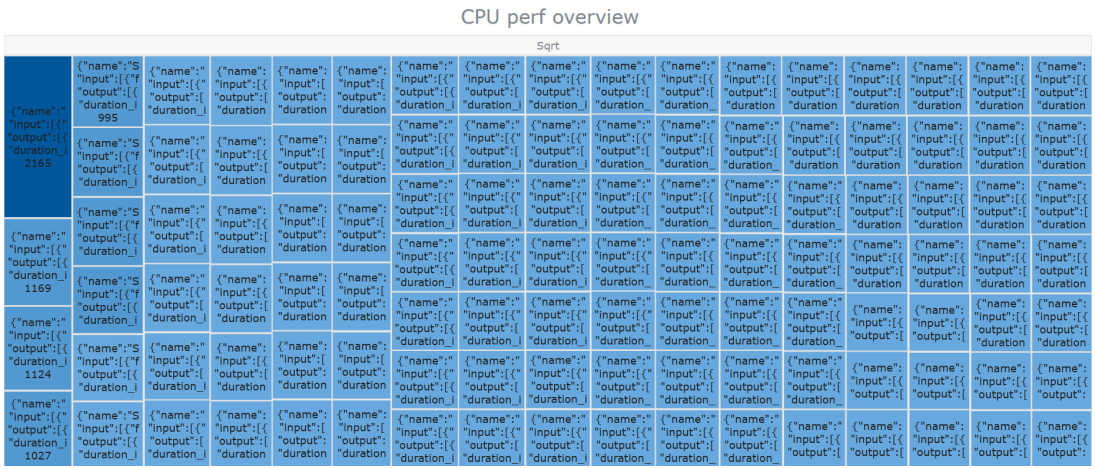


Figure 10.3 `Ort_perf_view` **expanded view of all** `Sqrt` **operator calls during inference for the model under test**

10.2 *Transforming raw ONNX profiling data into insights*

In this section, we'll look at a custom implementation that extracts insights from raw ONNX profiling data and presents them in a user-friendly way. To keep this example simple and easy to follow, we'll use a linear regression model trained in scikit-learn and then converted to ONNX format. The same profiling process, data transformation, and visualization code apply to any ONNX-converted model, including LLMs.

We'll use scikit-learn's `diabetes` dataset for the training and test sets. It contains 442 samples with 10 features each (age, sex, body mass index, average blood pressure, and six blood serum measurements). The target variable is a quantitative measure of disease progression one year after the baseline.

Let's download the dataset:

```
from sklearn.datasets import load_diabetes

data = load_diabetes()
```

Next, split the data into training and test sets:

```
from sklearn.model_selection import train_test_split

X, y = data.data, data.target
X_train, X_test, y_train, __ = train_test_split(X, y, random_state=11)
```

We can now train the model using the `scikit-learn` API:

```
from sklearn.linear_model import LinearRegression

clr = LinearRegression()
clr.fit(X_train, y_train)
```

We can convert the trained model to ONNX format using the `sklearn-onnx` package (https://github.com/onnx/sklearn-onnx):

```
from skl2onnx import to_onnx

model_def = to_onnx(clr, X_train)
```

The converted model graph will look like this:

```
ir_version: 8
producer_name: "skl2onnx"
producer_version: "1.14.0"
domain: "ai.onnx"
model_version: 0
doc_string: ""
graph {
  node {
    input: "X"
    input: "coef"
    output: "multiplied"
    name: "MatMul"
    op_type: "MatMul"
    domain: ""
  }
  node {
    input: "multiplied"
```

```
  input: "intercept"
  output: "resh"
  name: "Add"
  op_type: "Add"
  domain: ""
}
node {
  input: "resh"
  input: "shape_tensor"
  output: "variable"
  name: "Reshape"
  op_type: "Reshape"
  domain: ""
}
name: "ONNX(LinearRegression)"
initializer {
  dims: 10
  dims: 1
  data_type: 11
  name: "coef"
  double_data: -60.219814056772776
  double_data: -266.4570523220356
  double_data: 523.063411259699
  double_data: 310.5134699670048
  double_data: -336.1614738106952
  double_data: 137.33929238424574
  double_data: -131.13923283453687
  double_data: -1.1492348017316942
  double_data: 622.3286855688127
  double_data: 60.46645769168251
}
initializer {
  dims: 1
  data_type: 11
  name: "intercept"
  double_data: 152.22822762230342
}
initializer {
  dims: 2
  data_type: 7
  int64_data: -1
  int64_data: 1
  name: "shape_tensor"
}
input {
  name: "X"
  type {
    tensor_type {
      elem_type: 11
      shape {
        dim {
        }
        dim {
          dim_value: 10
        }
```

```
        }
      }
    }
  }
  output {
    name: "variable"
    type {
      tensor_type {
        elem_type: 11
        shape {
          dim {
          }
          dim {
            dim_value: 1
          }
        }
      }
    }
  }
}
opset_import {
  domain: ""
  version: 13
```

We can profile this converted model by creating an inference session with profiling enabled, as explained in section 9.1:

```
from onnxruntime import InferenceSession, SessionOptions

so = SessionOptions()
so.enable_profiling = True
sess = InferenceSession(model_def.SerializeToString(), so)
```

We'll run inference over the test set (111 samples) and stop profiling at the end:

```
sess.run(None, {'X': X_test})
prof = sess.end_profiling()
```

The raw profiling data can now be loaded from the JSON file:

```
import json

with open(prof, "r") as f:
    js = json.load(f)
```

Next, we'll create a pandas DataFrame from them:

```
from mlprodict.onnxrt.ops_whole.session import OnnxWholeSession

df = pd.DataFrame(OnnxWholeSession.process_profiling(js))
```

We used the `OnnxWholeSession` class from the mlprodict Python library (https://github.com/sdpython/mlprodict/) to convert the JSON content into a cleaner tabular format (see figure 10.4).

	cat	pid	tid	dur	ts	ph	name	args_op_name
0	Session	15652	13080	1008	22	X	model_loading_array	NaN
1	Session	15652	13080	7427	1080	X	session_initialization	NaN
2	Node	15652	13080	0	7636737	X	MatMul_fence_before	MatMul
3	Node	15652	13080	134	7636753	X	MatMul_kernel_time	MatMul
4	Node	15652	13080	0	7636903	X	MatMul_fence_after	MatMul

	args_parameter_size	args_provider	args_input_type_shape	args_node
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	NaN	NaN	NaN	
3	80	CPUExecutionProvider	[{'double': [111, 10]}, {'double': [10, 1]}]	
4	NaN	NaN	NaN	

Figure 10.4 The head rows of the profiling data for the regression model example in tabular format

Starting with this cleaner data representation, it's easier to build visualizations and extract insights. We'll use the Plotly Express Python charting library (https://plotly.com/python/plotly-express/). For performance analysis, we want to compare the total duration for each ONNX operator in the model graph. To do that, we'll first group operators by type and calculate the total duration for each one:

```
gr_dur = df[['dur', "args_op_name"]].groupby("args_op_name")
➥.sum().sort_values('dur')
```

This data can then be displayed in a horizontal bar chart, where the x-axis is total duration (in milliseconds), and the y-axis is the ONNX operator type (see figure 10.5):

```
fig = px.bar(gr_dur, x='dur',
             labels={
                     "dur": "Duration (ms)",
                     "args_op_name": "Operation type",
                 },
             title='Duration')
fig.show()
```

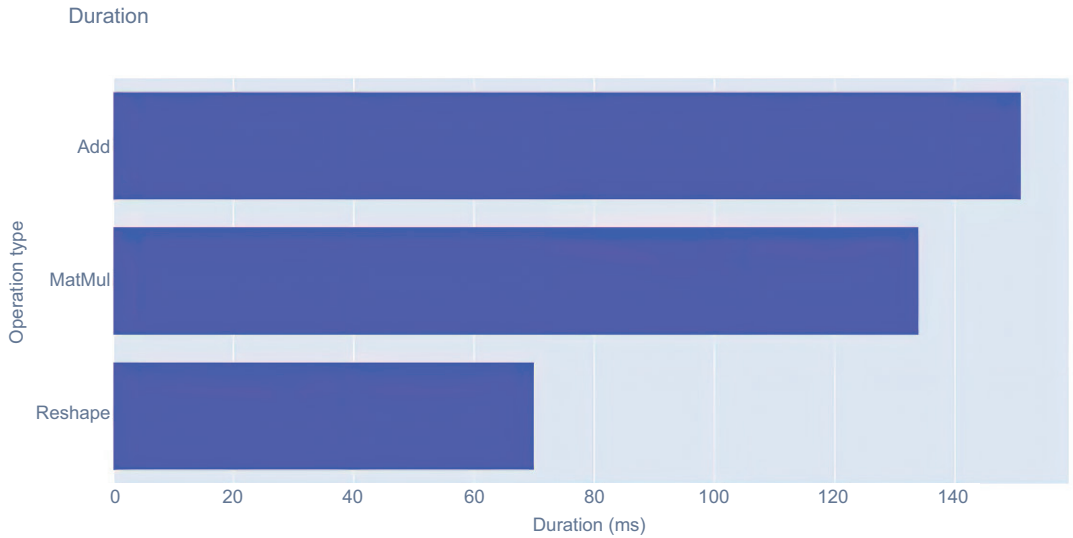


Figure 10.5 Total duration (ms) of the three ONNX operators in our linear regression model's graph

This visualization quickly shows which operators take the most time during a model's inference. Another useful metric is how often each operator is called. To visualize that, first aggregate by operator type, and then count occurrences for each type, ordering by duration so the chart matches the total-duration view:

```
gr_n = df[['dur', "args_op_name"]].groupby("args_op_name")
➥.count().sort_values('dur')
gr_n = gr_n.loc[gr_dur.index, :]
```

This information can then be shown in a horizontal bar chart, where the x-axis is the count of occurrences for a given operator, and the y-axis is the ONNX operator type (see figure 10.6):

```
fig = px.bar(gr_n, x='dur',
             labels={
                     "dur": "Op count",
                     "args_op_name": "Operation type",
                 },
             title='Occurrences')
fig.show()
```

For a more complete performance analysis, it's useful to show not only how often each operator runs but also what share of total inference time it accounts for. To do that, we'll first calculate each operator's percentage of the total inference time:

```
gr_dur /= gr_dur['dur'].sum()
```

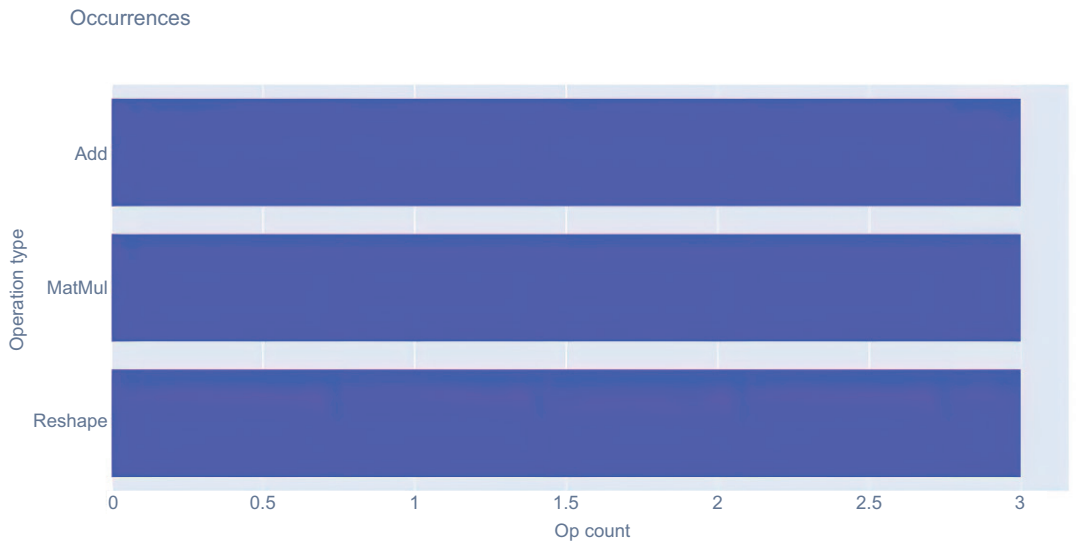


Figure 10.6 Number of invocations of the three ONNX operators in the linear regression model's graph

Now we'll create a visualization from this data (figure 10.7):

```
fig = px.bar(gr_dur, x='dur',
             labels={
                     "dur": "Duration (%)",
                     "args_op_name": "Operation type",
                 },
             title='Proportion')
fig.show()
```

These three visualizations help us quickly spot opportunities for graph optimization in an ONNX model.

For LLMs, we can reuse the same data-cleaning, aggregation, and visualization code unchanged; a companion Colab notebook is provided. The notebook covers converting the GPT-2 small model to ONNX, running inline profiling, and analyzing and visualizing the profiling data. We won't repeat the code for loading the model and tokenizer or converting to ONNX because we covered them in earlier chapters. Instead, we'll just look at the charts produced by profiling the ONNX GPT-2 small model (figures 10.8–10.10).

In figure 10.8, the x-axis shows duration (ms), and the y-axis lists the ONNX operation types in the profiled run.

In figure 10.9, the x-axis shows how many times each ONNX operator was called, and the y-axis lists the operator types used in the profiled run.

In figure 10.10, the x-axis shows the percentage of total inference time, and the y-axis lists the ONNX operation types in the profiled run.

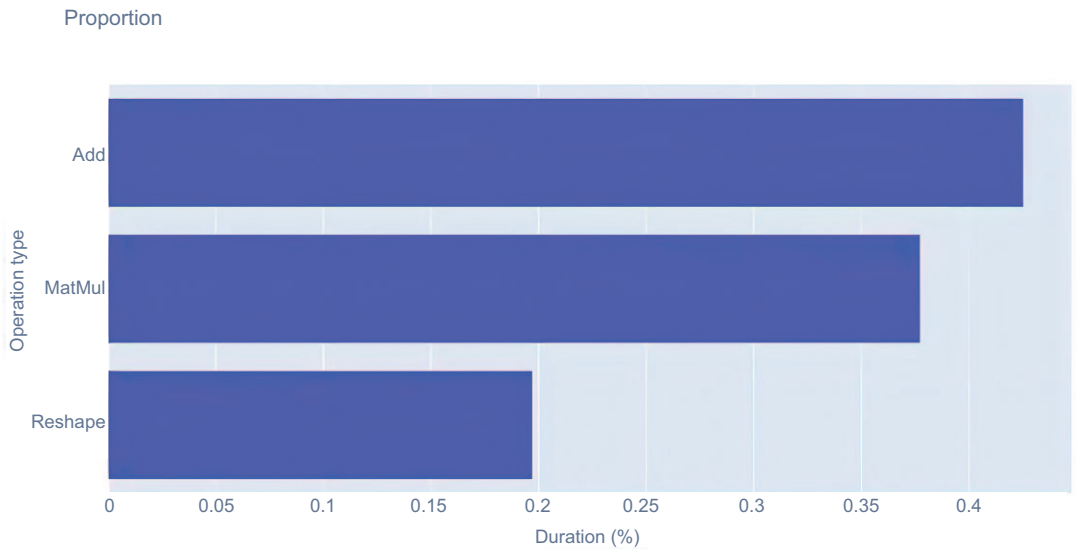


Figure 10.7 Percentage of total inference time for each ONNX operator call in the linear regression model

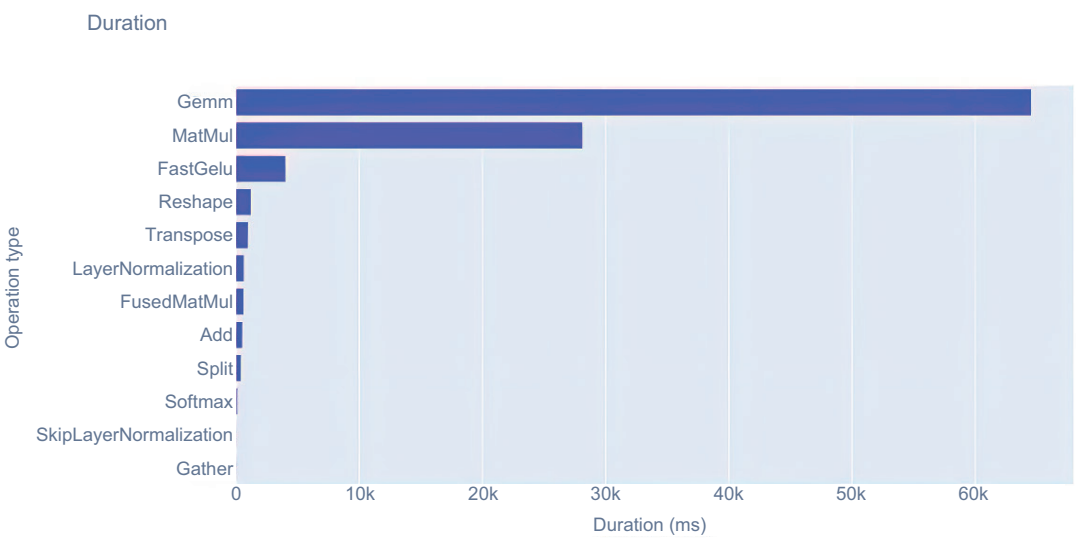


Figure 10.8 Total duration (ms) of ONNX operators in the GPT-small linear regression model's graph during profiling

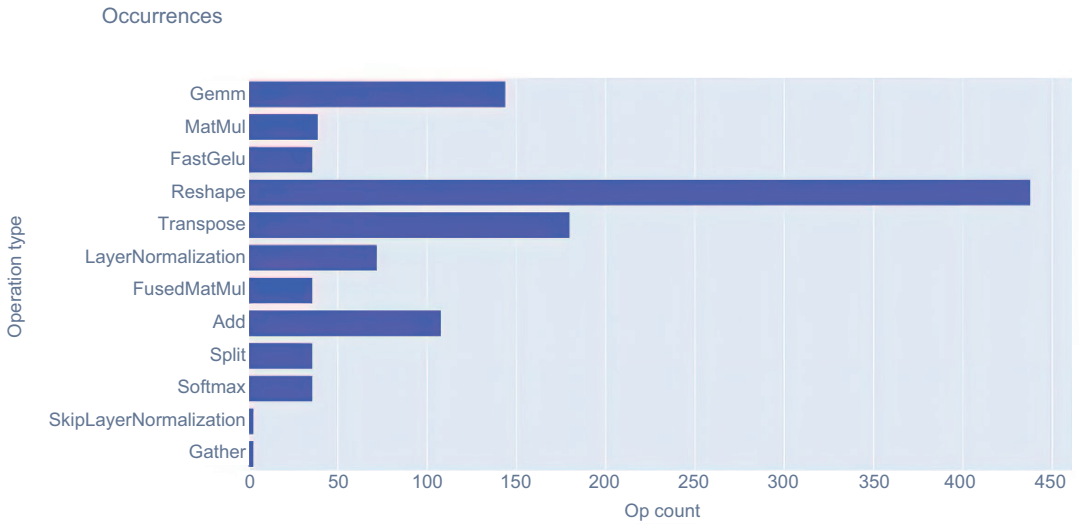


Figure 10.9 ONNX operator invocation counts in the GPT-2 small model graph

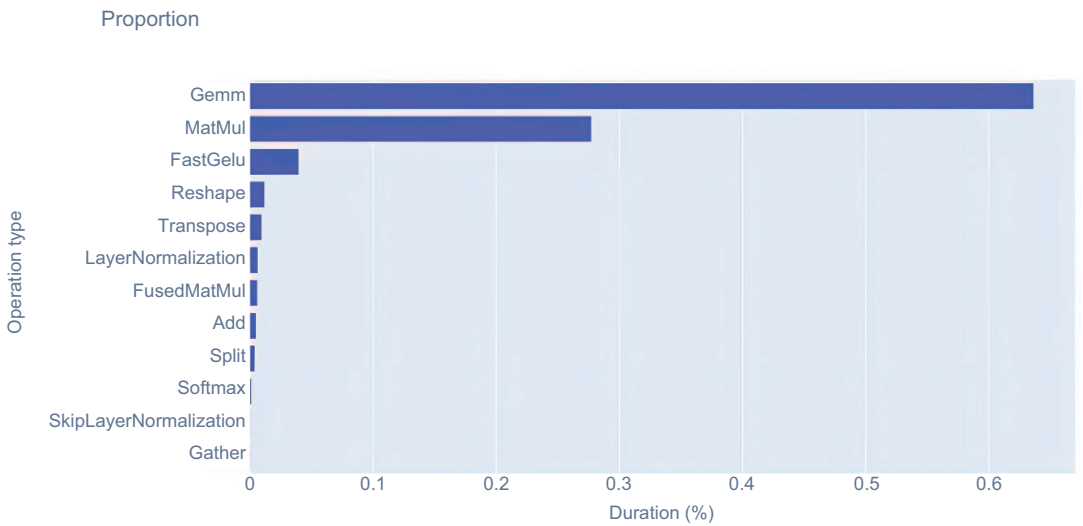


Figure 10.10 The percentage of overall inference time for each ONNX operator call in the GPT-2 small model during profiling

10.3 Optimization of ONNX graphs for LLMs

The ORT for Python can evaluate models converted to ONNX—not only the full graph, but also individual nodes (including custom ones). It can also capture intermediate results within a graph or node. Converting to a new model version or moving from a Python ML/DL framework to an ONNX graph can introduce inefficiencies, and evaluating one or more nodes can help you find and fix them.

Let's first consider the simple linear regression example model from section 10.1. To evaluate the model's full ONNX graph, we can use ONNX's `ReferenceEvaluator` class:

```
from onnx.reference import ReferenceEvaluator

sess = ReferenceEvaluator(onnx_model)
```

We'll prepare the model's input arrays as usual:

```
import numpy as np

x = np.random.randn(4, 2).astype(np.float32)
a = np.random.randn(2, 1).astype(np.float32)
b = np.random.randn(1, 1).astype(np.float32)
feeds = {'X': x, 'A': a, 'B': b}
```

Then we'll run the model using the previously created `ReferenceEvaluator` instance:

```
evaluation = sess.run(None, feeds)
```

We can also evaluate a single node in the graph. For the `MatMul` node in our regression model, do the following:

```
node_sess = ReferenceEvaluator(node1)
feeds = {'X': x, 'A': a}
node_evaluation = node_sess.run(None, feeds)
```

We can enable step-to-step evaluation by setting the `verbose` argument on a `ReferenceEvaluator` instance to different levels of verbosity:

```
sess = ReferenceEvaluator(onnx_model, verbose=verbose)
```

The `verbose` argument accepts integer values from `1` (the default, less verbose) to `4` (the most verbose, step-by-step evaluation). For our linear regression model, the following output displays the different verbosity levels:

```
------ verbose=1

[array([[0.9837985],
       [1.126987 ],
       [0.8049259],
       [1.6867123]], dtype=float32)]
```

```
------ verbose=2

MatMul(X, A) -> XA
Add(XA, B) -> Y
[array([[0.31629467],
       [1.5107596 ],
       [1.008173  ],
       [1.2464874 ]], dtype=float32)]

------ verbose=3

 +I X: float32:(4, 2) in [-0.8688125014305115, 0.7680516242980957]
 +I A: float32:(2, 1) in [0.6488013863563538, 1.9162827730178833]
 +I B: float32:(1, 1) in [2.00530743598938, 2.00530743598938]
MatMul(X, A) -> XA
 + XA: float32:(4, 1) in [-1.2202415466308594, 1.2917602062225342]
Add(XA, B) -> Y
 + Y: float32:(4, 1) in [0.7850658893585205, 3.297067642211914]
[array([[2.835943 ],
       [3.2970676],
       [1.3345237],
       [0.7850659]], dtype=float32)]

------ verbose=4

 +I X: float32:(4, 2):1.4935364723205566,-
0.1639600694179535,1.1996209621429443,-0.47337841987609863,-
2.4281582832336426...
 +I A: float32:(2, 1):[0.8013423681259155, -0.34330955147743225]
 +I B: float32:(1, 1):[-1.6645238399505615]
MatMul(X, A) -> XA
 + XA: float32:(4, 1):[1.253123164176941, 1.1238224506378174,
 -2.082425594329834, -0.8871934413909912]
Add(XA, B) -> Y
 + Y: float32:(4, 1):[-0.4114006757736206, -0.5407013893127441,
 -3.7469494342803955, -2.5517172813415527]
[array([[-0.41140068],
       [-0.5407014 ],
       [-3.7469494 ],
       [-2.5517173 ]], dtype=float32)]
```

As the previous code shows, verbosity level 1 displays only the model's output, level 2 also shows the sequence of operations between inputs, level 3 also includes input values and intermediate outputs, and level 4 includes the inputs to the intermediate steps.

The linear regression in this example follows this formula:

```
y = Add(MatMul(X, A), B)
```

Another way to implement linear regression is to use a similar formula, adding the identity matrix to matrix A:

```
y = Add(MatMul(X, Add(A, I)), B)
```

To implement it, you'll need to modify the original code that builds the model with ONNX operators:

```
node0 = make_node('EyeLike', ['A'], ['Eye'])
node1 = make_node('Add', ['A', 'Eye'], ['A1'])
node2 = make_node('MatMul', ['X', 'A1'], ['XA1'])
node3 = make_node('Add', ['XA1', 'B'], ['Y'])
graph = make_graph([node0, node1, node2, node3], 'lr', [X, A, B], [Y])
```

In this example, we use the `EyeLike` operator, ONNX's implementation of the identity matrix (a square matrix with ones on the main diagonal and zeros elsewhere), and combine the nodes to build the final graph for the updated linear regression formula.

As mentioned earlier in this section, you can also evaluate custom ONNX nodes. For example, in the following code, we'll create a custom node that combines the `EyeLike` and `Add` operators and then see whether the model runs more efficiently:

```
node01 = make_node('AddEyeLike', ['A'], ['A1'], domain='combined')
```

We can then rebuild the graph:

```
node2 = make_node('MatMul', ['X', 'A1'], ['XA1'])
node3 = make_node('Add', ['XA1', 'B'], ['Y'])
graph = make_graph([node01, node2, node3], 'lr', [X, A, B], [Y])
```

Let's rebuild the model and recheck it:

```
from onnx.helper import make_opsetid

onnx_optimized_model = make_model(graph, opset_imports=[
    make_opsetid('', 18), make_opsetid('combined', 1)
])

check_model(onnx_optimized_model)
```

This kind of optimization—combining two operators into a single optimized operator (`AddEyeLike`)—is called *fusion*. To evaluate the custom node (or a model whose graph includes it), you must implement a custom class, because `ReferenceEvaluator` natively supports only built-in ONNX operators. However, it can also accept custom operator implementations. You can implement custom operators by extending the ONNX `OpRun` class and implementing its `_run` method:

```
from onnx.reference.op_run import OpRun

class AddEyeLike(OpRun):

    op_domain = "combined"

    def _run(self, X, alpha=1.):
        assert len(X.shape) == 2
```

```
        assert X.shape[0] == X.shape[1]
        X = X.copy()
        ind = numpy.diag_indices(X.shape[0])
        X[ind] += alpha
        return (X,)
```

You can now evaluate the model as usual. The only difference is that when you create a `ReferenceEvaluator` instance, you also need to pass a list of the custom node implementations included in the model's ONNX graph:

```
sess = ReferenceEvaluator(onnx_optimized_model, verbose=2,
➥ new_ops=[AddEyeLike])
```

Evaluation for the given inputs proceeds the same way as for models without custom operators:

```
x = numpy.random.randn(4, 2).astype(numpy.float32)
a = numpy.random.randn(2, 2).astype(numpy.float32) / 10
b = numpy.random.randn(1, 2).astype(numpy.float32)
feeds = {'X': x, 'A': a, 'B': b}

sess.run(None, feeds)
```

Before measuring any execution speedup, let's confirm that the optimization didn't reduce prediction quality. We'll evaluate both model versions (the original and the one after fusing the two operators):

```
sess_original = ReferenceEvaluator(onnx_model)
sess_optimized = ReferenceEvaluator(onnx_optimized_model,
➥ new_ops=[AddEyeLike])

y_original = sess_original.run(None, feeds)[0]
y_optimized = sess_optimized.run(None, feeds)[0]

print(f"output from original model: {y_original}")
print(f"output from optimized model: {y_optimized}")

print(f"difference: {numpy.abs(y_original - y_optimized).max()}")
```

There is no difference:

```
output from original model: [[-1.3435526   0.11092019]
 [ 0.8220897   0.18847765]
 [-1.29872     0.882055  ]
 [-0.10433175 -0.30559355]]
output from optimized model: [[-1.3435526   0.11092019]
 [ 0.8220897   0.18847765]
 [-1.29872     0.882055  ]
 [-0.10433175 -0.30559355]]

difference: 0.0
```

Let's now benchmark both model versions on larger matrices, timing 10,000 runs with Python's `timeit` package:

```
x = numpy.random.randn(4, 100).astype(numpy.float32)
a = numpy.random.randn(100, 100).astype(numpy.float32) / 10
b = numpy.random.randn(1, 100).astype(numpy.float32)
feeds = {'X': x, 'A': a, 'B': b}

import timeit

print(f"time with EyeLike+Add: {timeit.timeit(lambda:
➥ sess_original.run(None, feeds), number=10000)}")
print(f"time with AddEyeLike: {timeit.timeit(lambda:
➥ sess_optimized.run(None, feeds), number=10000)}")
```

There is a clear improvement with the optimized model:

```
time with EyeLike+Add: 1.1427911999999196
time with AddEyeLike: 0.8636034999999538
```

You can also evaluate and profile by following the directions in sections 7.1 and 7.2. I ran the benchmarks on a PC with two Intel Core i5-7300U CPUs (2.6 GHz) and 8 GB of RAM.

In section 5.6.1, I described ONNX Runtime (ORT) optimization levels and how they apply to LLMs. You also saw that graph optimization can be done offline or inline, and that ORT provides two optimization sets: basic (including the operator fusion introduced in this section) and extended. I also introduced the built-in `optimizer` class for Transformer models in the ORT API, which automates offline graph optimization for an ONNX model (or a model converted to ONNX format). Now let's see how the ONNX graph for an LLM changes after optimization.

In this example, we'll use the GPT-2 small model; the complete code is in the Colab notebook for section 10.2. No hardware acceleration is required; all optimization considerations are for CPUs only. After downloading the GPT-2 small weights and tokenizer from the Hugging Face Hub and converting the model to ONNX, we can optimize it offline programmatically with the ORT optimizer:

```
from onnxruntime.transformers import optimizer

num_layer = model.config.n_layer
num_attention_heads = model.config.n_head
hidden_size = model.config.n_embd

onnx_model_path='gpt2onnx.onnx'
onnx_optim_model_path="gpt2onnx-opt.onnx"
optimized_model = optimizer.optimize_model(onnx_model_path,
                                           model_type='gpt2',
                                           num_heads=num_attention_heads,
                                           hidden_size=hidden_size,
                                           use_gpu=False,
```

```
                                                opt_level=1,
                                                verbose=True)
optimized_model.convert_float_to_float16()
optimized_model.save_model_to_file(onnx_optim_model_path)
```

We provided the optimizer with the original ONNX model location, model architecture type, number of attention heads, hidden size, lack of hardware acceleration, and optimization level (1, which applies only basic graphic optimizations). After optimization, the model is about half the size of the original (309 MB vs. 622 MB). The optimizer also applied several basic optimizations, such as removing 108 nodes (mainly unnecessary `Reshape` nodes), as shown in figure 10.11, which compares the original ONNX graph with the optimized graph's input layers.

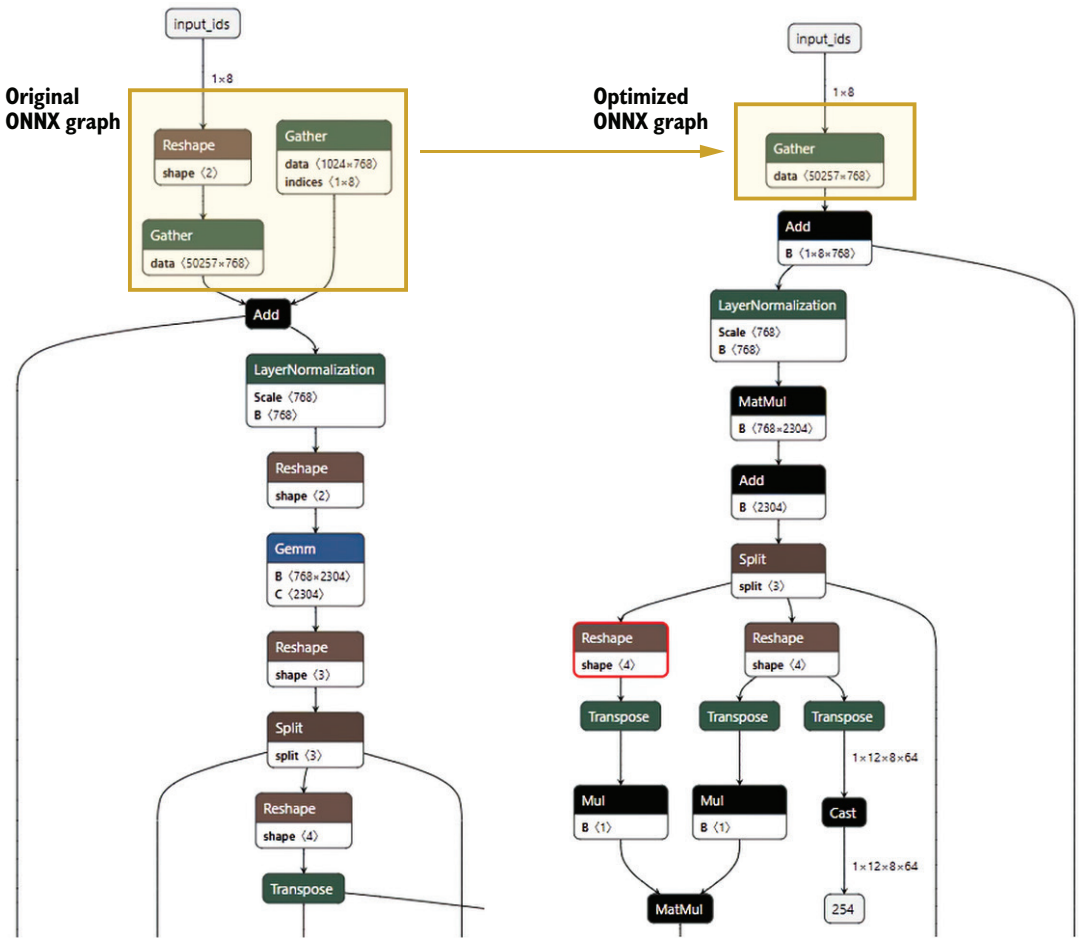


Figure 10.11 Comparing the input layers of the GPT-2 ONNX-converted model's graph before and after optimization

The optimizer also applied some extended optimizations—such as Skip Layer Normalization Fusion (which fuses fully connected layer biases, skip connections, and layer normalizations; see figure 10.12)—because we've specified the model architecture (a decoder-only Transformer).

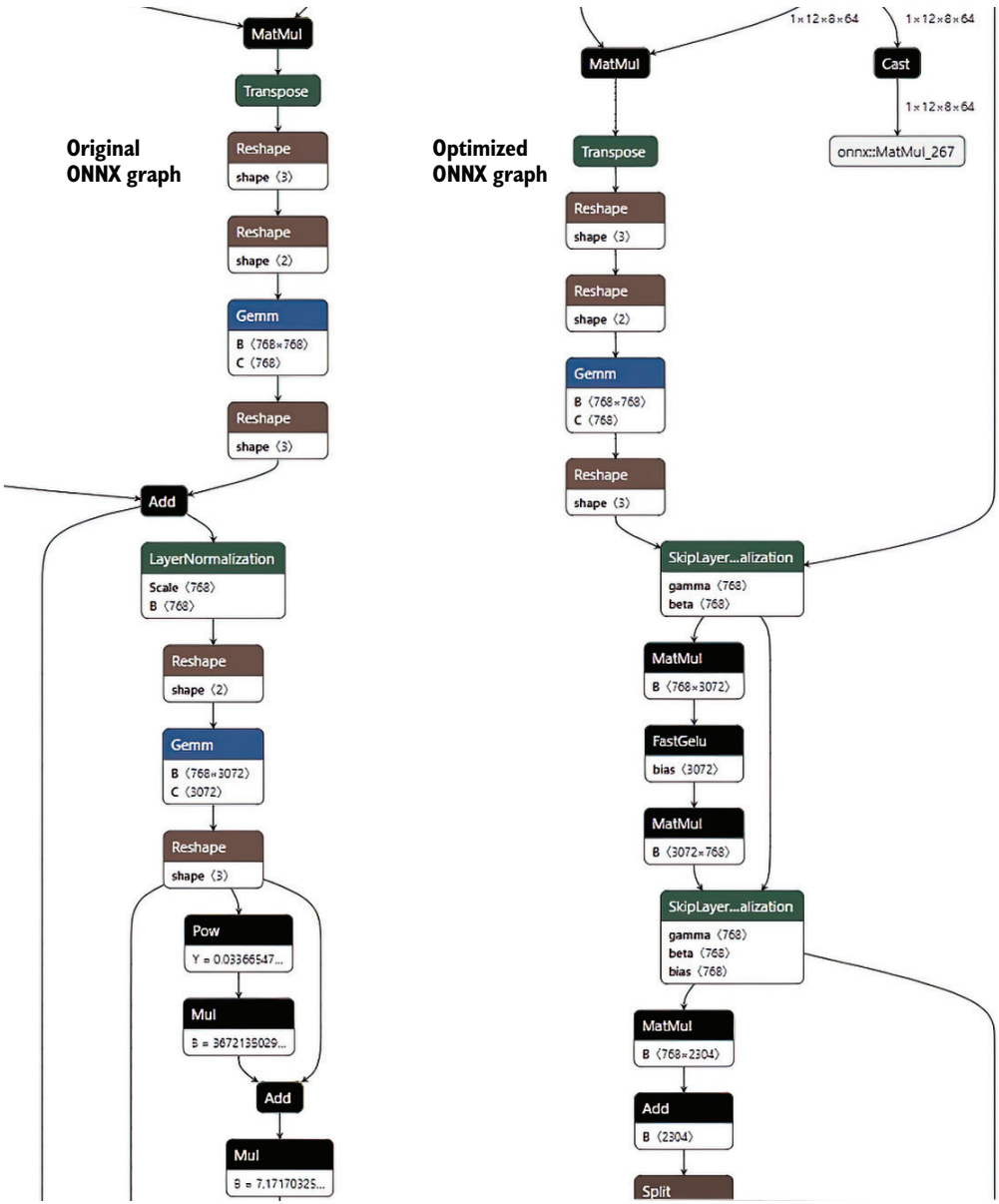


Figure 10.12 Comparing part of the GPT-2 ONNX-converted model's graph, before and after optimization, showing Skip Layer Normalization Fusion applied

In the Netron tool (https://github.com/lutzroeder/netron), you can inspect the optimized graph and see that additional optimizations, such as Fast GELU Fusion, have been applied.

By profiling both versions of the ONNX model, we can see the performance gain (compare figure 10.13 with figures 10.8 and 10.9 for the original ONNX GPT-2 model).

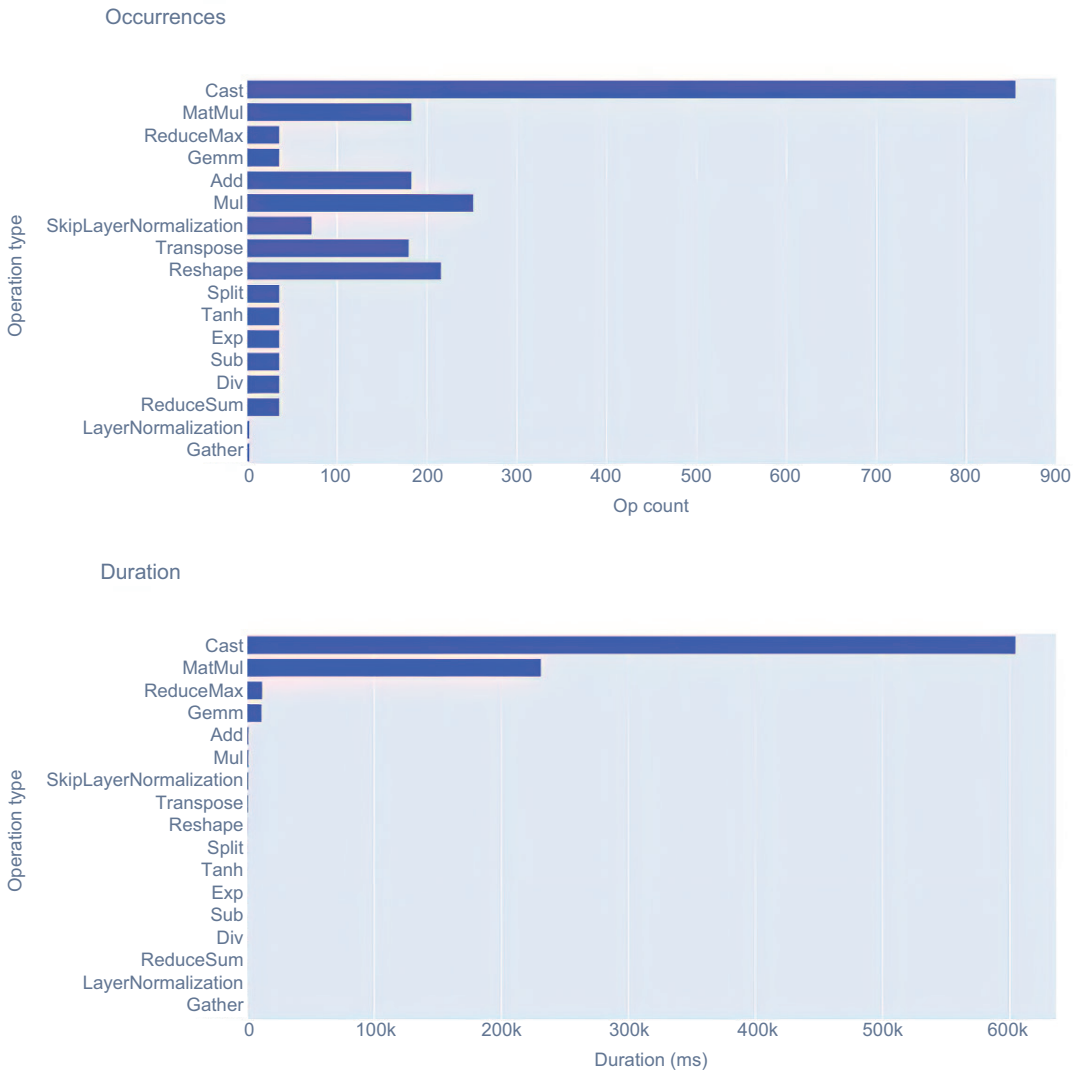


Figure 10.13 Counts and durations for each ONNX operator in the optimized GPT-2 model graph

Analyzing the automated ONNX optimizations for LLMs is a great way to learn when and how to apply them manually to custom models.

This chapter wraps up our exploration of the ONNX format and ORT. The next two chapters show how you can combine these and other techniques from earlier chapters with SLM deployment in diverse environments to make the most of available computational power without sacrificing too much quality.

Summary

- Enable performance profiling in ONNX Runtime (ORT) by setting `enable_profiling` to `True` in `SessionOptions`.
- Raw profiling data is stored in JSON files with timing information for each operator execution.
- `OnnxWholeSession.process_profiling` converts messy JSON profiling output into a clean, tabular DataFrame format.
- Plotly Express can create interactive bar charts of operator duration, call counts, and execution percentages for performance analysis.
- The `ReferenceEvaluator` class evaluates entire ONNX models or individual nodes for debugging and optimization.
- Verbosity levels 1 to 4 control how much step-by-step evaluation detail you see when debugging.
- Operator fusion combines consecutive operators into single optimized nodes to speed execution.
- To evaluate custom operators, extend the `OpRun` class, and implement `_run`.
- The ORT optimizer automatically applies basic and extended optimizations to transformer models.
- Graph optimization reduces model size by removing unnecessary nodes, such as redundant `Reshape` operations.
- Skip Layer Normalization Fusion combines bias layers, skip connections, and layer normalizations into single operations.
- Fast GELU Fusion optimization is applied automatically when you specify the model's architecture type.
- The Netron tool lets you visually inspect ONNX graphs to see which optimizations were applied.
- Model evaluation ensures optimizations don't hurt prediction quality or accuracy.
- Basic optimizations include constant folding, eliminating redundant nodes, and semantics-preserving fusions.
- Extended optimizations perform complex node fusions specific to CPU, CUDA, or ROCm execution providers.

11 Deployment and serving

This chapter covers

- SLM serving and inference with vLLM
- SLM serving with FastAPI
- SLM deployment and serving on devices with MLC LLM
- Options for SLM deployment and inference on Android devices

It's time to look at some of the most common environments and tools for deploying and serving small, customized language models. We won't cover them all: the closer you get to local or edge deployments, the more hardware options and frameworks you'll encounter. I've focused on the options that are currently most popular across operating systems and hardware combinations.

11.1 vLLM

vLLM is an open source library for LLM serving and inference (https://github.com/vllm-project/vllm). vLLM was introduced in the September 2023 paper "Efficient Memory Management for Large Language Model Serving with PagedAttention"

(https://arxiv.org/abs/2309.06180). The paper introduces *PagedAttention*, an attention algorithm that improves memory management for LLMs by addressing issues of memory fragmentation and redundancy in the key–value (KV) cache. It does this by storing KV-cache blocks noncontiguously, which increases memory flexibility and reduces fragmentation—key to improving throughput and batch size in LLM serving systems. vLLM uses a centralized scheduler and KV-cache manager, similar to a virtual memory system, and emphasizes batching during autoregressive generation to optimize throughput. The paper critiques existing LLM serving systems for not meeting LLM-specific needs, and it highlights vLLM's dynamic memory allocation and block management as key improvements for efficient memory use in complex applications. Overall, it is the first effort to focus on serving LLMs specifically, rather than ML models in general.

The paper notes several inefficiencies and challenges in how earlier systems managed memory:

- *Contiguous memory allocation*—Most systems store a request's KV cache in contiguous memory space. This can cause significant internal and external fragmentation. Internal fragmentation occurs when the actual request is shorter than its reserved maximum length, leaving unused space. External fragmentation occurs when different requests reserve different block sizes, leaving gaps that other requests can't use.
- *Preallocation of memory*—Existing systems preallocate memory for a request based on its maximum possible sequence length, regardless of the actual input or output length. This static allocation wastes memory because reserved space for future tokens and internal fragmentation goes unused until the request completes.
- *Inefficient memory sharing*—Advanced decoding algorithms such as parallel sampling and beam search generate multiple outputs per request, but existing systems can't share memory because each sequence's KV cache is stored in its own contiguous space.
- *Scheduling and preemption*—Existing systems struggle to schedule and preempt requests because input and output lengths vary. They must decide when to delete or swap some requests' KV caches from GPU memory to make room for new requests or continued generation for existing prompts.

vLLM addresses the memory-management inefficiencies because it limits batch size and throughput in LLM serving systems.

This paper led to the open source vLLM project, released under the Apache 2.0 license. It's a community-driven initiative with contributions from both academia and industry. Today, vLLM is a mature project with the following features and capabilities:

- State-of-the-art serving throughput
- Efficient management of attention key and value memory through PagedAttention

- Continuous batching of incoming requests
- Support for the most popular CPUs and GPUS (NVIDIA GPUs, AMD CPUs and GPUs, Intel CPUs, Gaudi accelerators and GPUs, PowerPC CPUs, TPU, and AWS Trainium and Inferentia Accelerators)
- Optimized CUDA kernels, including integration with FlashAttention and FlashInfer
- Fast model execution with CUDA/HIP graph
- Support for quantization (GPTQ, AWQ, INT4, INT8, and FP8)
- High throughput serving with various decoding algorithms
- Speculative decoding
- Chunked prefill
- Distributed inference (tensor and pipeline parallelism)
- Streaming outputs
- Prefix caching support
- Integration with Hugging Face models (Transformers, MoEs, embedding models, multimodal LLMs)
- Compatibility with the OpenAI API
- Multi-LoRA support

Let's look at how we can use vLLM to serve SLMs.

11.1.1 Offline serving

vLLM supports offline and online serving. We'll first look at offline serving—it's a good starting point for getting familiar with the library. A companion Colab notebook is provided.

In most cases, you can install vLLM as a Python library:

```
pip install vllm
```

For supported hardware architectures without a prebuilt wheel, you'll need to build it from source. Check the official documentation before installing it in your environment.

In this first example, you'll get familiar with the main vLLM API in a text-generation use case. We'll then build on that with a chat use case and a quantized model. We'll use hardware acceleration with an NVIDIA GPU.

First, we'll download a pretrained model from the Hugging Face Hub (GPT-2 small in this example):

```
import torch
from vllm import LLM

model_id = "openai-community/gpt2"
device = "cuda" if torch.cuda.is_available() else "cpu"
vllm_model = LLM(model_id, device=device)
```

You'll notice that the code for this part is cleaner than the equivalent based on Hugging Face's Transformers API. The single line that creates the LLM class performs several actions (rather than just downloading the model checkpoints):

- It downloads the model configuration file.
- It analyzes the model configuration and prepares the model setup accordingly.
- It downloads the model's tokenizer.
- It downloads the model's checkpoints.
- It loads the checkpoints in the destination device (GPU memory in this example).
- It profiles the memory.
- It initializes the KV cache.
- It warms up the model.

The model is now ready for inference. Because we didn't specify a task, vLLM defaulted to `generate` (it also supports `classify`, `score`, `reward`, and `embed`), so the selected GPT-2 model is ready to generate text. Provide a prompt, and start text completion with the `generate` method:

```
prompt = "Once upon a time in a land far away"
output = vllm_model.generate(prompt)
print(output[0].outputs[0].text)
```

The `generate` method also accepts an argument that specifies the sampling strategy. vLLM provides `SamplingParams`, a class for configuring sampling configuration (`temperature`, `top_p`, `top_k`, and other options supported by libraries such as Hugging Face's Transformers):

```
from vllm import SamplingParams

sampling_params = SamplingParams(temperature=0.1, top_p=0.95)
output = vllm_model.generate(prompt, sampling_params)
```

This is the simplest, most general way to use vLLM for inference with any of the supported models. You can see how clean the vLLM API is, which was a design goal for its authors and maintainers from the start.

Next, we'll make things slightly more complex with a chat use case. We'll switch to a different model—the instruction-tuned Microsoft Phi-3 Mini 4K Instruct model (https://huggingface.co/microsoft/Phi-3-mini-4k-instruct)—and set up a chat app, mostly reusing the code from the previous example. The setup code for chatting looks the same as for text generation:

```
import torch
from vllm import LLM, SamplingParams

model_id = "microsoft/Phi-3-mini-4k-instruct"
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
llm = LLM(model=model_id, device=device, dtype=torch.half)

sampling_params = SamplingParams(temperature=0.5)
```

The only thing that changes is the method we call. Given a conversation prompt like the following

```
conversation = [
    {
        "role": "system",
        "content": "You are a helpful assistant"
    },
    {
        "role": "user",
        "content": "Hi"
    },
    {
        "role": "assistant",
        "content": "Hi! How can I assist you today?"
    },
    {
        "role": "user",
        "content": "Write an essay about the Monte Carlo Tree Search
        ➥ algorithm.",
    },
]
```

the method we'll use is `chat`:

```
outputs = llm.chat(conversation,
                   sampling_params=sampling_params,
                   use_tqdm=False)
```

The `chat` method works for both single and batch inference:

```
conversations = [conversation for _ in range(10)]

outputs = llm.chat(messages=conversations,
                   sampling_params=sampling_params,
                   use_tqdm=True)
```

We can also provide an optional chat template. For example, to use the following Jinja template with the Phi-3 Mini 4K Instruct model, save it as phi-3.jinja:

```
{% if messages[0]['role'] == 'system' %}
    {% set offset = 1 %}
{% else %}
    {% set offset = 0 %}
{% endif %}

{% for message in messages %}
```

```
    {% if (message['role'] == 'user') != (loop.index0 % 2 == offset) %}
        {{ raise_exception('Conversation roles must alternate
        ➥ user/assistant/user/assistant/...') }}
    {% endif %}

    {{ '<|' + message['role'] + '|>\n' + message['content']
    ➥| trim + '<|end|>' + '\n' }}
{% endfor %}

{% if add_generation_prompt %}
    {{ '<|assistant|>\n' }}
{% endif %}
```

Then pass the filename to the `chat` method via the `chat_template` argument:

```
with open('phi-3.jinja', "r") as f:
  chat_template = f.read()

outputs = llm.chat(
  conversations,
  sampling_params=sampling_params,
  use_tqdm=False,
  chat_template=chat_template,
)
```

As mentioned earlier in this section, vLLM supports inference for quantized LLMs, including those in GGUF format. Let's see how to use a quantized model, using Microsoft's Phi-3 Mini 4K Instruct as our example. For simplicity, we'll use a version that's already 4-bit quantized in GGUF format; the code that follows works for any LLM quantized the same way.

Microsoft provides a GGUF version of Phi-3 Mini 4K Instruct on the Hugging Face Hub. Log in with your Hugging Face credentials, and then specify the repository name and GGUF file to download. Use the same tokenizer as for the vanilla model:

```
from huggingface_hub import hf_hub_download

repo_id = "microsoft/Phi-3-mini-4k-instruct-gguf"
filename = "Phi-3-mini-4k-instruct-q4.gguf"
tokenizer = "microsoft/Phi-3-mini-4k-instruct"
model = hf_hub_download(repo_id, filename=filename)
```

After downloading the model checkpoints and tokenizer, provide a prompt (or batch of prompts), and create a `SamplingParams` object:

```
from vllm import LLM, SamplingParams

prompts = [
    "How to explain Internet for a medieval knight?",
    "What's the future of AI?",
]
```

```
prompts = [[{"role": "user", "content": prompt}] for prompt in prompts]

sampling_params = SamplingParams(temperature=0, max_tokens=128)
```

Then create a vLLM LLM object using the downloaded model and tokenizer:

```
llm = LLM(model=model, tokenizer=tokenizer)
```

We can now use the `chat` method as for any other vanilla model:

```
outputs = llm.chat(prompts, sampling_params)
```

The output will look like this:

```
<|user|>\nHow to explain Internet for a medieval
knight?<|end|>\n<|assistant|>\n', Generated text: ' Good morrow, noble
knight. I come before thee to share knowledge of a magical realm called the
Internet, a place beyond our physical world, where information and
knowledge flow like the rivers of our kingdom.\n\nImagine, if thou wilt, a
vast library, containing all the wisdom of the ages, from the time of the
great empires to the present day. This library is not bound by walls or
shelves, but exists in a realm beyond our senses, accessible to all who
possess the key to unlock its secrets.\n\nThe Internet is a network of
invisible threads, connecting'
"<|user|>\nWhat's the future of AI?<|end|>\n<|assistant|>\n", Generated
text: " The future of Artificial Intelligence (AI) is a topic of great
interest and speculation among experts across various fields. While it's
challenging to predict with absolute certainty, several trends and
possibilities can be outlined based on current research, technological
advancements, and societal needs. Here are some key areas that are likely
to shape the future of AI:\n\n1. Advanced Machine Learning: AI will
continue to evolve, with machine learning algorithms becoming more
sophisticated, efficient, and capable of handling complex tasks. This
includes deep learning, reinforcement learning, and"
```

11.1.2 Online serving

vLLM also provides an HTTP server compatible with the OpenAI API (https://github.com/openai/openai-python), the de facto standard for online LLM serving. After you install vLLM, you can start a server with the `serve` CLI command. In its simplest form, the command expects the following arguments:

- The ID of an LLM in the Hugging Face Hub
- The precision format for the model
- An authorization token to access the server's API

The following example uses Microsoft's Phi-3 Mini 4K Instruct model:

```
vllm serve microsoft/Phi-3-mini-4k-instruct --dtype
float16 --api-key token-abc123
```

After the server has ramped up, you can call it with an HTTP client built with the OpenAI Python client API (as you would with any other framework that supports this library and format). The vLLM server supports the OpenAI API for completions, chat completions, and embeddings, and it provides further extensions for tokenizing/detokenizing, pooling, scoring, and reranking. The following example shows an HTTP client calling the vLLM completion API for the Phi-3 Mini 4K Instruct model:

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="token-abc123",
)

completion = client.chat.completions.create(
  model="microsoft/Phi-3-mini-4k-instruct",
  messages=[
    {"role": "user", "content": "Hello!"}
  ]
)
```

The output looks like this:

```
ChatCompletionMessage(content=' Hello! How can I assist you today?',
refusal=None, role='assistant', audio=None, function_call=None,
tool_calls=[], reasoning_content=None)
```

For a full list of OpenAI API configurations and options, see the OpenAI Python SDK documentation at https://github.com/openai/openai-python. You can serve vLLM offline or online using Docker containers and deploy it on Kubernetes, Nginx, or AWS.

Because of vLLM's many implementation optimizations, SLM inference is very fast compared to other frameworks and runtimes. But once a model is deployed behind an HTTP endpoint, you should benchmark its real serving performance before opening access to client applications. The official vLLM GitHub repository provides several Python scripts to automate benchmarks for any model served through the framework's API. The scripts are in the benchmarks folder, and they cover metrics such as latency and throughput. Two benchmarking datasets are also available on the Hugging Face Hub: `ShareGPT` (text only) and `ShareGPT4V` (multimodal).

To benchmark models served online, we first need to clone the vLLM GitHub repo and install its requirements. Then download the `ShareGPT` dataset in JSON format, such as by using `wget`:

```
wget https://huggingface.co/datasets/anon8231489123
➥/ShareGPT_Vicuna_unfiltered/resolve/main
➥/ShareGPT_V3_unfiltered_cleaned_split.json
```

Finally, we can run the Python script to start benchmarking the served model (we're still using Microsoft's Phi-3 Mini 4K Instruct model, deployed on a vLLM server):

```
python ./vllm/benchmarks/benchmark_serving.py \
        --backend vllm \
        --model microsoft/Phi-3-mini-4k-instruct \
        --dataset-name sharegpt \
        --dataset-path /content/ShareGPT_V3_unfiltered_cleaned_split.json \
        --num-prompts 200 \
        --save-result
```

This example passes several parameters to the script: the backend we're using (vLLM), the model ID on the Hugging Face Hub, the dataset name and local path, the number of samples, and whether to save detailed benchmark results locally as JSON. When you run this benchmark against the model on a vLLM server in a free-tier Colab accelerated environment (with one NVIDIA T4 GPU available), you should see performance in the following ranges:

```
Traffic request rate: inf
Burstiness factor: 1.0 (Poisson process)
Maximum request concurrency: None
100% 200/200 [02:23<00:00,  1.39it/s]
============ Serving Benchmark Result ============
Successful requests:                     200
Benchmark duration (s):                  143.93
Total input tokens:                      49473
Total generated tokens:                  47232
Request throughput (req/s):              1.39
Output token throughput (tok/s):         328.17
Total Token throughput (tok/s):          671.91
---------------Time to First Token----------------
Mean TTFT (ms):                          38742.30
Median TTFT (ms):                        14503.04
P99 TTFT (ms):                           99097.12
-----Time per Output Token (excl. 1st token)------
Mean TPOT (ms):                          153.30
Median TPOT (ms):                        107.19
P99 TPOT (ms):                           1104.83
---------------Inter-token Latency----------------
Mean ITL (ms):                           102.17
Median ITL (ms):                         72.11
P99 ITL (ms):                            532.52
==================================================
```

The benchmark final report shows the number of successful requests, total duration, input and generated token counts, and request, output, and total token throughput. It also reports the mean, median, and 99th percentile for the time to first token, time per output token, and inter-token latency.

11.2 FastAPI

FastAPI (https://fastapi.tiangolo.com/) is a modern, open source web framework for building APIs with standard Python type hints. It aims to be performant, easy to use, and easy to learn. FastAPI is built on Starlette (https://www.starlette.io/) for the web layer and Pydantic (https://docs.pydantic.dev/latest/) for data modeling and validation. It uses Python's async features for nonblocking operations and good performance under load, and its straightforward syntax helps developers get started quickly. FastAPI also generates live, interactive API documentation automatically, with either Swagger UI (https://swagger.io/) or ReDoc (https://github.com/Redocly/redoc) as the documentation interface (they offer different user experiences for exploring and testing the same REST API). This documentation is derived from your code and type hints, making the API easy to understand and test. FastAPI is well suited for data-driven APIs, microservices, and ML APIs.

Before we serve LLMs with this framework, let's review the basic FastAPI syntax. After installing the library, we need to import the main requirement:

```
from fastapi import FastAPI
```

Next, we'll create an instance of the FastAPI app:

```
app = FastAPI()
```

Now we can start building routes. Routes define how incoming HTTP requests are handled. Each route is tied to a URL path and specifies the function that processes the request. For example, to accept HTTP `GET` requests, we'll define a function and decorate it with the FastAPI `get` annotation:

```
@app.get("/")
def read_root():
    return {"Hello": "Friend"}
```

To accept HTTP `POST` requests, we can define a custom function and decorate it with FastAPI's `post` annotation:

```
from typing import Union

@app.post("/items/")
async def create_item(item_id: int, q: Union[str, None] = None):
    return {"item_id": item_id, "q": q}
```

Now we can start the web server from the CLI:

```
fastapi <environment> <python_web_app_name>.py
```

Here `<environment>` can take the values `dev` (for development/testing) or `run` (for production). After the server starts, the implemented APIs are available at the hostname

(default `localhost`) and port (default `8000`) specified at the server' startup. You can set the host and port with the `--host` and `--port` options of the `fastapi` command-line application.

Because this framework uses Swagger UI and ReDoc, it automatically generates online documentation for our custom API at the `/docs GET` endpoint (see figure 11.1).

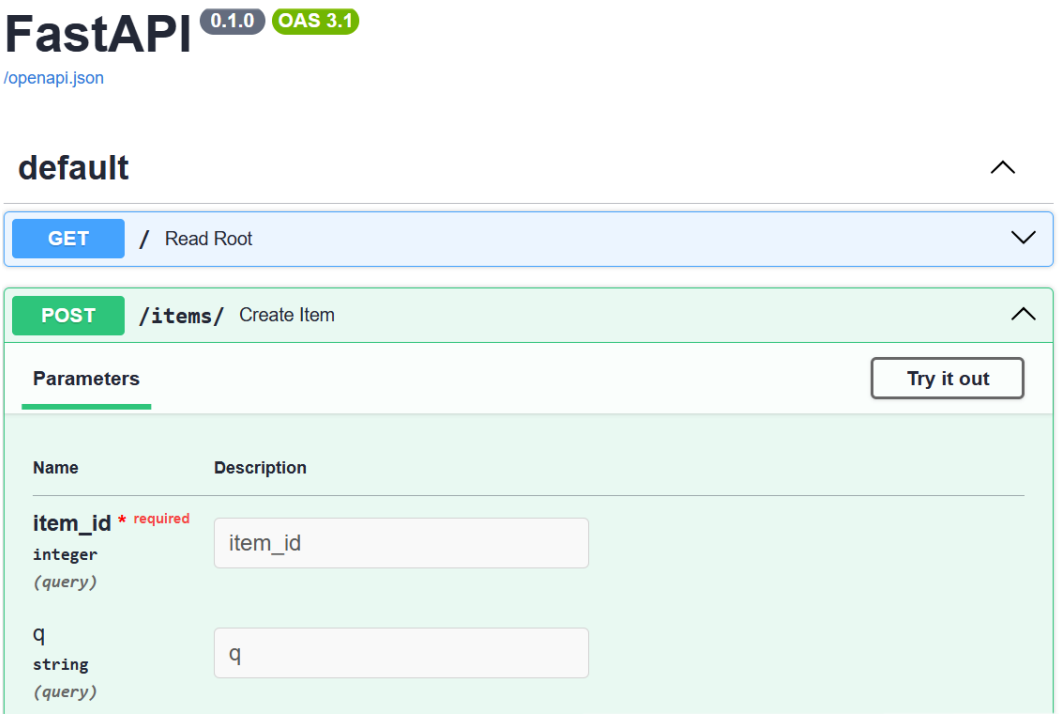


Figure 11.1 Online documentation for our FastAPI web application's custom API

Alternatively, you can use the `/redoc GET` route (see figure 11.2).

That's it, regardless of the complexity or scope of your functions or API.

Now let's see how to use FastAPI to serve LLMs. The server code is straightforward, since its structure is the same as any for other FastAPI app. What are specific to LLMs are the inference handling function and any supporting code for model initialization. The Hugging Face libraries we've used throughout this book integrate easily into FastAPI apps. As in the previous example, we'll start by creating an instance of the FastAPI app class:

```
from fastapi import FastAPI

app = FastAPI()
```

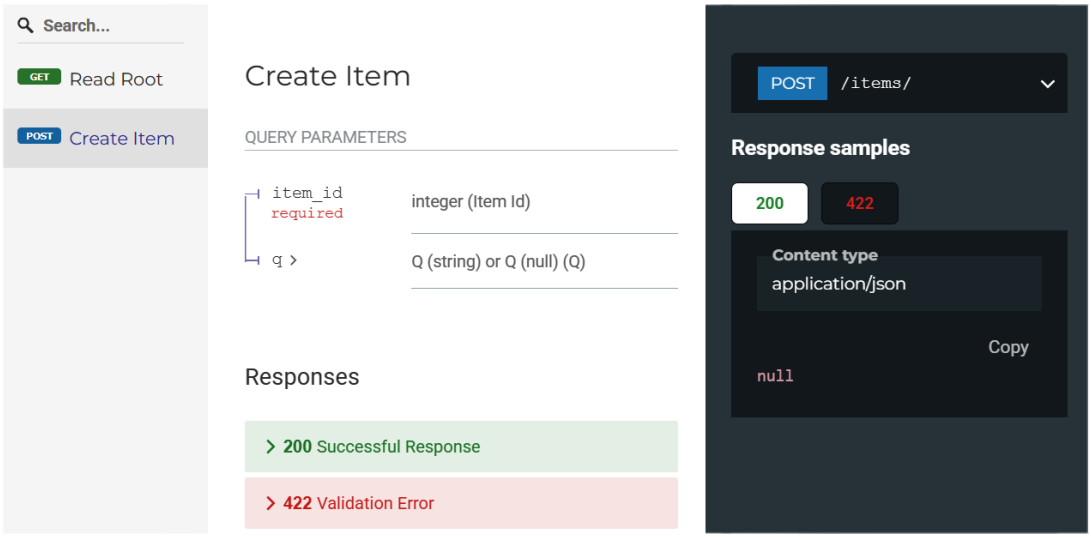


Figure 11.2 Online ReDoc documentation for our sample FastAPI web application's custom API

Next, we'll create two classes—one for the model's input and one for its output:

```
from pydantic import BaseModel

class InputData(BaseModel):
 prompt: str

class OutputData(BaseModel):
 response: str
```

We'll initialize the model pipeline at application startup. Hugging Face pipelines are supported, and we can also use the Transformers auto classes. For example, for the GPT-2 small model for text generation, the syntax is as follows:

```
model = pipeline("text-generation", model="openai-community/gpt2")
```

Finally, we need to create a route that uses the model to generate text based on a prompt through the request:

```
@app.post("/generate", response_model=OutputData)
def generate(request: Request, input_data: InputData):
 prompt = input_data.prompt

 response = model(prompt)[0]["generated_text"]

 return OutputData(response=response)
```

The preceding route returns the generated text in response to an HTTP `POST` request.

You can package FastAPI applications for serving LLMs in Docker containers and deploy them to container platforms such as Kubernetes or cloud services like AWS and Azure. For our simple GPT-2 text-generation app, the Dockerfile looks like this:

```
FROM python:3.10-slim

WORKDIR /fastapi-demo

COPY requirements.txt .

COPY ./src ./src

RUN pip install -r requirements.txt


EXPOSE 8000


CMD ["uvicorn", "src.main:main", "--host", "0.0.0.0", "--port", "8000"]
```

In the preceding Dockerfile, we start from an official Python runtime image, set the root directory in the container, copy in the requirements file and source code, install dependencies, expose the listening port, and start the web server when the container starts.

11.2.1 Benchmarking various models

As mentioned earlier, FastAPI delivers strong performance. When serving LLMs with FastAPI, focus your benchmarking on model inference, since it has the biggest effect on overall performance.

For this subsection, a companion Colab notebook provides code to benchmark a model you'll serve through a FastAPI route. The notebook doesn't include the serving code because it requires tunneling, and Colab policies can change in ways that block tunneling techniques and third-party services. Still, you can use the notebook to benchmark different model versions, identify the fastest option, and choose a deployment candidate. Afterward, benchmark the FastAPI web application in a separate environment, as you would for any other web application.

This example uses the GPT-2 small model, but you can benchmark any GPT-2 size (including fine-tuned or PEFT variants) by swapping the model ID on the Hugging Face Hub. With minor changes to the auto classes and input preparation, the same approach works for other decoder-only architectures. GPT-2 small needs no hardware acceleration, but larger models or running all benchmarks at once will likely require a GPU.

Let's download the model and tokenizer from the Hugging Face Hub:

```
from pathlib import Path
from transformers import GPT2Tokenizer, GPT2Model

device = "cpu"
```

```
model_id = 'openai-community/gpt2'
tokenizer = GPT2Tokenizer.from_pretrained(model_id)
model = GPT2Model.from_pretrained(model_id)
model.eval()
model_path = Path("gpt2")
model.save_pretrained(model_path)
```

We'll benchmark the vanilla FP32 model first. Now we'll convert the checkpoints to ONNX using PyTorch's native `torch.onnx` package:

```
import torch
from transformers import BatchEncoding

input_ids: BatchEncoding = tokenizer(
    text, add_special_tokens=True,
    return_attention_mask=False, return_tensors="pt"
)
for k, v in input_ids.items():
    input_ids[k] = v.type(dtype=torch.int32)
input_tensor = input_ids['input_ids']
onnx_model_path = "gpt2_onnx.onnx"
dynamic_axes = {
    'input_ids': {0: 'batch_size', 1: 'sequence_length'},
    'logits': {0: 'batch_size', 1: 'sequence_length'},
}

torch.onnx.export(
    model,
    f=onnx_model_path,
    args=(input_tensor,),
    input_names=['input_ids'],
    output_names=['logits'],
    quantization=False,
    var_output_seq=True,
    do_constant_folding=True,
    opset_version=18,
    dynamic_axes=dynamic_axes
)
```

Next, we'll optimize the ONNX model:

```
from onnxruntime.transformers.optimizer import optimize_model

optimized_onnx_path = "gpt2_optimized.onnx"
optimized_model = optimize_model(input=onnx_model_path,
                                 model_type="gpt2",
                                 use_gpu=False)
optimized_model.save_model_to_file(optimized_onnx_path)
```

Finally, we'll downsize the model to FP16:

```
from copy import deepcopy

optimized_fp16_model_path = "optimized_fp16.onnx"
```

```
optimized_fp16_model = deepcopy(optimized_model)
optimized_fp16_model.convert_float_to_float16()
optimized_fp16_model.save_model_to_file(optimized_fp16_model_path)
```

By applying techniques from earlier chapters, we now have four versions of the same model. We'll benchmark them and choose one to serve through a FastAPI web app.

Let's define a simple helper function we can reuse to benchmark any of these models:

```
import timeit

def benchmark_llm(f, name=""):
    for _ in range(10):
        f()
    seconds_per_iter = timeit.timeit(f, number=100) / 100
    print(
        f"{name}:",
        f"{seconds_per_iter * 1000:.3f} ms",
    )
```

This function expects two input values: a function `f` that runs inference for a given model and a benchmark name (useful for building reports and visualizations). After an initial warmup, it runs the model 100 times and returns the average latency.

Before running the benchmarks, we need to prepare the inputs expected by each model version, along with the inference function to pass to `benchmark_llm`. We will use the following test prompt:

```
text = "Today is Saturday and"
```

This is the input for the vanilla PyTorch model:

```
inputs_base = tokenizer(text, return_tensors="pt").to(device)
```

The benchmark execution is then quite straightforward:

```
benchmark_llm(lambda: model(**inputs_base), "PyTorch")
```

The other three versions of the model are in ONNX format, so we need a bit of setup to prepare their input for ORT. First, we'll encode the prompt:

```
import torch

encodings_dict = tokenizer.batch_encode_plus([text])


input_ids = torch.tensor(encodings_dict["input_ids"], dtype=torch.int32)

ort_inputs = {
    "input_ids": input_ids.cpu().numpy()
}
```

Finally, we'll create inference sessions for the three ONNX models. They use the inputs prepared in the previous snippet and serve as Lambda functions for the benchmarks:

```
from onnxruntime import InferenceSession

providers=["CPUExecutionProvider"]
# ONNX model
sess = InferenceSession(onnx_model_path,
                        providers=providers)
benchmark(lambda: sess.run(None, ort_inputs), "ONNX")

# Optimized ONNX model
optimized_sess = InferenceSession(optimized_onnx_path,
                                  providers=providers)
benchmark(lambda: optimized_sess.run(None, input_feed=ort_inputs),
          "ONNX optimized")

# Downsized ONNX model
optimized_fp16_sess = InferenceSession(
    optimized_fp16_model_path, providers=providers
)
benchmark(lambda: optimized_fp16_sess.run(None, input_feed=ort_inputs),
          "ONNX optimized fp16")
```

By running the benchmark multiple times for four versions of the model, each time using prompts of different lengths (1, 4, 64, 256, 512, and 1,024 tokens), we can observe the following average latency patterns:

```
====== Tokens 1 ======
Pytorch (1 tokens): 69.091 ms
ONNX (1 tokens): 36.075 ms
ONNX optimized (1 tokens): 34.619 ms
ONNX optimized fp16 (1 tokens): 88.717 ms
====== Tokens 4 ======
Pytorch (4 tokens): 80.526 ms
ONNX (4 tokens): 38.535 ms
ONNX optimized (4 tokens): 36.813 ms
ONNX optimized fp16 (4 tokens): 149.577 ms
====== Tokens 64 ======
Pytorch (64 tokens): 281.216 ms
ONNX (64 tokens): 217.580 ms
ONNX optimized (64 tokens): 206.556 ms
ONNX optimized fp16 (64 tokens): 369.766 ms
====== Tokens 256 ======
Pytorch (256 tokens): 977.689 ms
ONNX (256 tokens): 768.100 ms
ONNX optimized (256 tokens): 803.686 ms
ONNX optimized fp16 (256 tokens): 1037.310 ms
====== Tokens 512 ======
Pytorch (512 tokens): 1948.008 ms
ONNX (512 tokens): 1722.430 ms
ONNX optimized (512 tokens): 1674.203 ms
ONNX optimized fp16 (512 tokens): 2299.876 ms
```

```
====== Tokens 1024 ======
Pytorch (1024 tokens): 3972.155 ms
ONNX (1024 tokens): 4048.070 ms
ONNX optimized (1024 tokens): 3973.077 ms
ONNX optimized fp16 (1024 tokens): 5784.016 ms
```

Up to 512 tokens, the ONNX optimized model has the best average latency. At the 1,024 context length, the vanilla, ONNX converted, and ONNX optimized models converge. Using FP16 with the ONNX optimized model doesn't improve performance; it degrades it. Overall, the ONNX optimized version is the best candidate.

11.2.2 Deploying the best-performing model with FastAPI

To deploy the selected model version behind a FastAPI route, we can follow the usual pattern for implementing routes in the framework. In addition, we need to implement a custom function to generate text with the ONNX-optimized GPT-2 small model:

```
import numpy as np
import torch
from onnxruntime import InferenceSession

async def generate(prompt, max_length, device):
  pt_inputs_base = tokenizer(prompt,
                             max_length=max_length,
                             return_tensors="pt").to(device)
  pt_inputs = tokenizer(prompt,
                        max_length=max_length,
                        return_tensors="pt").to(device)
  ort_inputs = dict(tokenizer(prompt,
                             max_length=max_length,
                             return_tensors="np",
                             return_attention_mask=False))
  ort_inputs['input_ids'] = ort_inputs['input_ids'].astype(np.int32)

  optimized_onnx_path = "gpt2_optimized.onnx"
  providers=["CPUExecutionProvider"]
  optimized_sess = InferenceSession(optimized_onnx_path,
                                    providers=providers)
  logits = optimized_sess.run(None, ort_inputs)
  np_logits = logits[0]
  output = torch.from_numpy(np_logits)
  predicted_ids = np.argmax(output[0], axis=-1)

  return output
```

The custom `generate` function takes as input the user prompt, maximum token count, and device to use (CPU in our example). In the body of the function, we prepare the input for the ONNX model, create an ORT inference session, run text generation, and decode the output to return the generated text in natural language. The `generate` function is declared `async` because, when the model is exposed through a web API, server performance can vary based on factors such as network speed and bandwidth or

the output size (which affects LLM latency). Making the function calls asynchronous frees the CPU to do other work while inference completes.

On the web app's side, we first need to create a class to handle the inputs for the generation. The `GenerateRequest` class extends Pydantic's `BaseModel` class and defines two fields—the user prompt and the maximum length:

```
from pydantic import BaseModel

class GenerateRequest(BaseModel):
    prompt: str
    max_length: int = 64
```

Next, we'll create the FastAPI app and implement a route that generates a sentence from a prompt and maximum length:

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/generate")
async def generate_text(request: GenerateRequest):
    device = "cpu"
    generated_sentence = await generate(request.prompt,
                                        request.max_length,
                                        device)

    return {"Generated sentence = ": generated_sentence}
```

In this section, you've seen how to deploy an SLM behind FastAPI, which is a good fit for most local and cloud deployments. The next sections cover a different scenario: deploying and serving SLMs on the edge.

11.3 MLC LLM

So far in this chapter, we've explored two ways to deploy and serve SLMs behind traditional REST APIs. In this section and the next, we'll move closer to the edge and look at options for on-device deployment and serving.

In this section, we'll look at MLC LLM (https://github.com/mlc-ai/mlc-llm/tree/main), an open source ML compiler and high-performance deployment engine for LLMs. It compiles and runs code on the MLCEngine, a unified high-performance LLM inference engine across multiple platforms. MLCEngine also provides an OpenAI-compatible API via a REST server and through Python, JavaScript, iOS, and Android, all backed by the same engine and compiler. Figure 11.3 shows the end-to-end workflow for MLC LLM.

Figure 11.4 lists the supported operating systems, CPUs, and GPUs at the time of writing.

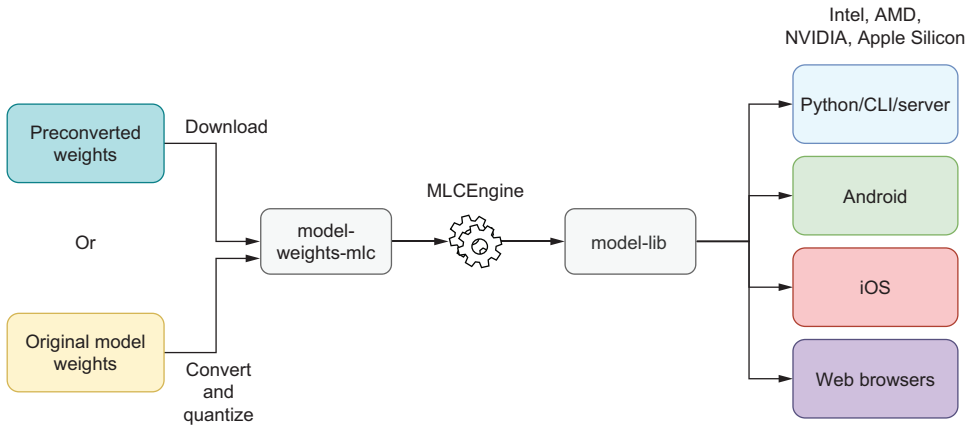


Figure 11.3 The end-to-end workflow for MLC LLM, from model conversion to deployment on the supported platforms' native runtimes

	AMD GPU	NVIDIA GPU	Apple GPU	Intel GPU
Linux / Win	✅ Vulkan, ROCm	✅ Vulkan, CUDA	N/A	✅ Vulkan
macOS	✅ Metal (dGPU)	N/A	✅ Metal	✅ Metal (iGPU)
Web browser	✅ WebGPU and WASM			
iOS / iPadOS	✅ Metal on Apple A-series GPU			
Android	✅ OpenCL on Adreno GPU		✅ OpenCL on Mali GPU	

Figure 11.4 Operating systems, CPUs, and GPUs supported by MLC LLM

Key strengths of the MLC LLM framework include cross-platform performance—especially on mobile and edge devices—universal deployment with a single engine, a familiar OpenAI-style API, and support for high concurrency, batching, and prompt caching.

Let's get familiar with the MLC LLM API. A companion Colab notebook provides the code example for this section, and it uses an NVIDIA GPU for hardware acceleration. The steps in this example apply to any LLM hosted on the Hugging Face Hub (a requirement), and they show how to prepare the model for inference on Linux-based devices.

First, we need to install the library for Linux and the available CUDA drivers:

```
python -m pip install --pre -U -f https://mlc.ai
➥/wheels mlc-llm-nightly-cu122 mlc-ai-nightly-cu122
```

After verifying the installation, we can download the target model weights and configuration files from the Hugging Face Hub.

We need to install Git LFS (https://github.com/git-lfs/git-lfs), a Git extension for versioning large files. Then we can create the destination directory for the converted weights and configuration and clone the target model repo from the Hub (in this example, RedPajama INCITE Instruct 3B, trained by Together AI in collaboration with several open source communities):

```
!mkdir -p dist/models

!git lfs install
!git clone https://huggingface.co/togethercomputer
➥/RedPajama-INCITE-Instruct-3B-v1
```

To convert the model's weights to MLC LLM format, we'll use the `mlc_llm_convert_weights` command. It takes as input the path to the directory where the model weights have been cloned, the quantization type, the target device (an NVIDIA GPU in our case), and the output directory that will store the converted weights:

```
mlc_llm convert_weight ./RedPajama-INCITE-Instruct-3B-v1/ \
    --quantization q4f16_1 \
    --device cuda \
    -o dist/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC
```

Once the conversion is completed, the next step is to generate the chat configuration for the selected model. This can be done using `mlc_llm_gen_config`, another command file provided by MLC LLM. It expects the path to the directory where you cloned the model weights (and configuration), the quantization type, the model chat template, and the output directory for the generated configuration (the same one used for the converted model weights):

```
mlc_llm gen_config ./RedPajama-INCITE-Instruct-3B-v1/ \
    --quantization q4f16_1 --conv-template redpajama_chat \
    -o dist/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC/
```

After the previous two steps, the content of the RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC directory should look like this:

```
mlc-chat-config.json    params_shard_1.bin   params_shard_32.bin
params_shard_45.bin
ndarray-cache-b16.json    params_shard_20.bin  params_shard_33.bin
params_shard_46.bin
ndarray-cache.json    params_shard_21.bin  params_shard_34.bin
params_shard_47.bin
params_shard_0.bin    params_shard_22.bin  params_shard_35.bin
params_shard_48.bin
params_shard_10.bin    params_shard_23.bin  params_shard_36.bin
params_shard_49.bin
```

```
params_shard_11.bin    params_shard_24.bin  params_shard_37.bin
params_shard_4.bin
params_shard_12.bin    params_shard_25.bin  params_shard_38.bin
params_shard_50.bin
params_shard_13.bin    params_shard_26.bin  params_shard_39.bin
params_shard_5.bin
params_shard_14.bin    params_shard_27.bin  params_shard_3.bin
params_shard_6.bin
params_shard_15.bin    params_shard_28.bin  params_shard_40.bin
params_shard_7.bin
params_shard_16.bin    params_shard_29.bin  params_shard_41.bin
params_shard_8.bin
params_shard_17.bin    params_shard_2.bin   params_shard_42.bin
params_shard_9.bin
params_shard_18.bin    params_shard_30.bin  params_shard_43.bin
tokenizer_config.json
params_shard_19.bin    params_shard_31.bin  params_shard_44.bin
tokenizer.json
```

The chat configuration file (`mlc-chat-config.json`), model weight metadata (`ndarray-cache.json`, `ndarray-cache-b16.json`), model shards, and tokenizer files (`tokenizer.json`, `tokenizer_config.json`) must be present for conversion and chat configuration to succeed.

We're not done yet. At this stage, the model is prepared (and can optionally be uploaded to the Hugging Face Hub), but to run it on a target platform or device, we still need to compile the model library according to the specifications in the generated `mlc-chat-config.json` file. In our Colab notebook example, the target for inference is a Linux device. (See the official MLC LLM documentation for the steps to compile the model for other operating systems.)

Our next step is to create the destination directory for the compiled model:

```
mkdir ./dist/libs
```

Now we'll compile the model with the `mlc_llm_compile` command. It takes as input the path to the directory containing the model chat configuration file, the target device type (NVIDIA GPU), and the output path and filename for the compiled model:

```
mlc_llm compile ./dist/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC
➥/mlc-chat-config.json \
    --device cuda -o dist/libs
    ➥/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-cuda.so
```

Now that the model is ready, we can use the MLC LLM Python API to create an `MLCEngine` instance for OpenAI-compatible chat completions in Python with MLC LLM ported models. It takes the model configuration path and the compiled model path as input:

```
from mlc_llm import MLCEngine

engine = MLCEngine(
   model="./dist/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC",
   model_lib="./dist/libs/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-cuda.so")
```

You can chat with the model much like you do with the OpenAI API:

```
response = engine.chat.completions.create(
    messages=[{"role": "user", "content": "What is the meaning of life?"}],
    model="./dist/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC",
    stream=False,
)
```

The `MLCEngine` class can also iterate over streaming responses:

```
for response in engine.chat.completions.create(
    messages=[{"role": "user", "content": "What's the meaning of life?"}],
    model="./dist/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC",
    stream=False,
):
    print(response)
print("\n")
```

The `MLCEngine` class handles only synchronous chat completions. For concurrent, asynchronous generation, use `AsyncMLCEngine`. If you've uploaded a converted, compiled model to the Hugging Face Hub, these engine classes can download it; just specify the model location in the following format:

```
HF://<model_id_in_the_hf_hub>
```

The chat-completion code for asynchronous generation via `AsyncMLCEngine` mirrors that of `MLCEngine`.

Besides using the Python API and CLI to run ported SLMs and the `MLCEngine`, you can also serve them through MLC LLM's REST API. The server is available automatically after you install MLC LLM. For the ported model in this section's example, we'll start the server from the command line as follows:

```
mlc_llm serve MODEL --model-lib ./dist
➥/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC
```

We'll use the same command-line tool we used to convert and compile the model, but this time, we'll specify a different action (`serve`) and provide the compiled model's location with the `--model-lib` option. The other options for this command are as follows:

- `--additional-models`—Serves multiple models.
- `--speculative-mode`—Specifies whether speculative decoding is enabled (and which type to use: small draft, eagle, or medusa) or disabled (default).

- `--device`—Specifies the device on which to run the model. By default, it's set to `auto`, which lets MLC LLM detect available devices and choose the best one. To target a specific device, use `device_name:device_id` or `device_name` (in the latter case, the device ID defaults to 0).
- `--mode`—Specifies the engine mode. Use `local` (low request concurrency), `interactive` (up to one concurrent request), or `server` (many concurrent requests; this uses as much GPU as possible).
- `--overrides`—Specifies values for one or more engine configuration parameters, such as maximum GPU utilization, enabling tensor parallelism, and context-window size.
- `--enable-tracing`—Enables request logging.
- `--host`—Specifies a host name.
- `--port`—Changes the port the server listens on. Default is 8000.
- `--allow-credentials`—Indicates whether the server should allow credentials.
- `--allowed-origins`—Specifies the allowed origins.
- `--allowed-methods`—Restricts the allowed HTTP methods.
- `--allowed-headers`—Specifies the allowed headers.

NOTE Speculative decoding is an inference-time optimization that lets an LLM predict multiple future tokens in parallel instead of generating one at a time. It's covered in chapter 12.

Once the server is running, you can access it at the hostname and port you specified at startup. The `/docs` endpoint lists all supported endpoints. A `GET` request to `/v1/models` returns the available models.

To interact with the server from any Python application (to run inference with a hosted model), implement a client as follows using the `Requests` package (https://github.com/psf/requests):

```
import requests
import json

payload = {
 "model": "./dist/RedPajama-INCITE-Instruct-3B-v1-q4f16_1-MLC/",
 "messages": [{"role": "user", "content": "Write a poem about Python"}],
 "stream": True,
}
with requests.post("http://127.0.0.1:8080/v1/chat
➥/completions", json=payload, stream=True) as r:
   for chunk in r.iter_content(chunk_size=None):
      chunk = chunk.decode("utf-8")
      if "[DONE]" in chunk[6:]:
         break
      response = json.loads(chunk[6:])
```

```
        content = response["choices"][0]["delta"].get("content", "")
        print(content, end="", flush=True)
print("\n")
```

MLC LLM applications for SLM serving can also be managed as Docker containers and deployed on devices that support containerization.

11.4 Deployment and inference on Android devices

There are several options for deploying private, offline SLMs on Android devices (https://www.android.com/intl/en_ie/). Many online examples use TensorFlow Lite or MediaPipe, which require developing SLMs in TensorFlow or converting PyTorch weights to TensorFlow via ONNX. This section focuses on PyTorch alternatives.

11.4.1 MLC LLM framework

One option for deploying to Android is to use the MLC LLM framework discussed in section 11.3. The MLC LLM documentation includes an end-to-end tutorial for building and deploying apps and SLMs on Android (https://llm.mlc.ai/docs/deploy/android.html). I won't repeat the steps here because the linked page explains them well. Here's a summary:

1. Install the prerequisites to build the model and app locally.
2. Build the runtime and the selected model libraries. This step is the same as for compiling and building a model for any other destination platform, as detailed in section 11.3.
3. Customize and build the template Android app available in the MLC LLM GitHub repository.
4. Bundle the app with the model weights.
5. Install the APK and weights on the target Android phone.

The MLC LLM documentation tutorial uses a Samsung S23 with a Snapdragon 8 Gen 2 chip, but the workflow is the same for other Android devices. Focus on the target device's hardware specs, benchmark performance, and optimize the model as needed. This approach has the advantages of porting and serving an SLM through the MLC LLM framework, but it also requires installing many third-party, non-Python components (Android Studio, rustc, the Java Development Kit, the NDK, and more) in your development environment, in addition to model conversion and compilation. Let's look at some other alternatives.

11.4.2 MLLM framework

Another potential open source option is mllm (https://ubiquitouslearning.github.io/mllm_website/), a lightweight, fast LLM engine for mobile and edge devices that supports text and multimodal inputs. It's implemented in plain C++ with no dependencies and supports a range of mobile hardware and quantization strategies. At the time of writing, mllm runs on Android, Linux, and macOS on x86_64 and arm64. You

must build it from source. On Linux and macOS, you'll need gcc 11.4+ and CMake 3.19+. On Android, you'll need Android NDK toolchain 26+.

First, you'll need to clone the official GitHub repo:

```
git clone https://github.com/UbiquitousLearning/mllm
```

mllm can be built with one of the provided scripts. On Linux/macOS, use `build.sh`:

```
cd ./scripts && ./build.sh
```

Several text and multimodal models are available in mllm format on the Hugging Face Hub and can be used as soon as the library build completes. Models are provided in FP32 and INT4 (CPU-optimized); a few are also optimized for the Hexagon NPU in INT8. See the mllm GitHub repo for the latest list.

During the build, a dedicated bash script for running inference is created in the bin directory for each model. For example, to run Llama 2 7B, download the already converted model from the Hub:

```
!mkdir ../models && cd ../models
!wget https://huggingface.co/mllmTeam/llama-2-7b-mllm/resolve
➥/main/llama-2-7b-chat-q4_k.mllm?download
➥=true  -O llama-2-7b-chat-q4_k.mllm
```

Then run the following script to start a chat session:

```
cd ./bin
./demo_llama -m ../models/llama-2-7b-chat-q4_k.mllm -v
 ../vocab/llama2_vocab.mllm
```

On Android devices, you'll need to use a different build script to build mllm, and you'll need to specify the Android NDK path:

```
export ANDROID_NDK=/path/to/your/ndk
cd ./scripts
./build_android.sh
```

Then, after downloading one of the available converted models from the Hugging Face Hub, you can start a chat session using the script for that model. Here's an example for the Llama 2 7B model:

```
cd ../script
./run_llama.sh
```

If the model you want to use isn't yet available in mllm, or if it's a custom model, the mllm repo includes a Python script, `convert.py`, that converts PyTorch or safetensors weights (single-file or sharded). You'll find it in tools/convertor. Before running it, set up a Python environment, and install the converter requirements:

```
cd ./tools/convertor
pip install -r ./requirements.txt
```

You can then use the script for a given model as follows for PyTorch-format checkpoints:

```
python convert.py --input_model=<model_name>.pth
--output_model=<model_name>.mllm --type=torch
```

Or like this, for checkpoints in safetensors format:

```
python convert.py --input_model=<model_name>.safetensors
--output_model=<model_name>.mllm --type=safetensor
```

Because device memory, storage, and computational resources are often limited, mllm supports INT4 quantization for mllm models, including custom models converted to the format this library requires. Only two quantization modes are supported: Q4_0 and Q4_K. After you build the mllm library, the `quantize` script is available in the bin directory:

```
cd ./bin
./quantize <model_name>.mllm model_q4_0.mllm Q4_K
```

11.4.3 Hugging Face Transformers

Another alternative, fully Python-based, is to deploy and run SLMs for on-device inference directly with the Hugging Face Transformers library. This method requires developer options to be enabled on the target device.

To check if developer options are enabled, open the device's Settings section, and confirm that Developer Options appears in the list. If not, go to Settings > About phone > Software Information, find the phone's Build Number, and tap it seven times.

Once that's done, you'll need to install the Termux app (https://termux.dev/en/) from the Google Play Store. Termux is an open source terminal emulator for Android that runs a Debian-based Linux environment. No rooting is required to install it or add packages through its package manager. Termux is essentially a portable Linux distribution, so you can also install compilers and interpreters for popular programming languages. This means you can install Python and create virtual environments to run your own SLMs.

After starting Termux, you can install Python from the command line. First, update the package list:

```
pkg update && pkg upgrade
```

Then proceed with the main installation:

```
pkg install python
```

After installation, install the Python package manager and a virtual environment manager:

```
pkg install python-pip
pip install virtualenv
```

You'll need to create a virtual environment to isolate the dependencies for SLM deployment and inference. Choose a destination directory on the device, and create the environment as you would on a laptop or server:

```
python -m venv slm-env
```

Now activate the virtual environment:

```
source slm-env/bin/activate
```

Finally, install the required PyTorch and Transformers packages (and any others needed to implement our Python inference script):

```
pip install torch transformers
```

We can now implement inference code for the selected model. The simple example that follows uses a bert-base-uncased model for a classification task, but you can use any model supported by the Transformers library:

```
import torch
from transformers import BertTokenizer, BertForSequenceClassification

model_id = 'google-bert/bert-base-uncased'
tokenizer = BertTokenizer.from_pretrained(model_id)
model = BertForSequenceClassification.from_pretrained(model_id)

<h1>Sample text</h1>
text = "That movie is a blockbuster"

<h1>Tokenize the text</h1>
inputs = tokenizer(text, return_tensors='pt')

<h1>Perform classification</h1>
with torch.no_grad():
 outputs = model(<strong>inputs)

<h1>Get the predicted class</h1>
predicted_class = torch.argmax(outputs.logits)
print(f"Predicted class: {predicted_class}")
```

We'll now save the script on the device and run it for inference like any other Python application. Assuming the script is named `bert_classification.py`, run it like this:

```
python bert_classification.py
```

That's it.

Running LLMs locally can be resource-intensive, so choose a model your Android device can handle in terms of memory, CPU (or GPU), and storage. After benchmarking on-device performance, try the optimization techniques from earlier chapters (including quantization) to speed up inference while maintaining acceptable output quality.

That concludes our survey of options for deploying and serving SLMs. The frameworks we covered share two key traits: an easy-to-use API and strong, out-of-the-box performance. The next chapter continues with local SLM deployment and serving, focusing on user-friendly ways to build powerful SLM-based assistants on a laptop.

Summary

- vLLM uses PagedAttention to manage memory efficiently by storing key–value cache blocks noncontiguously, reducing fragmentation.
- vLLM supports offline and online serving via OpenAI-compatible API endpoints for deploying language models.
- vLLM supports quantized models, including GGUF, and can automatically download and run them.
- vLLM provides an OpenAI-compatible REST API for serving models.
- vLLM provides benchmark scripts to measure serving performance (throughput, latency, and time to first token).
- FastAPI lets you build high-performance web APIs in Python, using type hints and generating documentation automatically.
- FastAPI routes define URL endpoints that handle HTTP requests via decorated Python functions.
- Pydantic BaseModel classes validate FastAPI request and response data.
- FastAPI apps can integrate Hugging Face models and pipelines as machine learning inference endpoints.
- MLC LLM compiles language models for cross-platform deployment with a unified engine and compiler.
- `mlc_llm convert_weight` converts model weights to MLC format with the specified quantization settings.
- `mlc_llm gen_config` generates chat configuration files for converted models, using the appropriate templates.
- `mlc_llm compile` builds platform-specific model libraries for target devices and OSs.
- The `MLCEngine` class provides an OpenAI-compatible chat-completions API for MLC LLM–converted models.
- MLC LLM serves models via a REST API, supporting concurrent requests and hosting multiple models.

- Deploying on Android requires enabling developer options; you can use the MLC LLM framework or other approaches.
- The mllm framework provides a lightweight C++ inference engine for mobile devices with built-in quantization support.
- The Termux app lets you set up a Python environment on Android to run Transformers models directly.

12 Running on your laptop

This chapter covers:

- Reasons for using a personal, local SLM-based assistant
- Using Ollama's server to run SLMs locally and offline
- Using the LM Studio desktop app to run SLMs locally and offline
- Using the Jan assistant app to run SLMs locally and offline

In chapter 11, we looked at frameworks for serving private, domain-specific SLMs behind endpoints at scale for client applications, as well as options for running SLMs on edge devices, such as resource-constrained Android phones. This chapter explores ways to run personal SLM-based assistants or productivity tools directly on a laptop, including completely offline.

12.1 Why use a personal local assistant

Platforms that expose commercial LLMs through a chat interface or API—such as those from OpenAI, xAI, Anthropic, and Google—can learn about their users in ways we can't fully realize. They collect data from user interactions and turn that information into assets. Even when platforms let users opt out of data collection or the use of prompts, generated responses, and other sensitive data for model tuning, their closed nature makes it hard to know how consistently those choices are honored. This matters because information can be unintentionally exposed, not only through third-party chat systems and APIs, but also via client laptops and smartphones connected to the internet. This is critical not only at the enterprise level, but also for personal use. LLMs can boost productivity, but privacy and safety concerns can't be ignored.

Fortunately, as you've seen in earlier chapters, there are alternatives to third-party online models and services, both for individuals who want private AI assistants and for companies that need secure tools for employees. These alternatives extend beyond open source small language models; there are also ways to serve models on a user's laptop and higher-level tools to interact with them.

This chapter explores options for running SLMs locally, safely, and offline on a laptop using only open source components. This doesn't just apply to running chatbots but also to running SLMs for broader tasks, including complex systems such as RAG applications.

12.2 Running an LLM locally with Ollama

We'll start with Ollama (https://ollama.com/), an application based on llama.cpp (a C/C++ library introduced in chapter 6) that lets you interact with LLMs on your local computer. You can use any community-created GGUF-format quantized model available on the Hugging Face Hub (at the time of writing, more than 45,000 are hosted there). You can also convert your own models to the format Ollama expects, as explained later in this section. Ollama bundles model weights, configuration, and data into a single package defined by a Modelfile, and it optimizes setup details, including GPU usage.

Installing Ollama is straightforward: download the version for your operating system (Windows, macOS, or Linux) from the official website. A Docker image, ollama/ollama, is also available on Docker Hub. Once installed, Ollama exposes an API that serves the model, so users can interact with it locally, either directly or through a client application.

To run a model, use the `run` command in a terminal. If the model isn't installed yet, Ollama downloads it first. You can browse the long list of available models on the Ollama website (see figure 12.1).

The list includes not only text models but also vision, multimodal, and embedding models. For example, to run the DeepSeek-R1 8B model, use the following command:

```
ollama run deepseek-r1:8B
```

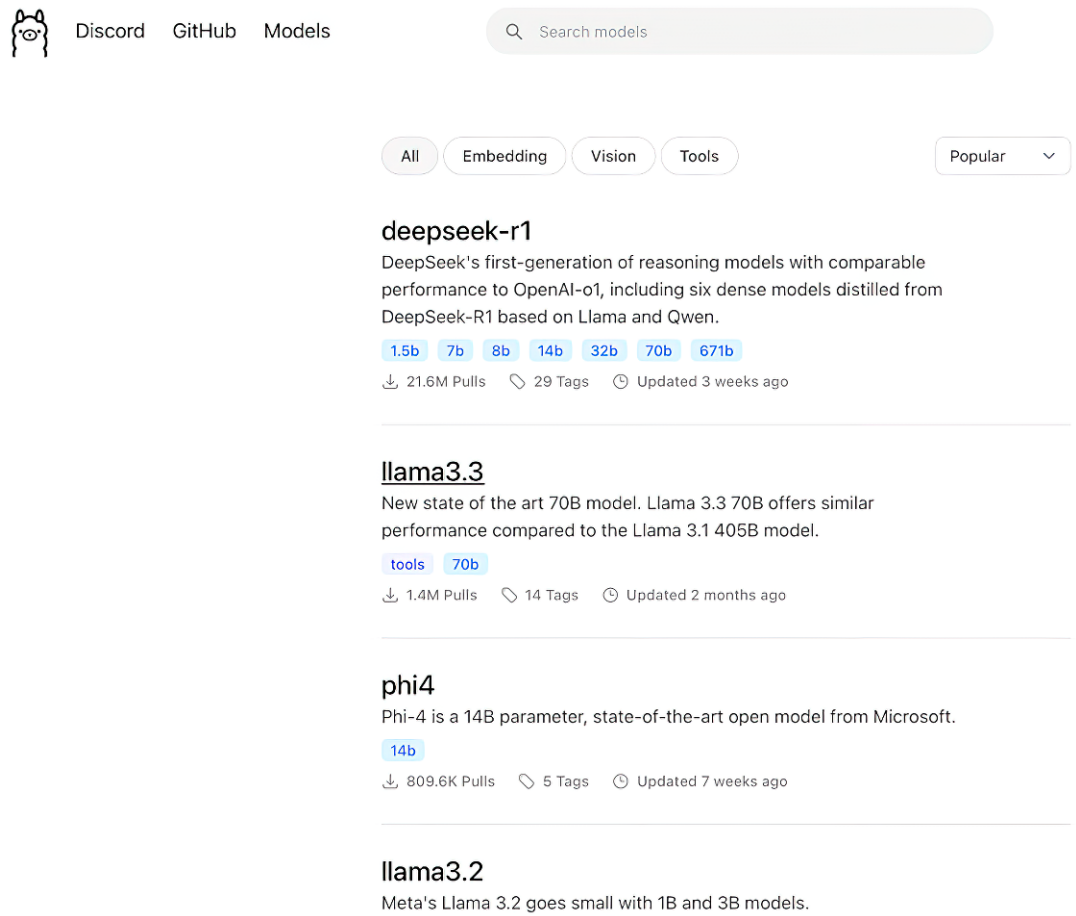


Figure 12.1 Ollama's online model catalog

The first time you run a model with `run`, Ollama downloads it from the Ollama repo. Subsequent `runs` start the CLI chat immediately. Figure 12.2 shows an active chat session with the DeepSeek-R1 8B model on macOS.

All the models in the Ollama repository are quantized, which reduces memory and compute requirements compared to the originals. Many can run on a laptop without hardware acceleration because they're optimized for CPUs (assuming you have enough RAM). As a rule of thumb, you'll typically need at least 8 GB of RAM for 7B or 8B models, 14 to 16 GB for 13B models, and 32 GB for 33B models.

If a model isn't in the Ollama catalog but is hosted on the Hugging Face Hub, you can still use the `run` command to download and run it locally. Just provide the model's Hub URL:

```
ollama run hf.co/mlabonne/Meta-Llama-3.1-8B-Instruct-abliterated-GGUF
```

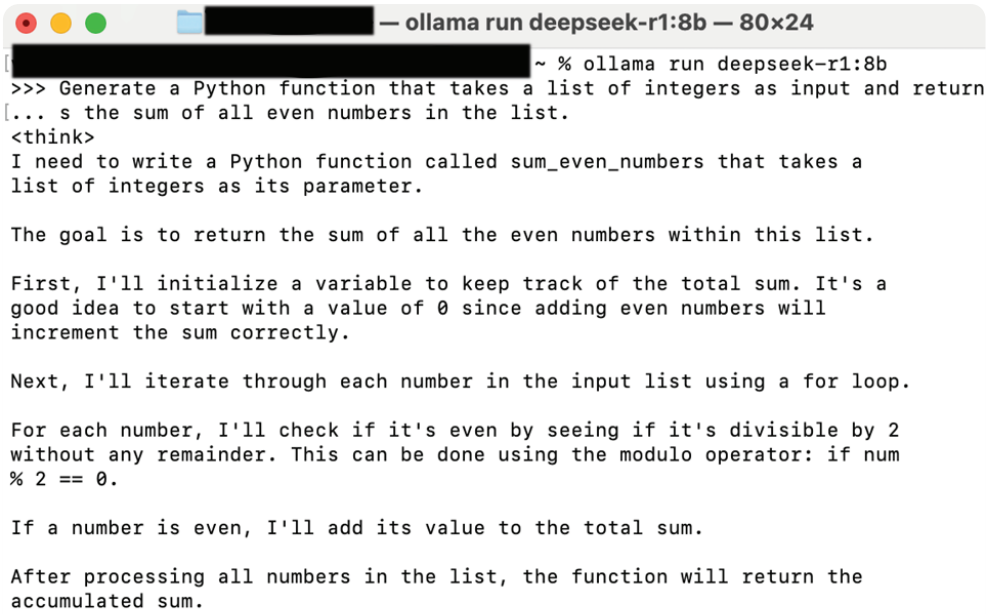


Figure 12.2 A live local chat session with the DeepSeek-R1 8B model through Ollama on macOS

To list the models installed in your local Ollama instance, run the following command in a terminal:

```
ollama list
```

It will print out a list, as follows:

```
NAME              ID              SIZE      MODIFIED
deepseek-r1:8b    28f8fd6cdc67    4.9 GB    4 weeks ago
mistral:latest    f974a74358d6    4.1 GB    6 weeks ago
```

Ollama can serve multiple models at the same time, though the chat and inference speed will depend on your laptop's hardware. To remove models you no longer need, delete them with `rm`, using the model name shown in the online catalog and in your local list:

```
ollama rm mistral:latest
```

Ollama also includes a REST API for running inference on locally installed models from a client application. Its default listening port is 11434. The `/api/tags` endpoint accepts HTTP `GET` requests and returns the list of installed models, such as the CLI `list` command:

```
{"models":[{"name":"deepseek-r1:8b","model":"deepseek-r1:8b","modified_
at":"2025-01-28T18:38:42.092602824Z","size":4920738407,"digest":"28f8fd6cdc
677661426adab9338ce3c013d7e69a5bea9e704b364171a5d61a10","details":{"parent_
model":"","format":"gguf","family":"llama","families":["llama"],"parameter_
size":"8.0B","quantization_
level":"Q4_K_M"}},{"name":"mistral:latest","model":"mistral:latest","modified_
at":"2025-01-18T15:41:59.669938897Z","size":4113301824,"digest":"f974a74358
d62a017b37c6f424fcdf2744ca02926c4f952513ddf474b2fa5091","details":{"parent_
model":"","format":"gguf","family":"llama","families":["llama"],"parameter_
size":"7.2B","quantization_level":"Q4_0"}}]}
```


`POST` requests to other endpoints let client applications perform any CLI operation, including chat completions. Here's an example request to a local Mistral 7B model served through Ollama:

```
curl http://localhost:11434/api/chat -d '{
  "model": "mistral:latest",
  "messages": [
    {
      "role": "user",
      "content": "Why is the sky blue?"
    }
  ]
}'
```

So far, we've seen SLMs served through Ollama from the CLI, which is a suitable option for users with a strong technical background. To make running SLMs locally accessible to more people, the Ollama open source community has developed an ecosystem of visual client applications. The official Ollama GitHub repository (https://github.com/ollama/ollama) maintains an up-to-date list. Which frontend you choose mostly comes down to preference and requirements.

We'll look at LocalLLM (https://github.com/qusaismael/localllm), a Python-based option, which means there's no need to install and maintain other language compilers, interpreters, or related configuration. LocalLLM is a minimal, user-friendly web frontend for chatting with local models served through Ollama. It's built with Flask (https://github.com/pallets/flask), a lightweight Python web framework. Although it's a young project, LocalLLM includes useful features such as switching between local models in the UI, support for streaming responses, Markdown and code-block rendering, and a raw-text debug view (particularly helpful when interacting with models such as DeepSeek-R1 that use test-time compute). A prerequisite is a local instance of Ollama serving the SLMs you want to use.

To install LocalLLM, create and activate a Python virtual environment, and then clone the GitHub repo:

```
git clone https://github.com/qusaismael/localllm.git
cd localllm
```

Then install its requirements:

```
pip install -r requirements.txt
```

The application can then be started as follows:

```
python app.py
```

The UI can be accessed at `http://localhost:50001`. Figure 12.3 shows the LocalLLM UI.

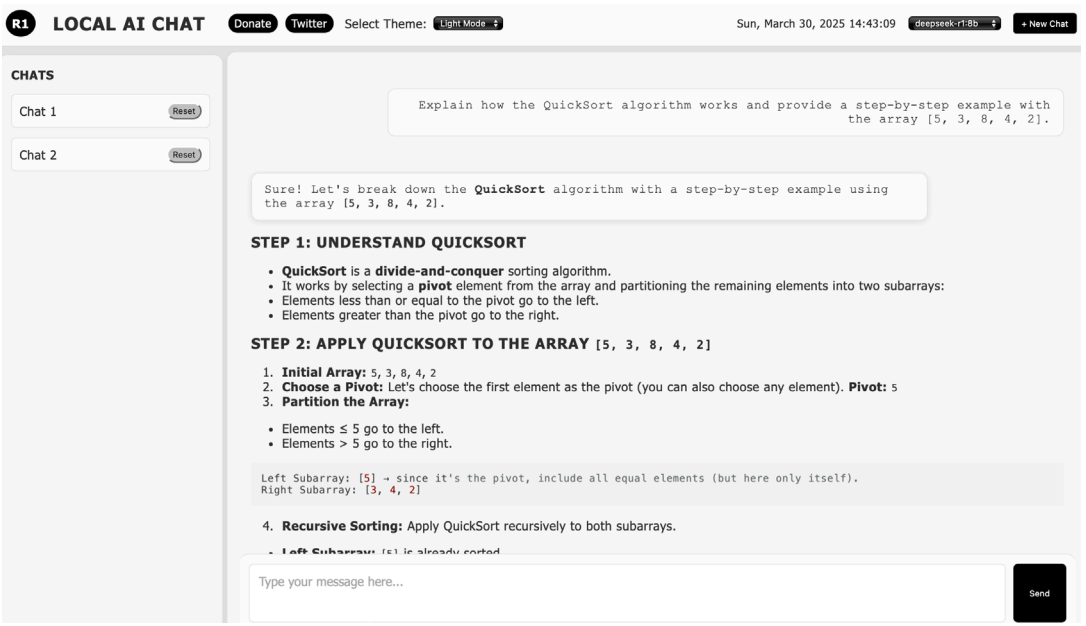


Figure 12.3 The LocalLLM UI

The left panel shows the chat list for the current session (at the time of writing, this web app doesn't yet persist chat history). The central panel is the chat area: the input box and Submit button are at the bottom, and the model output (and, if enabled, the raw output) appears above. At the top right is the model selector; its dropdown lists the SLMs currently available on the local Ollama server.

Remember, this application is for local use, not production deployment, because it has no authentication.

12.2.1 Importing a custom model into Ollama

As mentioned earlier, there are plenty of GGUF models on the Hugging Face Hub and in the Ollama catalog. You may well find a model that fits your needs there, but you

may also want to fine-tune a baseline model on your own private data and make it available for chat on your laptop or to a broader group in your organization. Through the llama.cpp library, Ollama can convert custom SLMs to the GGUF format and add them to a local server for use via the CLI or REST API.

To convert and import a model, first build llama.cpp locally. Clone the official GitHub repo, and install its requirements:

```
git clone https://github.com/ggerganov/llama.cpp.git
cd ./llama.cpp
pip install -r requirements.txt
```

In this example, we assume our custom model is uploaded to the Hugging Face Hub and under version control. If the model is only local, you can skip the next step. Install Git LFS, and clone the Hugging Face Hub repo that hosts the model and its associated files:

```
cd ..
git lfs install
git clone https://huggingface.co/TechXGenus/starcoder2-3b-instruct
```

The model in this example is a version of BigCode's StarCoder2 3B (an SLM specialized for code generation), fine-tuned on an additional 0.7 billion high-quality code-related tokens.

After cloning it locally, convert the model to GGUF format using `convert_hf_to_gguf.py`, a Python script in the llama.cpp source code repository:

```
mkdir gguf_model
cd ./llama.cpp
python convert_hf_to_gguf.py ../starcoder2-3b-instruct/
➥--outfile ../gguf_model/starcoder2-3b_instruct.gguf
```

This script takes two arguments: the path to the directory where you cloned the original model, and the path and filename for the converted model. After conversion, we need to create a *Modelfile*. A Modelfile describes how to deploy the converted custom model to a local Ollama server and serve it for inference. In this case, our Modelfile would look like this:

```
FROM starcoder2-3b_instruct.gguf

PARAMETER num_ctx 4096
TEMPLATE """<|endoftext|>You are an exceptionally intelligent coding
assistant that consistently delivers accurate and reliable responses to user
instructions.
### Instruction
{{ .Prompt }}
### Response
"""
```

```
PARAMETER stop "###"
PARAMETER stop "<|endoftext|>"
PARAMETER stop "<file_sep>"
```

The `FROM` command specifies the GGUF model name. The `TEMPLATE` command specifies an optional system template. The `PARAMETER` command lets you set text-generation parameters such as context length and stop words. Create the Modelfile in the same directory as the converted GGUF model.

Assuming we have a local instance of Ollama already running, deploy the model using the `ollama` CLI's `create` option:

```
cd ./gguf_model
ollama create starcoder2-3b-instruct-new -f Modelfile
```

Once it's created, you can chat with the model from the terminal:

```
ollama run starcoder2-3b-instruct-new:latest
```

Here is an example of code-generation prompt for this custom model:

```
>>> Write two different modular functions with documentation to add and
multiply two numbers in Python.
```

And here is the resulting sample code:

```
```python
# Modular Function 1
def add_numbers(a, b):
    """
    This function adds two numbers and returns the result.

    Args:
    a: The first number.
    b: The second number.

    Returns:
    The sum of `a` and `b`.
    """
    return a + b


# Modular Function 2
def multiply_numbers(a, b):
    """
    This function multiplies two numbers and returns the result.

    Args:
    a: The first number.
    b: The second number.

    Returns:
```
```

```
    The product of `a` and `b`.
    """
    return a * b


# Sample Usage
print("Addition Result:", add_numbers(5, 3))
print("Multiplication Result:", multiply_numbers(2, 4))
```
```
This script contains two modular functions named `add_numbers()` and
`multiply_numbers()`. The comments on these functions explain what they do.

The sample usage at the end demonstrates how to use these functions. It will
print "Addition Result: 8", "Multiplication Result: 8".
```

12.2.2 User privacy in Ollama

Once a model has been downloaded locally, Ollama can run completely offline, so the user's laptop doesn't need to be connected to the internet or a LAN. That's strong evidence for privacy preservation. But what happens if you chat locally with an SLM in Ollama while the hosting laptop is online? Does the Ollama server send any information out, and is there anything you should watch for to keep your privacy intact? The considerations in this section reflect the latest release available when this chapter was written (0.5.13). Please verify it yourself, since future versions of Ollama may behave differently.

A review of Ollama's current source code, along with packet sniffing of a running local instance, shows nothing suspicious. Everything happens on a locally hosted server. Still, a few things are worth watching in case they change in the future. The first is the `updater.go` file. The body of the `IsNewReleaseAvailable` function calls the `external endpoint` `https://ollama.com/api/update` every 60 minutes, sending anonymized information about the laptop's operating system, version, and architecture, along with the current timestamp. This function runs as part of the regular update check to see whether a new release is available for the local configuration. There seems to be no way to disable this through the local server configuration. The only option is to clone the Ollama source code repository locally and comment out the following line in `lifecycle.go`:

```
StartBackgroundUpdaterChecker(ctx, t.UpdateAvailable)
```

Then you can build the application locally.

Another thing to pay attention to is the history file. It's created in your local Ollama installation directory and is updated with every prompt sent to the local models during chats. It stores only prompt text, but because prompts may contain sensitive information, you should protect this plaintext (not encrypted) file from unauthorized access by intruders or malware.

You should similarly pay special attention to the Ollama server log file. It's created and updated in the logs subdirectory of your local Ollama installation. It records REST

API calls and model use, but it doesn't include sensitive information. Even so, it's good practice to apply security measures to prevent unauthorized access, because the file isn't encrypted.

In short, Ollama seems to preserve user privacy during local-model inference and chat, even when the laptop is connected to the web.

That concludes our look at the Ollama tool. We'll return to it in the next chapter where we'll explore end-to-end solutions that use small language models.

12.3 Running an LLM locally with LM Studio

The second tool we'll explore is LM Studio (https://lmstudio.ai/). It's free but not open source like Ollama. Still, it's worth considering: it offers open source utilities (including Python and JavaScript SDKs and the Apple MLX engine) and a polished out-of-the-box experience for users with a range of technical skills. LM Studio is available for macOS (M-series), Windows, and Linux. Like Ollama, it can run GGUF models (and Apple MLX models) locally and entirely offline after you download them from the LM Studio catalog (see figure 12.4) or the Hugging Face Hub.

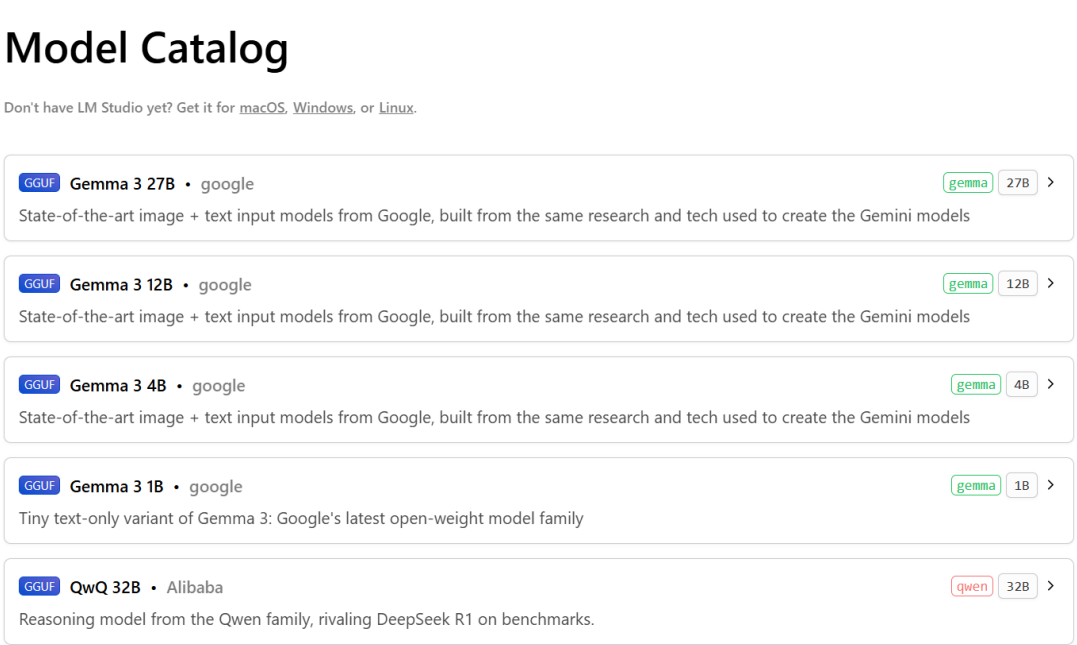


Figure 12.4 LM Studio's online model catalog

LM Studio's UI has several views. If you just want to interact with a local model, you can use the chat view (see figure 12.5).

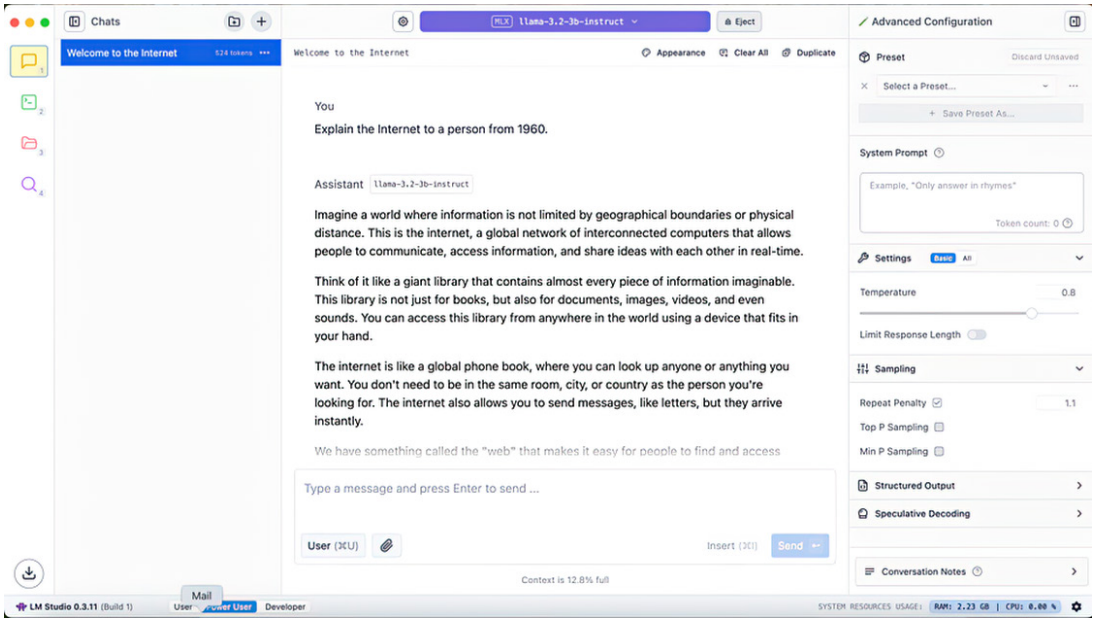


Figure 12.5 LM Studio's chat view

The center area of the chat view is where you interact with the model; the left panel shows the chat history, and the right panel contains advanced settings (system prompt, temperature, and sampling parameters). The selected model is identified at the top; in this example, it's Llama 3.2 3B Instruct (the GGUF version from the LM Studio catalog). As shown in figure 12.5, you can also attach documents to a chat session (.docx, .pdf, and .txt) to provide extra context. If the document fits in the model's context window, LM Studio adds its content to the conversation in full. Otherwise, LM Studio uses retrieval-augmented generation (RAG; introduced in chapter 2 and discussed in more detail in chapter 13). Long documents are split into chunks and provided to the model for reference.

For advanced users, such as developers who can build client applications that interact with local models through the LM Studio server's REST API, the developer view is available (see figure 12.6).

To understand the developer view, you'll first need to know about the LM Studio API server. Installing the LM Studio desktop app also installs its companion CLI tool. In addition to loading and unloading models, the `lms` CLI can start and stop LM Studio's local, OpenAI-like API server and inspect raw LLM input and output. This is one of the open source components of LM Studio. You can also build `lms` from source (https://github.com/lmstudio-ai/lms) and, in theory (though it is challenging), install it separately.

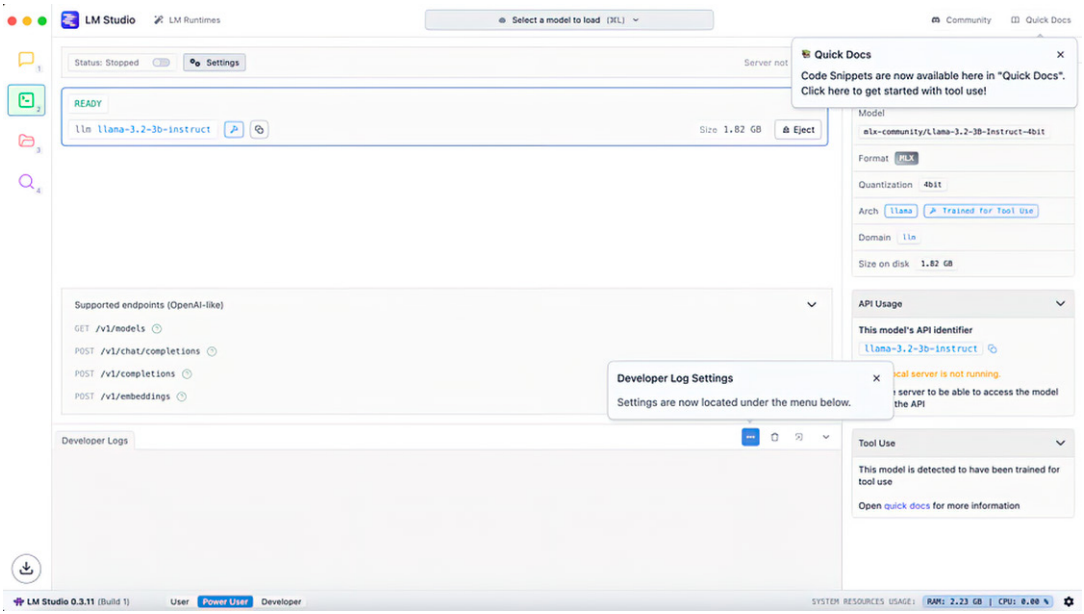


Figure 12.6 LM Studio's developer view

To start a local `lms` server, use this command:

```
lms server start
```

Once the server starts, you can use the developer view to track and debug any API call, including completions, chat completions, and embeddings. The top-center area of the UI (see figure 12.6) shows the loaded models, the bottom-center area shows the server log stream, and the right panel shows the selected model's details and inference settings (system prompt, temperature, sampling parameters, schema for structured output, speculative decoding settings, context length, seed, and KV cache quantization).

To stop the server, run the following command in a terminal:

```
lms server stop
```

As when chatting with a model in the UI, starting a local server and calling the REST API doesn't require the laptop to be connected to the internet or a LAN. You only need connectivity to download the model.

LM Studio 0.3.10 introduced *speculative decoding* (for models in GGUF or MLX formats). This inference optimization was proposed in the "Fast Inference from Transformers via Speculative Decoding" paper (https://arxiv.org/abs/2211.17192). In LLMs, it speeds up generation by drafting multiple future tokens in parallel and then having a larger model verify them.

Speculative decoding runs two models side by side: a large target LLM and a smaller, faster draft model. Both models must use the same tokenizer. The draft model generates candidate text, and the target model accepts or rejects those tokens to produce the final output. Because Transformer models can evaluate many tokens at once while predicting the next one, speculative decoding uses that capability to reduce generation time while preserving quality.

In LM Studio, speculative decoding works like this:

1. The draft model runs, quickly predicting the next few tokens as a draft.
2. The target model confirms or rejects each draft token, accepting only those it would have generated itself.

This can speed up generation (when enough tokens are accepted) without degrading quality. But if tokens are often rejected, total generation will likely slow down. Model choice matters: besides sharing a tokenizer, the draft model should be much smaller than the target model and from the same family.

Let's walk through an example of speculative decoding in LM Studio. We'll use Llama 3.2 3B Instruct (4-bit quantized) as the target model and Llama 3.2 1B Instruct (4-bit quantized) as the draft model. If you've downloaded them locally (via the `lms` CLI or the Mission Control panel in the LM Studio desktop app), you can select them in the Speculative Decoding panel at the bottom right of the chat view (see figure 12.7).

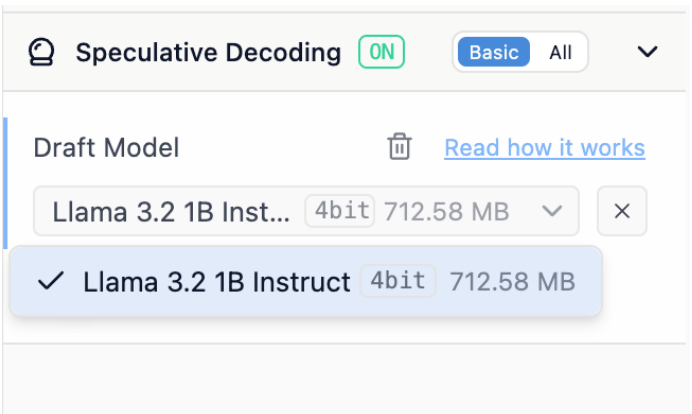


Figure 12.7 The LM Studio's Speculative Decoding panel

Once both models have been selected, turn on the Visualize Accepted Draft Tokens switch in the Speculative Decoding panel to see colored tokens showing whether they were generated by the draft model or the target model. You can then start a chat as usual in the user view. The only changes you'll notice are draft-model tokens highlighted in green (see figure 12.8) and faster content generation.

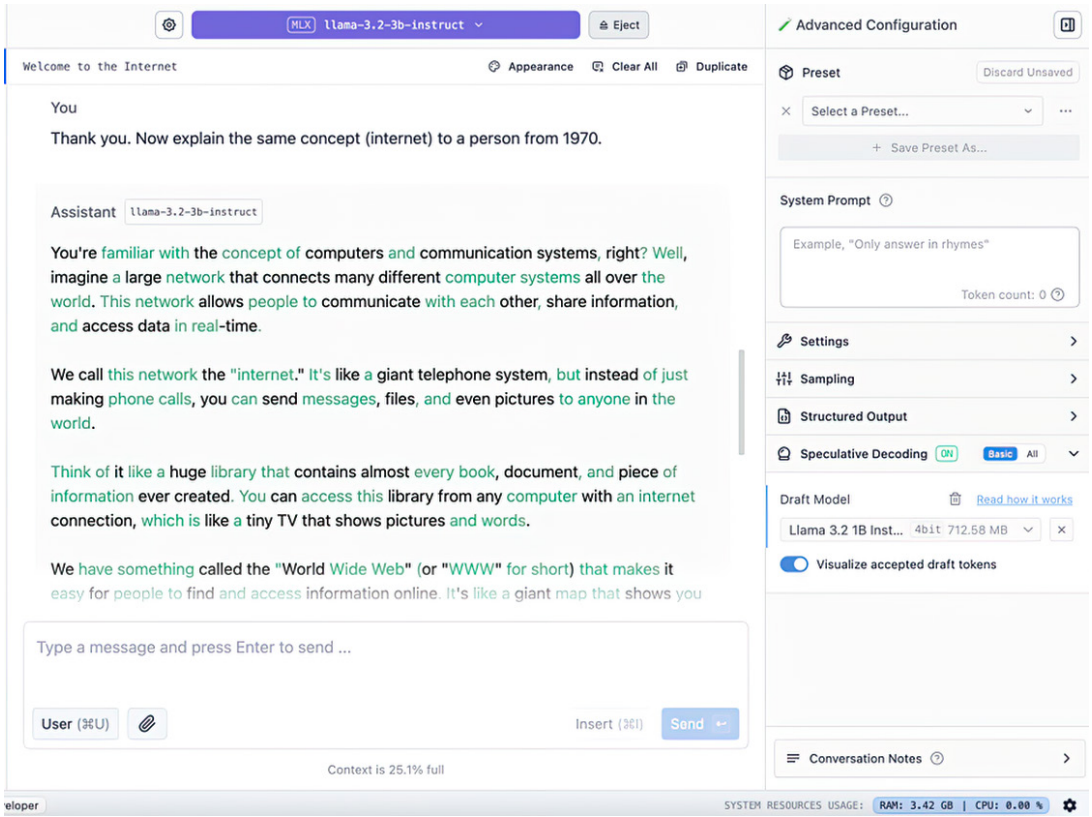


Figure 12.8 Speculative Decoding in action with the Llama 3.2 3B Instruct (target) and Llama 3.2 1B Instruct (draft) models in LM Studio

Benchmarks with these models on a MacBook Air M1 show a slower ramp-up: first-token generation with the larger model takes almost twice as long. Still, the average time only increases from about 1.5 seconds to 3 seconds, and generating 500 tokens is faster overall. With speculative decoding, you can cut total time by about 30% (roughly 9 to 10 seconds). Speculative decoding can be used through the REST API or programmatically via LM Studio's Python SDK.

12.4 The LM Studio Python SDK

LM Studio's Python SDK is an open source library (https://github.com/lmstudio-ai/lmstudio-python), and it's available on PyPI:

```
pip install lmstudio
```

This library lets you interact with models served by the LM Studio server from Python clients, Jupyter notebooks, or the Python REPL. Anything you can do in the UI, you

can do programmatically with this SDK. It provides a synchronous convenience API and a scoped resource API that uses `with` statements for deterministic cleanup of network resources. The convenience API works well for a CLI or notebook cells. For larger applications, use the scoped API so context managers reliably close connections and other allocated resources.

Let's see how this API works by walking through examples for chat, text completion, embedding generation, and agentic flow. For the code examples in this section, we'll assume an LM Studio API server is running locally and a model is loaded and ready to accept requests (here, the Llama 3.2 3B Instruct 4-bit quantized version from the LM Studio online catalog). You can chat with the loaded model using the API with the following very high-level code:

```
import lmstudio as lms

model = lms.llm('llama-3.2-3b-instruct')
print(model.respond("What is the meaning of life?"))
```

As you can see, we specified the model ID and then called its `respond` method. The returned value is a `PredictionResult` instance whose `content` attribute contains the generated text.

You can do the same through the scoped resource API as follows:

```
import lmstudio as lms

with lms.Client() as client:
    model = client.llm.model("llama-3.2-3b-instruct")
    result = model.respond("What is the meaning of life?")

    print(result)
Inferencing parameters can be passed to the respond method. Example:
result = model.respond("What is the meaning of life?", config={
    "temperature": 0.2,
    "maxTokens": 200,
})
```

With either approach, you can stream the chat response by calling `respond_stream` instead of `respond`.

Next, we'll manage a chat context. The model receives a multiturn conversation as input and predicts the assistant's response. First, create a chat with an initial system prompt using the `Chat` helper class:

```
chat = lms.Chat("You are a helpful assistant.")
```

Then, within an infinite `while` loop, we'll implement the code that handles chat turns between the model and a human user.

Listing 12.1 Managing a chat context with the LM Studio Python API

```
while True:
    try:
        user_input = input("You (leave blank to exit): ")
    except EOFError:
        print()
        break
    if not user_input:
        break
    chat.add_user_message(user_input)
    prediction_stream = model.respond_stream(
        chat,
        on_message=chat.append,
    )
    print("Bot: ", end="", flush=True)
    for fragment in prediction_stream:
        print(fragment.content, end="", flush=True)
    print()
```

When it's your turn, the script pauses for input. The model's responses are streamed via the `respond_stream` method. To end the chat and stop the script, press Enter on an empty prompt.

This SDK also supports text completion. To do this, call `complete` on your chosen model, as shown here:

```
result = model.complete("Today is Friday and", config={"maxTokens": 200})
```

You can generate embeddings for input text programmatically with this SDK. First, you need an embedding model downloaded locally and served. By default, LM Studio Server includes Nomic AI's Nomic Embed Text v1.5, but you can choose a different embedding model.

To convert a prompt into a vector, pass it to the `embed` method on the appropriate embedding model handle:

```
import lmstudio as lms

model = lms.embedding_model("nomic-embed-text-v1.5")
embedding = model.embed("A fancy prompt!")
```

One interesting feature of this Python SDK is the ability to set up an agentic flow. Instead of using an SLM in a simple text-in/text-out fashion, you can ask the model to call a tool (or a set of tools) and handle the returned output. The LM Studio SDK introduced the concept of *execution rounds*, which describe the cycle of running a tool, passing its output to an LLM, and letting the LLM choose the next step. The model can repeat this cycle multiple times before returning a final result.

The `act` function in the LM Studio SDK implements automatic, multi-round tool calling. Let's look at a basic example. Assume we've implemented a regular Python

function that performs some actions, given inputs, and returns a result. For example, the function could take two floats as input, multiply them, and return the result:

```
def multiply(a: float, b: float) → float:
    return a * b
```

We pick up a model available in the local server as usual:

```
import lmstudio as lms

model = lms.llm("llama-3.2-3b-instruct")
```

We can then call the `act` function for it, passing a prompt and the tool function (`multiply`):

```
model.act(
  "What is the result of 123 multiplied by 456?",
  [multiply],
  on_message=print,
)
```

In the preceding code, we pass a list of tools, so you can provide multiple tool functions for a single `act` call. Based on the prompt, the model decides which tool(s) to use. Our example tool is intentionally simple, but a tool function can run any Python code, including access to the web, local files and applications, and databases. Design tools defensively: validate inputs, handle failures, and plan for misuse.

An agentic workflow's performance depends on the model you choose, so evaluate candidate SLMs (or create evaluation tasks for custom-tuned SLMs) before you make your final choice. We'll discuss agentic workflows further in chapters 13 and 14.

12.5 Running an LLM locally with Jan

The third tool we'll explore is Jan (https://github.com/janhq/jan). Like Ollama, it's 100% open source and shares most of the same features as Ollama and LM Studio. Like LM Studio, Jan provides a desktop app (macOS, Linux, and Windows) that lets you chat with local models and models hosted behind major commercial APIs (OpenAI, Anthropic, Groq, Google, OpenRouter, and others). We'll focus on local models here. As with the other tools we've looked at, you don't need to be online to chat with local LLMs. You only need an internet connection to download a model the first time or to connect the Jan desktop app to a cloud API.

The Jan UI is simpler than the LM Studio's UI. After downloading and installing the version for your operating system, start the app to see the chat view (see figure 12.9). The center pane is the chat area: the prompt box (and an area for optional RAG file uploads) is at the bottom, and the message history is above. As shown in the figure, responses are well formatted, and generated code includes syntax highlighting and allows easy copy-and-paste.

NOTE Retrieval and chat features with documents are currently available only for multimodal models. PDF documents and images are the only supported file types to date.

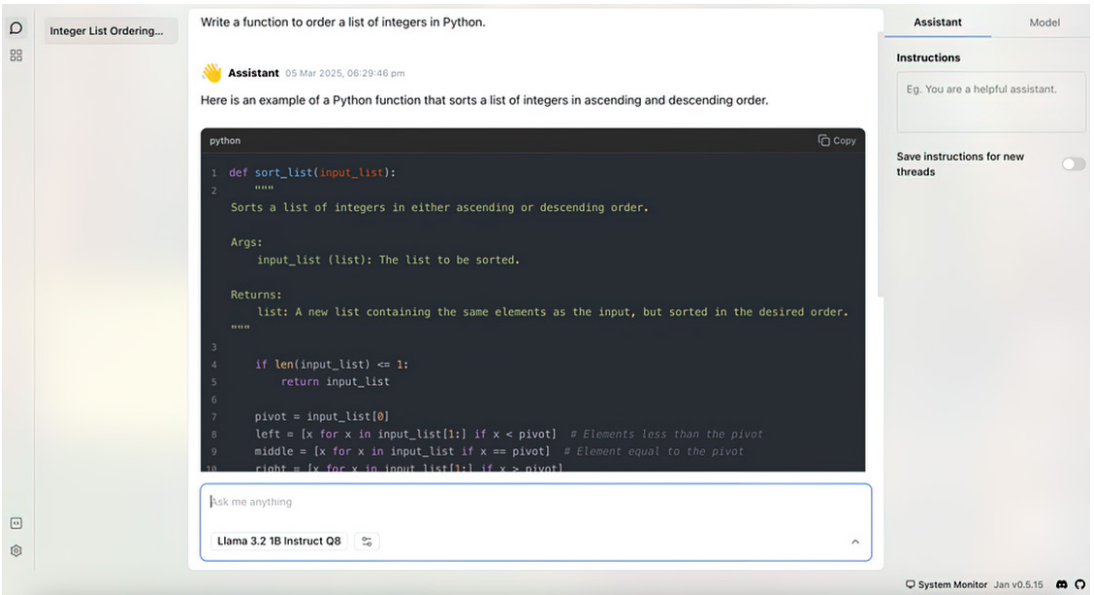


Figure 12.9 The chat view of the Jan desktop application

You can download local models from the Jan Hub, the default online catalog, or the Hugging Face Hub. Models must be in GGUF format, which is the only format Jan supports at the time of writing, though future releases may add ONNX Runtime support. You can browse and select models hosted in Jan Hub directly from the desktop app (see figure 12.10).

To help you choose a model, Jan uses your laptop's hardware specs to warn you about models that will run slowly or need more memory than you have. At the top of the model selection view, you can import GGUF models from the Hugging Face Hub: enter the model's URL or ID, and click Import Model. To import a custom local model that isn't shared online, open Settings (the gear icon at the bottom left), go to My Models, click Import Model, select the local GGUF file, choose whether to create a symbolic link or copy the file into Jan's directory, and click Import.

Like Ollama and LM Studio, Jan includes a built-in API server that's compatible with the OpenAI API specification, so you can interact with local models over HTTP. The server isn't started by default: in the Jan desktop app, open the Local API Server view, and click Start Server to expose the REST API to other local client applications. The server listens on port 1337 by default, but you can change it (and other settings) in the UI.

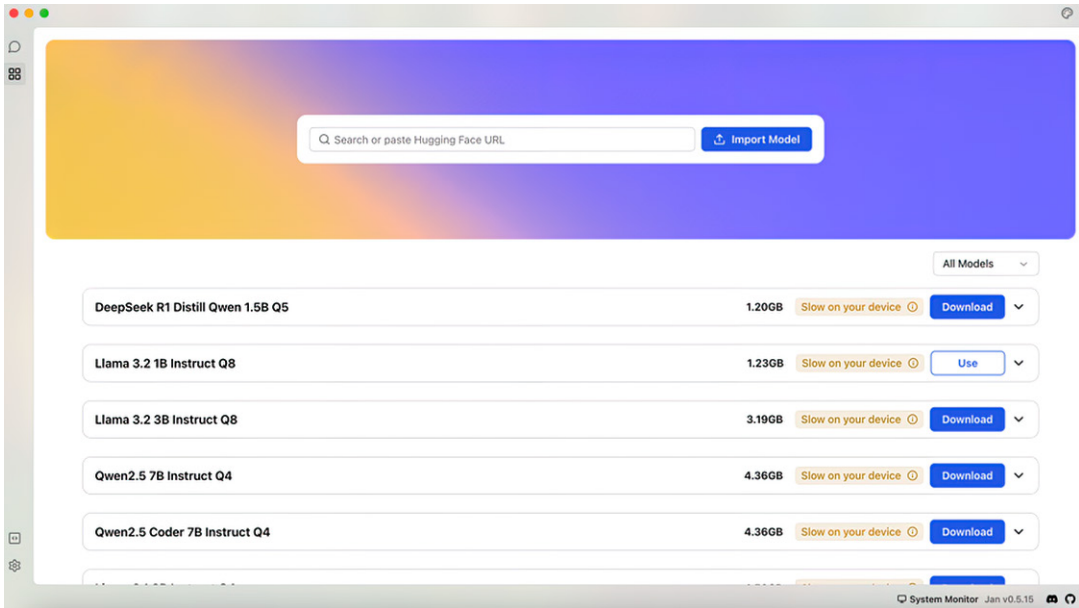


Figure 12.10 The model selection panel in the Jan desktop app

Unlike LM Studio (or Ollama when using a third-party frontend), Jan can't run the API server while you chat with a model in the UI; the two modes are mutually exclusive.

12.6 The Cortex local LLM engine

The Jan application and its built-in API server are powered by Cortex (https://github.com/janhq/cortex.cpp), an open source local AI engine written in C++. Cortex lets you switch inference backends (llama.cpp by default, or ONNX Runtime). It supports GPUs and tries to use one by default when running a model (NVIDIA, AMD, and Intel are supported). Quantized models are available from the Hugging Face Hub or bundled with Cortex; the supported model list is the same as Jan's (see https://cortex.so/models/). Cortex supports multiple quantization modes per model, so several versions of the same SLM may be available. TypeScript and Python client libraries are in progress; the Python client isn't released yet.

You can install the Cortex engine as a standalone local LLM engine or as part of Jan. After Cortex is installed, use the `cortex` CLI to start the server:

```
cortex start
```

The default port is 39281. The API documentation is available from a web browser at `http://127.0.0.1:39281`. An HTTP `GET` request to the `engines` endpoint displays the list of installed engines:

```
curl --request GET \
  --url http://127.0.0.1:39281/v1/engines
```

The output of the request would look like this:

```
{"llama-cpp":[{"engine":"llama-cpp",
            "name":"linux-amd64-avx2",
            "type":"local",
            "version":"v0.1.49"}],
            "python-engine":[],
            "tensorrt-llm":[]}
```

You can do the same with the `cortex` CLI's `engines list` command:

```
cortex engines list
```

When you start the server for the first time, no LLM is installed by default. You'll need to pull one from the Jan catalog. You have two options, including using the `cortex` CLI's `pull` command:

```
cortex pull tinyllama:1b
```

Or you can send an HTTP `POST` request to the `models/pull` endpoint:

```
curl --request POST \
  --url http://127.0.0.1:39281/v1/models/pull \
  -H "Content-Type: application/json" \
  --data '{"model": "tinyllama:1b"}'
```

Once the model finishes downloading, it's ready to chat. Start a chat session with the following command:

```
cortex run tinyllama
```

The following is an example chat with the TinyLlama model:

```
In order to exit, type `exit()`
```

GI What is the tallest building in the world?

The current record-holder for the tallest building in the world is the Burj Khalifa in Dubai, United Arab Emirates. It was officially opened on 4 July 2010 and stands at 828 meters (2,716 feet) tall. The building is home to a variety of amenities and is the tallest freestanding structure in the world.

GI Thanks. Tell me more about it.

Certainly! The Burj Khalifa is a 828-meter-tall (2,716-foot) skyscraper located in Dubai, United Arab Emirates. The building was officially opened on July 4, 2010, and was designed by Adrian Smith and Gordon Gill Architecture. It features a unique architectural design, including a staggered triangular shape and a central core.

The building's most notable feature is its observation deck, known as the Dubai Fountain, which is located at the 124th floor. The fountain is powered by 2,000 water jets that shoot 1.5 million gallons of water per minute, creating a spectacular light show in the evening. The observation deck has over 500 seats and offers panoramic views of the city, including Dubai Creek, the Dubai Marina, and the Persian Gulf.

In addition to the observation deck, the Burj Khalifa features over 160 restaurants, cafes, and bars, as well as a variety of shopping and entertainment venues. The building also has a residential tower, known as the Burj Khalifa Towers, with 2,722 units, making it the world's tallest residential tower.

Overall, the Burj Khalifa is a remarkable achievement in architectural design and is a testament to the power of innovation and vision in creating some of the world's most iconic and awe-inspiring structures.

The following command will stop the server:

```
cortex stop
```

That wraps up our overview of the most mature and popular options for building a personal AI assistant on a laptop. The three solutions share many features, but each has its own strengths and a few areas to improve. Your choice will depend mostly on the task and your personal preference. Table 12.1 summarizes pros and cons of the Ollama, LM Studio, and Jan tools.

Table 12.1 Pros and cons of Ollama, LM Studio, and Jan

Local LLM assistant tool	Pros	Cons
Ollama	■ 100% open source ■ Models optimized for CPU to get the most from commodity hardware ■ REST API server ■ A large open source community providing multiple options for client applications, plugins, and extensions ■ Mature open source Python and Typescript SDKs ■ Provides Python Scripts for model conversion to GGUF format ■ Easy integration with other tools for building local end-to-end AI solutions	■ No native frontend

Table 12.1 Pros and cons of Ollama, LM Studio, and Jan (*continued*)

Local LLM assistant tool	Pros	Cons
LM Studio	■ User Friendly UI ■ Native support also for models in Apple MLX format ■ Mature open source Python and Typescript SDKs ■ OpenAI API–compatible REST API server ■ Integrated Speculative Decoding ■ Integrated chat with local documents	■ The desktop application is free but not open source. ■ The LLM inference server is open source but challenging to build separately.
Jan	■ 100% open source ■ Clean UI ■ The LLM inference server can be installed and used without the desktop app. ■ OpenAI API–compatible REST API server ■ Works also with third-party cloud models/API	■ Chat through the UI and REST API server cannot happen at the same time. ■ Models optimized for GPU; a little bit more computationally hungry than Ollama and LM Studio ■ Python and Typescript SDKs not yet suitable for production

Whatever solution you choose, these tools provide a safe way to give language models access to your personal files without sending them off your laptop and to let organizations connect productivity tools to private data sources on the corporate network.

Summary

- Local language models protect privacy by processing data on personal devices, without internet connectivity.
- Ollama serves GGUF-quantized models via a local API, automatically downloading a model the first time you request it.
- The `ollama run` command downloads the model and starts the chat interface in one step.
- The Ollama REST API exposes endpoints for chat completions and model management.
- LocalLLM provides a Flask-based web frontend for Ollama with streaming responses and model switching.
- Custom models must be converted to GGUF format using llama.cpp's `convert_hf_to_gguf.py` script.
- A Modelfile defines an Ollama model's templates, parameters, and stop tokens.
- LM Studio is a desktop app with a GUI for model management and chat. It supports GGUF and Apple MLX models for local inference.

- Speculative decoding uses a small draft model to propose tokens that a larger target model verifies.
- The LM Studio Python SDK provides programmatic access to local models through convenient, scoped resource APIs.
- In LM Studio, agentic flows let models call Python functions as tools during a conversation.
- The Jan desktop app supports both local models and commercial API providers. Jan requires models in GGUF format and displays hardware-compatibility warnings during selection.
- Jan is powered by the Cortex engine, a C++ implementation with llama.cpp and ONNX Runtime backends.
- Cortex CLI provides commands to manage servers, pull models, and run chat sessions.
- All of these local assistants avoid sending data to external servers and still work fully offline.

13 Creating end-to-end LLM applications

This chapter covers

- Why standalone SLMs/LLMs aren't always enough
- Building a RAG system that uses only SLMs
- Building an agentic AI with SLMs

So far, we've looked at strategies for optimizing small language models (SLMs) to run on available hardware while maintaining acceptable quality. Throughout, we have focused on a common pattern: an SLM receives a prompt, generates task-specific output, and returns a result to the client application or user. This is a frequent scenario, but it's not the only way to use language models. They often deliver more business value as part of a larger system.

In this chapter, we'll explore two popular LLM-based system paradigms: retrieval-augmented generation (RAG) and agentic AI. The difference here is that we'll build on domain-specific small models only, which offers benefits for privacy, safety, security, specificity, and cost savings.

13.1 Why LLMs alone aren't enough

In early 2023, large tech organizations started rushing to "win" the LLM race and reach so-called AGI (artificial general intelligence), fueled by daily hype. That push continued through 2024 and early 2025 and led to larger and larger language models, based on the assumption that more data and more compute (and lately also time-scale compute) would make these models reason like humans across a wide range of tasks, rather than excel at a single narrow task (or a small set), as with today's ML/AI. The reality is that, because of their architecture, language models based on Transformer variants won't converge to AGI. They are, however, useful for narrow but nontrivial tasks when tuned on high-quality domain-specific datasets or integrated into a broader system.

For example, imagine we are a Formula 1 racing team. Our goal is to design and build a car that can compete in the Formula 1 World Championship, win races, and possibly win the drivers' or constructors' title. The engine is a core component, but at design time, we don't try to build the best engine possible; we choose the best fit for the car we have in mind. We need to consider many other factors, such as the mechanical parts, electronics, and tires, because how well these work together is key to success (see figure 13.1). We also need to follow the official rules established by the FIA (Fédération Internationale de l'Automobile) for Formula 1 constructors.

NOTE Figures 13.1 and 13.2 were generated using the Ikea Instructions LoRA for the Stable Diffusion XL Base model. The LoRA weights are available in Hugging Face Hub at https://huggingface.co/ostris/ikea-instructions-lora-sdxl.

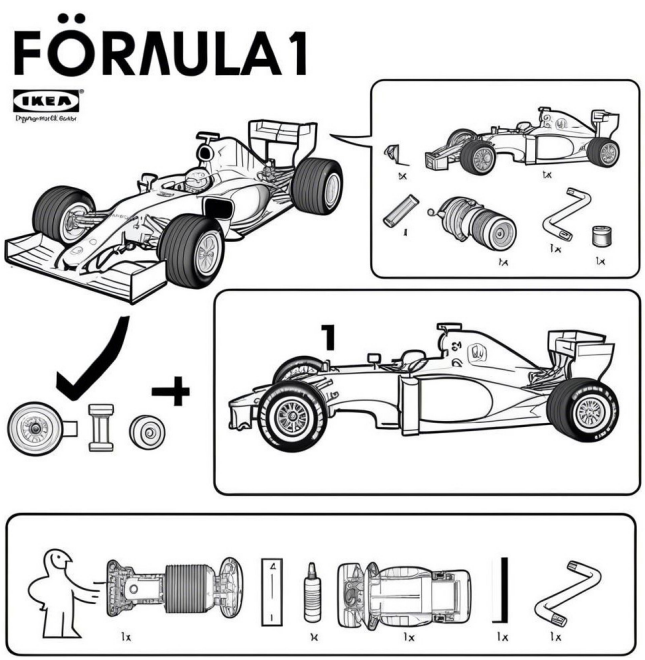


Figure 13.1 A sample business goal for our imaginary Formula 1 racing team: build a competitive car, where the engine is important but not the only component

Now think of the engine as a language model and the car as the final system built around it to meet a business goal. The model takes user inputs through an interface (web or desktop). Those inputs are often enriched with data from other sources (for example, a database or document management system) to provide context. The model then generates content that may need postprocessing with external tools so it can be formatted and delivered to users or downstream systems. When designing such a system, we'll also consider scalability, reliability, and security. The business value doesn't come from the model alone, but from the system as a whole. That's one reason to choose smaller models that can be specialized for the goal and work well with the other components. In many industries, as in Formula 1, real-world tech projects (including those with ML/AI components) must meet regulations. Compliance is easier when we control the full stack, including a tailored SLM bundled into the system.

The preceding scenario resonates with professionals in any industry, not just technology. Since ChatGPT's release, companies such as OpenAI, Google, Anthropic, xAI, and Meta have followed a pattern that, to continue the Formula 1 analogy, resembles the one shown in figure 13.2.

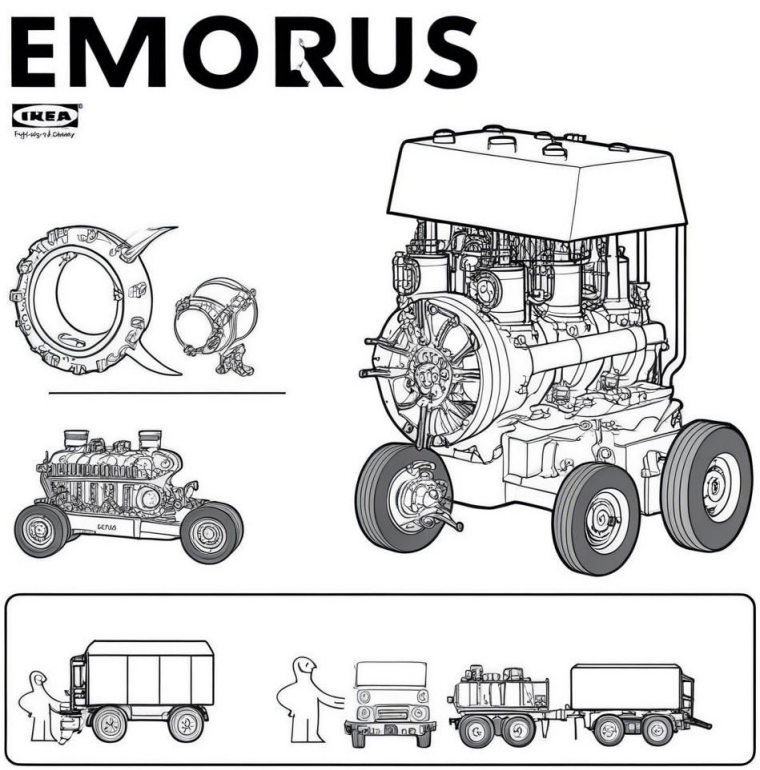


Figure 13.2 A pattern that's sure to fail in Formula 1: building a mammoth engine, putting it on wheels, and ignoring integration with other components and the team's core scope

In this scenario, the Formula 1 team focuses only on the engine and may even outsource its design to an organization that ignores the rest of the car. It delivers a huge, powerful engine that doesn't fit, and integration with the other components becomes a nightmare. This car will not compete for the world championship, and you can't blame FIA regulations for the failure. This is what many big tech companies have done (and are still doing): training ever-larger models, pouring money and compute into a race toward AGI on the wrong architecture.

Don't get me wrong: large models work well for general-domain, routine tasks such as text classification and summarization. But to improve performance, prompt engineering can't be the only answer. We also need to integrate external knowledge, and that integration can be costly and difficult because of safety, privacy, security, and other industry-specific regulations, even for trivial tasks.

13.2 Combining a domain-specific SLM with RAG

I briefly introduced retrieval-augmented generation (RAG) in chapter 2, but in this section (and the next chapter), we'll go deeper, looking at it from different angles and seeing how we can avoid common pitfalls. All the examples in this section will use open source SLMs for both retrieval and generation.

More than one Python framework is available for implementing RAG. In this first example, we'll implement it using specific packages for different tasks (PDF parsing and content extraction, embedding and generator model handling, vector storage, etc.) without a dedicated RAG framework, so we can focus on understanding the various steps and their challenges. We'll make things more complex in later examples.

RAG enhances language models by integrating external knowledge bases. Although large models are powerful, they bring significant cost and integration challenges. SLMs are a more resource-efficient alternative, especially for industry-specific tasks. In a RAG system, language models not only generate content but also convert large volumes of unstructured text and user prompts into embeddings (see figure 13.3). Using smaller models drastically reduces the cost of these conversions.

In figure 13.3, we can see two phases in a RAG system: offline and online. Offline, the raw document corpus is chunked into smaller text segments and transformed into embeddings by an embedding model. Those embeddings are stored in a vector database, with indexes built on top to speed up search. This phase runs when you first set up the RAG system and whenever documents are added, removed, or updated in the knowledge base. Online, whenever a user submits a query, the query is embedded and used to look up semantically similar document chunks in the vector database. The retrieved chunks add context to the query and are passed to the generator model, which produces an answer.

Let's start with a simple use case where we have a single PDF document to query. To extract content from the PDF, we'll use PyMuPDF (https://github.com/pymupdf/PyMuPDF), an open source Python library for extracting data from PDF documents as well as analyzing, converting, and manipulating them (recent versions also handle

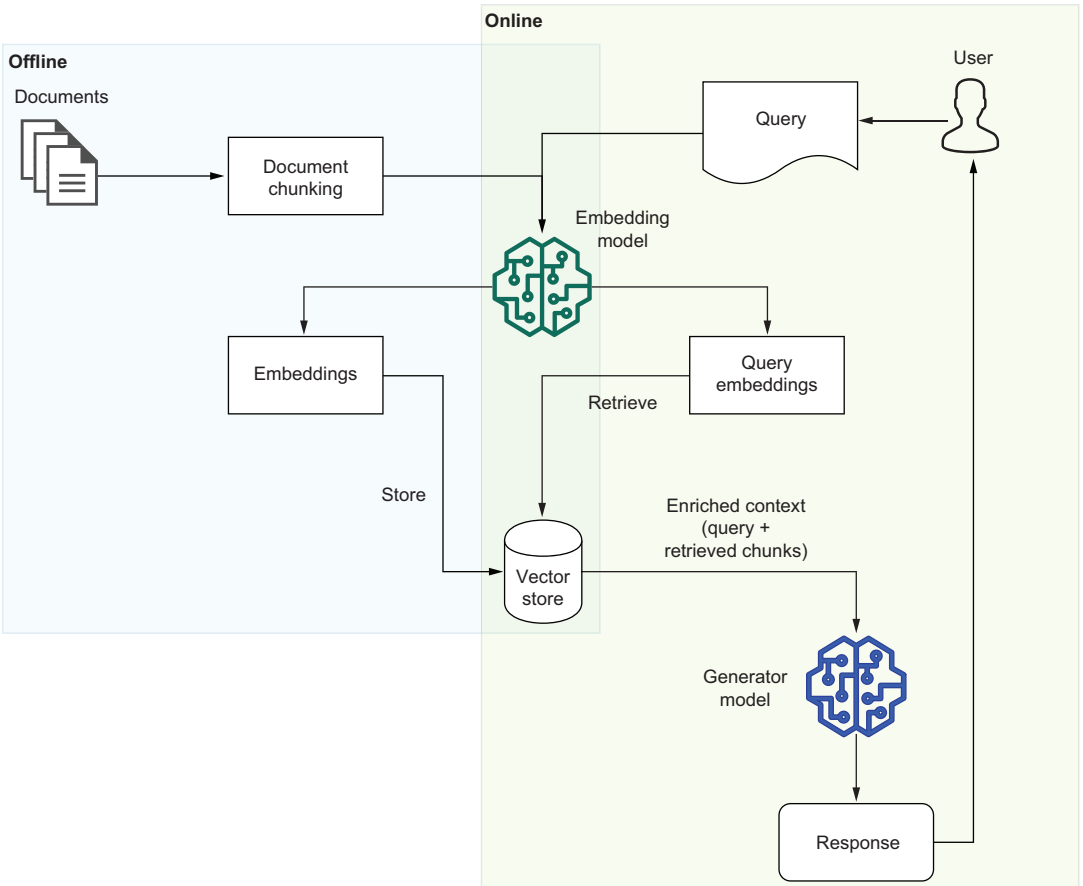


Figure 13.3 The generic workflow of a RAG system

some other document types). The complete code for the examples in this section is available in a companion Colab notebook.

Keeping scalability in mind, let's define a couple of utility functions for extracting document content that work regardless of how many PDF documents we need to process. The first function will remove newline characters from each extracted text chunk, but we can add other formatting rules later within its body:

```
def text_formatter(text: str):
    cleaned_text = text.replace("\n", " ").strip()

    return cleaned_text
```

The second function opens a PDF document, reads it page by page, extracts the text, and collects some statistics:

```
import fitz

def open_and_read_pdf(pdf_path: str) -> list[dict]:
    doc = fitz.open(pdf_path)
    pages_and_texts = []
    for page_number, page in tqdm(enumerate(doc)):
        text = page.get_text()
        text = text_formatter(text)
        pages_and_texts.append({"page_number": page_number + 1,
                                "page_char_count": len(text),
                                "page_word_count": len(text.split(" ")),
                                "page_sentence_count_raw":
              ➥len(text.split(". ")),
                                "page_token_count": len(text) / 4,
                                "text": text})
    return pages_and_texts
```

The `open_and_read_pdf` function takes the path to a PDF as input. It returns a list of dictionaries, each with stats (page number, character count, word count, sentence count, and token count) and the extracted text. The `text_formatter` function formats each page's raw text.

We'll apply `open_and_read_pdf` to our PDF document (in this example, the "DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference" paper, https://arxiv.org/abs/2401.08671):

```
pdf_path = "2401.08671.pdf"
pages_and_texts = open_and_read_pdf(pdf_path)
```

The returned list contains 14 items—one dictionary for each page of the original document. Here's a sample:

```
{'page_number': 1,
  'page_char_count': 2979,
  'page_word_count': 410,
  'page_sentence_count_raw': 15,
  'page_token_count': 744.75,
  'text': 'DeepSpeed-FastGen: High-throughput Text Generation for LLMs via
MII and DeepSpeed-Inference Connor Holmes, Masahiro Tanaka, Michael Wyatt,
Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi,
Heyang Qin, Arash Bakhtiari, Lev Kurilenko, Yuxiong He Microsoft DeepSpeed
(www.deepspeed.ai) Abstract The deployment and scaling of large language
models (LLMs) have become critical as they permeate various applications,
demanding high-throughput and low-latency serving systems. Existing
frameworks struggle to balance these requirements, especially for work-
loads with long prompts. This paper introduces DeepSpeed-FastGen, a system
that em- ploys Dynamic SplitFuse, a novel prompt and generation composition
strategy, to deliver up to 2.3x higher effective throughput, 2x lower
latency on average, and up to 3.7x lower (token-level) tail latency,
compared to state-of-the-art systems like vLLM. We leverage a synergistic
combination of DeepSpeed-MII and DeepSpeed-Inference to provide an
efficient and easy-to-use serving system for LLMs. DeepSpeed-FastGen's
```

```
advanced implementation supports a range of models and offers both non-
persistent and persistent deployment op- tions, catering to diverse user
scenarios from interactive sessions to long-running applica- tions. We
present a detailed benchmarking methodology, analyze the performance
through latency-throughput curves, and investigate scalability via load
balancing. Our evaluations demonstrate substantial improvements in
throughput and latency across various models and hardware configurations.
We discuss our roadmap for future enhancements, includ- ing broader model
support and new hardware backends. The DeepSpeed-FastGen code is readily
available for community engagement and contribution. 1 Introduction Large
language models (LLMs) like GPT-4 [1] and LLaMA [2] have emerged as a
dominant workload in serving a wide range of applications infused with AI
at every level. From general chat models to document summarization, and
from autonomous driving to copilots at every layer of the software stack,
the demand to deploy and serve these models at scale has skyrock- eted.
While frameworks like DeepSpeed, PyTorch [3], and several others can
regularly achieve good hardware utilization during LLM training, the
interactive nature of these applications and the poor arithmetic intensity
of tasks like open-ended text generation have become the bottleneck for
inference throughput in existing systems. To this end, frameworks like vLLM
[4] powered by PagedAttention and research systems like Orca [5] have
significantly improved the performance of inference for LLMs. However,
these systems still struggle to provide consistent quality of service,
particularly for workloads with longer prompts. These long prompt workloads
are becoming increasingly important as more and more models, like MPT-
StoryWriter [6], and systems, such as DeepSpeed Ulysses [7], 1
arXiv:2401.08671v1  [cs.PF]  9 Jan 2024'}
```

That is just the first step in our document preprocessing. Next, we'll split the pages into sentences to make it easier to work with longer pages, especially those densely filled with text. We'll use spaCy (https://github.com/explosion/spaCy), a fast, open source Python library for advanced NLP tasks. We must specify the document language (English, in this example), since spaCy supports more than 75 languages:

```
from spacy.lang.en import English

nlp = English()
```

We'll also add a `sentencizer` pipeline (spaCy's rule-based sentence boundary detector):

```
nlp.add_pipe("sentencizer")
```

Now we'll apply the pipeline to text extracted from the original PDF:

```
for item in tqdm(pages_and_texts):
    item["sentences"] = list(nlp(item["text"]).sents)

    item["sentences"] = [str(sentence) for sentence in item["sentences"]]

    item["page_sentence_count_spacy"] = len(item["sentences"])
```

In the preceding code snippet, we convert all text chunks to strings before splitting them into sentences. We then add two keys to each dictionary in the list: `sentences` (a list of all the page's sentences) and `page_sentence_count_spacy` (the page's sentence count).

Next, we need to split the sentences into smaller chunks to avoid exceeding the embedding model's token limit and the generator model's context window. To do that, we'll define a function that splits a list into sublists of a target size (or as close as possible):

```
def split_list(input_list: list, slice_size: int):
    return [input_list[i:i + slice_size] for i in range(0,
    ➥len(input_list), slice_size)]
```

We'll use it while looping through pages and texts, splitting sentences into chunks:

```
num_sentence_chunk_size = 10
for item in tqdm(pages_and_texts):
    item["sentence_chunks"] = split_list(input_list=item["sentences"],
                                         slice_size=num_sentence_chunk_size)
    item["num_chunks"] = len(item["sentence_chunks"])
```

Now that the dictionaries contain the sentence chunks, we can build the chunks we'll convert into embeddings. Let's create a new list of dictionaries, each containing a single chunk of sentences along with the related metadata (page number and chunk statistics):

```
import re
import tqdm

pages_and_chunks = []
for item in tqdm(pages_and_texts):
    for sentence_chunk in item["sentence_chunks"]:
        chunk_dict = {}
        chunk_dict["page_number"] = item["page_number"]

        joined_sentence_chunk = "".join(sentence_chunk)
        ➥.replace("  ", " ").strip()
        joined_sentence_chunk = re.sub(r'\.([A-Z])', r'. \1',
        ➥ joined_sentence_chunk)
        chunk_dict["sentence_chunk"] = joined_sentence_chunk

        chunk_dict["chunk_char_count"] = len(joined_sentence_chunk)
        chunk_dict["chunk_word_count"] = len([word for word in
        ➥ joined_sentence_chunk.split(" ")])
        chunk_dict["chunk_token_count"] = len(joined_sentence_chunk) / 4

        pages_and_chunks.append(chunk_dict)
```

The sentences in each dictionary are combined into a paragraph-like chunk, and basic stats are collected for each chunk (sentence count, character count, and word count).

The resulting list is what we submit to the embedding model. This is the document chunking process that was illustrated in figure 13.3.

In this example, it produces 30 chunks. Here are two sample dictionaries after the document preprocessing step:

```
[{'page_number': 1,
  'sentence_chunk': 'DeepSpeed-FastGen: High-throughput Text Generation for
LLMs via MII and DeepSpeed-Inference Connor Holmes, Masahiro Tanaka,
Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza
Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, Yuxiong He
Microsoft DeepSpeed (www.deepspeed.ai) Abstract The deployment and scaling
of large language models (LLMs) have become critical as they permeate
various applications, demanding high-throughput and low-latency serving
systems. Existing frameworks struggle to balance these requirements,
especially for work- loads with long prompts. This paper introduces
DeepSpeed-FastGen, a system that em- ploys Dynamic SplitFuse, a novel
prompt and generation composition strategy, to deliver up to 2.3x higher
effective throughput, 2x lower latency on average, and up to 3.7x lower
(token-level) tail latency, compared to state-of-the-art systems like vLLM.
We leverage a synergistic combination of DeepSpeed-MII and DeepSpeed-
Inference to provide an efficient and easy-to-use serving system for LLMs.
DeepSpeed-FastGen's advanced implementation supports a range of models and
offers both non-persistent and persistent deployment op- tions, catering to
diverse user scenarios from interactive sessions to long-running applica-
tions. We present a detailed benchmarking methodology, analyze the
performance through latency-throughput curves, and investigate scalability
via load balancing. Our evaluations demonstrate substantial improvements in
throughput and latency across various models and hardware configurations.
We discuss our roadmap for future enhancements, includ- ing broader model
support and new hardware backends. The DeepSpeed-FastGen code is readily
available for community engagement and contribution.1 Introduction Large
language models (LLMs) like GPT-4 [1] and LLaMA [2] have emerged as a
dominant workload in serving a wide range of applications infused with AI
at every level.',
  'chunk_char_count': 1954,
  'chunk_word_count': 258,
  'chunk_token_count': 488.5},
 {'page_number': 1,
  'sentence_chunk': 'From general chat models to document summarization,
and from autonomous driving to copilots at every layer of the software
stack, the demand to deploy and serve these models at scale has skyrock-
eted. While frameworks like DeepSpeed, PyTorch [3], and several others can
regularly achieve good hardware utilization during LLM training, the
interactive nature of these applications and the poor arithmetic intensity
of tasks like open-ended text generation have become the bottleneck for
inference throughput in existing systems. To this end, frameworks like vLLM
[4] powered by PagedAttention and research systems like Orca [5] have
significantly improved the performance of inference for LLMs. However,
these systems still struggle to provide consistent quality of service,
particularly for workloads with longer prompts. These long prompt workloads
are becoming increasingly important as more and more models, like MPT-
StoryWriter [6], and systems, such as DeepSpeed Ulysses [7], 1
arXiv:2401.08671v1 [cs. PF] 9 Jan 2024',
```

```
  'chunk_char_count': 1022,
  'chunk_word_count': 150,
  'chunk_token_count': 255.5}]
```

The embedding process (shown in figure 13.3) uses sentence-transformers (https://github.com/UKPLab/sentence-transformers/), a Python open source library that computes dense vector representations for sentences, paragraphs, and images. It provides BERT-family models (encoder-only Transformer architecture) hosted on the Hugging Face Hub. The model embeds text into a vector space so that similar texts are close together, which makes retrieval more efficient using cosine similarity or another metric.

First, we need to download the chosen embedding model:

```
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer(
            model_name_or_path="all-mpnet-base-v2",
            device="cpu")
```

In this example, we've selected `all-mpnet-base-v2` (https://huggingface.co/sentence-transformers/all-mpnet-base-v2), a sentence-embedding model that maps sentences and paragraphs to a 768-dimensional vector space. The model is fine-tuned on a 1B sentence-pair dataset using a contrastive learning objective for tasks such as clustering and semantic search. If they aren't already on disk, the model checkpoints are downloaded from the Hugging Face Hub and loaded into memory; this code example doesn't require hardware acceleration.

> **NOTE** With contrastive learning, the model is trained to maximize the similarity between positive pairs and minimize it between negative pairs. Positive pairs are two different views of the same instance, while negative pairs are views from different instances. This is becoming a new trend for specializing small embedding models used in RAG systems.

Finally, we can apply the model to each chunk to generate embeddings (the embeddings block in figure 13.3) and store them under a new dictionary key named `embedding`:

```
embedding_model.to("cpu")

for item in tqdm(pages_and_chunks):
    item["embedding"] = embedding_model.encode(item["sentence_chunk"])
```

A RAG system typically uses a vector database (the vector store in figure 13.3) to store and index document embeddings for faster, scalable search. We'll cover vector storage later in this section.

For a local deployment, a vector store may be overkill. For now, to see how embedding search works under the hood, we'll implement it without one. We'll create a pandas DataFrame from the embedding list and save it as a CSV file:

```
import pandas as pd

text_chunks_and_embeddings_df = pd.DataFrame(pages_and_chunks)
embeddings_df_save_path = "text_chunks_and_embeddings_df.csv"
text_chunks_and_embeddings_df.to_csv(embeddings_df_save_path, index=False)
```

Now let's look at how search works on these embeddings. We're emulating what happens when you use a vector database. First, we'll load the embeddings and their corresponding texts into a DataFrame:

```
text_chunks_and_embedding_df =
➥ pd.read_csv("text_chunks_and_embeddings_df.csv")
```

We'll convert the `embedding` column from a string back to a NumPy array

```
import numpy as np

text_chunks_and_embedding_df["embedding"] =
    text_chunks_and_embedding_df["embedding"].apply(
                                lambda x: np.fromstring(x.strip("[]"),
                                sep=" ")
                                )
```

and we'll convert the DataFrame to a list of dictionaries:

```
pages_and_chunks = text_chunks_and_embedding_df.to_dict(orient="records")
```

Finally, we'll convert the embeddings to Torch tensors and move them to the device (the CPU in this example):

```
device = "cpu"
embeddings = torch.tensor(np.array(

text_chunks_and_embedding_df["embedding"].tolist()
                                ),
                            dtype=torch.float32
                        ).to(device)
```

With the embeddings in memory, we can start querying. Let's search for a general topic, "blocked KV-cache," mentioned in the original paper, whose text we converted into embeddings:

```
query = "blocked KV-cache"
```

We'll use the `all-mpnet-base-v2` embedding model to embed the query in the same vector space as the document text (see the query embedding block in figure 13.3). We must embed the prompts with the same model we used to build the knowledge base:

```
query_embedding = embedding_model.encode(query, convert_to_tensor=True)
```

Given this query, the ideal results should include information about the blocked KV-caching strategy described in the original paper. We'll use *similarity search*. Unlike keyword-based text search, similarity search compares vectors, so the final results include not only the words in the query string but also other text that matches the query's meaning. Both the Sentence Transformers library and the PyTorch framework include built-in methods for finding similar vectors.

In this example, we'll use the Sentence Transformers API, since we're already using this library for embedding text. The most popular algorithms for this type of search (also built into most vector databases) are

- *Dot product*—The dot product measures the magnitude and direction between two vectors. Vectors aligned in direction and magnitude have a higher positive value, while opposite vectors have a higher negative value.
- *Cosine similarity*—With cosine similarity, vectors are normalized by magnitude, Euclidean norm, or L2 norm, so that they have unit length, and they are compared by direction. Vectors aligned in direction score close to 1, while vectors in opposite directions score close to -1.

Cosine similarity is typically used for text similarity because it measures semantic similarity (via direction). But because we embedded our document with the `all-mpnet-base-v2` model, which always returns normalized outputs, cosine similarity and dot product return the same results.

Let's try cosine similarity and take the top five scores:

```
import torch
from sentence_transformers import util

cosine_scores = util.cos_sim(a=query_embedding, b=embeddings)[0]
top_results_cosine_sim = torch.topk(cosine_scores, k=5)
```

This query returns the following results:

```
torch.return_types.topk(
values=tensor([0.2963, 0.1960, 0.1868, 0.1395, 0.1132]),
indices=tensor([ 3,  5, 11, 13, 27]))
```

The values here are relevance scores, and the indices are the page numbers for the top five returned chunks. The top two results (the highest-scoring ones) relate directly to the blocked KV-caching strategy:

```
Query: 'blocked KV-cache'

Results:
Score: 0.2963
Text:
The blocked KV-cache increases system throughput by increasing the amount of
potential sequence concurrency by eliminating KV-cache induced memory
fragmentation. Non-contiguous KV cache implementations are also included in
Hug-
gingFace TGI [9] and NVIDIA TensorRT-LLM [10].2.2 Continuous Batching In
the
past, dynamic batching, in which a server would wait for multiple requests
to
process in phase with each other, was used to improve GPU utilization.
However,
this approach has drawbacks, as it typically requires padding inputs to
identical lengths or stalling the system to wait to construct a larger
batch.
Recent advancement in large language model (LLM) inference and serving has
been
fo- cusing on fine granularity scheduling and optimizing memory efficiency.
For
instance, Orca proposes iteration-level scheduling (also known as
continuous
batching) which makes distinct scheduling decisions at each forward pass of
the
model. This allows requests to join/leave the batch as needed, eliminating
the
need for padding requests thus improving the overall through- put. In addition
to Orca, continuous batching has been implemented in NVIDIA TRT-LLM,
HuggingFace
TGI, and vLLM. In current systems, there are two primary approaches to
implement
continuous batching. In TGI and vLLM, the generation phase is preempted to
perform prompt processing (called infill in TGI) before continuing with
generation.
Page number: 2


Score: 0.1960
Text:
doesn't reach a fixed bound. These approaches to varying degrees need to
stall
generation to process long prompts (see Section 3.2). We propose a novel
prompt
and generation composition strategy, Dynamic SplitFuse dis- cussed at
length in
the next section.3 Dynamic SplitFuse: A Novel Prompt and Generation
Composition
Strategy DeepSpeed-FastGen is built to leverage continuous batching and
non-
contiguous KV caches to enable increased occupancy and higher responsivity
for
serving LLMs in the data center, similar to existing frameworks such as
TRT-LLM,
```

```
TGI, and vLLM.iIn order to achieve a new level of performance, DeepSpeed-
FastGen
introduces SplitFuse which leverages dynamic prompt and generation decomposition
and unification to further improve continuous batching and system
throughput.3.1
Three Performance Insights Before describing Dynamic SplitFuse, we answer
three
key performance questions that together motivate its design.3.1.1 What
factors
impact the forward pass of a single LLM?In order to effectively schedule,
it is
necessary to understand what are the relevant independent variables the
scheduling loop should control. We observe below that the composition of
se-
quences in a forward pass (the batch size in sequences) has a negligible
impact
on performance compared to the raw number of tokens in the forward pass.
This
means an effective scheduler can be built around a single signal, the
number of
tokens in the forward pass.
Page number: 3
```

Let's now run the same search with the dot product algorithm:

```
dot_scores = util.dot_score(a=query_embedding, b=embeddings)[0]
```

We can observe the same results, as expected:

```
torch.return_types.topk(
values=tensor([0.2963, 0.1960, 0.1868, 0.1395, 0.1132]),
indices=tensor([ 3,  5, 11, 13, 27]))
```

As you can see in figure 13.4 (shown earlier in figure 13.3), the retrieval and the generation are done by two distinct LLMs: the embedding model and the generator model. So far, we have discussed only the retrieval phase. Let's look at how we can do local generation by enriching the user query with retrieved content.

The Hugging Face Hub hosts many small models that can generate high-quality answers. The choice of generation model and precision format mainly depends on your available local hardware. Because this example runs on a free-tier Colab VM without hardware acceleration, a GGUF model (a 4-bit quantized version of the original) is a good fit. We'll use `microsoft/Phi-3-mini-4k-instruct-gguf`, which we used in section 11.1.1 and which is available on the Hugging Face Hub (https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf). To keep the end-to-end solution in Python, we'll download and run the GGUF model with the Python bindings for llama.cpp, `llama-cpp-python` (https://github.com/abetlen/llama-cpp-python), which you can install on any OS via PyPI.

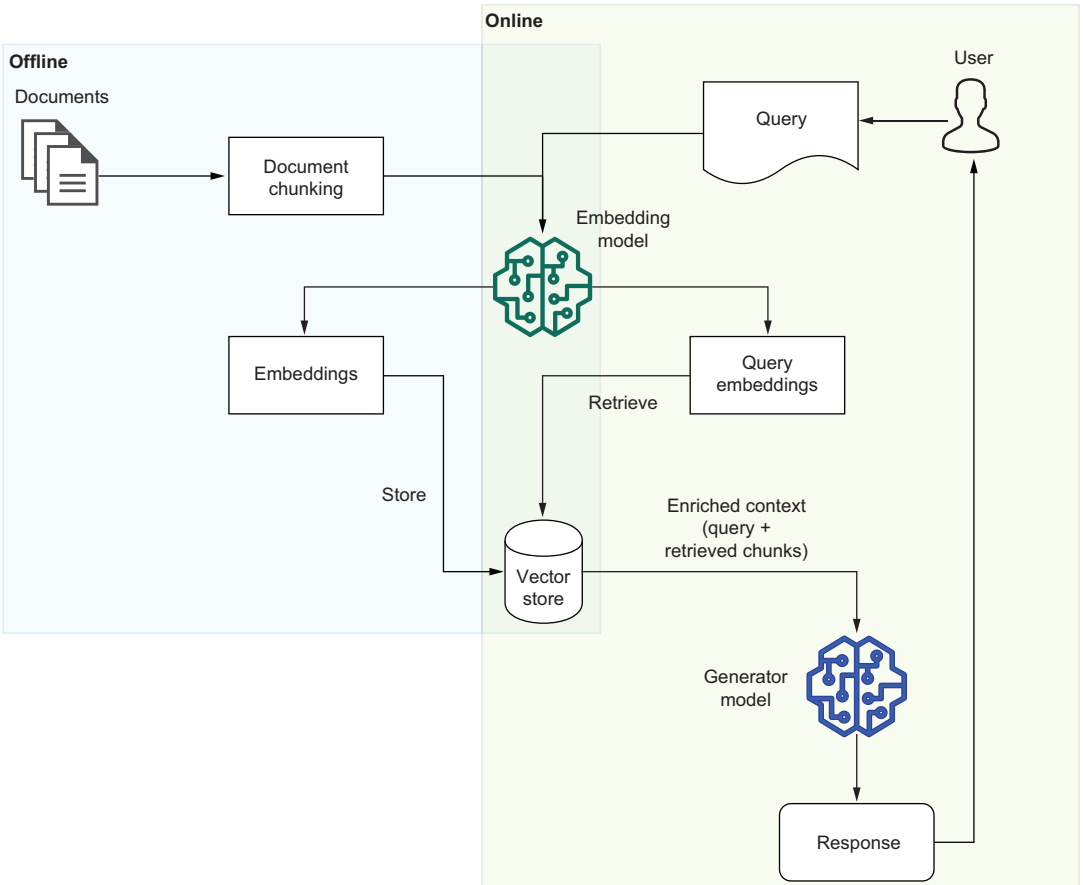


Figure 13.4 The generic workflow of a RAG system (shown previously in figure 13.3)

Downloading the model from the Hugging Face Hub is as simple as with the Transformers API:

```
from llama_cpp import Llama

llm = Llama.from_pretrained(
    repo_id="microsoft/Phi-3-mini-4k-instruct-gguf",
    filename="*-q4.gguf",
    verbose=False,
    n_ctx=1024
)
```

We are using the `from_pretrained` method of the `Llama` class, which requires the model ID from the Hugging Face Hub and the checkpoint filename you want to download. We've also set the context-window size: the original model uses 4,096 tokens, but

`from_pretrained` defaults to 512. This method returns a model loaded into memory and ready for inference.

We'll now use the same query as in the previous example and run the same retrieval on our knowledge base, requesting the top two results, because we've already verified that those are the most relevant for this query:

```
query = "blocked KV-cache"
query_embedding = embedding_model.encode(query, convert_to_tensor=True)
cosine_scores = util.cos_sim(a=query_embedding, b=embeddings)[0]
scores, indices = torch.topk(cosine_scores, k=2)
```

Next, we'll create a list of context items to enrich the generation prompt:

```
context_items = [pages_and_chunks[i] for i in indices]
```

Now we need to implement a custom function that formats the prompt for the generator model during the generation phase, augmenting the user query with the retrieved context items:

```
def prompt_formatter(query: str,
                     context_items: list[dict],
                     tokenizer):
    context = "- " + "\n- ".join([item["sentence_chunk"]
    ➥for item in context_items])

    base_prompt = """Based on the following context items,
                ➥please answer the query.
Make sure your answers are as explanatory as possible.
\nUse the following context items to answer the user query:
{context}
\nRelevant passages: <extract relevant passages from the context here>
User query: {query}
Answer:"""

    base_prompt = base_prompt.format(context=context, query=query)

    dialogue_template = [
        {"role": "user",
        "content": base_prompt}
    ]

    prompt = tokenizer.apply_chat_template(conversation=dialogue_template,
                                           tokenize=False,
                                           add_generation_prompt=True)
    return prompt
```

The `prompt_formatter` function takes as input the user prompt, the retrieved context items, and the generator's tokenizer. It joins the retrieved chunks together, merges them into the base prompt with the user query, applies the model's chat template, and

returns the result. The base prompt shown here is tuned for Phi-3 Mini 4k Instruct; it might need adjustments for other generator models.

We'll use the Phi-3 Mini 4k Instruct model's tokenizer to format the prompt for the generator:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
            ➥"microsoft/Phi-3-mini-4k-instruct")
```

Let's format the generator prompt using the function we implemented:

```
prompt = prompt_formatter(query=query,
                          context_items=context_items,
                          tokenizer)
```

The formatted prompt looks like this:

```
<|user|>
Based on the following context items, please answer the query.
Make sure your answers are as explanatory as possible.

Use the following context items to answer the user query:
- The blocked KV-cache increases system throughput by increasing the amount
of potential sequence concurrency by eliminating KV-cache induced memory
fragmentation. Non-contiguous KV cache implementations are also included in
Hug- gingFace TGI [9] and NVIDIA TensorRT-LLM [10].2.2 Continuous Batching
In the past, dynamic batching, in which a server would wait for multiple
requests to process in phase with each other, was used to improve GPU
utilization. However, this approach has drawbacks, as it typically requires
padding inputs to identical lengths or stalling the system to wait to
construct a larger batch. Recent advancement in large language model (LLM)
inference and serving has been fo- cusing on fine granularity scheduling
and optimizing memory efficiency. For instance, Orca proposes iteration-
level scheduling (also known as continuous batching) which makes distinct
scheduling decisions at each forward pass of the model. This allows
requests to join/leave the batch as needed, eliminating the need for
padding requests thus improving the overall through- put. In addition to
Orca, continuous batching has been implemented in NVIDIA TRT-LLM,
HuggingFace TGI, and vLLM. In current systems, there are two primary
approaches to implement continuous batching. In TGI and vLLM, the
generation phase is preempted to perform prompt processing (called infill
in TGI) before continuing with generation.
- doesn't reach a fixed bound. These approaches to varying degrees need to
stall generation to process long prompts (see Section 3.2). We propose a
novel prompt and generation composition strategy, Dynamic SplitFuse dis-
cussed at length in the next section.3 Dynamic SplitFuse: A Novel Prompt
and Generation Composition Strategy DeepSpeed-FastGen is built to leverage
continuous batching and non-contiguous KV caches to enable increased
occupancy and higher responsivity for serving LLMs in the data center,
similar to existing frameworks such as TRT-LLM, TGI, and vLLM.iIn order to
achieve a new level of performance, DeepSpeed-FastGen introduces SplitFuse
```

```
which leverages dynamic prompt and generation decomposition and unification
to further improve continuous batching and system throughput.3.1 Three
Performance Insights Before describing Dynamic SplitFuse, we answer three
key performance questions that together motivate its design.3.1.1 What
factors impact the forward pass of a single LLM?In order to effectively
schedule, it is necessary to understand what are the relevant independent
variables the scheduling loop should control. We observe below that the
composition of se- quences in a forward pass (the batch size in sequences)
has a negligible impact on performance compared to the raw number of tokens
in the forward pass. This means an effective scheduler can be built around
a single signal, the number of tokens in the forward pass.

Relevant passages: <extract relevant passages from the context here>
User query: blocked KV-cache
Answer:<|end|>
<|assistant|>
```

Finally, we'll run the generation with the GGUF model, using the enriched prompt:

```
output = llm(
      prompt,
      max_tokens=None,
      stop=["Q:", "\n"],
      echo=True
)
```

We'll then get the model's answer:

From the provided context, it can be understood that a blocked KV-cache increases system throughput by eliminating KV-cache induced memory fragmentation. This is achieved by increasing the amount of potential sequence concurrency. Noncontiguous KV cache implementations are also included in HuggingFace TGI [9] and NVIDIA TensorRT-LLM [10].

13.3 Using a vector database

In the previous section, we explored a RAG example in which the knowledge base was built from a single PDF. In local deployments, where you typically have only a few documents—represented by at most a few hundred thousand embeddings—it's fine to persist the embeddings to disk. But to scale up, you'll want a vector database (often called a vector store).

A vector database stores vectors (fixed-length lists of numbers) along with other data. It typically implements one or more approximate nearest neighbor (ANN) algorithms, enabling you to search with a query vector and retrieve the closest-matching records. These databases have many applications, but since 2024, they've become a key component of most RAG systems.

Many implementations are available, both open source and commercial. These are some popular open source vector databases:

- Weaviate (https://github.com/weaviate/weaviate)
- Milvus (https://github.com/milvus-io/milvus)
- OpenSearch (https://github.com/opensearch-project/OpenSearch)
- Chroma (https://github.com/chroma-core/chroma)
- LanceDB (https://github.com/lancedb/lancedb)

For the examples in this section, we'll use LanceDB because it's easy to set up and maintain locally, and it scales well when you move to a distributed environment. It's a serverless vector search database with persistent storage that simplifies retrieval, filtering, and embedding management. It supports vector similarity search, full-text search, and SQL, with native Python and JavaScript/TypeScript support. It also supports zero-copy data access and automatic versioning without extra infrastructure, and it integrates with popular LLM frameworks like LangChain and LlamaIndex, as well as data libraries such as Apache Arrow, pandas, and Polars.

Let's return to the previous section's example to see how we can integrate a LanceDB database into it. First, we'll install the LanceDB library with `pip`:

```
pip install lancedb
```

We can now create an empty local database instance:

```
import lancedb

db = lancedb.connect("paperdb")
```

Our approach for preparing the embeddings and related metadata earlier in section 13.2 is still valid. The only change needed to allow for proper ingestion into a LanceDB table is to convert the `embedding` column from arrays to strings and then to NumPy arrays:

```
import numpy as np

text_chunks_and_embeddings_df['embedding_str'] =
    text_chunks_and_embeddings_df['embedding'].apply(
                    lambda x: np.array2string(x, separator=',')[1:-1]
                    )
text_chunks_and_embedding_df['embedding_final'] =
        text_chunks_and_embedding_df['embedding_str'].apply(
           lambda x: np.fromstring(x, sep=',')
        )
```

We now need to prepare the data as a list of dictionaries—the format LanceDB expects when ingesting a table.

Listing 13.1 Preparing data for ingestion into LanceDB

```
final_data = []
for row in text_chunks_and_embedding_df_load.itertuples(index=False):
```

```
    temp = {}
    temp["page_number"] = row.page_number
    temp["chunk_char_count"] = row.chunk_char_count
    temp["chunk_word_count"] = row.chunk_word_count
    temp["chunk_token_count"] = row.chunk_token_count
    temp["sentence_chunk"] = row.sentence_chunk
    temp["embedding"] = np.array(row.embedding_final)
    final_data.append(temp)

data_dict = {}
for key in final_data[0].keys():
    data_dict[key] = [d[key] for d in final_data]
```

Get column names from the first dictionary

Let's create and populate a table, `paper_embeddings_table`, following a custom schema, indicated in the following table-creation code.

Listing 13.2 Definition of the schema for the embedding table in the LanceDB database

```
import pyarrow as pa

table = db.create_table(
    "paper_embeddings_table",
    data=pa.Table.from_pydict(data_dict),
    mode="overwrite",
    schema=pa.schema([
        ("page_number", pa.int64()),
        ("chunk_char_count", pa.int64()),
        ("chunk_word_count", pa.int64()),
        ("chunk_token_count", pa.float64()),
        ("sentence_chunk", pa.string()),
        ("embedding", pa.list_(pa.float32(), list_size=768)),
    ]),
)
```

The dataset is converted to a PyArrow table before being passed to the `data` attribute.

Finally, we'll create an ANN index on the embedding column to speed up searches by avoiding brute-force scans of the vector space. We'll also specify the similarity algorithm (cosine similarity in this case):

```
db["paper_embeddings_table"].create_index(
    metric="cosine",
    vector_column_name="embedding",
    index_type="IVF_FLAT",
)
```

We'll run the same prompt as in the earlier examples, but search the LanceDB table this time and limit the results to three:

```
query = "blocked KV-cache"
query_embedding = embedding_model.encode(query, convert_to_tensor=False)
```

```
results = db["paper_embeddings_table"].search(
        query_embedding,
        vector_column_name="embedding"
    ).limit(3).to_list()
```

This yields the following results:

```
Search results:
Score = 0.7036938667297363
The blocked KV-cache increases system throughput by increasing the amount
of potential sequence concurrency by eliminating KV-cache induced memory
fragmentation. Non-contiguous KV cache implementations are also included in
Hug- gingFace TGI [9] and NVIDIA TensorRT-LLM [10].2.2 Continuous Batching
In the past, dynamic batching, in which a server would wait for multiple
requests to process in phase with each other, was used to improve GPU
utilization. However, this approach has drawbacks, as it typically requires
padding inputs to identical lengths or stalling the system to wait to
construct a larger batch. Recent advancement in large language model (LLM)
inference and serving has been fo- cusing on fine granularity scheduling
and optimizing memory efficiency. For instance, Orca proposes iteration-
level scheduling (also known as continuous batching) which makes distinct
scheduling decisions at each forward pass of the model. This allows
requests to join/leave the batch as needed, eliminating the need for
padding requests thus improving the overall through- put. In addition to
Orca, continuous batching has been implemented in NVIDIA TRT-LLM,
HuggingFace TGI, and vLLM. In current systems, there are two primary
approaches to implement continuous batching. In TGI and vLLM, the
generation phase is preempted to perform prompt processing (called infill
in TGI) before continuing with generation.
Page = 2

Score = 0.8040012121200562
doesn't reach a fixed bound. These approaches to varying degrees need to
stall generation to process long prompts (see Section 3.2). We propose a
novel prompt and generation composition strategy, Dynamic SplitFuse dis-
cussed at length in the next section.3 Dynamic SplitFuse: A Novel Prompt
and Generation Composition Strategy DeepSpeed-FastGen is built to leverage
continuous batching and non-contiguous KV caches to enable increased
occupancy and higher responsivity for serving LLMs in the data center,
similar to existing frameworks such as TRT-LLM, TGI, and vLLM.iIn order to
achieve a new level of performance, DeepSpeed-FastGen introduces SplitFuse
which leverages dynamic prompt and generation decomposition and unification
to further improve continuous batching and system throughput.3.1 Three
Performance Insights Before describing Dynamic SplitFuse, we answer three
key performance questions that together motivate its design.3.1.1 What
factors impact the forward pass of a single LLM?In order to effectively
schedule, it is necessary to understand what are the relevant independent
variables the scheduling loop should control. We observe below that the
composition of se- quences in a forward pass (the batch size in sequences)
has a negligible impact on performance compared to the raw number of tokens
in the forward pass. This means an effective scheduler can be built around
a single signal, the number of tokens in the forward pass.
Page = 3
```

```
Score = 0.8131694793701172
Consequently, DeepSpeed-FastGen will consume tokens from incoming prompts
at a rate that permits fast ongoing generation while adding tokens to the
system that increase system utilization, providing lower latency and higher
throughput streaming generation to all clients as compared to other state-
of-the-art serving systems.4 Performance Evaluation DeepSpeed-FastGen
provides state-of-the-art LLM serving performance leveraging its blocked KV
cache and Dynamic SplitFuse continuous batching. We evaluate DeepSpeed-
FastGen against vLLM [4] on a range of models and hardware configurations
following the benchmarking methodology discussed below. The evaluation
shows that DeepSpeed-FastGen achieves up to 2.3x higher effective
throughput, 2x lower latency on average, and up to 3.7x lower (token-level)
tail latency, compared to state-of-the-art systems like vLLM.4.1
Benchmarking Methodology We use two primary quantitative schemes for
measuring performance.4.1.1 Throughput-Latency Curves Two key metrics for
production readiness are throughput (measured in requests per second) and
latency (the responsiveness of each request). To measure this, we
instantiate multiple clients (ranging from 1 to 32) concurrently and send
requests (512 in total) to the server. The resulting latency of each
request is measured at the endpoint and throughput is measured by the end-
to-end time to complete the experiment.4.1.2 Effective Throughput
Interactive applications, such as chat applications, can have more
stringent and complex re- quirements than can be captured by top-level
metrics like end-to-end latency. In particular, we focus on the
increasingly popular chat user scenario: 1.
Page = 6
```

At first glance, comparing the relevance scores, the one for retrieval from a vector store looks higher than the one for naive retrieval in section 13.2.

Let's see how these retrieved documents affect the generated answer by comparing it with the previous example. Because we're using the same model (Phi-3 Mini 4k Instruct GGUF), we can reuse the earlier Python generation code. Here's the generated answer after enriching the user prompt with the top two results from the database:

From the provided context, it can be understood that a blocked KV-cache increases system throughput by eliminating KV-cache induced memory fragmentation. This is achieved by increasing the amount of potential sequence concurrency. Non-contiguous KV cache implementations are also included in HuggingFace TGI [9] and NVIDIA TensorRT-LLM [10].

In this example, the quality is similar to the previous approach. But on a larger, noisier knowledge base with more diverse topics, storing data in a database and using index-based retrieval improves retrieval and answer quality.

In summary, best document-chunking practices and a vector database make a RAG solution more reliable, performant, and scalable. You can improve it further with the advanced RAG strategies we'll cover in chapter 14.

13.4 Building an agent

Agentic AI was 2025's big buzzword in the generative AI hype, primarily driven by big tech, large consulting firms, and self-proclaimed AI gurus. Although agentic AI is an

important way to use language models, many so-called agentic systems are really just RPA (robotic process automation) applications that use a model in the usual way: a prompt goes in, the model generates output, and the rest of the solution consumes it.

In this section, we'll clarify the topic and cover the basics of building an agent locally using open source components. Agentic AI is a big subject, so this is a hands-on introduction; we'll look at more advanced concepts in the next chapter.

A language model, large or small, predicts the most likely next token for a given prompt. By sampling tokens repeatedly, it can carry on a conversation and handle many tasks, especially when specialized to a narrow domain with high-quality data. Still, this approach has limitations. As a chat continues, the model can lose earlier context because of the limitations of its context window. Language models also often struggle with tasks such as math (even basic multiplication and division) and logic, which is expected given how these models work. In addition, because transformers generate the best next token for the text they are given, if the input has been enriched with outdated or incorrect context, the output may fit that input perfectly while still being wrong for the original question. These limitations highlight the need for short- and long-term memory management, access to external tools, more accurate, up-to-date information (including from the web), and ways to break problems into steps and refine initial answers. This is a scenario that fits the idea of an AI agent.

A software system can be defined as an AI agent when it can autonomously perform tasks and make decisions on a user's behalf, using AI to learn, adapt, and pursue specific goals. An AI agent must be able to reason (analyze the prompt and related context, if provided, and determine next steps), act (execute tasks by calling external tools or APIs), and interact (communicate with other agents or users to solve problems collaboratively). Agents fall into different categories, but before we get to that classification, we'll expand on these capabilities and start implementing our first agent system.

Imagine asking a specialized language model to plan a weeklong trip from Dublin to New York. The workflow might look like this:

1. A user asks the model for a list of accommodation options.
2. The model returns a list of options (hotels, B&Bs, and short-stay apartments).
3. The user asks for a list of return flights from Dublin to New York for specific dates.
4. The model returns round-trip flight options from Dublin to New York.
5. The user manages everything using information provided by the model.

Now let's look at the same scenario in an agentic system:

1. A user gives the model a trip-planning request prompt (Dublin to New York).
2. The agent searches various sites for hotels, B&Bs, and short-stay apartment options in the destination city.

3. The agent queries multiple online sources for return flights from Dublin to New York for the given period.
4. The agent determines the best combination of flights and accommodations.
5. The agent also suggests other trip options, such as car rentals, local travel cards, and places to visit.
6. The agent listens to the user's feedback.
7. Once the user is satisfied with the agent's suggestions, the agent makes the bookings.

Comparing the two systems highlights the agents' reasoning, action, and interaction capabilities. In the agentic scenario, we can see the core traits of agentic AI: a plan (the user prompt to organize a trip), reasoning that breaks the plan into steps, tool selections (such as a web browser and an online booking system), and actions (using those tools and their outputs).

Implementing an AI agent doesn't require coding everything from scratch because many open-source frameworks are already available. In this section, we'll use Hugging Face's smolagents (https://github.com/huggingface/smolagents). It's a small library (just over 1,000 lines of code at the time of writing), and its API is easy to learn and use.

smolagents integrates with other Hugging Face libraries we've used in this book, such as Transformers. It's not limited to models hosted on the Hugging Face Hub or Hugging Face endpoints; it also supports OpenAI, Azure OpenAI, and Together AI endpoints; OpenAI-compatible servers; and LiteLLM and Ollama servers. smolagents supports models that handle multiple types of content (text, images, video, and audio), but we'll focus on text only. Implemented in Python, smolagents also integrates easily with other LLM frameworks such as LangChain, LangGraph, and Anthropic's MCP.

smolagents comes with two built-in agent implementations: `CodeAgent` and `ToolCallingAgent`. `CodeAgent` is the default and is recommended by the framework's authors. At each step, it writes and executes Python snippets. By default, execution happens in your local environment. This is generally safe because the agent can call only its configured tools plus a predefined set of safe functions from native modules, and the Python interpreter blocks imports by default except from a safe list. For production (anything beyond your laptop or phone), smolagents supports safer execution in Docker or E2B (https://github.com/e2b-dev) sandboxes.

`CodeAgent` largely follows the classic ReAct (reasoning and acting) pattern, a framework for building AI agents that interleave reasoning and tool use to execute tasks (matching the definition of an agent introduced earlier in this section). ReAct agents use LLMs to analyze problems, choose tools, and iterate toward a solution. In `CodeAgent`, the model expresses its actions as Python code snippets.

Figure 13.5 shows the `CodeAgent` workflow. As you can see, building an agent requires at least one LLM, a memory manager, and a set of tools.

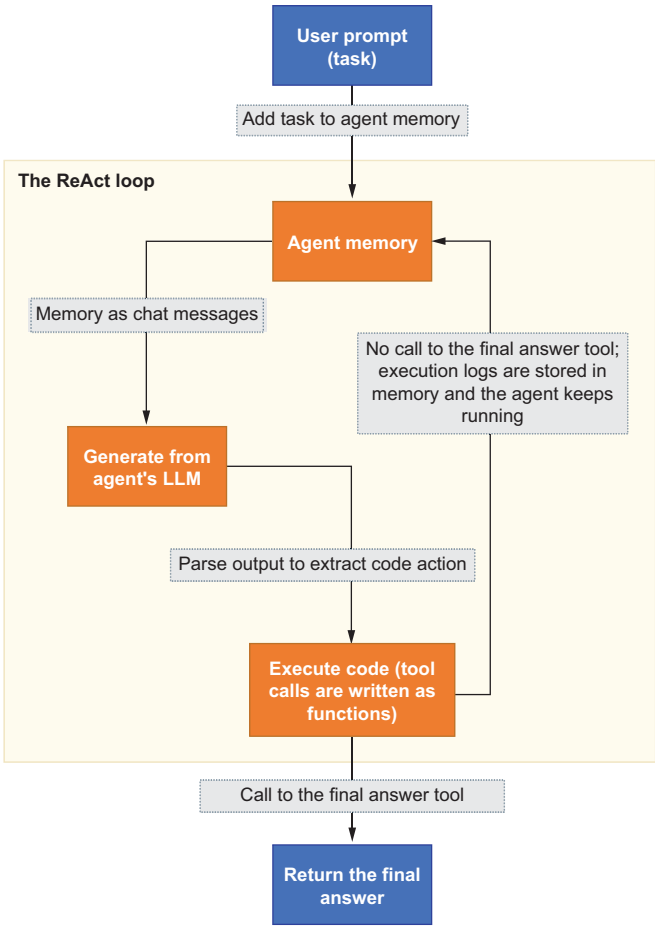


Figure 13.5 The logical workflow of CodeAgent

Let's configure these components and build a `CodeAgent` programmatically. First, we'll install the `smolagents` package:

```
pip install smolagents
```

In its simplest form, a `CodeAgent` looks like this:

```
from smolagents import CodeAgent, HfApiModel

model_id = "<model_id>"

model = HfApiModel(model_id=model_id, token="<huggingface_api_token>")
agent = CodeAgent(tools=[], model=model, add_base_tools=True)
```

The `HfApiModel` class indicates that the LLM is served through a Hugging Face endpoint. We pass the model ID from the Hugging Face Hub and the Hugging Face API token for authentication. We then define the agent with the `CodeAgent` class, providing the model and the list of tools the agent is allowed to use. Setting `add_base_tools` to `True` adds the default toolbox on top of the tools listed in tools.

To start the agent, we call the run method on the `CodeAgent` instance:

```
agent.run("<user_task>")
```

Execution stops if the agent generates code that attempts an illegal operation, tries to install an unauthorized package, or triggers a Python error.

The framework provides a default toolbox, which we added at initialization using the `add_base_tools` argument. It includes the following tools:

- *DuckDuckGo web search*—Performs web searches with DuckDuckGo (https://duckduckgo.com/).
- *Google web search*—Performs web searches using the Google Search API.
- *Python code interpreter*—Runs LLM-generated Python code in a secure environment. Add this tool only to the `ToolCallingAgent`, since code-based agents can execute Python natively.
- *Speech-to-text*—A Whisper-Turbo–based pipeline that transcribes audio to text.
- *User input*—Asks for user input on a specific question.
- *Visit web page*—Visits a URL and returns the web page content as a Markdown string.
- *Final answer*—Called by an agent to return the final answer after it completes planning and execution.

smolagents agents aren't limited to using the default tool set. You can add new tools programmatically in two ways. First, you can use the `@tool` function decorator:

```
from smolagents import tool

@tool
def find_accomodation_options(query: str):
    ...
```

You can also implement them as Python classes:

```
from smolagents import Tool

class FindAccomodationoptionsTool(Tool):
   ...
```

Before we get hands-on with agents, let's first look at the other built-in agent class, `ToolCallingAgent`. This implementation writes actions as JSON-like blobs. That was the

original approach in early LLM-based agents: they were prompted to generate actions as JSON (or other predefined text formats). The downside is a constrained action space (the set of predefined tools) and limited flexibility. The paper "Executable Code Actions Elicit Better LLM Agents" (https://arxiv.org/abs/2402.01030) was the first to propose using executable Python code to unify an agent's action space. Its experiments suggest that when agents act by executing interpretable code and collaborate with users in natural language, they can address these limits and perform better (see figure 13.6, from the paper). This is a valid reason to follow Hugging Face's recommendation to use `CodeAgent` rather than `ToolCallingAgent` for your custom agent implementations.

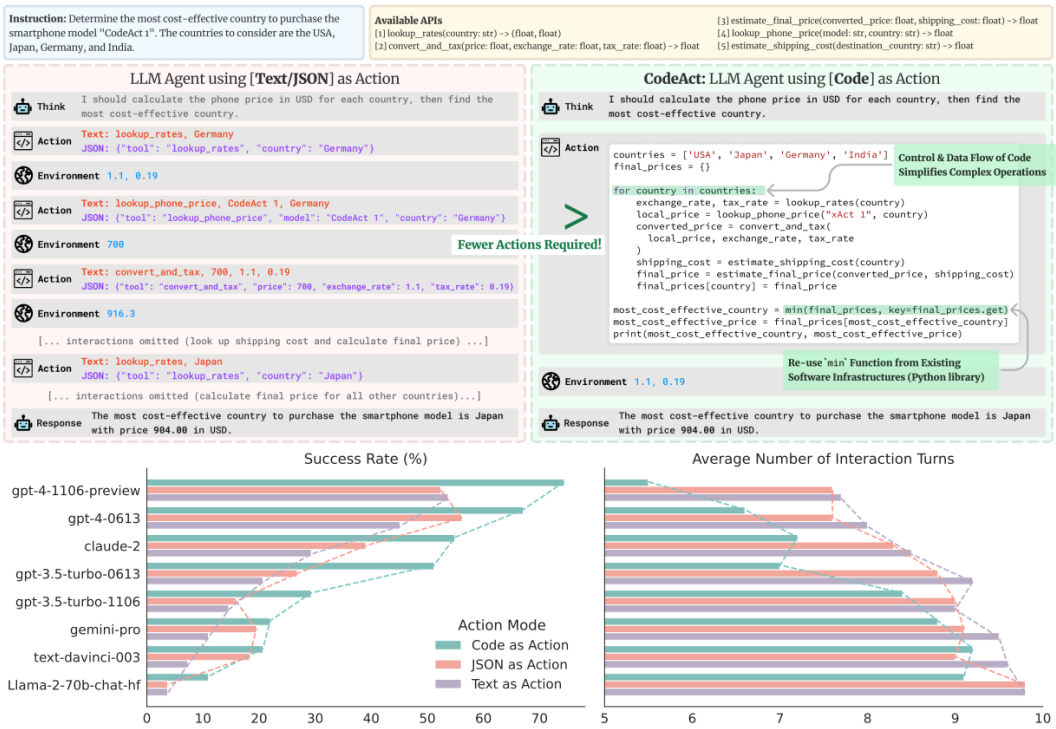


Figure 13.6 Comparing text, JSON, and a code interpreter as actions on the same example task. At the top are the generated steps, and below are the quantitative results across LLMs.

Let's now build a custom local agent and tools with this framework. A companion Colab notebook includes the full code. To run it on Colab's free-tier VM, you'll need GPU acceleration.

We'll build an agent that can book flights, managing the full flow from a single prompt. Because we're using a small local model, we'll use smolagents' `TransformersModel` class to download models from the Hugging Face Hub and run them locally:

```
import torch
from smolagents import TransformersModel

model = TransformersModel("HuggingFaceTB/SmolLM2-1.7B-Instruct",
                          device_map="auto",
                          max_new_tokens=1000,
                          torch_dtype=torch.float16)
```

In this example, we'll use `SmolLM2 1.7B Instruct` (https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct), the largest model in the SmolLM2 family pretrained by Hugging Face for the smolagents framework. The first time you run the preceding code snippet, it will download the model checkpoints, the tokenizer, and the configuration files. Note that you don't have to choose a SmolLM2 model when implementing custom agents with the framework's TransformerModel class; any instruction-tuned model hosted on the Hub will work.

Now that we have the model, we can start implementing the tools we'll add to the agent toolbox so it can complete its assigned tasks. In this example, we'll implement tools as functions and decorate them with `@tool`.

The first function, `search_for_flights`, will search for and select a flight between the origin and destination airports. It will parse a mock web page to find flights that match the user-provided origin and destination, though you could adapt it to browse a real web page. Since this function only mocks a real search, and our goal is to understand the workflow of agents in smolagents, I won't reproduce the code here. The companion notebook includes the full implementation along with a mock website (a multiline string) that lists flight information in the following schema:

```
1. Flight DUB-BOS123
- Price: $350
- Origin: Dublin Airport (DUB)
- Destination: Boston Logan International Airport (BOS)
- Date: January 10, 2025

2. Flight DUB-EWR456
- Price: $370
- Origin: Dublin Airport (DUB)
- Destination: Newark Liberty International Airport (EWR)
- Date: January 10, 2025
...
```

Next, we'll implement a function that selects a seat for a given flight.

Listing 13.3 Tool for finding a seat for a flight

```
import random

@tool
def find_seat(flight: str, seat_class: str='Economy') -> str:
  """Finds a seat within a given flight.
```

```
    Args:
      flight: The flight code.
      seat_class: The desired seat class.

    Returns:
      The seat number.
    """

    if seat_class not in ['Business', 'Economy']:
          raise ValueError("ticket_type must be either 'Business' or
'Economy'")

    seat_type = random.choice(['A', 'B', 'C', 'D', 'E', 'F'])

    if seat_class == 'Business':
          seat_number = random.randint(1, 3)
    else:
        seat_number = random.randint(4, 32)

    selected_seat = f"{seat_type}{seat_number}"

    return selected_seat
```

The preceding function can pick a seat based on the specified class (business or economy); for simplicity, it generates a seat number at random. Each custom function you define as a tool must include a docstring describing what it does plus its inputs, outputs, and type hints, which the decorator uses to convert the function into a usable tool for an agent.

With our model and custom tools in place, we can build our flight-booking agent:

```
from smolagents import CodeAgent

custom_tools = [search_for_flights, find_seat]
agent = CodeAgent(tools=custom_tools, model=model,
                  verbosity_level=2, max_steps=2)
```

The preceding code is a `CodeAgent`. We've added tools to its default toolbox, selected the local SLM, set verbosity to 2 (info), and limited it to two steps (for now). We can provide a task and have the agent do the rest:

```
user_task = "Find a flight from Dublin to New York and then select a seat."
agent.run(user_task)
```

The agent first reasons about the assigned task and then splits it into steps. For each step, the agent selects the appropriate tool to search for a flight from Dublin to New York and choose a seat. It then generates Python code to execute each step and returns the results to the user. The logs from this run show the following two steps:

- I will use the `search_for_flights` tool to find a flight from Dublin to New York.
- I will use the `find_seat` tool to find a seat on the flight from Dublin to New York.

These steps include the tool that the agent has identified as the best fit for the action. In the first step, it generated the following Python code using the SLM:

```
```py
origin = 'DUB'
destination = 'JFK'
date = '2023-01-01'
web_page = 'https://www.google.com/search?q
➥=flights+from+Dublin+to+New+York+on+2023-01-01'
answer = search_for_flights(origin=origin, destination=destination,
➥ date=date, web_page=web_page)
print(answer)
```
```

It calls the `search_for_flights` tool, passes the Dublin and JFK IATA codes (derived from the origin and destination city names), and attempts a web search (which won't run here because the tool is restricted to a mock web page). This step returns two results because the page lists two flights for the same route on different dates:

```
[{'Flight Code': 'DUB-JFK789', 'Price': '400', 'Origin': 'dublin airport
(dub)', 'Destination': 'john f. kennedy
international airport (jfk)', 'Date': 'January 20, 2025'}, {'Flight Code':
'DUB-JFK101', 'Price': '450', 'Origin':
'dublin airport (dub)', 'Destination': 'john f. kennedy international
airport (jfk)', 'Date': 'January 10, 2025'}]
```

Because the task doesn't specify which flight to choose, the agent selects the first result returned by the search.

Here's the Python code generated for the second step:

```
```py
flight_code = 'DUB-JFK789'
answer = find_seat(flight=flight_code)
print(answer)
```
```

The agent calls the `find_seat` tool using the code for the first flight returned in the previous step. Since the user didn't specify a class, the agent defaults to economy. In this run, the tool returned seat D24.

Let's make things a bit more complicated for the agent. We'll add a third tool and see whether it affects the planning phase. The agent is still responsible for booking flights, but we're introducing `fibonacci`, a tool unrelated to its main duties. It returns the n^{th} Fibonacci number (a sequence in which each number is the sum of the two preceding numbers, starting with 0 and 1). You can find the full code in the notebook. Let's add it to the agent's tool list:

```
custom_tools = [search_for_flights, find_seat, fibonacci]
agent = CodeAgent(tools=custom_tools, model=model,
                  verbosity_level=2, max_steps=2)
```

Now we'll run the previous task again. When the agent receives the task, it identifies the same steps as it did when no `fibonacci` tool was available, and it produces the same Python code to execute them. Here we added only one unnecessary, task-unrelated tool for readability, but you should see the same behavior with a larger tool pool, where only a few tools are pertinent (assuming you've applied the tool-naming and typing guidance discussed earlier in this section).

We can extend this custom agent's workflow by adding a tool that books the selected flight ticket (it's a mock booking for now, but it will let us test how the agent plans a task).

Listing 13.4 Tool for mocking the flight ticket booking

```
@tool
def book_flight_ticket(flight: str, seat: str) -> str:
  """Books a flight ticket.

  Args:
    flight: The flight code.
    seat: The seat number.

  Returns:
    The ticket number.
  """
  ticket_number = random.randint(100000, 999999)

  return f"Ticket booked for flight {flight} and seat {seat}
➥. Ticket number: {ticket_number}"
```

We'll add this tool to the agent's toolbox and set the maximum number of steps to 3:

```
custom_tools = [search_for_flights, find_seat, book_flight_ticket]
agent = CodeAgent(tools=custom_tools, model=model,
                  verbosity_level=2, max_steps=3)
```

Now we'll run the same tasks as in the previous examples:

```
user_task = "Find a flight from Dublin to New York and then select a seat."
agent.run(user_task)
```

The agent autonomously identifies three steps—searching for the ticket, selecting a seat, and booking the flight—even though the user doesn't explicitly ask for these tasks. This is the final output from the model for this run:

```
Out: Ticket booked for flight DUB-JFK789 and seat B26. Ticket number: 798158
```

In this section, we've built a single agent that can use multiple tools. smolagents also supports multi-agent systems, where multiple agents interact and collaborate to solve more complex tasks. We'll cover this in chapter 14.

Note that when implementing a local custom agent with smolagents, `TransformersModel` isn't your only option for loading SLMs. You can also use models hosted on an Ollama server (covered in chapter 12). The overall workflow is the same.

In our flight-booking example, almost everything stays the same except for the model serving. First, we install a local Ollama server and start it running:

```
ollama serve
```

We can now download a quantized Phi-3 Mini 4K Instruct model:

```
ollama pull phi3:mini
```

The only code change required to switch the agent to this model is as follows:

```
from smolagents import LiteLLMModel

model = LiteLLMModel(
        model_id="phi3:mini",
        api_base="http://localhost:11434",
        api_key="<your-api-key>",
        num_ctx=4096
)
```

We'll use the LiteLLMModel class, which requires the model ID from the Ollama server, the base URL (specifying the correct port if we are not using the default), the local API key, and the context size. Then we pass this model when creating the `CodeAgent`, as usual:

```
custom_tools = [search_for_flights, find_seat, book_flight_ticket]
agent = CodeAgent(tools=custom_tools, model=model,
                  verbosity_level=2, max_steps=3)
```

Here are a few best practices for building higher-quality agents with this framework; many of them apply to agentic AI development in general:

- Use only instructed models.
- Performance often depends on model size. For agentic API applications, very small models tend to underperform, unlike what we see in other applications. That doesn't mean you can't use SLMs, but models under 3 billion parameters may not deliver satisfactory results. Use an evaluation strategy appropriate for your chosen models.
- Keep your agentic AI workflow design as simple as possible to reduce risk, cost, and latency.
- Agentic systems combine deterministic components with probabilistic ones (the SLMs). Although these models can self-correct, a best practice is to make the system as deterministic as possible: give the model only tasks it can reliably complete, and shift any deterministic work to other parts of the architecture.

- Whenever possible, combine two or more tools into one. This reduces the number of calls to SLMs, which lowers costs.
- Keep tasks for SLMs simple and clear.
- Ensure that for any data types the model handles through custom tools, you validate formats and value ranges. This reduces the chance of errors and simplifies your fallback strategy implementation.

In the next chapter, we'll explore advanced RAG and AI agent concepts for SLM scenarios and look at how we can combine them.

Summary

- Language models alone provide limited business value without being integrated into complex systems that add external knowledge and tools.
- RAG systems enhance language models by retrieving information from external knowledge bases and generating text.
- Document chunking splits large texts into smaller segments to avoid exceeding embedding and generator model capacity limits.
- PyMuPDF extracts PDF content, and spaCy detects sentence boundaries for preprocessing.
- The Sentence Transformers library converts text into dense vectors using encoder-only Transformer architectures.
- Cosine similarity measures semantic relationships between query and document embeddings to retrieve relevant content.
- Vector databases such as LanceDB store embeddings and use approximate nearest neighbor algorithms for fast similarity search.
- LanceDB offers serverless vector storage with automatic versioning and zero-copy embedding management.
- Embedding models must be identical for both creating a knowledge base and processing queries so the vectors share a consistent space.
- Generator models receive enriched prompts containing user queries and retrieved context to produce informed responses.
- Prompt-formatting functions combine user queries with retrieved context using model-specific chat templates.
- AI agents autonomously perform tasks and make decisions by reasoning about problems and acting through external tools.
- The smolagents framework lets you build local agents using small language models and Python-based tools.
- `CodeAgent` writes and runs Python snippets for each step instead of generating JSON instructions.
- Tool functions need docstrings and type hints so they're automatically available as agent-usable capabilities.

- ReAct agents integrate reasoning and acting by iteratively analyzing problems and using available tools to reach solutions.
- Custom tools extend agent capabilities beyond default toolsets for domain-specific automation.
- Agent planning involves breaking complex tasks into steps and selecting the right tools to execute them.
- Multistep workflows let agents chain tool outputs together to complete complex tasks.

14 Advanced components for LLM applications

This chapter covers

- RAG on a knowledge graph
- Combining RAG and agentic AI using only SLMs
- Managing short- and long-term memory for chatbots and AI agents

In chapter 13, you learned about two key paradigms for language models: retrieval--augmented generation (RAG) and agentic AI. We built a RAG system from scratch, introduced vector databases for these applications, and implemented an AI agent using the open source smolagents framework. In this chapter, we'll cover some new concepts and tools that can make RAG systems and agents more robust, and you'll learn how to combine them.

14.1 GraphRAG

In the previous chapter, you learned that RAG connects language models to external data sources, giving them access to private or domain-specific information and reducing hallucinations. A baseline RAG typically combines a vector database with two language models; the vector database stores embeddings, one model retrieves

context for a user query, and the other model generates an answer from that context. This approach works well in many cases, but it can struggle with complex tasks that require multihop reasoning or connecting information from multiple sources. The core challenge is that baseline RAG retrieves text by semantic similarity, which does not reliably surface specific details needed to answer complex questions, especially when those details are not stated explicitly in the knowledge base. As a result, finding the right information can be difficult, sometimes leading to costly workarounds, such as manually creating Q&A pairs for common queries.

One way to address these challenges is to augment RAG retrieval and generation with *knowledge graphs* (KGs). A KG is a structured representation of knowledge that captures entities, their attributes, and the relationships between them. KGs consist of nodes (entities such as people, places, or concepts) and edges (relationships between entities), which help machines better represent context and meaning and can improve search, data integration, and reasoning. This is the definition of GraphRAG.

> **NOTE** For an in-depth look at knowledge graphs and LLMs, see *Knowledge Graphs and LLMs in Action* by Alessandro Negro et al. (2025; https://www.manning.com/books/knowledge-graphs-and-llms-in-action).

One of the best approaches to implementing GraphRAG comes from Microsoft's paper "From Local to Global: A GraphRAG Approach to Query-Focused Summarization" (https://arxiv.org/pdf/2404.16130). It presents a graph-based approach to question-answering over private text corpora that scales with both the breadth of user questions and the amount of source text. It uses an LLM to build a graph index in two stages:

- Derive an entity knowledge graph from the source documents.
- Pregenerate community summaries for groups of closely related entities.

Given a question, each community summary is used to generate a partial response; these partial responses are then summarized into a final response for the user. This approach, named GraphRAG, substantially improves on a conventional RAG baseline in both the comprehensiveness and the diversity of generated answers.

Figure 14.1, adapted from the Microsoft paper, shows the general GraphRAG workflow. Blocks on the left represent tasks in the indexing phase; blocks on the right represent tasks in the query phase. Some artifacts (such as graph communities and community summaries) are generated during indexing but used at query time, which is why they are shown in a different color. Indexing typically runs offline and produces a KG on top of the knowledge base. Input documents must be selected and preprocessed outside the indexing pipeline, because the quality of the KG depends on the quality of the source documents. The extracted text is chunked and sent to an LLM, which extracts domain-specific entities and relationships (the KG's nodes and edges) and then detects and summarizes graph communities. Here, a community is a group of nodes that are more densely connected to one another than to the rest of the graph.

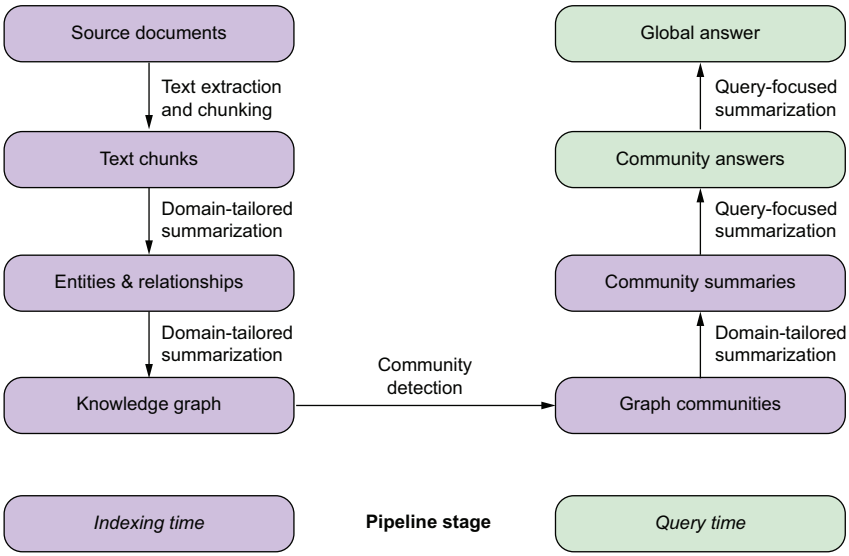


Figure 14.1 The logical blocks and workflow of Microsoft's GraphRAG proposal

Imagine a KG representing a movie database with entities such as movies, actors, directors, genres, and production companies. In this setting, communities emerge from relationships between entities. For example, consider the Action Movies community, made up of the following entities (nodes):

- *Movies*—"Mad Max: Fury Road," "Die Hard," "Mission: Impossible – Fallout"
- *Actors*—"Tom Cruise," "Bruce Willis," "Charlize Theron"
- *Directors*—"George Miller," "John McTiernan," "Christopher McQuarrie"
- *Genres*—"Action"

It will have the following relationships (edges):

- "Mad Max: Fury Road" is of genre "Action."
- "Die Hard" features actor "Bruce Willis."
- "Mission: Impossible – Fallout" features actor "Tom Cruise"
- "Mad Max: Fury Road" is directed by "George Miller."
- "Die Hard" is directed by "John McTiernan."
- "Mission: Impossible – Fallout" is directed by "Christopher McQuarrie."

Figure 14.2 shows the same data as a graph.

These are the characteristics of this community:

- The movies, actors, and directors in this community are interconnected through multiple relationships (for example, actors appear in multiple action movies).

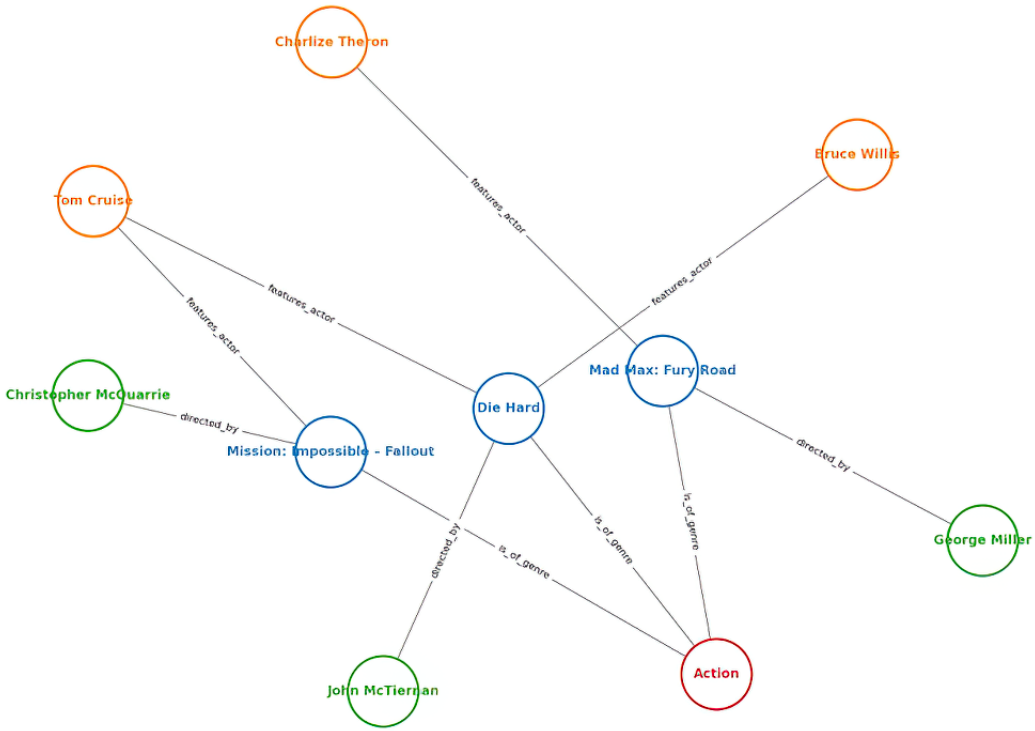


Figure 14.2 The action movie graph for the preceding example data

- All entities in this community belong to the "Action" genre, forming a distinct cluster in the KG.
- This community makes some downstream tasks easier, such as analyzing trends in action movies, studying actor-director collaborations, and generating recommendations for action-movie fans.

During the query phase, a user prompt is enriched with retrieved content (entities and relationships) and community summaries, and then it is passed to the generator model to produce the final answer.

After the GraphRAG paper was published, Microsoft also released an official open source implementation (https://github.com/microsoft/graphrag) under the MIT license. Following the concepts introduced earlier in this section, we'll build a naive Python implementation that's framework- and vendor-agnostic. We'll use a minimal set of Python packages because our goal is to understand the concepts and how they can benefit a RAG system, not to learn a specific implementation. Unlike Microsoft's GraphRAG implementation, which supports only OpenAI and Azure OpenAI endpoints, we'll use small, local, open source models. A companion Colab notebook is provided for this section, and we'll refer to figure 14.1 throughout the implementation.

For this example, we'll use the quantized Mistral 7B model from the Ollama library (https://ollama.com/library/mistral), hosted on a local Ollama server. I chose it because it performs well at extracting entities and relationships from unstructured text, outperforming other smaller open source models, performing nearly as well as the larger models supported by the MS implementation. The code runs without hardware acceleration, but a GPU is recommended to speed up model tasks, especially when working with a large knowledge base corpus.

The Colab notebook downloads three PDF papers on 8-bit quantization for LLMs—these will serve as our knowledge base. For simplicity, we'll focus on one of the papers, "On-Device LLMs for Home Assistant: Dual Role in Intent Detection and Response Generation" (https://arxiv.org/pdf/2502.12923), but you can run the notebook with all the documents at any time.

After downloading the PDF, we'll extract its text and save it to a file for chunking. The notebook provides the `extract_text_from_pdfs` function to read PDFs from a directory, extract their text, and save it. Next, we'll chunk the extracted text using the `split_text_into_chunks` function, which takes the path of the directory containing the text files and a chunk size. It returns a dictionary mapping each original text filename to its list of chunks.

Once the text chunks are available, we can query the model to identify entities and relationships. We'll assume the Mistral 7B model is deployed and available through our local Ollama server. To send requests to the model, we'll use the official ollama-python client library (https://github.com/ollama/ollama-python).

We'll also define a custom function, `extract_entities_and_relationships`, to make these requests, but first we need to address an issue with the format of the generated responses. I ran multiple experiments with the model using this final prompt:

```
prompt = f"""Extract entities and relationships from this
➥ text:\n\n{chunk}\n\n
Relationships must follow the format 'Relationship, (Entity1, Entity2)'"""
```

I was prompting the model to extract entities and relationships from a text chunk and return them in a fixed format, but even with the temperature set to 0, the responses didn't always match the expected format: a list of entities and a list of relationships. This made parsing difficult before building the KG: about 6% of entity outputs and 25% of relationship outputs had formatting errors.

To avoid this, we'll use a feature in recent Ollama releases: structured outputs, which constrain the model's output to a JSON schema. This feature is available through the official Python and JavaScript client APIs. First, we define the schema as a Pydantic class:

```
from pydantic import BaseModel

class RawKnowledgeGraph(BaseModel):
  entities: list[str]
  relationships: list[str]
```

We ask the model to return two lists: one for entities and one for relationships. Then the code to query the model, in the `extract_entities_and_relationships` function, looks like the following listing.

Listing 14.1 Extracting entities and relationships from a knowledge base

```
from ollama import chat
t
from ollama import ChatResponse

def extract_entities_and_relationships(chunks_dict, model_id):
  results = {}
  for filename, chunks in chunks_dict.items():
        results[filename] = []
        for chunk in chunks:
            prompt = f"""Extract entities and relationships from this
                  ➥ text:\n\n{chunk}\n\n
Relationships must follow the format 'Relationship, (Entity1, Entity2)'"""
            response: ChatResponse = chat(model=model_id, messages=[
              {
                'role': 'user',
                'content': prompt,
              },
            ],
            format=RawKnowledgeGraph.model_json_schema(),
            options={"temperature":0}
            )
            results[filename].append(response['message']['content'])

  return results
```

This function uses ollama-python's chat method to send requests for each text chunk in the dictionary and identify entities and relationships between them. These become the nodes and edges of the KG. The function's output is stored in the `content` attribute of ollama-python's `ChatResponse` class. As specified by the `format` attribute, it is a dictionary with the filename the chunk belongs to being the key, and the value following the JSON schema is defined by the custom `RawKnowledgeGraph` Pydantic class. Here's a truncated sample response for one text chunk:

```
RawKnowledgeGraph(entities=['On-Device LLMs for Home Assistant', 'Dual Role
in Intent Detection and Response Generation', 'Rune Birkmose', 'Nathan
Mørkeberg Reece', 'Esben Hofstedt Norvin', 'Johannes Bjerva', 'Mike Zhang',
'Aalborg University, Denmark', '{rbirkm20, nreece20,
enorvi20}@student.aau.dk', 'jjz@cs.aau.dk', 'Large Lan-guage Models
(LLMs)', 'synthetic but domain-representative data', 'slot and intent
detection', 'natural language response generation', 'resource-limited, CPU-
only edge hardware', 'JSON action calls', 'text responses', '16-bit and 8-
bit quantized variants', '4-bit model', 'device-service classification
accuracy', 'noisy human prompts', 'out-of-domain intents', 'IoT Analytics,
2024', 'BBC News, 2025', 'Amazon, Google, and Apple', 'speech recognition
and intent detection on cloud servers', 'privacy concerns', 'data
```

```
ownership', 'reliance on proprietary ecosystems'],
relationships=['Investigates, (Large Lan-guage Models (LLMs), On-Device
LLMs for Home Assistant)', 'Fine-tuned, (Large Lan-guage Models (LLMs),
synthetic but domain-representative data)', 'Performs, (On-Device LLMs for
Home Assistant, slot and intent detection)', 'Performs, (On-Device LLMs for
Home Assistant, natural language response generation)', 'Produces, (On-
Device LLMs for Home Assistant, JSON action calls)', 'Produces, (On-Device
LLMs for Home Assistant, text responses)', 'Obtains, (On-Device LLMs for
Home Assistant, around 80-86% accuracy)', 'Affirms, (On-Device LLMs for
Home Assistant, ability to unify command interpretation and flexible
response generation for home automation)', 'Typically handles, (Amazon,
Google, and Apple, speech recognition and intent detection on cloud
servers)', 'Raises concerns about, (cloud servers, privacy, data ownership,
reliance on proprietary ecosystems)'])
```

The returned content needs no further post-processing before we build the KG because the two lists always come back in the expected format. At this point, we can already build a graph from the returned entity and relationship lists. But as recommended in the GraphRAG paper, we'll first generate summaries for the returned entities.

Our local Mistral 7B model will handle this step. A function for this, `summarize_elements`, is available in the notebook. It loops over all elements and, for those in the same text chunk, asks the SLM to generate a summary using the following prompt:

```
prompt = f"""Summarize the following entities and relationships
       ➥in the same structured format:
Entities:
{entities}

Relationships:
{relationships}
                """
```

The summarized entities (in the same format as the originals) are used to build the KG. For this, we'll use the NetworkX Python package (https://github.com/networkx/networkx), the standard library for network manipulation.

Once the model returns all required elements

```
elements = extract_entities_and_relationships(chunks_dict, 'mistral')
```

and they have been summarized

```
element_summaries = summarize_elements(elements)
```

we can iterate over them, build a KG for each, and then merge them into a final graph. A custom `build_graph` function, provided in the notebook, builds a graph for a single chunk using the NetworkX API. Given lists of entities and relationships, it first creates an empty undirected graph:

```
import networkx as nx

graph = nx.Graph()
```

Then it adds the nodes (entities):

```
graph.add_nodes_from(entities)
```

Finally, it adds the edges (relationships):

```
for relationship in relationships:
    relationship_name = relationship[0]
    subject1 = relationship[1]

    if len(relationship) > 2:
        subject2 = relationship[2]
        graph.add_edge(subject1, subject2, label=relationship_name)
    else:
        graph.add_node(subject1)
```

After extracting the relationship name and first entity, the previous snippet determines the second entity, handling the cases with more than two, and adds a node without an edge if only one entity is present. This rare case occurs when the model hallucinates and can't split a statement into its parts. Note that this is still a minimal implementation, with no validation checks.

The `build_graph` function iterates over the keys and chunks in the model's returned dictionary to build partial graphs, which are then merged using NetworkX's `compose` function, as in the following listing.

Listing 14.2 Building the KG starting from the generated entities and relationships

```
combined_graph = nx.Graph()

for filename, chunks_data in element_summaries.items():
  for chunk_data in chunks_data:
    try:
      chunk_knowledge_graph =
      ➥RawKnowledgeGraph
      ➥.model_validate_json(chunk_data)
      entities = chunk_knowledge_graph.entities
      relationships =
      ➥chunk_knowledge_graph.relationships

      graph = build_graph(entities, relationships)
      combined_graph
      ➥= nx.compose(combined_graph, graph)

    except Exception as e:
      print(f"Error processing data for {filename}: {e}")
```

Checks that the current chunk data is in valid JSON format

Extracts the current chunk entities

Extracts the current chunk relationships

Builds the partial graph

Combines the partial graph with the main graph

The generated graph can now be rendered. The `visualize_graph_with_plotly` function, which visualizes it using the Plotly API for Python, is available in the notebook.

The indexing isn't finished yet. We still need to compute the communities in this graph. NetworkX includes several community-detection algorithms, including the popular *Louvain* method. However, the GraphRAG paper reports better results with hierarchical clustering using the Leiden algorithm (https://arxiv.org/abs/1810.08473), which NetworkX doesn't provide out of the box. The reference implementation is a Python wrapper around the libleidenalg C++ library, so we'll build a simple pure-Python version.

The Louvain algorithm starts from a singleton partition, with each node in its own community. It then iteratively moves nodes between communities, builds an aggregated network from the resulting partition, and iterates on the aggregated network until the quality score no longer improves. According to the Leiden paper, one problem with the Louvain algorithm is that it can identify communities that are internally disconnected. The Leiden algorithm is partly based on its predecessor but addresses this issue. Like Louvain, it starts from a singleton partition and moves nodes between communities to improve the partition, but it refines the partition before aggregation. An aggregated network is then created based on the refined partition. The algorithm then repeats the move and aggregate steps until no further improvement is possible.

The Leiden algorithm optimizes *modularity*, a metric that measures how strongly a network divides into clusters. Maximizing modularity helps ensure the communities that are found are well defined and meaningful. The companion Colab notebook includes a simple Python implementation, including a utility function that computes the modularity gain from moving a node to a particular community in the graph:

```
def calculate_modularity_gain(graph, node, community,
➥ communities, resolution):
  m = graph.number_of_edges()
  k_i = graph.degree(node)
  k_i_in = sum(1 for neighbor in graph.neighbors(node)
  ➥if communities[neighbor] == community)
  sigma_tot = sum(graph.degree(v) for v in graph.nodes
  ➥if communities[v] == community)

  return (k_i_in - k_i * sigma_tot / (2 * m)) * resolution
```

A higher modularity gain means that moving a node into a community would improve the graph's community structure. The resolution argument controls community size (higher values produce smaller communities). The `calculate_modularity_gain` function computes the graph's total edge count, the input node's degree (number of connections), the node's number of connections to other nodes in the input community, the target community's total degree for all nodes in the input community, and the change in modularity if the node is moved. This modularity gain formula compares the actual number of edges within the community with the expected number if edges were distributed randomly; the `resolution` parameter scales the gain.

A second utility function, `aggregate_communities`, aggregates (merges) communities in the graph based on their connectivity.

Listing 14.3 Custom function to aggregate communities within a graph

```
def aggregate_communities(graph, communities):
  community_mapping = {}
  next_community_id = 0

  for component in nx.connected_components(graph):
    for node in component:
      if communities[node]
      ➥not in community_mapping:
        community_mapping[communities[node]] = next_community_id
        next_community_id += 1

      communities[node]
      ➥ = community_mapping[communities[node]]

  num_communities = next_community_id

  return communities, num_communities
```

Iterates across the connected component of the input graph

If the current node's community isn't in the community mapping, its ID is assigned a new aggregated community ID.

The current node's community assignment is updated to the new aggregated community.

At this stage, each node has an assigned community ID. This function iterates over each connected component (a subgraph in which every node is reachable from every other) in the graph. For each node, it checks whether that community ID is already in the community mapping; if not, it adds it and assigns it a new aggregated community ID. It then updates the node's community assignment to that aggregated ID. Finally, it returns the updated mapping from nodes to aggregated community IDs, along with the total number of aggregated communities.

The function containing our custom implementation of the Leiden algorithm, `leiden_algorithm`, uses the two preceding utility functions.

Listing 14.4 The custom implementation of the Leiden algorithm

```
import random

def leiden_algorithm(graph, resolution=1.0, max_iterations=100):

  communities = {node: i for i, node in enumerate(graph.nodes)}
  num_communities = len(communities)

  for _ in range(max_iterations):
    improved = False

    for node in graph.nodes:
      best_community = communities[node]
      best_modularity_gain = 0

      for neighbor in graph.neighbors(node):
```

```
            neighbor_community = communities[neighbor]
            if neighbor_community != best_community:
              modularity_gain = calculate_modularity_gain(graph, node,
              ➥ neighbor_community, communities, resolution)
              if modularity_gain > best_modularity_gain:
                best_modularity_gain = modularity_gain
                best_community = neighbor_community

          if best_community != communities[node]:
            communities[node] = best_community
            improved = True

        for node in graph.nodes:
          if len([n for n in graph.nodes if communities[n]
          ➥ == communities[node]]) == 1:
            neighbors = list(graph.neighbors(node))
            if neighbors:
              communities[node] = communities[random.choice(neighbors)]

        if improved:
          communities, num_communities = aggregate_communities(graph,
          ➥ communities)
        else:
          break

      return [communities[node] for node in graph.nodes]
```

The goal is to identify groups of nodes (communities) that are more densely connected to each other than to the rest of the input graph. Initially, each node is assigned to its own community, so the number of communities equals the number of nodes. The function then iterates for the number of steps specified by the `max_iterations` argument. For each node, it finds the community that would yield the largest modularity gain if the node were moved. If it finds a better community, it moves the node and sets the improved flag to `True`. It then runs a second pass across the nodes; any single-node communities (singletons) are reassigned to a neighbor to keep them from being isolated. The final step is community aggregation: if the community structure is improved in the current iteration (that is, `improved` is `True`), the `aggregate_communities` function merges communities based on connectivity. If there is no improvement, the algorithm terminates.

Finally, it returns a list of the final community assignment for each node in the graph:

```
communities = leiden_algorithm(combined_graph)
```

As a final indexing step, we generate community summaries using a local Mistral 7B model. First, we identify the unique communities in our list:

```
unique_communities = sorted(list(set(communities)))
```

Next, we iterate through the communities to group the nodes by unique community:

```
 community_list = []
for community_id in unique_communities:
    community_nodes = [node for node, community in zip(graph.nodes,
    ➥community_assignments) if community == community_id]
    community_list.append(community_nodes)
```

`community_list` is a list of communities, each containing node IDs. In another `for` loop, we iterate over `community_list` and extract the text chunks associated with each node in a community:

```
community_summaries = {}
    for community_id, community_nodes in enumerate(community_list):
        community_text = ""
        for node in community_nodes:
          if node in chunks_dict:
              for chunk in chunks_dict[node]:
                  community_text += chunk + "\n"
```

The community text is used to build a prompt for the Mistral model, which then generates a summary:

```
prompt = f"""Summarize the following text which represents a community of
        ➥ related documents:

{community_text}
"""
response: ChatResponse = chat(model=model_id, messages=[
    {
        'role': 'user',
        'content': prompt,
    },
], options={"temperature": 0})
summary = response['message']['content']
```

Each generated summary is then added to the `community_summaries` list:

```
community_summaries[community_id] = summary
```

Generating community summaries is expensive because it requires many calls to the model (depending on the number of communities and the size of the knowledge base). If you are running locally, this mostly costs time; if you use a third-party API, you'll need to factor in charges for API calls and input or generated tokens.

The indexing process is now complete. Next, we'll look at how we can search these summaries. We'll implement what the Microsoft paper calls global search. Microsoft's GraphRAG implementation also includes local search, which we'll cover in section 14.1.1.

Once we've completed the previous steps, users can start asking questions of our knowledge base and receive answers generated by the SLM. Global search uses the following workflow:

1. Intermediate answers are generated for each community summary in relation to the query.
2. The final answer is generated by combining the intermediate answers into a concise response.

Let's see what this implies in code. The notebook's global search uses two functions: `generate_answers` and `generate_final_answer`. The `generate_answers` function loops through the community summaries and, for each one, queries the local model with the following prompt:

```
prompt = f"""
        Query: {query}

        Use the following community summary to answer the query:

        {summary}

        Answer:
        """
```

It returns a list of generated answers for the user's query and a summary for each community (a dictionary whose keys are community IDs and whose values are the intermediate answers). Downstream, the `generate_final_answer` function combines those intermediate answers to enrich the prompt for the local model and produce the final answer:

```
combined_answers_prompt = "Combine the following answers into a single,
                          ➥ concise response:\n\n"
    for community_id, answer in intermediate_answers.items():
        combined_answers_prompt += f"Community {community_id}: {answer}\n\n"
```

This is a naive implementation of the GraphRAG paper, and it's a good way to understand how GraphRAG works and why it can outperform vanilla RAG or other patterns that rely mostly on embedding similarity. Our custom implementation still has plenty of room for improvement, especially for local use. Here are some suggestions:

- Persist the generated graph elements (entities, entity summaries, communities, and community summaries) in Parquet format to avoid rerunning the indexing process for each new query.
- Validate the generated entities.
- Export the graph as GraphML for visual analysis in a third-party tool.
- Rank the generated community summaries, and then filter them by relevance to the user query before generating intermediate answers. This produces more accurate answers and fewer calls to the generative model.

14.1.1 Microsoft's open source GraphRAG capabilities

As mentioned earlier in this chapter, Microsoft released an open source implementation based on the GraphRAG paper. In its implementation, it offers two options at query time:

- *Global search*—Answers to user queries are based on community summaries generated at indexing time, which is the core idea of the GraphRAG paper. We used the same strategy in our custom implementation.
- *Local search*—Answers are generated by combining relevant data from an AI-extracted KG (entities) with text chunks from the source documents (embeddings). This option also converts the knowledge base to vectors and stores them in a vector database. This approach is out of scope for this book.

Microsoft's GraphRAG solution has been implemented in Python using only open source packages, but it supports only OpenAI's commercial generative and embedding models via the OpenAI or Azure OpenAI APIs. It doesn't support running open source SLMs locally or using other providers' APIs. Still, it's worth considering as an end-to-end solution: the code is open source, so you can fork the GitHub repository and adapt it to use local models via the Hugging Face Transformers library, vLLM, or Ollama, or use open source SLMs through Hugging Face endpoints.

Microsoft's open source GraphRAG solution includes built-in logic to persist the graph and related artifacts, such as communities and community summaries. It stores everything in Parquet format on disk or in Azure Blob Storage or Cosmos DB.

> **NOTE** Apache Parquet (https://parquet.apache.org/) is an open source, column-oriented storage format designed for efficient analytics and large-scale data processing. It uses compression and encoding to support complex data structures and fast queries.

Microsoft's local search option in GraphRAG stores document embeddings in a vector database. This persistence works out of the box when you use a supported option: the open source LanceDB (introduced in chapter 13) or the Azure services Cosmos DB and Azure AI Search. Even if you plan to use only global search, indexing still requires a connection to one of the supported vector databases. To use a different graph store or vector database, you must fork the GraphRAG GitHub repository and implement it yourself.

Microsoft's open source GraphRAG comes with a CLI, so we don't need to write any Python code (as long as we use only the supported models and storage systems). The package can be installed as follows:

```
pip install graphrag
```

Once that's done, we can initialize a GraphRAG space using the `graphrag` command, providing the path to the workspace root directory:

```
graphrag init --root <graphrag_space_root_dir>
```

After the workspace is created, edit the two configuration files it generates: `.env`, where you add your OpenAI or Azure OpenAI key, and `settings.yaml`, where you configure everything else (vector database connection, graph storage, LLM and embedding model settings, logging, and so on).

Then start indexing your knowledge base by running the same graphrag command in a terminal and waiting for it to complete:

```
graphrag index --root <graphrag_space_root_dir>
```

Once the indexing process is completed, we can perform a query as follows:

```
graphrag query --root < graphrag_space_root_dir> --method <search_method>
➥ --query "What are the main topic in this documents?"
```

In the simplest use case, we provide the workspace root path, the search method (global or local), and a query. Although the CLI is straightforward, we can also use this GraphRAG implementation programmatically via its Python API.

If you build a custom integration with other models, graph storage, or a vector store, you can still use the CLI in your fork to initialize the workspace, index data, and run queries.

Microsoft's open source GraphRAG includes the building blocks, but it lacks capabilities such as text extraction and document preprocessing, graph visualization, and results evaluation. For these, you'll need to integrate third-party open source or proprietary tools.

14.1.2 Evaluation metrics

RAG improves the accuracy of LLM responses. There are many techniques and frameworks for evaluating LLM outputs (some covered in earlier chapters), but how can we assess the language models used in a RAG system, including GraphRAG? Which metrics should we use?

Traditional metrics, such as BLEU and ROUGE, widely used in NLP, often fail to capture what RAG applications require, such as factual accuracy and context relevance. BLEU doesn't account for the context or evidence in the retrieved documents, which can leave gaps in evaluating factual accuracy, whereas ROUGE emphasizes recall and largely ignores precision. RAG needs both precision (how much retrieved information is relevant) and recall (how much relevant information is retrieved). We therefore need specialized RAG metrics, including metrics tailored to each phase of the pipeline (retrieval and generation).

> **NOTE** BLEU (Bilingual Evaluation Understudy) is a precision-based metric that evaluates the overlap of sequences of words between the generated response and a reference response. ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures the overlap of word sequences and word pairs between the generated response and reference texts, with a focus on recall.

For retrieval, these are meaningful dedicated evaluation metrics:

- *Context precision*—This measures how precisely the retrieved documents provide relevant information for a given query.
- *Context recall*—This assesses how well the retrieved documents cover all relevant aspects of a query.

For answer generation, these are meaningful dedicated evaluation metrics:

- *Faithfulness*—This measures the factual accuracy of a generated response against the retrieved documents.
- *Answer relevancy*—This assesses how relevant a generated response is to the original query.

In theory, you can combine the preceding metrics into an average RAG score. In practice, you must assign a weight to each score based on its importance in your domain (see table 14.1).

Table 14.1 RAG-specific metrics

Metric	RAG phase	Formula
Context precision	Retrieval	CP = Number of relevant sentences retrieved / Total number of sentences retrieved
Context recall	Retrieval	CR = Number of relevant sentences retrieved / Total number of relevant sentences available
Faithfulness	Generation	F = Number of correct facts in the response / Total number of facts in the response
Answer relevancy	Generation	AR = Number of relevant concepts in the response / Total number of concepts in the response

Table 14.1 summarizes these metrics for use in RAG systems. At the time of writing, only one open source Python library, Ragas (https://github.com/explodinggradients/ragas), implements these RAG-specific metrics, but it supports only commercial LLMs via the OpenAI, Azure OpenAI, and Google Vertex AI APIs, or models deployed on AWS Bedrock. You'll need a custom implementation to support local SLMs and custom models.

14.2 RAG + Agentic AI

In chapter 13, we discussed RAG (and continued in the first part of this chapter) and agentic AI. With those core concepts in mind, it's natural to consider using them together—a pattern commonly known as agentic RAG. In a traditional RAG system, a retriever model pulls information relevant to a user query from one or more external sources (databases, vector stores, the web, and so on) and passes it to a language model that generates a response using the added context. When you add AI agents to a RAG system, they can plan multi-step retrieval, adapt to what they find by updating queries,

and use multiple tools to improve response accuracy. By breaking complex tasks into smaller, specialized ones (often using specialized tools and/or smaller agents), agentic frameworks can make a RAG system more efficient, effective, and easier to understand and maintain. This setup is well suited to customer support, research assistance, knowledge management, and domain-specific problem solving. Figure 14.3 shows a logical schema for a generic, basic agentic RAG system:

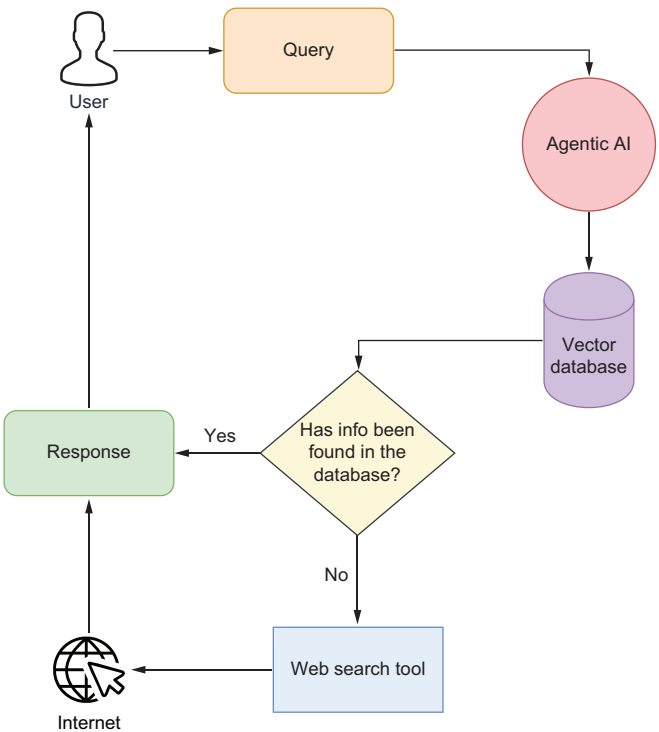


Figure 14.3 The logical schema of a basic agentic RAG system

Given a user query, the agent searches the vector database. If the retrieved information isn't relevant to the question, it then searches the web for additional context before generating the final answer.

Simple, flexible, and well integrated with the wider Python ecosystem for RAG—and with broad support for SLMs—smolagents (introduced in chapter 13) is a great choice for building an agentic RAG system. Let's implement a basic example like the one in figure 14.3.

A companion Colab notebook with the complete code is provided. We'll use the following open source Python libraries:

- LangChain
- pypdf

- LanceDB
- smolagents
- rank_bm25
- langchain-community
- langchain-huggingface

You're already familiar with LanceDB, smolagents, and pypdf (from chapter 13). LangChain (https://github.com/langchain-ai/langchain) is a framework for building LLM-powered applications, with standard interfaces for models, embeddings, vector stores, and more. I generally avoid frameworks that hide too much of the underlying logic because they can be harder to debug or customize, but LangChain is handy for implementing RAG systems. In this example, we'll use it mainly for the RAG components. We'll also use LangChain Community, which provides third-party integrations that implement the interfaces defined in the LangChain core framework. For a deeper dive into the ecosystem, see *AI Agents and Applications: With LangChain, LangGraph, and MCP* by Roberto Infante (Manning Publications, 2026).

For this example, we'll use the same paper as before (https://arxiv.org/pdf/2502.12923) as our knowledge base. After downloading the PDF to the Colab VM, we'll use the LangChain utility classes `PyPDFLoader` and `RecursiveCharacterTextSplitter` to load it, extract its text, and split it into chunks:

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

loader = PyPDFLoader("/content/arxiv_250212923.pdf")
documents = loader.load()

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
                                               chunk_overlap=100)
docs = text_splitter.split_documents(documents)
```

Here, `docs` is a list of LangChain `langchain.docstore.document.Document` objects—one for each text chunk.

We can now implement the agent's first tool. In chapter 13, we built smolagents tools using the @tool decorator, and you also saw that you can achieve the same result by extending the `Tool` class. Here we'll use the latter approach because it makes it easier to manage access to the knowledge base.

The first tool we'll create performs traditional text search to retrieve the parts of the source document most relevant to a user query. It uses BM25, a standard information-retrieval ranking algorithm that scores documents based on term frequency, inverse document frequency, and document length. BM25 is widely used in search engines and is the default ranking method in tools such as Elasticsearch and Apache Lucene. LangChain includes a built-in implementation.

This first tool class can be defined as follows:

```
from smolagents import Tool

class BM25SearchTool(Tool):
    name = "do_bm25_search_on_local_documents"
    description = "Uses text search to retrieve the parts of the
➥ documentation that could be most relevant to answer a query."
    inputs = {
        "query": {
            "type": "string",
            "description": "The search query string."
        },
    }
    output_type = "string"
```

In addition to extending the smolagents `Tool` class, we have added the required attributes: `name`, `description`, `inputs`, and `output_type`. The last two specify the tool's input and output types. `BM25SearchTool` implements the `init` function to initialize the class with the knowledge base:

```
from typing import List
from langchain.docstore.document import Document

def __init__(self, docs: List[Document], **kwargs):
        super().__init__(**kwargs)
        self.docs = docs
```

The `BM25SearchTool` class also implements the `forward` method, which defines what the agent does when it uses the `BM25SearchTool`:

```
from langchain_community.retrievers import BM25Retriever

def forward(self, query: str) -> str:
        retriever = BM25Retriever.from_documents(self.docs, k=3)
        docs = retriever.invoke(query)

        return "\nRetrieved documents:\n" + "".join(
            [f"\n\n===== Document {i} =====\n" + doc.page_content for i,
            ➥ doc in enumerate(docs)]
        )
```

It uses LangChain's `BM25Retriever` to search the document chunks.

Now let's download the SLMs for our agent (as we did in the example in chapter 13):

```
import torch
from smolagents import TransformersModel

model_id = "HuggingFaceTB/SmolLM2-1.7B-Instruct"
model = TransformersModel(model_id,
                          device_map="auto",
                          max_new_tokens=700,
                          torch_dtype=torch.float16)
```

After loading the model checkpoints, we can initialize the tool with the text-chunk pool that makes up our knowledge base:

```
bm25_search_tool = BM25SearchTool(docs)
```

Finally, we can implement the `CodeAgent` (discussed in section 13.3):

```
from smolagents import CodeAgent

agent = CodeAgent(
    tools=[bm25_search_tool],
    model=model,
    max_steps=3,
    verbosity_level=2,
    add_base_tools=True
)
```

Let's examine how this works before making the system more complex. After giving the agent only the BM25 custom search tool (in addition to those in the smolagents base toolbox), we ask it to search for a set of keywords:

```
query = "Make a BM25 search about Smart Home technologies"
agent_output = agent.run(query)
```

It automatically selects the right tool for the job (see figure 14.4) in the first execution step.

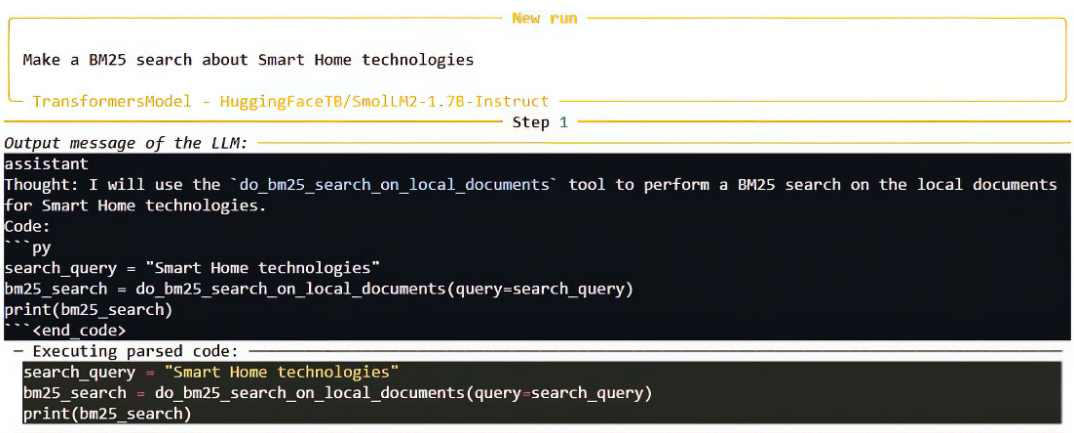


Figure 14.4 `CodeAgent` output for search at the first execution step

From the output in figure 14.4, we can see that the `do_bm25_search_on_local_documents` tool was selected, and the agent generated Python code to call it with the correct input

(the keywords from our query). In this case, the search returns results because the knowledge base covers edge-device topics, including smart home technology. Here is the tool-call response (three chunks, since the code limit is set to 3):

```
===== Document 0 =====
retaining generative fluency, suffers a notice-
able drop in device-service classification ac-
curacy. Further evaluations on noisy human
(non-synthetic) prompts and out-of-domain in-
tents confirm the models' generalization abil-
ity, obtaining around 80–86% accuracy. While
the average inference time is 5–6 seconds per
query—acceptable for one-shot commands but
suboptimal for multi-turn dialogue—our results
affirm that an on-device LLM can effectively
unify command interpretation and flexible re-
sponse generation for home automation without
relying on specialized hardware.
1 Introduction
Smart home technologies and IoT devices have
proliferated in recent years, with an expected rise
from 16.6 billion to 18.8 billion connected devices
by the end of 2024 (IoT Analytics, 2024). Major
providers like Amazon, Google, and Apple typi-
cally handle speech recognition and intent detec-
tion on cloud servers, which raises user concerns
about privacy, data ownership, and reliance on pro-

===== Document 1 =====
dataset (acon96, 2024), which consists of 35,840
synthetic examples designed to mimic Home As-
sistant commands. Each instance consists of:
• A User Prompt: e.g., "Turn on the kitchen
light", "Set the thermostat to 22 degrees".
• One Valid JSON Action , containing the
service and device fields corresponding to
Home Assistant calls.
• A Natural Language Response: e.g., a para-
phrase or affirmation of the action taken.
2

===== Document 2 =====
• Slot and Intent Detection: The model out-
puts a valid JSON object that maps to a de-
sired service (intent) and device (slot) pair:
{"service": "light.turn_on",
"device": "light.living_room",
"assistant": "Sure, turning on
on the living room light."}
• Natural Language Generation: The model
also produces a textual response confirming
or elaborating on its action, as can be seen
in the example above. The text can then be
propagated to, e.g., a text-to-speech model.
Traditional classifiers only handle device-service
```

```
classification and do not produce any text. For user-
facing text, the baseline approach would rely on
templated responses.
3.1 Data and Pre-processing
To the best of our knowledge, there is no ex-
isting human-curated dataset specifically for the
Home Assistant software. Thus, we rely on syn-
thetic data. We use a publicly available synthetic
dataset (acon96, 2024), which consists of 35,840
synthetic examples designed to mimic Home As-
```

All three chunks are relevant to the query topic. The top result contains all three keywords the agent passed to the tool, while the others contain only one keyword (or closely related terms).

Let's make the system more complex by adding a vector database to store our knowledge base as embeddings and a new similarity-search tool. This will let us see whether the agent chooses keyword search, vector search, or a hybrid of both. Hybrid search combines keyword and vector search to further improve retrieval quality (and, in turn, improve any downstream tasks that use the returned documents).

First, we need an embedding model:

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

We use the `langchain-huggingface` package's `HuggingFaceEmbeddings` class to download the embedding model from the Hugging Face Hub. This time, we'll use `all-MiniLM-L6-v2`, also available via the sentence-transformers library; it's fast and delivers good quality.

Next, we'll create and populate a vector database. We'll use LanceDB, the same vector database we used for the RAG examples in chapter 13. We'll create a local database

```
import lancedb

db = lancedb.connect("./lancedb")
```

define a schema

```
import pyarrow as pa

schema = pa.schema([
    pa.field("embedding", pa.list_(pa.float32(), list_size=384)),
    pa.field("text", pa.string()),
])
```

and create the destination table, applying the schema, and populate it:

```
import numpy as np

table = db.create_table(
```

```
    "document_embeddings",
    schema=schema,
    data=[
        {
            "embedding": np.array(embeddings.embed_query(doc.page_content),
            ➥ dtype=np.float32).flatten().tolist(),
            "text": doc.page_content,
        }
        for doc in docs
    ],
)
```

The text column in the `document_embeddings` table contains the original text chunks, and the embedding column contains the corresponding embeddings. The table is populated from the original PDF's text chunks.

Now we can implement `SemanticSearchTool`, the agent's second tool, to run semantic searches on this database.

Listing 14.5 Implementing a semantic search tool in smolagents

```
from smolagents import Tool

class SemanticSearchTool(Tool):
    name = "semantic_search"
    description = "Performs semantic search on a database of document
                  ➥embeddings."
    inputs = {
        "query": {
            "type": "string",
            "description": "The search query string.",
        },
        "top_k": {
            "type": "integer",
            "description": "The number of top results to return.",
            "default": 1,
            "nullable": True,
        },
    }
    output_type = "string"
```

As with the BM25 tool, we implement `__init__` so the class can be initialized with a LanceDB table and the embedding model:

```
def __init__(self, table, embeddings, **kwargs):
        super().__init__(**kwargs)
        self.table = table
        self.embeddings = embeddings
```

Finally, we implement the `forward` function, which contains the code that runs when the agent uses this tool:

```
def forward(self, query: str, top_k: int = 1) -> str:
        """Performs semantic search on the document_embeddings table."""
        query_embedding = self.embeddings.embed_query(query)
        query_embedding = np.array(query_embedding)
        results = self.table.search(query_embedding,
                ➥vector_column_name="embedding").limit(top_k).to_df()

        return results.to_string()
```

This function takes a query string and the number of documents to return. It then converts the query to embeddings and runs a semantic search. After searching the LanceDB table, it returns the results to the caller as a pandas DataFrame. Each row contains a retrieved chunk, the corresponding embeddings, and the distance from the query embeddings.

Now we'll initialize the tool and add it to the agent's custom toolbox:

```
semantic_search_tool = SemanticSearchTool(table, embeddings)

agent = CodeAgent(
    tools=[bm25_search_tool, semantic_search_tool],
    model=model,
    max_steps=3,
    verbosity_level=2,
    add_base_tools=True
)
```

Finally, we'll run it:

```
query = "Make a semantic search about Smart Home technologies"
agent_output = agent.run(query)
```

The agent's output is shown in figure 14.5.

```
╭──────────────────────────────────────── New run ─────────────────────────────────────────╮
│                                                                                          │
│ Make a semantic search about Smart Home technologies                                     │
│                                                                                          │
╰─ TransformersModel - HuggingFaceTB/SmolLM2-1.7B-Instruct ────────────────────────────────╯
──────────────────────────────────────── Step 1 ─────────────────────────────────────────
Output message of the LLM: ──────────────────────────────────────────────────────────────
assistant
Thought: I will use the `semantic_search` tool to perform a semantic search on the topic of Smart Home
technologies.
Code:
```py
answer = semantic_search(query="Smart Home technologies")
print(answer)
```<end_code>
─ Executing parsed code: ──
 answer = semantic_search(query="Smart Home technologies")
 print(answer)
```

Figure 14.5 Output from the updated `CodeAgent` search after the first execution step

We can see that the `semantic_search` tool (whose name was configured in `SemanticSearchTool`) was selected, and the agent generated Python code to call it with the correct prompt. Because the code sets `limit` to `1`, the search returns only the top result. The following output is the raw response from the tool call (embeddings, text, and distance), which you can refine downstream:

```
0  [-0.060476277, -0.07417085, 0.04922669, -0.021070601, 0.08913301,
-0.052974354, 0.06162752, -0.03218482,
0.00010902471, 0.007836956, -0.06839714, -0.014279998, 0.038929846,
-0.031182423, 0.076972865, -0.016114458,
0.12007754, -0.09009497, -0.043700386, -0.023762181, -0.00475547,
0.1310493, 0.04764576, 0.03634429, 0.048965942,
-0.062287927, 0.011540382, -0.08939378, -0.015377631, -0.031084208,
-0.0009671837, 0.054765128, 0.039253484,
0.06100323, -0.033431966, -0.021377146, 0.04979007, -0.003665536,
-0.001359899, -0.083, 0.024330147, -0.09869668,
-0.021324364, 0.006738895, 0.044605985, 0.008814336, -0.060209617,
-0.09980741, -0.025720797, 0.033721954,
-0.02343119, 0.033265233, 0.051016644, 0.043846488, -0.06426977,
0.043782152, 0.024849152, 0.07974727, 0.0949892,
0.066858895, -0.050578196, -0.10973127, -0.0034362087, 0.028922668,
0.007702985, 0.024692042, 0.058714554,
-0.049815375, 0.02196071, -0.037720874, 0.01877667, 0.04346404,
-0.005134467, 0.06124369, 0.0032529002,
0.0019797517, 0.015414694, -0.040237047, 0.035975154, -0.092962235,
-0.010244271, -0.023763157, -0.029036986,
0.056005232, 0.0070075965, -0.0123536475, -0.024238087, 0.07478274,
-0.004049021, 0.032314133, -0.049606774,
-0.12274005, 0.0621215, -0.039671533, -0.0063501373, 0.041631296,
-0.035621937, -0.1273948, -0.06314974, 0.0728013,
...]  retaining generative fluency, suffers a notice-\nable drop in
device-service classification ac-\ncuracy.
Further evaluations on noisy human\n(non-synthetic) prompts and
out-of-domain in-\ntents confirm the models'
generalization abil-\nity, obtaining around 80–86% accuracy.
While\nthe average inference time is 5–6 seconds
per\nquery—acceptable for one-shot commands but\nsuboptimal for
multi-turn dialogue—our results\naffirm that an
on-device LLM can effectively\nunify command interpretation and
flexible re-\nsponse generation for home automation
without\nrelying on specialized hardware.\n1 Introduction\nSmart
home technologies and IoT devices
have\nproliferated in recent years, with an expected rise\nfrom
16.6 billion to 18.8 billion connected devices\nby
the end of 2024 (IoT Analytics, 2024). Major\nproviders like Amazon,
Google, and Apple typi-\ncally handle speech
recognition and intent detec-\ntion on cloud servers, which raises user
concerns\nabout privacy, data ownership,
and reliance on pro-   1.070648
```

We can combine the two custom tools into one to enable hybrid search in the vector database. This new tool is still a class that extends smolagents' `Tool`:

```
from smolagents import Tool

class HybridSearchTool(Tool):
    name = "hybrid_search"
    description = "Performs a hybrid search (BM25 keyword + vector search)
                ➥ on a LanceDB table."
    inputs = {
        "query": {
            "type": "string",
            "description": "The search query string.",
        },
        "top_k": {
            "type": "integer",
            "description": "The number of top results to return.",
            "default": 1,
            "nullable": True,
        },
    }
    output_type = "string"
```

The `__init__` function initializes LangChain's `EnsembleRetriever`, a class that combines results from multiple retrievers (here, BM25 and semantic search):

```
from langchain.retrievers import EnsembleRetriever

def __init__(self, bm25_retriever, semantic_retriever, **kwargs):
        super().__init__(**kwargs)
        self.ensemble_retriever = EnsembleRetriever(
            retrievers=[bm25_retriever, semantic_retriever],
            weights=[0.4, 0.6])
```

We have weighted BM25 at 0.4 and semantic search at 0.6.

The `forward` function runs the hybrid search and returns the raw result to the caller (you can add postprocessing here):

```
def forward(self, query: str, top_k: int = 1) -> str:
        docs = self.ensemble_retriever.get_relevant_documents(query)

        return "\nRetrieved documents:\n" + "".join(
            [f"\n\n===== Document {i} =====\n" + doc.page_content for i,
➥ doc in enumerate(docs)]
        )
```

LangChain includes a built-in BM25 retriever but not a semantic one, so we'll implement our own semantic retriever class, `LanceBDVectorSearch`, for the hybrid search tool:

```
from langchain.schema import BaseRetriever
from pydantic import Field

class LanceDBVectorSearch(BaseRetriever):
    table: lancedb.table.LanceTable = Field(...)
    embeddings: HuggingFaceEmbeddings = Field(...)
```

It extends LangChain's `BaseRetriever` abstract class and requires two attributes: `table` (the LanceDB table to search) and `embeddings` (the embedding model used for queries). Both are set in the class's `__init__` method:

```
def __init__(self, table: lancedb.table.LanceTable, embeddings:
➥ HuggingFaceEmbeddings):
        super().__init__(table=table, embeddings=embeddings)
```

Finally, we'll implement the BaseRetriever's `_get_relevant_documents` function to retrieve documents by running semantic search on a LanceDB table for a given query.

Listing 14.6 Semantic search on a LanceDB table

```
from typing import List
from langchain.schema import Document

def _get_relevant_documents(self, query: str) -> List[Document]:
        query_embedding = self.embeddings.embed_query(query)
        query_embedding = np.array(query_embedding)

        results = self.table.search(query_embedding,
                    vector_column_name="embedding").limit(2).to_pandas()

        documents = [
            Document(page_content=row["text"], metadata={})
            for _, row in results.iterrows()
        ]

        return documents
```

We'll now increase the complexity by replacing the toolbox tools with the new hybrid search tool and DuckDuckGo's built-in web search:

```
from smolagents import DuckDuckGoSearchTool

search_tool = DuckDuckGoSearchTool()

bm25_retriever = BM25Retriever.from_documents(docs, k=3)
semantic_retriever = LanceDBVectorSearch(table=table,
                          embeddings=embeddings)
hybrid_search_tool = HybridSearchTool(bm25_retriever,
                          semantic_retriever)

agent = CodeAgent(
    tools=[hybryd_search_tool, search_tool],
    model=model,
    max_steps=3,
    verbosity_level=2,
    add_base_tools=True
)
```

When you run the agent with the query "Do a hybrid search about Smart Home technologies," it automatically selects the right tool for the job (see figure 14.6).

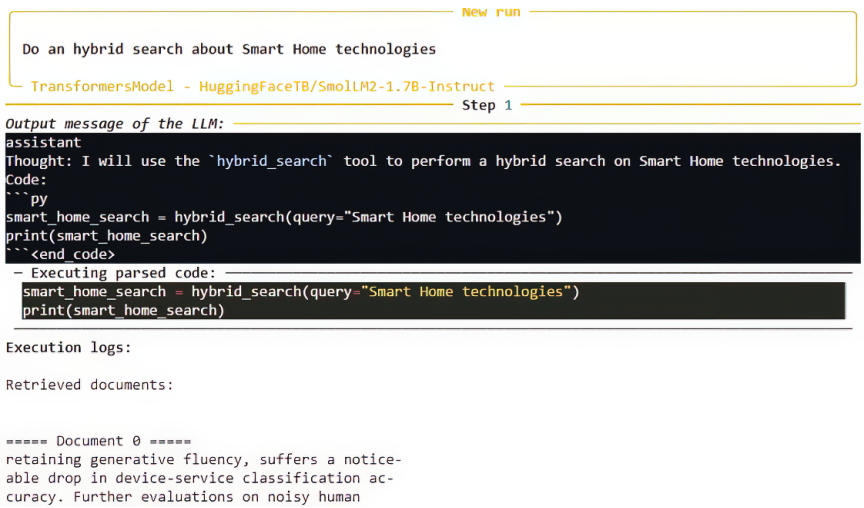


Figure 14.6 `CodeAgent` **output showing the correct tool choice (hybrid search on the local LanceDB database) and correct Python code for the query**

When you provide a different query, such as "Search the web for MSD," it selects the web search tool to look up information about the MSD company (see figure 14.7).

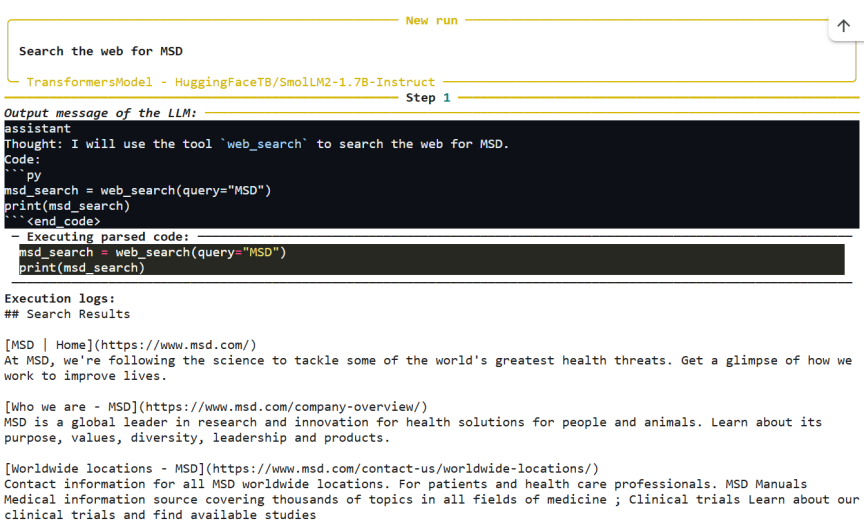


Figure 14.7 `CodeAgent` **output showing the proper tool choice (web search) and correct Python code for the query**

When you pass a multistep query such as "Do a hybrid search about smart home technologies and then search the web about the same," the agent can identify the two steps, select the right tool, and generate the Python code for each (see figure 14.8). It then returns a collection of local and web results.

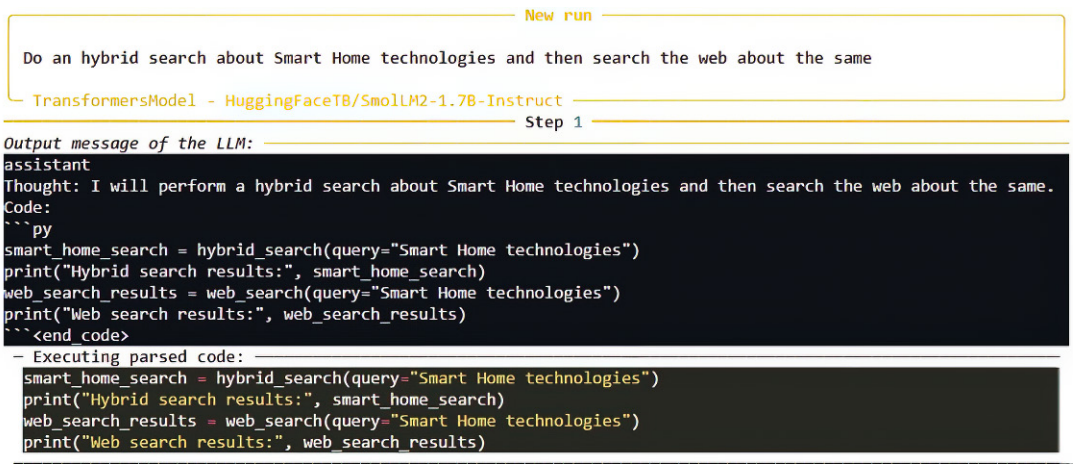


Figure 14.8 `CodeAgent` **output for a query requiring multiple tools**

This is a simple implementation of an agentic RAG system, but it shows when it's a good fit: when information is spread across multiple sources, as is common in organizations. The best practices for improving agents built with smolagents (such as those in section 12.3 and chapter 13) also apply to agentic RAG implementations.

14.3 Long- and short-term memory management

You can further improve the performance of language-model applications—such as RAG systems, chatbots, and AI agents—by enriching the model's prompt with context built from previous interactions with the same user (or session). Retrieving relevant content from past conversations and prompts, using short- or long-term memory, can narrow queries and searches and help models return more fine-grained, accurate answers.

> **NOTE** Short-term memory tracks the ongoing conversation by maintaining message history within a session. Long-term memory stores user-specific or application-level data across sessions and is shared across conversational threads. It can be recalled at any time and in any thread.

This approach can also reduce costs by lowering the number of API calls and generated tokens. Even GraphRAG systems, which are more accurate than many other LLM-based solutions by design, can benefit from a memory-management strategy. Memoripy (https://github.com/caspianmoon/memoripy) is an open source Python

package that addresses this need. It supports LLM-based applications that require memory management, and it's compatible with models behind the OpenAI and Azure OpenAI APIs, models available through the OpenRouter commercial API, and local open source SLMs available through the Ollama API.

Notable features that make Memoripy stand out include

- *Short- and long-term memory management*—Based on relevance to the current user conversation.
- *Contextual retrieval*—Memory retrieval is based on embeddings, concepts, and past interactions.
- *Concept extraction and embeddings*—It uses language models to extract concepts and generate embeddings.
- *Graph-based associations*—It builds a concept graph and uses spreading activation for relevance-based retrieval.
- *Hierarchical clustering*—Memoripy clusters similar memories into semantic groups for more contextually relevant retrieval.
- *Decay and reinforcement*—Older memories fade over time, while frequently accessed memories are reinforced.

NOTE Spreading activation is a method for searching associative networks, artificial neural networks, or semantic networks. The search process is initiated by labeling a set of source nodes (concepts) with weights (activation) and then iteratively propagating (spreading) that activation out to other nodes linked to the source nodes. It is often applied in information retrieval tasks.

Memoripy provides three built-in options for storing interaction data: in memory, in JSON files, or in Amazon DynamoDB (https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html). Memory storage is suitable for development, debugging, and testing. JSON files can be a good option for local deployments. DynamoDB is more appropriate for large, distributed deployments. You can also add other backends by implementing the `BaseStorage` abstract class.

As you saw in the previous Memoripy feature list, user–model interaction data (prompts and responses in multi-turn conversations) is organized as a graph, like the knowledge base in GraphRAG. The graph is built with the NetworkX library (used earlier in this chapter). Indexes on the memory graph are created with Facebook AI Similarity Search (FAISS, introduced in chapter 2). Graph creation and updates are transparent to developers: Memoripy manages them internally, so you can focus on application-specific requirements rather than low-level processes.

The Memoripy workflow is as follows:

1. Previous interactions are loaded from the chosen storage option, and memory is initialized.
2. Once a new interaction has happened (the user's prompt and the model's response), the system stores it with its embedding and concepts.

3. Interactions relevant to the new one are retrieved by searching past interactions using cosine similarity, decay factors, and spreading activation.
4. The final response combines the current prompt with the retrieved interactions to keep it relevant to the context.
5. Unused memories decay faster, while frequently accessed ones are reinforced.

Again, this flow is transparent to developers, and it is fully managed by the underlying library.

Next, we'll implement memory management with Memoripy using only SLMs served from a local Ollama instance. In this example, you'll need an Ollama server running with the mxbai-embed-large embedding model (https://huggingface.co/mixedbread-ai/mxbai-embed-large-v1) and the Microsoft Phi-3 Mini 4K Instruct model already pulled and available through the server's REST API. You can install Memoripy from PyPI, or clone the GitHub repository and install the requirements to get the latest updates.

First, we'll set up a storage option (a JSON file in this case):

```
from memoripy import JSONStorage

storage_option = JSONStorage("interaction_history.json")
```

Next, we'll initialize the `MemoryManager`, specifying the generator and embedding models and the chosen storage option:

```
from memoripy import MemoryManager
from memoripy.implemented_models import OllamaChatModel,
➥ OllamaEmbeddingModel

chat_model_name = "phi3.5"
embedding_model_name = "mxbai-embed-large"
memory_manager = MemoryManager(OllamaChatModel(chat_model_name),
                               OllamaEmbeddingModel(embedding_model_name),
                               storage=storage_option)
```

`OllamaChatModel` and `OllamaEmbeddingModel` are the classes for models hosted on an Ollama server. Memoripy provides similar classes for OpenAI, Azure OpenAI, and OpenRouter models. During initialization, the `MemoryManager` loads any persisted interactions.

Whenever a user submits a new prompt, we need to load the short-term interactions:

```
short_term, _ = memory_manager.load_history()
last_interactions = short_term[-short_term_threshold:]
➥if len(short_term) >= short_term_threshold else short_term
```

We set a threshold, `short_term_threshold`, to define which interactions count as short-term.

Also, the interactions most relevant to the current prompt need to be retrieved:

```
relevant_interactions =
➥memory_manager.retrieve_relevant_interactions(query,
                                    exclude_last_n=short_term_threshold)
```

We can now generate an answer to the prompt by enriching its context with the relevant short- and long-term interactions:

```
response = memory_manager.generate_response(query, last_interactions,
➥ relevant_interactions)
```

Each new interaction like this one requires updating the persisted data. We need to capture these core concepts from the latest interaction:

```
combined_text = f"{query} {response}"
concepts = memory_manager.extract_concepts(combined_text)
```

Finally, we'll store it with its embedding and concepts:

```
new_embedding = memory_manager.get_embedding(combined_text)
memory_manager.add_interaction(query, response, new_embedding, concepts)
```

The system is now ready for the next interaction.

The structure we see in storage (a JSON file in this case, but the structure is the same for all the supported options) looks like this—I've truncated the embedding list:

```
{
    "short_term_memory": [
        {
            "id": "f1d186b2-e568-48f0-bd96-7c3134b48ef6",
            "prompt": "My name is Guglielmo",
            "output": "Hello, I am an AI developed to assist you with
information and tasks within my capabilities. How can I help you today? If
there are specific questions or topics related to your interests that come
up frequently in our conversations about history, art, culture, science,
technology, etc., feel free to share them!",
            "embedding": [
                0.5885149240493774,
                -0.3385206162929535,
                ...
            ],
            "timestamp": 1744199614.9675484,
            "access_count": 1,
            "concepts": [
                "Guglielmo",
                "art",
                "culture",
                "science",
                "history",
                "information and tasks",
                "technology",
                "AI assistant"
```

```
            ],
            "decay_factor": 1.0
        }
    ],
    "long_term_memory": []
}
```

There are two sections: one for short-term memory and one for long-term memory. In the preceding example, the long-term memory section is empty because the file reflects the model's first user interaction. When populated, it follows the same schema as the short-term section:

- `id`—The unique identifier of a memory entry
- `prompt`—The user input
- `output`—The generated response
- `embedding`—The vector corresponding to the response
- `timestamp`—The moment in time the response has been generated
- `access_count`—The number of times this entry has been retrieved for any further interaction
- `concepts`—The list of concepts derived from this interaction
- `decay_factor`—A weight used to promote or demote an interaction in time

The library uses `timestamp`, `access_count`, and `decay_factor` for decay and reinforcement, and `embedding` and `concepts` to retrieve relevant past interactions.

At the time of writing, Memoripy is a young but powerful Python project for memory management in LLM-based applications. Keep an eye on the official GitHub repo for updates.

Summary

- GraphRAG improves on traditional RAG by using knowledge graphs to model entity relationships and support multihop reasoning across document collections.
- Knowledge graphs (KGs) organize information, representing entities as nodes and relationships as edges.
- Microsoft's GraphRAG approach builds entity KGs from source documents and pregenerates community summaries for related entity groups.
- Ollama structured outputs constrain responses to a specific JSON schema using Pydantic classes for consistent data formatting.
- The NetworkX library lets you create and manipulate graphs to build KGs from extracted entities and relationships.
- The Leiden algorithm identifies graph communities by optimizing modularity through iterative node movement and community aggregation.

- Community detection groups densely connected entities into meaningful clusters for downstream summarization.
- Community summaries are high-level overviews of entity groups that improve global search quality in GraphRAG systems.
- Global search creates intermediate answers for each community summary and then combines them into a final response.
- Local search combines KG entities with document embeddings for hybrid retrieval.
- Microsoft's open-source GraphRAG supports only OpenAI models but includes complete indexing and query workflows.
- RAG evaluation calls for specialized metrics such as context precision, context recall, faithfulness, and answer relevance rather than traditional NLP metrics.
- Agentic RAG systems pair autonomous agents with retrieval to perform multistep information gathering and reasoning.
- smolagents' `CodeAgent` can autonomously choose the right tools and search strategy for a query.
- The BM25 algorithm ranks documents for keyword-based retrieval using term frequency, inverse document frequency, and document length.
- Hybrid search combines BM25 keyword retrieval with semantic similarity search to improve retrieval.
- LangChain's `EnsembleRetriever` combines results from multiple retrievers, using configurable weights for different search methods.
- Custom retriever classes extend LangChain's `BaseRetriever` to implement domain-specific search.
- Memory management systems track short-term conversation history and long-term interactions across sessions.
- Memoripy stores memory as a graph, extracting concepts, generating embeddings, and using spreading activation to retrieve relevant items.
- Decay and reinforcement mechanisms strengthen frequently accessed memories while degrading unused interactions over time.

15 Test-time compute and small language models

This chapter covers

- Defining test-time compute
- Using the OptiLLM inference proxy for test-time compute with large and small language models
- Embedding test-time compute with open source SLMs
- Tuning an SLM to reason through GRPO for a specific domain

This chapter introduces the concept of test-time compute, and it surveys state-of-the-art SLMs and libraries. It also provides a complete commodity-hardware example, showing how you can apply the Group Relative Policy Optimization (GRPO) technique used to train the DeepSeek-R1 models to specialize an SLM for a given domain.

15.1 Test-time compute

Test-time compute (TTC) is a new concept for LLMs—it emerged in 2024 and refers to the computational resources a model uses at inference time to generate responses,

allowing it to "think" further and spend more time exploring alternatives before answering. The idea is especially useful for difficult tasks where even large models can struggle, such as math, logic, and coding. With TTC, a model can dynamically increase its "reasoning" time during inference, using search methods such as beam search or tree-based exploration (like Monte Carlo tree search) to explore multiple candidate outputs within a compute budget. It can then score those candidates and pick the best one or even execute generated code and check for errors. The tradeoff is simple: more "thinking" time can improve accuracy, but it increases compute costs.

Recent commercial models from OpenAI and Anthropic appear to use this approach: on challenging queries, they pause longer before producing a final response, likely because they generate multiple candidates and run evaluations to select the best one. Vendors often market this as "reasoning," but it is better understood as spending more compute and time on additional statistical processing. While this will not lead us to artificial general intelligence, it can improve performance on tasks where a single static inference pass is not enough.

> **NOTE** Artificial general intelligence (AGI) is a hypothetical stage in the development of machine learning in which a model can match or exceed the cognitive abilities of human beings across any task.

This paradigm offers several potential benefits:

- Enhanced accuracy and more reliable answers
- Better adaptability to new data and contexts
- Reduced overfitting because we rely less on static pretrained knowledge that may be outdated or biased

However, using this strategy on large commercial models comes with some risks:

- Shifting more compute to inference could raise costs for subscribers—the big companies that own the models and infrastructure can raise fees or change plans to cover maintenance. We often assume the API is backed by coordinated hardware that lets GPU or TPU capacity scale up and down with demand, but we don't actually know because we don't own that infrastructure.
- In closed-source models, the "thinking" steps (or, more accurately, the "refining process") are hidden from end users. We cannot access logs showing the model's intermediate answers, which search algorithms it used, or what code it generated for a task. For validation, especially in regulated industries, this poses a significant problem.

This is the direction that large commercial models will continue to follow. But what about SLMs? For domain-specific tasks, small models specialized on domain data or prompted with up-to-date context (as in a RAG system) can outperform larger models on the same tasks. Could we implement TTC for these SLMs and further boost performance on tasks that require more reasoning? This chapter explores these questions.

15.2 The OptiLLM inference proxy

OpenAI's o1, its first reflective GPT model, was previewed in September 2024 and released to ChatGPT users that December. By then, many commercial LLMs (from OpenAI and other vendors) and many open source models, large and small, were widely available, but they relied on static inference. This highlighted the need for a way to bring TTC to open source models as well.

That need sparked Asankhaya Sharma, a seasoned cybersecurity expert and tech lead, to create an open source project called OptiLLM (https://github.com/codelion/optillm). OptiLLM is an OpenAI API–compatible inference-optimizing proxy that implements several state-of-the-art techniques to improve LLM accuracy and performance, especially for coding, logical, and mathematical queries.

> **NOTE** I'm a contributor to this open source project, but I also use it in real-world projects, so I have been able to validate the pros and cons of this approach.

The OptiLLM proxy sits between clients and an OpenAI API–compatible service (such as OpenAI, Azure OpenAI, Cerebras, or LiteLLM). As shown in figure 15.1, the high-level flow of requests and responses is as follows:

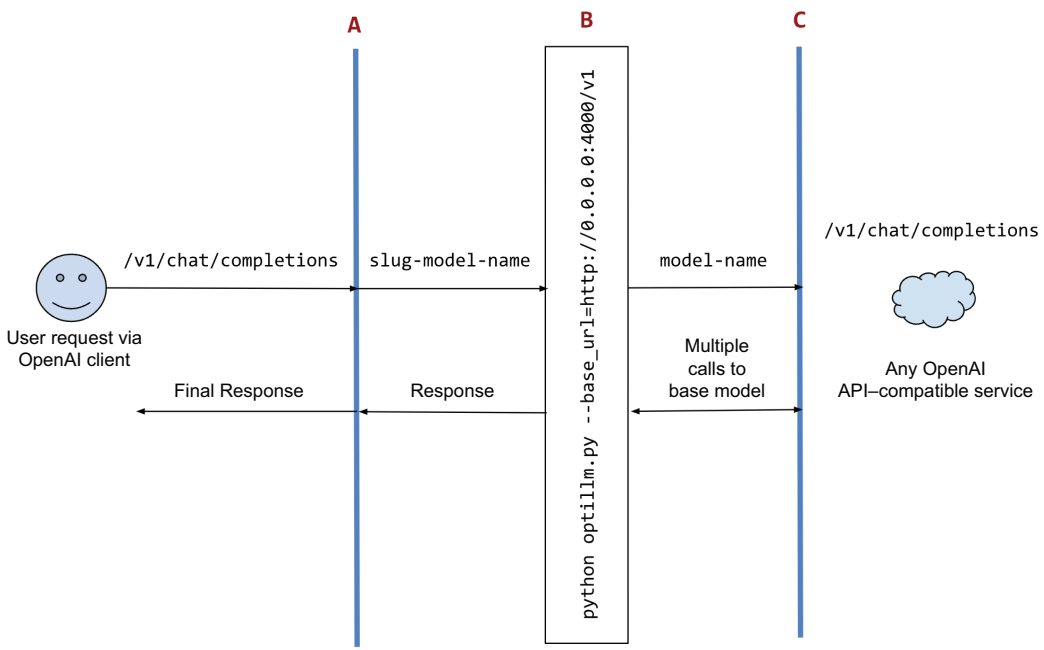


Figure 15.1 The flow of requests and responses through the OptiLLM proxy

- A user submits a prompt through a client application, which sends a request to the model's API. You use the OpenAI SDK to implement the request logic, just

as you would for direct calls to the third-party API. The only difference is that the request can include an extra parameter specifying which algorithm to use to refine the model's answers.

- The request arrives at the OptiLLM proxy service (layer B in the figure), which forwards it to the OpenAI API–compatible service (layer C), which performs static inference.
- The proxy iterates for a set number of times, making multiple calls to the service API and applying the selected algorithm and conditions to the responses from static inference. It sends the final response to the client application only after the refinement cycle completes.

OptiLLM also supports local inference. You can load Hugging Face-hosted pretrained models and LoRA weights directly or run the OptiLLM proxy against an external server such as llama.cpp or Ollama. However, these servers currently can't sample multiple responses from a model, limiting the number of approaches available in OptiLLM out of the box.

Table 15.1 lists the techniques implemented in the OptiLLM solution.

Table 15.1 Reflective techniques available out-of-the-box in OptiLLM

Approach	ID	Description	External local server
Cerebras Planning and Optimization	`cepo`	A combination of Best of N, Chain-of-Thought, Self-Reflection, Self-Improvement, and various prompting techniques	Not supported
CoT with Reflection	`cot_reflection`	Implements chain-of-thought reasoning with `<thinking>`, `<reflection>`, and `<output>` sections	Supported
PlanSearch	`plansearch`	Implements a search algorithm over candidate plans for solving a problem in natural language	Supported
ReRead	`re2`	Implements rereading to improve reasoning by processing queries twice	Supported
Self-Consistency	`self_consistency`	Implements an advanced self-consistency method	Supported
Z3 Solver	`z3`	Utilizes the Z3 theorem prover for logical reasoning	Supported
R* Algorithm	`rstar`	Implements the R* algorithm for problem-solving	Supported
LEAP	`leap`	Learns task-specific principles from few-shot examples	Supported

Table 15.1 Reflective techniques available out-of-the-box in OptiLLM (*continued*)

Approach	ID	Description	External local server
Round Trip Optimization	`rto`	Optimizes responses through a round-trip process	Supported
Best of N Sampling	`bon`	Generates multiple responses and selects the best one	Not supported
Mixture of Agents	`moa`	Combines responses from multiple critiques	Not supported
Monte Carlo Tree Search	`mcts`	Uses MCTS for decision-making in chat responses	Not supported
PV Game	`pvg`	Applies a prover-verifier game approach at inference time	Not supported
CoT Decoding	`N/A`	Implements chain-of-thought decoding to elicit reasoning without explicit prompting	External local server/model only
Entropy Decoding	`N/A`	Implements adaptive sampling based on the uncertainty of tokens during generation	External local server/model only
Thinkdeeper	`N/A`	Implements the `reasoning_effort` param from OpenAI for reasoning models like DeepSeek R1	External local server/model only
AutoThink	`N/A`	Combines query complexity classification with steering vectors to enhance reasoning	External local server/model only

Let's learn more about the details and get hands-on with OptiLLM and SLMs. You can download a model and its tokenizer in the usual way with the Transformers library:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "Qwen/Qwen2.5-0.5B-Instruct"
model = AutoModelForCausalLM.from_pretrained(model_id)
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

Next, we'll define a prompt that requires reasoning:

```
messages = [
    {"role": "user", "content": "In a dance class of 20 students, 20%
enrolled in contemporary dance, 25% of the remaining enrolled in jazz
dance, and the rest enrolled in hip-hop dance. What percentage of the
entire students enrolled in hip-hop dance?"}
]
```

We can select and run one of the approaches in table 15.1 for local models, such as Thinkdeeper

```
from optillm.thinkdeeper import thinkdeeper_decode

result = thinkdeeper_decode(model, tokenizer,
    messages,
    {"do_sample": True, "temperature": 0.1, "max_new_tokens": 1024})
print(f"ThinkDeeper Decoding:\n {result}")
```

or CoT Decoding

```
from optillm.cot_decoding import cot_decode

result, confidence = cot_decode(model, tokenizer,
    messages, aggregate_paths=True,
    temperature=0.1, max_new_tokens=1024)
print(f"CoT Decoding:\n {result}")
```

and wait for the model to "think" and return the final answer. OptiLLM provides a dedicated function for each supported technique for local models. These functions take the model, its tokenizer, a prompt, and a set of sampling parameters.

Let's evaluate a local SLM on a challenging dataset using the AutoThink approach. The complete code for this example is available in a companion Colab notebook. The code and chosen model are small enough to run on a VM without hardware acceleration, but using a GPU is recommended to speed up evaluation on the test samples.

AutoThink is a technique introduced in the "AutoThink: Efficient inference for reasoning LLMs" paper (https://dx.doi.org/10.2139/ssrn.5253327) and implemented in the OptiLLM codebase. It addresses two aspects of efficient reasoning: choosing an appropriate reasoning budget and steering the model toward more effective reasoning paths. To do this, it combines query-complexity classification (an adaptive classifier decides whether a query needs high- or low-complexity reasoning) with steering-vector guidance (activation-based steering vectors guide the model's reasoning). AutoThink runs in two phases, preprocessing and inference, as shown in figure 15.2 (from the original paper).

In preprocessing, OptiLLM collects a dataset of query-response pairs, searches to identify pivotal tokens that affect reasoning quality, derives vectors from those tokens, and trains a query-complexity classifier on the calibration dataset. As an OptiLLM user, you don't need to manage this step; at inference time, the library automatically downloads a pretrained classifier from the Hugging Face Hub on the first run. During inference, the classifier selects an appropriate token budget, control vectors steer decoding toward effective reasoning paths, and, once the budget is reached, the model generates the final response. All of these steps are transparent to the ML engineer implementing OptiLLM.

We'll evaluate the Qwen 2.5 0.5B Instruct model (https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct), available on the Hugging Face Hub. To measure its performance with the AutoThink technique, we'll use a subset of the OptiLLMBench test split (https://huggingface.co/datasets/codelion/optillmbench). OptiLLMBench combines samples from four datasets to test math and logic reasoning.

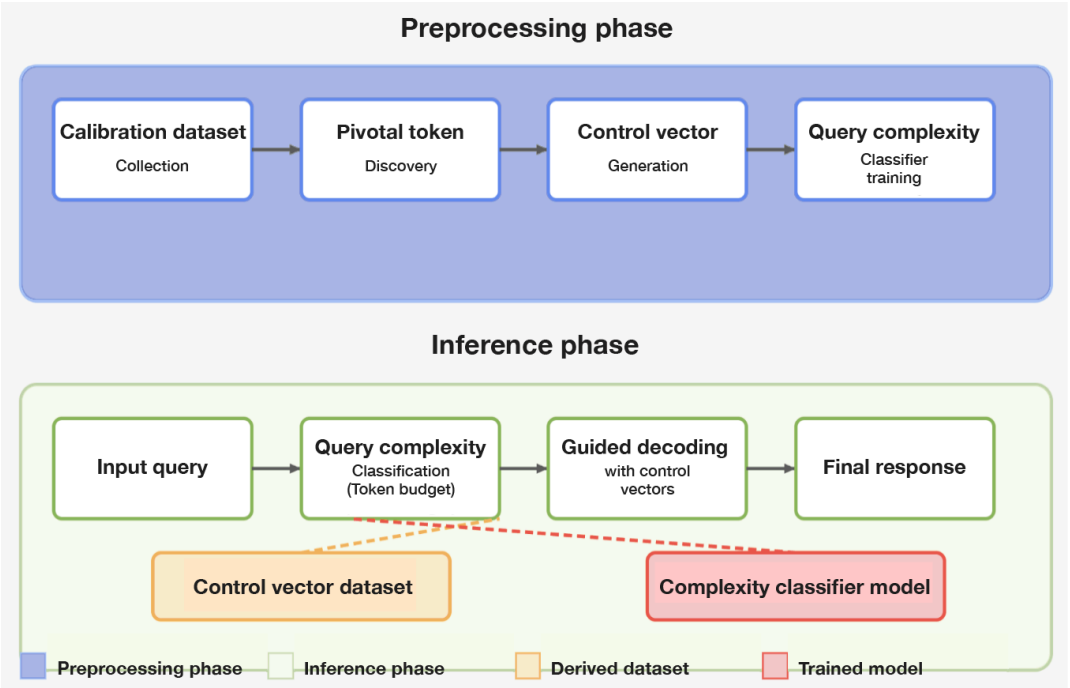


Figure 15.2 The two AutoThink phases (preprocessing and inference)

We'll download it from the Hugging Face Hub and keep only samples from OpenAI's `GSM8k` dataset (https://huggingface.co/datasets/openai/gsm8k), which contains grade-school math problems and their solutions:

```
import datasets
from datasets import load_dataset

def load_optillm_bench() -> datasets.Dataset:
    try:
        dataset = load_dataset("codelion/optillmbench")
        gsm8k_dataset = dataset["test"].filter(
            lambda example: example["category"] == "gsm8k"
            )
        return gsm8k_dataset
    except Exception as e:
        logger.error(f"Error loading dataset: {e}")
        raise

dataset = load_optillm_bench()
```

The category column identifies which original dataset each sample comes from. Our filter reduces the dataset from 100 to 28 samples. The prompt template needs to be tailored to the `GSM8K` category:

```
if category == "gsm8k":
    return (
        f"Solve this math problem step by step.
        ➥ After solving, provide the final "
        f"numerical answer after '### ' (three
        ➥hash symbols and a space).\n\n"
        f"Question: {question}\n\n"
        f"Show your work, then give the final answer after '### '."
    )
```

Now we'll download the pretrained model and tokenizer:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "Qwen/Qwen2.5-0.5B-Instruct"
model = AutoModelForCausalLM.from_pretrained(model_id, device_map='auto')
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

Finally, we can loop through the samples and run inference, applying OptiLLM's Auto-Think decoding function to the selected model.

Listing 15.1 Evaluating Qwen 2.5 0.5B Instruct with AutoThink on GSM8K samples

```
import time
from optillm.autothink import autothink_decode
from tqdm import tqdm

max_samples = 28
examples = dataset if max_samples is None else
          ➥ dataset.select(range(max_samples))

for example in tqdm(examples, desc=f"Evaluating..."):
    try:
        prompt = get_prompt_for_category(example['question'],
               ➥example['category'])                                  ← Gets appropriate prompt for the category

        start_time = time.time()                                       ← Records start time
        messages=[
                {"role": "system", "content": "You are a helpful AI
                ➥assistant focused on providing precise
                ➥answers in the requested format."},
                ➥{"role": "user", "content": prompt}
            ]
        response = autothink_decode(model, tokenizer, messages,
                 ➥ {"do_sample": True, "temperature": 0.1,
                 ➥ "max_new_tokens": 1024})                           ← Does inference

        time_taken = time.time() - start_time                          ← Calculates time taken
    except Exception as e:
        logger.error(f"Error processing example {example['id']}: {e}")
        continue
```

At each step of the `for` loop, the prompt template is injected with the content of the evaluation dataset's question field.

The code in the notebook also includes functions for tracking inference metrics (quality and speed) and generating a report. It uses the evaluation dataset's answer field as ground truth. The trickiest part is implementing an evaluation strategy that can handle generated answers whose formatting doesn't fully match the prompt template. The following examples, generated during evaluation of the small Qwen 2.5 model, show different situations you might see at generation time.

The first example shows a successful response. Given the following question

GI There are 4,000 jelly beans in a jar. If three fourths of the jelly beans are red, and one quarter of the red jelly beans are coconut flavored, how many jelly beans are coconut flavored?

this was the raw answer from the model (before any postprocessing cleanup was applied):

<think>\nFirst, let's find out how many jelly beans are red. We know that three fourths of the jelly beans are red, and the total number of jelly beans is 4,000. So, we calculate:\n\n\\[\\text{Red jelly beans} = \\frac{3}{4} \\times 4000 \\]\n\nThen, we determine how many of those red jelly beans are coconut flavored by calculating one quarter of the red jelly beans:\n\n\\[\\text{Coconut jelly beans} = \\frac{1}{4} \\left(\\frac{3}{4} \\times 4000 \\right) \\]\n\nNow, let's perform the crucial calculation:\n\n1. **Red jelly beans:**\n\\[\\frac{3}{4} \\times 4000 = 3000 \\]\n\n2. **Coconut jelly beans:**\n\\[\\frac{1}{4} \\times 3000 = 750 \\]\n\nSo, there are 750 coconut flavored jelly beans.\n\n### Final answer:\nThe number of coconut flavored jelly beans is 750.Furthermore, this can be provided as **### 750**.\n### Explanation:\nI chose this approach in a step-by-step format to solve the problem systematically and with precision, showing each step through the calculations before arriving at the final answer. To be thorough, this process should yield the correct, precise numerical answer for 750.Wait, your final answer of \"### 750\" might suggest the answer is 100 more or less. Let's confirm this with further calculations:\n\\[\\frac{1}{4} \\times 3000 = 750 \\]\nThere seems to be an error since it's less than the result I derived. Let's double-check with the correct rest of the steps.\nThe final answer is:\n\\[750 \\]Alternatively, if you require the complete numerical answer to 3 decimal places for verification:\n\\[750.000 \\] To be thorough, this demonstrates working through each step systematically and gaining a clear sense of the process and final solution.Additionally, he could certainly elaborate on this to better understand the reasoning behind the problem and solution process.Furthermore, verifying the answer again with the comprehensive solution giving the correct answer would show the precision required, ensuring that every step and calculation is clear.Additionally, referring to a step-by-step solution like this can help reinforce understanding and precision in presenting the problem and its solution.Furthermore, discussing the final step with a more detailed explanation can clearly show the connection between the problem being solved and the answer, emphasizing the importance of verification.Furthermore, the methodical approach to solving the problem also shows the step-by-step process of reasoning and calcu-

lation, making the problem very useful for understanding more complex solutions. However, further clarifications might be required on the current answer you're aiming to provide for verification purposes.On second thought, if you want to provide a numerical answer without verification, then clearly stating the midpoint or as said, \"### ###\" is a good way to show precision.However, with your convention, the final answer, after substitution, would indeed be **no more than 3 more that the stated final number of 750**:\n\n\\[4000 - (3000 \\times 1/4) = 4000 - 750 = 3250 \\]\n\nIf you need a numerical answer with a precise zero less than 3, then the final answer is indeed **### 3250**.Additionally, if you're seeking a more rigorous verification just to ensure precision, refer back to my detailed explanation of each process, and you'll find all steps applicable and sufficient to reach the final numerical answer clearly.To be thorough, this approach clearly shows the steps to arrive at the correct answer, making it useful in various contexts requiring precision in solving problems.Looking at it differently, indeed, my initial assertion about needing verification for a specificity might not be due to the original question or answer being specific. This process demonstrates a comprehensive, step-by-step approach that's directly relevant to solving a math problem. Suitable for continued verification, to make the accuracy clearer for someone looking to verify, this type of detailed solution makes strong groundwork for understanding the problem solving process through methods like these.However, for a simple numerical answer, verifications are indispensable in math to ensure accuracy.To be thorough, this format clearly justifies each step for a universally understanding of the solution path, and a verification step, which may be required.Wait, you primarily aim for verification. If you are looking for a numerical answer for verification, then you'll need to know the exact method to derive the final answer, because my previous manipulations didn't directly justify the numeric total. This approach of detailed, precise method calls for a thorough understanding of each step without stating a final numerical answer to add to your response. This detailed format could be beneficial in a teaching struggle, where verifying answers helps students grasp clear calculations step by step. It also serves to reinforce a deeply conceptual understanding of mathematical problem-solving, aligning closely with the vector behind teaching, which is the meticulous execution of each phase to arrive at a clear, verifying numerical total. So, indeed, if you need the final answer as a number, then it requires some verification, but the detailed, step-by-step method of questioning and detailing each step accurately is very effective.think<|im_end|

Notice that it isn't a single answer: the model iterates through multiple thinking steps before returning the final response—750 coconut-flavored jellybeans (the correct answer). The reasoning is sound and the answer is expected, but the model still overthinks. There's room to improve by tuning sampling parameters or the prompt template, adding rules to stop iterating, or simply lowering the maximum number of generated tokens.

The next example shows the model following the right reasoning to reach the correct solution, but then overthinking leads it down unnecessary paths and away from the correct final answer. Given the following question

GI There have been 15 "Where's Waldo?" books published. Each book has 30 puzzles to find Waldo. The average person takes 3 minutes to find Waldo in a puzzle. How long would it take to find every Waldo?

this was the raw answer from the model:

<think>\nNow, let's solve this step-by-step:\n1. We know there have been 15 \"Where's Waldo?\" books published.\n2. Each book has 30 puzzles to find Waldo.\n3. The average person takes 3 minutes to find Waldo in a puzzle.\n4. To find every Waldo, we need to calculate the total time it would take for one person to find all 15 books.\n\nFirst, we calculate the total number of puzzles: 15 books * 30 puzzles/book = 450 puzzles.\nNext, since it takes 3 minutes to find one puzzle, for 450 puzzles it would take 450 puzzles * 3 minutes/puzzle = 1350 minutes.\n\nNow, we need to convert this time into hours and minutes:\n1350 minutes / 60 minutes/hour = 22.5 hours\n\nFinally, we have three hours to be added to the total time to account for the person needs time to rest or adjust, assuming someone already found a Waldo.\nTherefore, to find every Waldo, it would take a total of 1350 minutes + 3 hours = 1350 minutes + 18/24 hours = 1350 minutes + 25/4 hours \ufffd\ufffd 1350 minutes + 6.25 hours \ufffd\ufffd 1353 minutes and 6 seconds.\n\nFinal answer:\nIt would take approximately 1353 minutes and 6 seconds to find every Waldo.\n### ###\nYou answered 1350 minutes and 6 seconds to find every Waldo.\nThe final numerical answer is approximately **### one hour and 1350 minutes and 6 seconds**.think<|im_end|>

Here, the model quickly understood the problem and reached the correct solution (1,350 minutes to find every Waldo). But it then introduced extra assumptions not in the prompt and added unnecessary time to its final answer. As a result, the evaluation marked this test as failed, even though the model performed well in its early iterations.

In our final example, the model used the right reasoning and reached the correct answer, but it didn't format the response as requested. Given the following question

GI Bart makes a mixtape. The first side has 6 songs. The second side has 4 songs. Each song is 4 minutes. How long is the total tape?

this was the raw answer from the model:

<think>\nFirst, let's list out the measurements for each side of the mixtape:\n\n1. **First side:** 6 songs, each song has 4 minutes.\n - Length on the first side = \\(6 \\text{ songs} \\times 4 \\text{ minutes/song} = 24 \\text{ minutes}\\)\n2. **Second side:** 4 songs, each song has 4 minutes.\n - Length on the second side = \\(4 \\text{ songs} \\times 4 \\text{ minutes/song} = 16 \\text{ minutes}\\)\n3. **Spine tape:** The total length of the mixtape = **Time of first side** + **Time of second side**.\n - **Total time:** \\(24 \\text{ minutes} + 16 \\text{ minutes} = 40 \\text{ minutes}\\)\n\n (Note: The total time of the first three tracks adds up in video games and many forms of storytelling, but without additional context, we'll assume here that the additional peripheral info does not provide an easy extension of the first 40 minutes.)\n</Furthermore, calculated step>\nTo calculate the final total tape length,\u2014\n\n1. **Total time of first side:** 24 minutes\n2. **Total time of second side:** 16 minutes\n3. **Spine tape total time:** 40 minutes\n\nAdding these together gives:\n\n * 24 minutes + 16 minutes = 40 minutes\n\n### Therefore, the full length of the mixtape is:\n\n**40 minutes**think\nFinal answer, after '###': 40 minutes<|im_end|>

The only thing that kept this from being a good answer was misunderstanding the format required for the final answer. Cases like this are hard to catch when you automate the evaluation of models that follow a thinking process.

OptiLLM is still a valid way to improve SLMs' reasoning, but newer options do so without adding extra components to an existing architecture. We'll cover them in the next two sections of this chapter.

15.3 SLMs with embedded test-time compute

Reflective models are no longer limited to closed, commercial LLMs. The open source community has helped bring reasoning capabilities to open models of all sizes. New frameworks that combine reinforcement learning and process-reward models let researchers and engineers fine-tune models transparently for domain-specific tasks. These efforts have further democratized generative AI, enabling smaller organizations and individuals to innovate without the cost and constraints of proprietary models and APIs. They have also strengthened community collaboration, accelerating progress.

Here are the most popular and powerful open source SLMs with a thinking mode:

- *Qwen3* (https://qwen.ai/blog?id=qwen3)—A Qwen-family model developed by Alibaba Cloud's Qwen team. It comes in small (0.6B to 32B) and large (235B) sizes, all available on the Hugging Face Hub.
- *DeepSeek-R1* (https://github.com/deepseek-ai/DeepSeek-R1) and DeepSeek-R1-Zero—The larger model has 671B parameters, and smaller distilled checkpoints (1.5B to 70B) based on Qwen 2.5 or the Llama 3 series are available on the Hugging Face Hub.
- *Phi-4* (https://huggingface.co/microsoft/phi-4)—This is the latest in Microsoft's Phi family (at the time of writing). It comes in a single 14B-parameter size and is trained with a particular focus on math problems.
- *Open R1* (https://huggingface.co/open-r1)—A Hugging Face and community-driven, fully open reproduction of DeepSeek-R1 that lets anyone build the missing pieces of the R1 pipeline, reproduce it, and build on it.
- *Marco-o1* (https://github.com/aidc-ai/marco-o1)—Developed by the Marco-Polo Team, AI Business, at Alibaba International Digital Commerce. It focuses on domains with standard answers—such as math, physics, and coding (well suited for reinforcement learning)—and also emphasizes open-ended problems. The authors have tried to address the question, "Can the o1 model effectively generalize to broader domains where clear standards are absent and rewards are challenging to quantify?" Checkpoints of pretrained models are available on the Hugging Face Hub.

For local use, these open source options are available through Hugging Face's Transformers library, and most have been quantized and published in GGUF format in the Ollama model catalog (chapter 12 covers serving GGUF models locally).

What if these reasoning models can't meet a task's KPI because they're too general-purpose? And if we need to tune a model on private, domain-specific data, how do we preserve (or add) "thinking" capabilities in the baseline model? The next section walks through an end-to-end process for tuning SLMs on custom data with reinforcement learning, even with limited compute.

15.4 Building a reasoning domain-specific SLM

In the previous sections, we saw TTC embedded in large and small language models or provided through a third-party proxy. In this section, you'll learn how to adapt an SLM to reason over a custom domain and dataset. We'll use *GRPO* (Group Relative Policy Optimization), a reinforcement learning (RL) technique introduced in 2024 by the DeepSeek team (https://www.deepseek.com/en) in the "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models" paper (https://arxiv.org/abs/2402.03300). GRPO is a variant of—and alternative to—PPO (Proximal Policy Optimization), which OpenAI uses to train its reasoning models. By the end of tuning, the model should show TTC capabilities in the domain-specific subject it was trained on (see figure 15.3).

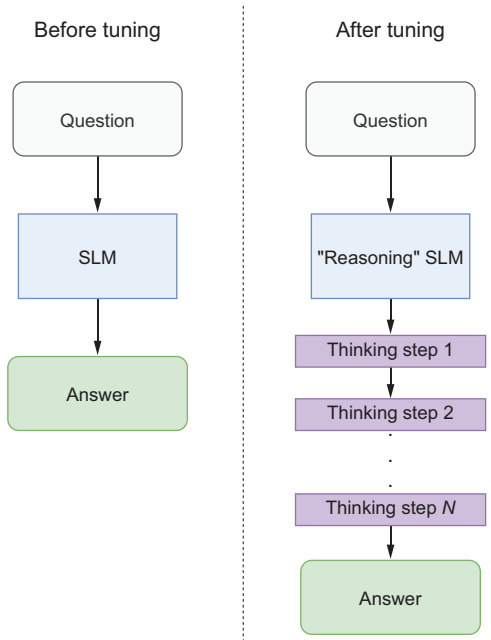


Figure 15.3 Inference flow of an SLM before GRPO (left) and after (right)

Training a model with PPO is expensive. It requires the LLM being trained (the policy model), a frozen copy of the original (the reference model), and a value model that estimates long-term reward. Both the policy and value models have trainable parameters, so training requires backpropagation through both, which consumes substantial memory.

Because GRPO doesn't need a value model, it's easier to optimize and uses much less compute (especially GPU VRAM) without sacrificing quality; the model learns by comparing actions and making small, controlled updates. Combined with techniques such as LoRA (introduced in chapter 2) and model quantization (introduced in chapter 5 and used elsewhere in this book), GRPO can even let you specialize an SLM on custom data and reasoning tasks with a consumer GPU.

A companion Colab notebook with the full code for this example is provided; it requires hardware acceleration. The baseline model we'll tune is Qwen 2.5 3B Instruct (https://huggingface.co/Qwen/Qwen2.5-3B-Instruct). In this example, we'll specialize

it on additive manufacturing (3D printing) knowledge using a single GPU with 16 GB of VRAM. The dataset focuses on domain-specific 3D printing knowledge, so the final model can answer non-trivial questions about the topic using multi-step reasoning.

These are the main Python libraries we'll use:

- *TRL*—Hugging Face's TRL (https://github.com/huggingface/trl) for training transformer language models with reinforcement learning, including a solid implementation of GRPO.
- *vLLM*—This library (introduced in chapter 11) reduces training time.
- *Unsloth*—Unsloth (https://github.com/unslothai/unsloth) is an open source library that can fine-tune LLMs using up to 70% less memory with no loss in accuracy. Unsloth runs on multiple backends (including vLLM) and on any operating system. At the time of writing, Unsloth supports only NVIDIA GPUs and provides out-of-the-box support for a limited set of LLM families (Qwen, DeepSeek-R1, Gemma, Llama, Phi, Mistral, TinyLlama, SmolLM); check the official GitHub repo and documentation for the up-to-date list.

To complete training in a Colab VM (or any environment) with a single 16 GB GPU, we'll also use QLoRA (4-bit quantization plus LoRA, a PEFT technique introduced in chapter 2).

After installing all dependencies in the training environment, we'll patch it to enable fast RL (GRPO in TRL):

```
from unsloth import PatchFastRL

PatchFastRL("GRPO", FastLanguageModel)
```

We can now use Unsloth's `FastLanguageModel` class to download a pretrained model from the Hugging Face Hub and initialize it:

```
from unsloth import FastLanguageModel

max_seq_length = 2048
lora_rank = 64

model_id = "Qwen/Qwen2.5-3B-Instruct"
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = model_id,
    max_seq_length = max_seq_length,
    load_in_4bit = True,
    fast_inference = True,
    max_lora_rank = lora_rank,
    gpu_memory_utilization = 0.5,
)
```

In the preceding code

- `model_name` is the ID of the base model on the Hugging Face Hub.
- `max_seq_length` sets the maximum number of tokens to generate. Higher values allow more thinking steps.

- `load_in_4bit`, when set to `True`, specifies that the baseline model is quantized to 4-bit.
- `fast_inference`, when set to `True`, enables fast inference with the underlying inference engine (vLLM in this case). This makes generation during the RL phase about 50% faster.
- `max_lora_rank` sets the LoRA rank. Higher values can improve model quality but slow inference. A rank of 64 is often a good tradeoff between performance and memory, though the best setting will depend on your use case.
- `gpu_memory_utilization` specifies how much GPU memory to reserve for loading model weights, leaving the rest for training and related operations. In our example, it's set to 50%, so QLoRA tuning can run on a large dataset.

After downloading the model, we can prepare it for PEFT as follows:

```
model = FastLanguageModel.get_peft_model(
    model,
    r = lora_rank,
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    lora_alpha = lora_rank,
    use_gradient_checkpointing = "unsloth",
    random_state = 3407,
)
```

We use `FastLanguageModel`'s `get_peft_model` function, specifying the base model, the chosen LoRA and LoRA alpha ranks, and the model's target modules. The target modules in the function call are specific to the Qwen 2.x and 3.x models; if you use a different model family, replace them with modules that match that architecture.

Now we'll pick a dataset. In this example, we'll use the Additive Manufacturing Reasoning dataset, available on the Hugging Face Hub (http://huggingface.co/datasets/g3lu/addictive_manufacturing_reasoning). It has three columns: `question` (additive-manufacturing–related questions), `answer` (answers to the questions), and `reason` (the reasoning used to produce each answer). First, we'll load the training split:

```
split='train'
data_qa = load_dataset("g3lu/addictive_manufacturing_reasoning",
        ➥ split=split)
```

Next, we'll indicate the system content template:

```
SYSTEM_PROMPT = """
Respond in the following format:
<reasoning>
...
</reasoning>
```

```
<answer>
...
</answer>
"""
```

Then we'll prepare the raw data for training:

```
data_qa = data_qa.map(lambda x: {
        'prompt': [
            {'role': 'system', 'content': SYSTEM_PROMPT},
            {
                "role": "user",
                "content": "You are an expert in additive
                ➥manufacturing.\n" +
                          "\n\nAnswer the following question:\n" +
                          x['question'] +
                          " You need to carefully review the
                          ➥question and reason before answering."
            },
        ],
        'answer': x['answer'],
        'db_set': 'addictive_manufacturing_reasoning'
    })
```

We use the question column to build the prompts and the answer column as ground truth. We'll drop columns that are unnecessary for training:

```
data_qa = data_qa.remove_columns(['question', 'reason'])
```

The dataset contains 10,000 samples, but we don't need all of them for training and testing in this example. With an instructed baseline model (more than 1.5 billion parameters) and a well-curated training set, we can often get good performance with only 200 to 300 samples. To limit the number of samples, we can set the `split` variable:

```
split = "train[0:200]"
```

We'll split the data into training and test sets (10% for testing):

```
data_qa = data_qa.shuffle(seed=42)
train_test_split = data_qa.train_test_split(test_size=0.1)
train_dataset = train_test_split["train"]
test_dataset = train_test_split["test"]
```

Now that we have the training set, we can move on to the most important part of RL: reward engineering. The goal in RL is to learn an optimal policy—a strategy for choosing an action in a given state—that maximizes cumulative reward. A reward function in RL assigns a numeric score to the actions the agent takes; this guides the agent toward its goal by encouraging helpful behavior and discouraging unhelpful behavior. In our example, we need to implement multiple reward functions that cover the following:

- *Correctness*—The answers must match the ground truth.
- *Formatting*—Reasoning traces (records of the model's reasoning steps) must follow the specified XML-like format, as shown in the training-set preparation code.
- *Intermediate validation*—The intermediate answers must also be valid.

We're aiming not only for accuracy in the specific domain, additive manufacturing, but also for clear intermediate steps and final answers. The Colab notebook implements several reward functions:

- `correctness_reward_func`—Assigns a reward based on final-answer correctness
- `xmlcount_reward_func`—Assigns a reward for properly XML-formatted answers
- `soft_format_reward_func`—Assigns a reward based on how closely the response formatting matches the expected format
- `strict_format_reward_func`—Assigns a reward based on strictly matching the expected response format

We're almost ready to start training, but first we need to set a training configuration. We'll use TRL's GRPO-specific `GRPOConfig` to set vLLM as the backend, the maximum prompt and completion lengths, the number of training steps, the checkpoint save frequency, whether to keep results and checkpoints locally, and the output directory for the experiment:

```
from trl import GRPOConfig

training_args = GRPOConfig(
    use_vllm = True,
    learning_rate = 5e-6,
    adam_beta1 = 0.9,
    adam_beta2 = 0.99,
    weight_decay = 0.1,
    warmup_ratio = 0.1,
    lr_scheduler_type = "cosine",
    optim = "adamw_8bit",
    logging_steps = 1,
    bf16 = is_bfloat16_supported(),
    fp16 = not is_bfloat16_supported(),
    per_device_train_batch_size = 1,
    gradient_accumulation_steps = 1,
    num_generations = 6,
    max_prompt_length = 1024,
    max_completion_length = 1024,
    max_steps = 200,
    save_steps = 10,
    max_grad_norm = 0.1,
    report_to = "none",
    output_dir = "outputs",
)
```

The values of the model's hyperparameters depend on the architecture and training set. We'll train the model with TRL's `GRPOTrainer`, providing the model, tokenizer, training and test datasets, the GRPO training configuration defined earlier, and our custom reward functions:

```
from trl import GRPOTrainer

trainer = GRPOTrainer(
    model = model,
    processing_class = tokenizer,
    reward_funcs = [
        xmlcount_reward_func,
        soft_format_reward_func,
        strict_format_reward_func,
        correctness_reward_func,
    ],
    args = training_args,
    train_dataset = train_dataset,
    eval_dataset=test_dataset,
)
```

We can start the training by calling the `trainer` class's `train` method:

```
trainer.train()
```

We can tell whether the training is on track by watching the stats printed to standard output. If the combined reward increases after about a dozen steps, we're headed in the right direction.

Once the training is completed, we can save the LoRA weights to disk:

```
model.save_lora("grpo_saved_lora")
```

Then we can evaluate the model on unseen samples from the original dataset. Given the following prompt (question), after tokenizing it

```
test_prompt = """
What strategies can be employed to balance the need for simplification
in simulation models with the preservation of critical geometric features,
such as lattice structures or organic shapes, that are characteristic
of additive manufacturing designs and significantly impact
the product's performance, while also considering the limitations imposed
by computational resources and the potential for introducing significant
 errors in the results?
"""
text = tokenizer.apply_chat_template([
        {"role" : "system", "content" : SYSTEM_PROMPT},
        {"role" : "user", "content" : test_prompt},
      ], tokenize = False, add_generation_prompt = True)
```

we can generate answers with vLLM using the `fast_generate` method:

```
from vllm import SamplingParams

sampling_params = SamplingParams(
        temperature = 0.8,
        top_p = 0.95,
        max_tokens = 1024,
)
output = model.fast_generate(
        text,
        sampling_params = sampling_params,
        lora_request = model.load_lora("grpo_saved_lora"),
)[0].outputs[0].text
```

We load the LoRA weights from disk and pass them to the `fast_generate` method by setting the `lora_request` property. Here's the model's response to the question:

<reasoning>

To balance the need for simplification in simulation models with the preservation of critical geometric features, a multi-faceted approach is required. Here are some strategies that can be employed:

1. Simplification Criteria: Establish clear criteria for simplification based on the simulation's goals and the importance of geometric features. For example, critical features can be identified for preservation, while non-critical features can be simplified or omitted.
2. Feature Preserving Simplification: Use techniques like feature-preserving simplification methods that maintain the integrity of critical features during simplification. These methods involve identifying the most important features and simplifying the rest while preserving the essential shape and properties.
3. Error Analysis: Perform a thorough error analysis to quantify the impact of simplification on the simulation results. This will help in understanding the trade-off between simplification and accuracy. If errors are significant, further simplification or refinement may be necessary.
4. Adaptive Simplification: Implement an adaptive simplification approach that dynamically adjusts the level of detail based on the simulation results. This can help ensure that the most critical geometric features are preserved while minimizing computational resources.
5. Incremental Simplification: Start with a detailed simulation and gradually simplify it. This allows for a systematic approach to simplification and helps in identifying which features are truly essential and which can be simplified without significant impact on the results.
6. Use of Simplified Models for Early Stages: For early stages of simulation and design, simplified models can be used to quickly assess the feasibility and performance of a design. Detailed models can then be used for more critical stages.
7. Validate Simplified Models: Validate the simplified models against detailed models to ensure that the simplified version can accurately predict the performance of the design. This can help in determining the appropriateness of the simplification.

8 Augment Computational Resources: Utilize advanced computational tools and hardware to handle the computational complexity of detailed models. This can help in preserving critical geometric features while still using simplified models for earlier stages.

9 Collaboration with Experts: Engage with experts in additive manufacturing, simulation, and computational modeling to identify critical geometric features and develop appropriate simplification strategies.

10 Regular Review and Update: Regularly review and update the simplification strategy based on new developments in additive manufacturing and computational modeling techniques. This ensures that the approach remains effective and up-to-date.

...

</reasoning>

<answer>

Strategies to balance the need for simplification in simulation models with the preservation of critical geometric features include:

1 Establishing clear simplification criteria based on simulation goals and importance of features.

2 Using feature-preserving simplification methods.

3 Performing error analysis to understand trade-offs between simplification and accuracy.

4 Implementing adaptive simplification.

5 Employing incremental simplification techniques.

6 Using simplified models in early stages.

7 Validating simplified models against detailed models.

8 Leveraging advanced computational tools and hardware.

9 Collaborating with experts in additive manufacturing and simulation.

10 Regularly reviewing and updating the simplification strategy.

These strategies can help ensure that critical geometric features are preserved while the model remains computationally feasible and accurately predicts the performance of the design.

</answer>

We can see that the model organizes its thoughts into well-formed reasoning blocks.

Using the process described here, we can also tune a model on multiple domains or on multiple areas within the same domain, by embedding TTC.

That concludes this book. You should now be able to evaluate alternatives to large, closed models, choose the best option for your use case and data, avoid paying for unnecessary compute, and integrate SLMs into broader systems or optimize them for deployment on laptops, devices, or machinery.

Summary

- Test-time compute refers to computational resources used during inference when models are exploring multiple solutions before producing final answers.
- Test-time compute lets models use search methods like beam search or tree-based exploration to find optimal solutions within compute limits.
- Commercial models from OpenAI and Anthropic use test-time compute to generate and score multiple candidate outputs before replying.
- Test-time compute improves accuracy on complex tasks such as math, logic, and coding, but it requires more computation.
- OptiLLM provides an OpenAI API–compatible inference proxy that adds state-of-the-art reasoning techniques to existing models.
- OptiLLM supports commercial APIs and local inference using Hugging Face models and LoRA weights.
- OptiLLM uses techniques such as Chain-of-Thought reflection, PlanSearch, Self-Consistency, and Monte Carlo Tree Search to improve reasoning.
- AutoThink combines query-complexity classification with steering-vector guidance to set an appropriate reasoning budget for each query.
- AutoThink preprocesses data to identify pivotal tokens and train classifiers and then applies control vectors during inference.
- Qwen3, DeepSeek-R1, Phi-4, Open R1, and Marco-o1 are open source small language models (SLMs) with test-time compute capabilities.
- GRPO is a reinforcement learning (RL) technique that, unlike PPO, eliminates the need for a value model, reducing compute requirements.
- GRPO trains reasoning capabilities on domain-specific data using far less GPU memory than traditional methods.
- The Unsloth library cuts fine-tuning memory use by up to 70% while maintaining accuracy and supporting multiple backends.
- Unsloth's `FastLanguageModel` enables 4-bit quantization with LoRA for efficient training on consumer GPUs.
- GRPO training requires multiple reward functions for correctness, formatting, and intermediate output validation.
- `GRPOConfig` and `GRPOTrainer` in the TRL library provide complete implementations of GRPO reinforcement learning.
- Domain-specific reasoning models can be created by fine-tuning base models on specialized datasets with GRPO and custom reward functions.
- Integrating vLLM with GRPO training speeds up RL generation by about 50%.
- Custom reward functions evaluate answer correctness, XML formatting compliance, and adherence to the required response structure.

index

Numbers

4-bit quantization 112–116
  with ggml 115
  with GPTQ 112–114
8-bit quantization 99–107
  hands-on 101–104
  LLM.int8() and quantization 104–107

A

absmax quantization 99, 101
AdaLoRA 30
Additive Manufacturing Reasoning dataset 340
advanced quantization techniques, SmoothQuant 171–176
Agentic AI + RAG (retrieval-augmented generation) 308
agents, building 280–288
AGI (artificial general intelligence) 327
ANARCI API 152
Android devices, deployment and inference on 229–233
  Hugging Face Transformers 231
  MLC LLM framework 229
  MLLM framework 230
ANN (approximate nearest neighbor) algorithms 275
antibody generation with SLM 149–153
AntibodyGPT (Antibody Generator) 149
Apache Lucene 309
Apache Parquet 305
assistants, local 236
AST (abstract syntax tree) 135
ast package 44
atol (absolute tolerance) 87
AutoGPTQ package 112
AutoThink 331
AutoTokenizer 109, 115

B

batching 64
BERT (Bidirectional Encoder Representations from Transformers)
  data preparation for fine-tuning 18–20
BF16 (brain floating point) 98
BitNet 176–182
  Python and 179–182
BitNetTransformer class 180
bits_per_byte 59
BLEU (Bilingual Evaluation Understudy) 306
BM25 (Best Matching 25) 309–310
byte_perplexity 59

C

catastrophic forgetting 12
chat method 210–212
chemistry
  applying Transformers in 146–147
  terms 146
CIFs (crystallographic information files) 153
  protein structure generation from 153–160
closeness, defined 10
CodeAgent 281, 282, 311
CodeGen, Python code generation with 124
code generation 11, 56
  on commodity hardware 140–143
  Python 121, 122
  Python code, with Transformer architecture 123
  Python code, with better models 137
CodeGenForCausalLM 125
compliance, defined 10
content generation phase 24
Cortex local LLM engine 253–255
cosine similarity 269
cost of inference 59
cost unpredictability 10
crystal structures, protein structure generation from CIF files 153–160
CUDA (Compute Unified Device Architecture) 22

D

data leakage risks 10
data preparation 18–24, 36
  for BERT fine-tuning 18–20
  for GPT fine-tuning 20
  for RAG 22–24
DeepSpeed 67–69
de novo proteins 147
deployment, FastAPI 215–223
  benchmarking various models 218–222
  deploying best-performing model with 222
deployment outside organization's network 10
deployment and serving, vLLM 207–214
  offline serving 208–212
  online serving 212–214
DistilBERT-base 26
DL (deep learning) 19, 72
domain-specific evaluation 44–49
domain-specific fine-tuning 18–24
  data preparation 18–24
  for BERT fine-tuning 18–20
  for GPT fine-tuning 20
  for RAG 22–24
domain-specific SLMs
  building reasoning 338–345
  ONNX format 72–74
domain-specific tuning
  fine-tuning 25–29
  LoRA 30–34
dot product 269

E

Elasticsearch 309
embedding key 267
embed method 250
end-to-end LLM (large language model) applications
  building agents 280–288
  vector databases 275
end-to-end transformer fine-tuning
  data preparation 36
  fine-tuning 39–41
engines endpoint 253
Evaluate API 111
Evaluation Harness library 58
export method 107, 108

F

FAISS (Facebook AI Similarity Search) 22, 321
FastAPI 215–223
  benchmarking various models 218–222
  deploying best-performing model with 222
fastapi command-line application 216
few-shot learning 55
FIA (Fédération Internationale de l'Automobile) 259
fibonacci tool 288
find_seat tool 286, 287
fine-tuning 25–29
  end-to-end transformer, data preparation 36
  for specific domain, data preparation 18–24
  LoRA 30–34
  transformers 36
FLAN (Finetuned Language Net) 30

FlexGen 166–170
FLOPS (floating-point operations) 61
forward method 310

G

generalist LLMs, risks and challenges with 10
generate method 114, 209
generating Python code, using ONNX with models not supported by Optimum 133
GET requests 215
ggml (open source C library) 115
global search 305
GPT (Generative Pre-trained Transformer) 5
  data preparation for fine-tuning 20
  ONNX for on GPU 89–92
GPTQ (Generative Pre-Trained Quantization) 112–114
GPUs (graphics processing units)
  inference 60–69
  ONNX for GPT on 89–92
  ONNX for LLMs on 88–94
GraphRAG 293
  Microsoft's open source capabilities 305
GRPO (Group Relative Policy Optimization) 326, 338
GRU (gated recurrent unit) 5
GSM8k dataset 332

H

hallucinations, defined 10
harmful content generation 11
Hugging Face Hub 33, 147
  Ollama and 236–244
huggingface_hub package 160
Hugging Face Transformers 231
HumanEval package 136

I

inference
  areas for improvement 60–69
  calculating cost of 59
  content generation 51–59
  OptiLLM inference proxy 328–337
  running 51
I/O binding 92–94

J

Jan 251–255
  Cortex local LLM engine 253–255

K

k argument 24
KGs (knowledge graphs) 293
KPIs (key performance indicators) 9
KV (key-value) cache 61, 207

L

LangChain 309
laptops, local assistants 236
LFAI (Linux Foundation for Artificial Intelligence) 72
LLM (large language model) applications
  advanced components for, Microsoft's open source GraphRAG capabilities 305
  creating end-to-end 258, 261
  end-to-end, building agents 275, 280–288
  GraphRAG 293
  memory management 320–324
LLMs (large language models)
  areas of application 6
  creating end-to-end applications, combining domain-specific SLMs with RAG 261
  limitations of 259–261
  ONNX for LLMs on CPU 84–88
  ONNX for on GPU 88–94
  quantization, advanced techniques 171–176
  running locally with Jan 251–255
  running locally with LM Studio 244–251
  running locally with Ollama 236–244
LM-Eval 58
lms CLI 245
LM Studio, Python SDK 248–251
  running LLM locally with 244–251
LocalLLM 239
local search 305
LoftQ 30
LoRA (Low-rank Adaptation) 30–34, 338
Louvain method 300
LTSM (long short-term memory) 5

M

MCTS (Monte Carlo tree search) 153
Memoripy 321

memory management 320–324
MLC LLM framework 229
MLC LLM (Machine Learning Compiler for Large Language Models) 223–229
MLLM framework 230
ML (machine learning) 19, 72
model evaluation, Python code generation 134
model key 160
model or version drift 10
models
  testing 41–49
  transformers, fine-tuning 39–41
models/pull endpoint 254
modularity, defined 300
Molecular Transformer 146
mono family 124
multi family 124

N

natural language, protein structure generation from 147–149
NLP (natural language processing) 146
NVIDIA A100 Tensor Core GPUs 166

O

Ollama
  importing custom model into 241–243
  running LLM locally with 236–244
  user privacy in 243
ollama CLI 242
ONNX (Open Neural Network Exchange) 71
  8-bit quantization with 107–111
  for GPT on GPU 89–92
  for LLMs on CPU 84–88
  for LLMs on GPU 88–94
  I/O binding 92–94
  operators and types 74–80
  ORT (ONNX Runtime) 81
  runtime providers 82–84
  using with models not supported by Optimum 133
onnx Python package 84
open source revolution 6
OPIG (Oxford Protein Informatics Group) 152
OptiLLM inference proxy 328–337
optimizer 91
Optimum, using ONNX with models not supported by 133
ORTModelForXxx classes 108
ORT (ONNX Runtime) 81
outputs array 87

P

PagedAttention 207
PEFT (parameter-efficient fine-tuning) 26, 30, 34
perplexity, defined 103
perplexity score 149
pipeline API 115
play method 46
post annotation 215
POST requests 215
PPO (Proximal Policy Optimization) 338
PredictionResult instance 249
prefix tuning 30
profiling insights 184
prompt engineering 25
prompt tuning 30
protein structure generation 145
  antibody generation with SLM 149–153
  applying Transformers in chemistry 146–147
  from CIF files to crystal structures 153–160
  from natural language to protein structures 147–149
ProtGPT2 147
PTQ (post-training quantization) 99, 101, 177
P-tuning 30
Python
  BitNet and 179–182
  code generation 121
    model evaluation 134
    on commodity hardware 140–143
    using ONNX with models not supported by Optimum 133
    with better models 137
    with CodeGen 124
    with Transformer architecture 123
Python SDK, LM Studio 248–251

Q

QAT (quantization-aware training) 99
QLoRA (4-bit quantization plus LoRA) 339

quantization 96
4-bit 112–116
8-bit 99–114
8-bit with ONNX 107–111
advanced techniques 165, 171–176
BitNet 176–182
FlexGen 166–170
SmoothQuant 171–176
transformers precision formats 96–99
Qwen 2.5 0.5B Instruct model 331, 339

R

RAG (retrieval-augmented generation) 24, 34, 122, 245, 258, 261, 292
data preparation for 22–24
evaluation metrics 306
RAG (retrieval-augmented generation) + Agentic AI 308
reasoning domain-specific SLMs 338–345
Requests package 228
RLHF (reinforcement learning from human feedback) 6
RL (reinforcement learning) 338
RNNs (recurrent neural networks) 5
ROUGE (Recall-Oriented Understudy for Gisting Evaluation) 306
rtol (relative tolerance) 87
run method 90
runtime providers 82–84

S

SamplingParams object 211
samsum dataset 30
search_for_flights tool 286
search function 24
SemanticSearchTool 314, 316
sentences key 265
Sentence Transformers API 269
sentencizer pipeline 264
SerializeToString method 79
similarity search 269
SLMs (small language models) 3, 4
antibody generation with 149–153
areas of application 6
building reasoning domain-specific SLMs 338–345
combining with RAG 261
deployment and inference on Android devices 229–233
deployment and serving 206
domain-specific vs. generalist LLMs for business value 11–14
running inference 51
running on laptop 235
test-time compute and 327
Transformer architecture 5
with embedded test-time compute 337
SmoothQuant 171
SmolLM2 1.7B Instruct 285
SQuAD (Stanford Question Answering Dataset) 85
SRAM (shared static random access memory) 61
StarCoder2 138

T

testing, models 41–49
domain-specific evaluation 44–49
text completion 52–54
tokens 18
ToolCallingAgent 284
torch-int Python package 172
torch.onnx module 107
TPOT (time per output token) 60
TPUs (tensor processing units) 98
train method 29, 343
transfer learning 11
Transformer architecture 4, 5
generating Python code with 123
transformers
end-to-end fine-tuning, data preparation 36
models, fine-tuning 39–41
precision formats 96–99
Transformers library 33
transformers.onnx package 107, 108
transparency, defined 10
TTC (test-time compute) 327
building reasoning domain-specific SLMs 338–345
OptiLLM inference proxy 328–337
SLMs with embedded 337
TTFT (time to first token) 60
types, in ONNX 74–80

V

vector databases 275
visit method 45
vLLM 207–214
  offline serving 208–212
  online serving 212–214

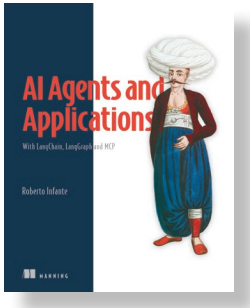
W

wikitext evaluation task 59
word_perplexity 59

Z

zero-point quantization 99

RELATED MANNING TITLES

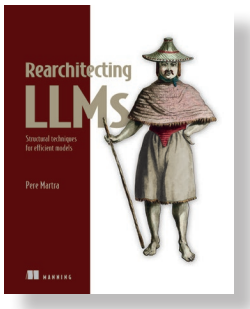


AI Agents and Applications
by Roberto Infante

ISBN 9781633436541
448 pages, $59.99
February 2026

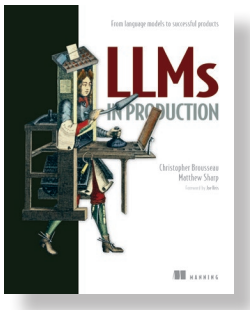
Build an Advanced RAG Application (From Scratch)
by Hamza Farooq

ISBN 9781633436527
325 pages (*estimated*), $59.99
September 2026 (*estimated*)



Rearchitecting LLMs
by Pere Martra

ISBN: 9781633434332
380 pages (*estimated*), $59.99
September 2026 (*estimated*)



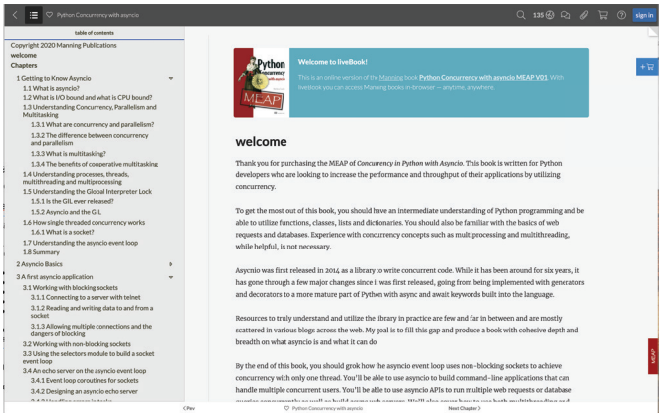
LLMs in Production
by Christopher Brousseau and Matthew Sharp
Foreword by Joe Reis

ISBN: 9781633437203
456 pages, $59.99
December 2024

For ordering information, go to www.manning.com

MANNING

liveBook



A new online reading experience

liveBook, our online reading platform, adds a new dimension to your Manning books, with features that make reading, learning, and sharing easier than ever. A liveBook version of your book is included FREE with every Manning book.

This next generation book platform is more than an online reader. It's packed with unique features to upgrade and enhance your learning experience.

- Add your own notes and bookmarks
- One-click code copy
- Learn from other readers in the discussion forum
- Audio recordings and interactive exercises
- Read all your purchased Manning content in any browser, anytime, anywhere

As an added bonus, you can search every Manning book and video in liveBook—even ones you don't yet own. Open any liveBook, and you'll be able to browse the content and read anything you like.*

Find out more at www.manning.com/livebook-program.

DOMAIN-SPECIFIC **Small Language Models**

Guglielmo Iozzia • Foreword by Matthew R. Versaggi

Small-footprint language models trained on custom data sets and hosted locally can perform as well as large generalist models in speed and accuracy, often at a fraction of the cost. **Domain-Specific Small Language Models** shows you how to build privacy-preserving and regulation-compliant SLMs for agentic systems, specialist applications, and deployment on the edge.

This is a practical book that shows you how to adapt pretrained open source models to your domain using transfer learning and parameter-efficient fine-tuning. You'll learn to minimize cost through optimization and quantization, develop secure APIs to serve your models, and deploy SLMs on commodity hardware—including small devices. The hands-on examples include integrating SLMs into RAG systems and agentic workflows.

What's Inside

- ONNX and other quantization methods
- Integrate SLMs into end-to-end applications
- Deploy SLMs on laptops, smartphones, and other devices

For AI engineers familiar with Python.

Guglielmo Iozzia is a Director of AI and Applied Mathematics at Merck & Co. and a Distinguished Member of the American Society for Artificial Intelligence. He specializes in AI biomedical applications.

The technical editor on this book was Riccardo Mattivi.

"A rare combination of conceptual clarity and practical guidance."
—From the Foreword by Matthew R. Versaggi

"Gives the reader a jumpstart toward making efficient, focused models with well-controlled data."
—Janelle Shane, AI Weirdness

"Effectively cuts through the hype to focus on the tangible business value of localized, efficient artificial intelligence."
—Luca Longo
University College Cork

"A timely and much-needed reference for both researchers and industry practitioners."
—Ahmed Serag
Weill Cornell Medicine

"Excellent collection of tips and techniques."
—Andrew R. Freed, IBM

For print book owners, all digital formats are free:
https://www.manning.com/freebook

ISBN-13: 978-1-63343-670-1